Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
Câmpus Birigui

Camila Albieri Mattos

Plataforma Personalizável para Criação e Compartilhamento de Coleções Digitais

Birigui
2026

Camila Albieri Mattos

Plataforma Personalizável para Criação e Compartilhamento de Coleções Digitais

Trabalho de Conclusão de Curso apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, como requisito parcial para conclusão do curso de Engenharia da Computação.

Área de Concentração: Engenharia da Computação

Orientadora: Dra. Helen de Freitas Santos

Birigui

2026

RESUMO

Este trabalho apresenta o desenvolvimento de uma plataforma *web* para a criação e o compartilhamento de coleções digitais de qualquer natureza, motivada pela rigidez das ferramentas usuais de registro, que tratam cada registro como um conjunto de campos fixos de aparência padronizada. Na plataforma, o acervo organiza-se em categorias, coleções e itens, e a ficha de cada item é definida pelo próprio usuário em um editor visual de *layouts*, com elementos de seis tipos. Completam a proposta a personalização da interface por paletas de cores e tipografia e as funcionalidades sociais de amizade, acompanhamento de coleções, *feed* e notificações, sob a privacidade definida em cada coleção. O desenvolvimento partiu da especificação dos requisitos e do modelo de domínio, seguiu uma arquitetura em camadas, com interface em React, API em NestJS e banco de dados PostgreSQL, e foi verificado por testes automatizados, verificação manual e medições de desempenho. A plataforma foi publicada e está em operação. Os 546 testes automatizados, de unidade e de ponta a ponta, foram aprovados, os vinte e quatro requisitos funcionais especificados para esta versão foram atendidos e as métricas de desempenho foram cumpridas com folga no ambiente local de medição: com a rede limitada a 10 Mbps, as telas medidas atingiram o maior elemento pintado em menos de meio segundo no percentil 75, e o editor respondeu às interações medidas em até 48 milissegundos. Conclui-se que a plataforma reúne, em um só lugar, a organização de coleções, a liberdade de definir a estrutura e a aparência de cada registro e o compartilhamento com outras pessoas.

Palavras-chave: Coleções digitais. Personalização de interfaces. Editor de *layouts*. Aplicação *web*. Compartilhamento.

ABSTRACT

This work presents the development of a web platform for creating and sharing digital collections of any kind, motivated by the rigidity of the usual record-keeping tools, which treat each record as a set of fixed fields with a standardized appearance. In the platform, the user's holdings are organized into categories, collections and items, and the card of each item is defined by the user in a visual layout editor with six types of elements. The proposal is completed by the customization of the interface through color palettes and typography and by social features of friendship, collection following, feed and notifications, under the privacy set for each collection. The development started from the specification of the requirements and the domain model, followed a layered architecture, with a React interface, a NestJS API and a PostgreSQL database, and was verified through automated tests, manual verification and performance measurements. The platform has been published and is in operation. All 546 automated tests, unit and end-to-end, passed, the twenty-four functional requirements specified for this version were met, and the performance metrics were met by a wide margin in the local measurement environment: with the network limited to 10 Mbps, the measured screens painted their largest element in less than half a second at the 75th percentile, and the editor responded to the measured interactions within 48 milliseconds. It is concluded that the platform brings together, in a single place, the organization of collections, the freedom to define the structure and the appearance of each record, and sharing with other people.

Keywords: Digital collections. Interface customization. Layout editor. Web application. Sharing.

LISTA DE FIGURAS

Figura 1 – Gerenciamento de Coleções 117
Figura 2 – Interações Com Itens da Coleção 118
Figura 3 – Personalização da Plataforma 118
Figura 4 – Layout e Templates 119
Figura 5 – Perfil e Social 120
Figura 6 – Autenticação 121
Figura 7 – Notificações 122
Figura 8 – Modelo de Domínio 185
Figura 9 – Esquema da tela inicial 187
Figura 10 – Esquema da tela de coleções do usuário 188
Figura 11 – Esquema da listagem de itens de uma coleção 188
Figura 12 – Esquema da biblioteca de layouts 189
Figura 13 – Esquema do editor de layout com a área de edição vazia 190
Figura 14 – Esquema do editor de layout com uma ficha composta 191
Figura 15 – Esquema da tela de amigos 192
Figura 16 – Esquema do perfil de outro usuário 193
Figura 17 – Esquema do perfil do próprio usuário 194
Figura 18 – Esquema da tela de configurações de aparência 195
Figura 19 – Diagrama de implantação do sistema 244
Figura 20 – Diagrama de atividades da publicação de uma nova versão . . . 255

LISTA DE QUADROS

Quadro 1 – Configuração do equipamento utilizado no desenvolvimento . . 46
Quadro 2 – Conceitos centrais do modelo de domínio 67
Quadro 3 – Descrição dos Campos dos Requisitos Funcionais 68
Quadro 4 – Grupos de Requisitos Funcionais 68
Quadro 5 – Classificação de Prioridade dos Requisitos 69
Quadro 6 – Classificação de Operação dos Requisitos 69
Quadro 7 – Atores do Sistema 69
Quadro 8 – RF-01 – Cadastrar Usuário 70
Quadro 9 – RF-02 – Autenticar Usuário (Login e Logout) 71
Quadro 10 – RF-03 – Alterar Senha na Área Restrita 72
Quadro 11 – RF-04 – Recuperar Senha via E-mail 73
Quadro 12 – RF-05 – Gerenciar Perfil do Usuário 74
Quadro 13 – RF-06 – Gerenciar Categoria 75
Quadro 14 – RF-07 – Gerenciar Coleção 77
Quadro 15 – RF-08 – Gerenciar Privacidade da Coleção 80
Quadro 16 – RF-09 – Gerenciar Item 81
Quadro 17 – RF-10 – Mover Item entre Coleções 82
Quadro 18 – RF-11 – Exibir Ficha de Item em Modo Leitura 83
Quadro 19 – RF-12 – Visualizar Itens de uma Coleção 84
Quadro 20 – RF-13 – Exibir Tela Principal de Categorias 86
Quadro 21 – RF-14 – Gerenciar Layout Personalizado 88
Quadro 22 – RF-15 – Associar Layout a Coleção 90
Quadro 23 – RF-16 – Compartilhar Layout 92
Quadro 24 – RF-17 – Modificar Layout de Item Individualmente 93
Quadro 25 – RF-18 – Gerenciar Aparência (Tema e Cores) 95
Quadro 26 – RF-19 – Gerenciar Tipografia Global 97
Quadro 27 – RF-20 – Gerenciar Amizade 97
Quadro 28 – RF-21 – Seguir Coleção Visível 99
Quadro 29 – RF-22 – Exibir Feed de Atualizações 100

Quadro 30 – RF-23 – Visualizar Perfis e Coleções de Usuários 101
Quadro 31 – RF-24 – Gerenciar Notificações do Sistema 102
Quadro 32 – RNF-01 – Usabilidade 105
Quadro 33 – RNF-02 – Desempenho de Carregamento 106
Quadro 34 – RNF-03 – Segurança de Senhas e Dados Pessoais 107
Quadro 35 – RNF-04 – Compatibilidade e Responsividade 108
Quadro 36 – RNF-05 – Personalização Visual sem Quebras de Layout 109
Quadro 37 – RNF-06 – Desempenho na Movimentação de Itens 110
Quadro 38 – RNF-07 – Desempenho do Editor de Layout 111
Quadro 39 – RNF-08 – Disponibilidade do Sistema 112
Quadro 40 – RNF-09 – Acessibilidade 113
Quadro 41 – RNF-10 – Segurança e Integridade dos Dados 114
Quadro 42 – Especificação do Caso de Uso: Fazer Login 123
Quadro 43 – Especificação do Caso de Uso: Realizar Logout 126
Quadro 44 – Especificação do Caso de Uso: Alterar Senha 128
Quadro 45 – Especificação do Caso de Uso: Recuperar Senha via E-mail . . 131
Quadro 46 – Especificação do Caso de Uso: Editar Perfil 134
Quadro 47 – Especificação do Caso de Uso: Personalizar Interface 137
Quadro 48 – Especificação do Caso de Uso: Criar Categoria 140
Quadro 49 – Especificação do Caso de Uso: Criar Coleção 144
Quadro 50 – Especificação do Caso de Uso: Gerenciar Privacidade da Coleção 147
Quadro 51 – Especificação do Caso de Uso: Criar Item 150
Quadro 52 – Especificação do Caso de Uso: Exibir Ficha de Item em Modo Leitura 153
Quadro 53 – Especificação do Caso de Uso: Visualizar Itens da Coleção . . . 156
Quadro 54 – Especificação do Caso de Uso: Visualizar Dashboard de Categorias 159
Quadro 55 – Especificação do Caso de Uso: Criar Layout 162
Quadro 56 – Especificação do Caso de Uso: Compartilhar Layout 167
Quadro 57 – Especificação do Caso de Uso: Adicionar Amigo 170
Quadro 58 – Especificação do Caso de Uso: Gerenciar Solicitações de Amizade 174
Quadro 59 – Especificação do Caso de Uso: Visualizar Perfil de Usuário . . . 177
Quadro 60 – Especificação do Caso de Uso: Visualizar Feed de Atualizações 181
Quadro 61 – Tecnologias e versões utilizadas na implementação 214

Quadro 62 – Estrutura de diretórios do *backend* 218
Quadro 63 – Estrutura de diretórios do *frontend* 221
Quadro 64 – Distribuição dos testes de unidade por módulo 228
Quadro 65 – Caso de teste CT01: validar a revogação da sessão no *logout* . . 229
Quadro 66 – Caso de teste CT02: validar a redefinição de senha por *token* temporário 230
Quadro 67 – Caso de teste CT03: validar o encerramento de acompanhamentos ao restringir a privacidade 231
Quadro 68 – Caso de teste CT04: validar o acesso a coleções conforme privacidade e amizade 232
Quadro 69 – Caso de teste CT05: validar a criação da ficha própria de um item 233
Quadro 70 – Caso de teste CT06: validar a paleta em uso 234
Quadro 71 – Caso de teste CT07: validar a resposta a uma solicitação de amizade 235
Quadro 72 – Caso de teste CT08: validar o formato real da imagem enviada 236
Quadro 73 – Testes de ponta a ponta da API 238
Quadro 74 – Alvos de implantação do sistema 247
Quadro 75 – Variáveis de ambiente de produção 248
Quadro 76 – `Dockerfile` da API 249
Quadro 77 – Declaração do contêiner da API e configuração do Caddy . . . 251
Quadro 78 – Síntese da validação dos requisitos funcionais 264
Quadro 79 – Tempo de resposta da API, em milissegundos (n = 50 por rota) 265
Quadro 80 – Carregamento das telas no navegador a 10 Mbps, em milissegundos desde o início da navegação (n = 10 por linha) 265
Quadro 81 – Síntese da validação dos requisitos não funcionais 269

LISTA DE ABREVIATURAS E SIGLAS

1D	Uma dimensão
2D	Duas dimensões
3D	Três dimensões

LISTA DE SÍMBOLOS

α	Letra grega minúscula Alfa
β	Letra grega minúscula Beta

SUMÁRIO

1 INTRODUÇÃO 16
1.1 Justificativa 17
1.2 Objetivos 18
1.2.1 Objetivo Geral 18
1.2.2 Objetivos Específicos 18
1.3 Problema 19
1.4 Organização Deste Trabalho 19

2 REVISÃO DA LITERATURA 21
2.1 Coleções e Colecionismo Digital 21
2.1.1 Colecionismo e suas Motivações 21
2.1.2 Colecionismo Digital 23
2.1.3 Propriedade Psicológica 25
2.1.4 Registro e Memória 26
2.2 Personalização de Interfaces 27
2.3 Manipulação Direta e Editores Visuais 30
2.4 Redes Sociais e Compartilhamento de Coleções 32
2.5 Engenharia de Requisitos 35
2.6 Modelagem Conceitual com UML 36
2.7 Arquitetura de Aplicações *Web* 38
2.8 Considerações sobre o Capítulo 42

3 MATERIAIS E MÉTODOS 44
3.1 Metodologia da Pesquisa 44
3.1.1 Pesquisa Bibliográfica 44
3.1.2 Pesquisa Aplicada 45
3.2 Materiais 46
3.2.1 Equipamentos 46
3.2.2 Softwares 46

3.3 Ferramentas Utilizadas . . . 47
3.3.1 Visual Studio Code . . . 47
3.3.2 Node.js e NestJS . . . 47
3.3.3 PostgreSQL e Docker . . . 48
3.3.4 Prisma . . . 48
3.3.5 DBeaver . . . 49
3.3.6 React e Vite . . . 49
3.3.7 Figma . . . 50
3.3.8 Astah Community . . . 51
3.4 Métodos de Desenvolvimento . . . 51
3.4.1 Elicitação e especificação de requisitos . . . 51
3.4.2 Modelagem conceitual . . . 52
3.4.3 Definição arquitetural . . . 52
3.4.4 Prototipação . . . 53
3.4.5 Implementação . . . 53
3.4.6 Gestão do projeto . . . 54
3.4.7 Estratégia de Testes . . . 54

4 DESCRIÇÃO GERAL DO PRODUTO . . . 56
4.1 Perspectiva do Produto . . . 56
4.2 Funções do Produto . . . 56
4.3 Características dos Usuários . . . 57
4.4 Restrições Gerais . . . 58
4.5 Diferenciais do Produto . . . 58

5 ELICITAÇÃO DE REQUISITOS E ANÁLISE . . . 59
5.1 Requisitos do Usuário . . . 59
5.1.1 Organização em Pacotes . . . 62
5.2 Requisitos do Sistema . . . 64
5.2.1 Requisitos Funcionais . . . 65
5.2.2 Requisitos Não-Funcionais . . . 104
5.2.3 Restrições, Suposições e Dependências . . . 114
5.2.4 Requisitos Adiados . . . 115

5.3 Casos de Uso . . . 116
5.3.1 Diagrama de Casos de Uso . . . 116
5.3.2 Especificação dos Casos de Uso . . . 122
5.3.2.1 Fazer Login . . . 123
5.3.2.2 Realizar Logout . . . 126
5.3.2.3 Alterar Senha . . . 128
5.3.2.4 Recuperar Senha via E-mail . . . 131
5.3.2.5 Editar Perfil . . . 134
5.3.2.6 Personalizar Interface . . . 137
5.3.2.7 Criar Categoria . . . 140
5.3.2.8 Criar Coleção . . . 144
5.3.2.9 Gerenciar Privacidade da Coleção . . . 147
5.3.2.10 Criar Item . . . 150
5.3.2.11 Exibir Ficha de Item em Modo Leitura . . . 153
5.3.2.12 Visualizar Itens da Coleção . . . 156
5.3.2.13 Visualizar Dashboard de Categorias . . . 159
5.3.2.14 Criar Layout . . . 162
5.3.2.15 Compartilhar Layout . . . 166
5.3.2.16 Adicionar Amigo . . . 170
5.3.2.17 Gerenciar Solicitações de Amizade . . . 174
5.3.2.18 Visualizar Perfil de Usuário . . . 177
5.3.2.19 Visualizar Feed de Atualizações . . . 181
5.4 Modelo de Domínio . . . 184

6 PROJETO DE SOFTWARE . . . 186
6.1 Projeto de Interface . . . 186
6.2 Projeto de Dados . . . 195
6.2.1 Mapeamento Objeto-Relacional . . . 196
6.2.2 Estrutura das Tabelas no Banco de Dados . . . 197
6.2.2.1 Detalhamento da Estrutura das Tabelas . . . 199

7 IMPLEMENTAÇÃO . . . 213
7.1 Ambiente de Desenvolvimento . . . 213

7.2 Servidor de Dados . . . 214
7.3 Servidor de Aplicação . . . 215
7.3.1 Armazenamento de Arquivos . . . 217
7.4 Organização do Código do *Backend* . . . 218
7.5 Aplicação Cliente . . . 220
7.5.1 Comunicação com a API . . . 221
7.5.2 Editor de Layouts . . . 222
7.5.3 Aparência e Tipografia . . . 223
7.6 Verificação Durante a Implementação . . . 224
7.7 Limitações da Implementação . . . 224

8 TESTE . . . 226
8.1 Ambiente e Execução dos Testes . . . 226
8.2 Visão Geral dos Testes . . . 227
8.3 Casos de Teste . . . 227
8.4 Testes de Ponta a Ponta da API . . . 237
8.5 Resultados e Limitações dos Testes Automatizados . . . 238
8.6 Verificação Manual . . . 239
8.7 Medições de Desempenho . . . 240
8.7.1 Ambiente e Acervo . . . 240
8.7.2 Tempo de Resposta da API . . . 240
8.7.3 Medições no Navegador . . . 241
8.7.4 Cálculo dos Percentis . . . 242

9 IMPLANTAÇÃO . . . 243
9.1 Infraestrutura . . . 243
9.2 Configuração dos Serviços . . . 248
9.2.1 Contêiner da API . . . 248
9.2.2 Superfície Exposta . . . 250
9.2.3 Continuidade de Execução . . . 252
9.3 Procedimento de Implantação . . . 252
9.3.1 Provisionamento Inicial . . . 252
9.3.2 Publicação de Novas Versões . . . 254

9.3.3 Reversão 255

10 RESULTADOS E DISCUSSÃO 257
10.1 A Plataforma em Operação 257
10.2 Validação dos Requisitos Funcionais 257
10.2.1 Acesso ao Sistema 258
10.2.2 Coleção 259
10.2.3 Editor de *Layout* 260
10.2.4 Aparência 260
10.2.5 Interação Social 261
10.2.6 Notificação 261
10.2.7 Síntese 262
10.3 Validação dos Requisitos Não Funcionais 262
10.3.1 Resultados das Medições 263
10.3.2 Requisitos Mensuráveis 265
10.3.3 Síntese da Validação 268
10.4 Validação em Uso 271
10.5 Discussão e Implicações 271

11 CONCLUSÕES 273
11.1 Objetivos Alcançados 273
11.2 Limitações 274
11.3 Ameaças à Validade 275
11.4 Trabalhos Futuros 275

REFERÊNCIAS BIBLIOGRÁFICAS 277

1 INTRODUÇÃO

Colecionar é uma prática presente em diferentes fases da vida e nos mais diversos interesses: há quem reúna livros lidos, filmes e séries assistidos, jogos concluídos, miniaturas de carros, moedas ou discos. Mais do que acumular objetos ou títulos, a coleção envolve escolher o que entra nela, organizá-la segundo critérios próprios e, muitas vezes, exibi-la a outras pessoas. Com a digitalização do cotidiano, essas etapas passaram a acontecer também em meio digital, e o ambiente em que a coleção é mantida pode tanto favorecer quanto dificultar as práticas de aquisição, curadoria e exibição que dão sentido a ela (WATKINS; SELLEN; LINDLEY, 2015).

Registrar uma coleção não se limita a listar o que se possui ou o que se consumiu. Para muitos colecionadores, o registro funciona como memória: a data em que um filme foi assistido, a nota dada a uma temporada de série, as impressões sobre um livro ou os detalhes de uma peça adquirida permitem revisitar a coleção mais tarde, comparar opiniões antigas com as atuais e perceber a própria trajetória. A forma como essas informações são apresentadas também importa, pois a coleção é, em boa medida, uma expressão de quem a mantém.

Nesse contexto, este trabalho apresenta uma plataforma *web* personalizável para criação e compartilhamento de coleções digitais. O acervo do usuário é organizado em três níveis: **Categorias**, que agrupam temas amplos (como “Filmes” ou “Carros”); **Coleções**, subdivisões de uma categoria onde os registros são cadastrados e onde se definem a privacidade e o layout; e **Itens**, que representam cada unidade da coleção. A apresentação dos itens é definida por um editor de layouts no estilo de ferramentas como o Canva, no qual o usuário posiciona e redimensiona Elementos de seis tipos (texto, imagem, forma, ícone, classificação e data) e monta modelos reutilizáveis para suas coleções. Completam a proposta a personalização global da interface, por meio de paletas de cores e tipografia, e uma dimensão social, na qual o usuário pode adicionar amigos, seguir coleções de outras pessoas e acompanhar suas atualizações em um *feed*.

1.1 Justificativa

A ideia deste trabalho surgiu de uma prática pessoal de registro de leituras. Ao concluir cada livro, anotavam-se detalhes como a data de término e as impressões sobre a história, com o objetivo de consultá-los depois, ao reler a obra ou simplesmente ao relembrar o que já havia sido lido. Voltar a esses registros permitia comparar a opinião de uma leitura anterior com a percepção atual sobre o enredo e os personagens.

Ao longo dos anos, diferentes meios foram experimentados para manter esse registro, sem que nenhum atendesse plenamente à necessidade. Modelos de acompanhamento de leituras encontrados na internet precisavam ser copiados à mão para o papel, o que dava trabalho e raramente alcançava o visual desejado. As planilhas eletrônicas resolviam o armazenamento, mas ofereciam poucas possibilidades de personalização, reduzindo cada registro a uma linha de tabela. Tampouco se encontrou, entre os aplicativos experimentados, um que unisse o registro organizado à liberdade de definir sua aparência.

A mesma necessidade apareceu, em seguida, em outros tipos de coleção: filmes, séries, jogos e *hobbies* em geral. Cada um deles pede informações diferentes (um filme pode ter diretor e data em que foi assistido; um carro de coleção, fabricante, escala e ano), e as ferramentas de campos fixos não se adaptam bem a essa variedade. Faltava, portanto, um espaço em que o colecionador pudesse decidir não apenas *o que* registrar, mas *como* cada registro seria apresentado, expressando sua personalidade por meio de cores, fontes e do layout das fichas.

Essa motivação encontra respaldo na literatura sobre personalização de interfaces. Marathe e Sundar (2011) observaram que, ao personalizar aspectos como fontes e cores, os usuários passam a ver a interface como reflexo de sua identidade, e que esse sentimento reforça a percepção de controle sobre a interação. Uma plataforma que une a organização de coleções à personalização visual das fichas atende, assim, tanto à necessidade prática de registrar e consultar o acervo quanto ao desejo de que esse acervo tenha a cara de quem o mantém.

1.2 Objetivos

Esta seção apresenta o objetivo geral do trabalho e os objetivos específicos que o compõem.

1.2.1 Objetivo Geral

Desenvolver uma plataforma *web* que permita ao usuário criar e organizar coleções pessoais de qualquer natureza em categorias, coleções e itens, personalizar a apresentação de cada item por meio de layouts reutilizáveis e compartilhar suas coleções com outros usuários.

1.2.2 Objetivos Específicos

- Levantar e especificar os requisitos funcionais e não funcionais da plataforma, bem como seus casos de uso;
- Elaborar o modelo de domínio do sistema e, a partir dele, o projeto de dados, além do projeto de interface;
- Implementar a organização do acervo em categorias, coleções e itens, com controle de privacidade no nível da coleção;
- Construir um editor visual de layouts baseado em Elementos de seis tipos, que permita associar layouts a coleções e personalizar a ficha de itens individualmente;
- Oferecer a personalização global da interface por meio de paletas de cores e tipografia;
- Implementar as funcionalidades sociais de amizade, acompanhamento de coleções, *feed* de atualizações e notificações;
- Verificar o funcionamento do sistema por meio de testes automatizados, testes manuais e medições de desempenho.

1.3 Problema

As ferramentas usualmente empregadas para registrar coleções pessoais, como planilhas eletrônicas, aplicativos de listas e cadernos, tratam cada registro como um conjunto de campos fixos com aparência padronizada. Essa rigidez dificulta a adaptação a acervos de naturezas distintas, que exigem informações diferentes, e não permite ao colecionador decidir como cada registro é apresentado. Em consequência, quem deseja manter coleções variadas precisa recorrer a várias ferramentas ou aceitar registros que não refletem a forma como gostaria de ver seu acervo.

O problema que este trabalho se propõe a resolver é, portanto, a falta, entre as ferramentas usuais de registro e nos trabalhos revisados no Capítulo 2, de uma plataforma que reúna, em um só lugar, a organização de coleções de qualquer tipo, a liberdade de definir a estrutura e a aparência de cada registro e a possibilidade de compartilhar essas coleções com outras pessoas.

1.4 Organização Deste Trabalho

Este texto está organizado em onze capítulos. O Capítulo 2 reúne a fundamentação teórica que sustenta o trabalho, abordando o colecionismo digital, a personalização de interfaces, a manipulação direta, as redes sociais, a engenharia de requisitos, a modelagem conceitual e a arquitetura de aplicações *web*. O Capítulo 3 descreve os materiais e os métodos empregados, da pesquisa bibliográfica à estratégia de testes, e apresenta o equipamento e as ferramentas utilizados no desenvolvimento. O Capítulo 4 traz a visão panorâmica do produto, com sua perspectiva, suas funções, os perfis de usuário atendidos e as restrições que delimitam o escopo. O Capítulo 5 especifica os requisitos do usuário e do sistema, os casos de uso e o modelo de domínio, e é nele que se definem os quatro conceitos centrais da plataforma. O Capítulo 6 trata do projeto de software, com o projeto de interface e o projeto de dados, detalhando o mapeamento objeto-relacional e a estrutura das tabelas do banco. O Capítulo 7 descreve a implementação, incluindo o ambiente de desenvolvimento, a organização do código e as decisões tomadas durante a construção do sistema. O Capítulo 8 apresenta os testes realizados, com

seus resultados, e o método das medições de desempenho. O Capítulo 9 trata da implantação da aplicação. O Capítulo 10 discute os resultados obtidos à luz dos objetivos estabelecidos. Por fim, o Capítulo 11 apresenta as conclusões e aponta possibilidades de trabalhos futuros.

2 REVISÃO DA LITERATURA

Este capítulo reúne os fundamentos que sustentam a plataforma em duas partes. A primeira trata do que o colecionador precisa, e a segunda, de como isso se constrói. A Seção 2.1 apresenta as coleções e o colecionismo digital, e as Seções 2.2, 2.3 e 2.4 discutem a personalização de interfaces, a manipulação direta em editores visuais e as redes sociais e o compartilhamento de coleções. Em seguida, as Seções 2.5, 2.6 e 2.7 abordam a engenharia de requisitos, a modelagem conceitual com UML e a arquitetura de aplicações *web*. Por fim, a Seção 2.8 retoma as três práticas do colecionador e aponta a lacuna que a plataforma proposta ocupa.

2.1 Coleções e Colecionismo Digital

Esta seção trata da prática de colecionar e de suas motivações, de sua passagem para o meio digital, do sentimento de posse sobre bens digitais e do registro como forma de memória.

2.1.1 Colecionismo e suas Motivações

Colecionar é uma prática antiga, com indícios que remontam à Pré-História (RIGALT, 2020; SUNDSTRÖM; ALBUQUERQUE, 2020). Sundström e Albuquerque (2020) associam esses primeiros indícios à sobrevivência, pois identificar e guardar um novo instrumento era essencial para o desenvolvimento humano, e Rigalt (2020) trata o instinto de posse como inerente à condição humana. Já nesse período, porém, objetos com características especiais eram guardados e recebiam valor simbólico mesmo quando não estavam ligados à sobrevivência (OLIVEIRA, 2017). Na Antiguidade, o rei assírio Assurbanipal reuniu mais de 20 mil tabuletas de argila em sua biblioteca em Nínive, e gregos e romanos colecionavam como forma de prestígio e de poder (RIGALT, 2020; OLIVEIRA, 2017; FINARDI; SOUSA, 2020).

A prática se transformou ao longo dos séculos. Na Idade Média, as coleções tinham viés religioso e exibiam as riquezas de reis e governantes (SUNDSTRÖM; ALBUQUERQUE, 2020). A partir do Renascimento, nobres europeus passaram a guardar peças raras e exóticas em gabinetes de curiosidades, e muitas dessas coleções formaram o núcleo dos primeiros museus públicos (FINARDI; SOUSA, 2020; OLIVEIRA, 2017; RIGALT, 2020). No século XX, a produção em massa tornou o colecionismo um fenômeno de massas, e muitos objetos passaram a ser fabricados já para serem colecionados (OLIVEIRA, 2017; SUNDSTRÖM; ALBUQUERQUE, 2020; FINARDI; SOUSA, 2020).

Ao longo dessa história, a literatura procurou delimitar o que distingue colecionar de simplesmente acumular. Belk et al. (1988) separam o colecionador do acumulador pela seletividade e mostram que uma coleção pode reunir objetos, ideias ou experiências. Estudos recentes mantêm essa base. Silva, Nunes e Strehlau (2020) retomam a definição de Belk de colecionar como um processo ativo, seletivo e apaixonado, sem função utilitária, e destacam que manipular, catalogar e expor os itens sustentam o sentimento de posse e a expressão de si. Correia et al. (2026), ouvindo 29 colecionadores brasileiros, descrevem o colecionismo como aquisição, classificação, exibição e cuidado, e como construção de identidade e de memória. Sundström e Albuquerque (2020) opõem o acúmulo, sem ordem, ao colecionar, que busca representar quem coleciona. Rava (2023) observa, porém, que essa fronteira depende de interpretações que mudam com o tempo e que, no meio digital, a abundância de itens torna a seleção cada vez menos nítida.

As motivações para colecionar também são variadas. A coleção expressa quem coleciona, pois reflete a personalidade do colecionador e funciona como uma extensão do seu eu (SILVA; NUNES; STREHLAU, 2020; OLIVEIRA, 2017; CORREIA et al., 2026). Ela responde ainda ao desejo de controle, e Cao, Brucks e Reimann (2025) mostram, em estudos de laboratório e de campo, que esse desejo leva as pessoas a se engajar no colecionismo porque a coleção oferece estrutura e ordem. Catalogar os itens, por sua vez, dá ao colecionador um meio concreto de acompanhar seus objetivos (SILVA; NUNES; STREHLAU, 2020).

Muitas vezes, colecionar é alimentado pela nostalgia e pelo desejo de resgatar memórias da infância, motivação relatada por colecionadores brasileiros e por

consumidores norte-americanos (CORREIA et al., 2024; CORREIA et al., 2026; JORGENSEN; SORENSEN; SPILINEK, 2023; SILVA; NUNES; STREHLAU, 2020). A coleção também preserva a história pessoal e coletiva, já que reúne registros de acontecimentos passados cujo valor pode ser pessoal ou ligado à história de famílias, comunidades e nações (FINARDI; SOUSA, 2020), e funciona como um arquivo pessoal que reflete a trajetória do colecionador (SILVA; NUNES; STREHLAU, 2020). Para muitos objetos, reuni-los numa coleção é a única forma de salvaguardá-los para as gerações futuras (OLIVEIRA, 2017).

Cuidar da coleção traz ainda bem-estar e satisfação. Colecionadores descrevem a atividade como um lazer agradável, que proporciona contentamento e no qual manusear, organizar e inventariar os itens é fonte de satisfação (OLIVEIRA, 2017). Alcançar um objetivo da coleção gera um sentimento de autoeficácia e eleva a autoestima (SILVA; NUNES; STREHLAU, 2020), e completar um conjunto é descrito pelos próprios colecionadores como uma sensação muito boa (CORREIA et al., 2026). Cao, Brucks e Reimann (2025) acrescentam que a coleção pode servir como forma de enfrentamento em tempos de incerteza e mudança rápida, nos quais traz conforto e sentido.

2.1.2 Colecionismo Digital

No meio digital, essas práticas ganham novas formas. Watkins, Sellen e Lindley (2015), a partir de entrevistas com 20 pessoas no Reino Unido, identificam três práticas que dão sentido a uma coleção, a aquisição, a curadoria e a exibição, e mostram que os recursos do ambiente digital podem tanto favorecer quanto dificultar cada uma delas. A aquisição corresponde a incorporar um novo item ao conjunto, como registrar um livro recém-lido. A curadoria envolve organizar, classificar e anotar o que já foi reunido, e a exibição consiste em mostrar a coleção a si mesmo e aos outros, o que, segundo os autores, comunica quem é o colecionador e reafirma sua identidade. Rava (2023), retomando Winget, observa que, como os itens digitais são fáceis de obter, o esforço do colecionador se desloca da aquisição para a organização e o compartilhamento, a ponto de ser possível colecionar o que não se possui, como as músicas reunidas em listas de reprodução nos serviços de *streaming*. Para a autora, organizar e exibir a coleção no espaço digital é o que lhe

confere significado.

Essas práticas aparecem também entre colecionadores brasileiros. Ribeiro e Salcedo (2022) mostram que, no colecionismo filatélico, as mídias digitais trouxeram outros modos de classificar, expor, leiloar, negociar e trocar selos, e que os colecionadores passaram a participar de exposições filatélicas pela internet. Entre leitores, o Goodreads oferece uma estante virtual, uma lista à qual se adicionam e da qual se retiram livros, semelhante à estante em que se guardam os livros em casa (SILVA et al., 2022).

As ferramentas digitais, porém, nem sempre acompanham essas práticas. No estudo de Watkins, Sellen e Lindley (2015), uma participante lamentava que o programa em que guardava suas músicas as listasse em ordem alfabética, sem permitir destacar os álbuns favoritos, o que tornava a exibição pouco representativa de seu gosto. Mardon, Denegri-Knott e Molesworth (2023) relatam um caso semelhante, o de uma colecionadora que organizava seus discos de vinil em categorias próprias, como músicas para jantares, mas não conseguia organizar nem exibir a coleção digital do mesmo modo, e por isso a sentia apenas em parte como sua. Watkins, Sellen e Lindley (2015) observam que muitas aplicações impõem esquemas de organização que enfraquecem o senso de controle sobre a coleção e defendem ferramentas mais flexíveis, nas quais o usuário imponha suas próprias estruturas, e Razali e Ishak (2025) acrescentam que as plataformas para colecionadores costumam ser voltadas a domínios específicos, oferecem pouca personalização e deixam de lado o compartilhamento e o contato entre colecionadores.

É nesse espaço que se situa a plataforma proposta neste trabalho, que reúne as três práticas em um mesmo ambiente. O usuário cadastra os itens de coleções de qualquer natureza, organiza-os em categorias e coleções definidas por ele e escolhe, por meio de layouts, como cada item é apresentado, além de compartilhar suas coleções com outras pessoas. A curadoria e a exibição são retomadas nas Seções 2.2 a 2.4, dedicadas à personalização de interfaces, à manipulação direta e às redes sociais.

2.1.3 Propriedade Psicológica

Manter uma coleção digital, porém, não produz automaticamente o mesmo vínculo que a coleção física. Odom et al. (2012), ao entrevistar pessoas sobre o que guardavam em serviços *on-line*, observaram que colocar algo na internet pode significar abrir mão do controle sobre aquilo que se valoriza e perder a noção do que se tem, de onde está e de quem tem acesso, a ponto de ficar difícil fazer um inventário do que se possui. Esse vínculo é o que Morewedge et al. (2021) chamam de propriedade psicológica, o sentimento de que algo é "meu", que não depende da propriedade legal e que atende tanto ao desejo de controle quanto ao de expressar a própria identidade. Mardon, Denegri-Knott e Molesworth (2023) explicam que esse sentimento se forma por meio de práticas como guardar, organizar, exibir e cuidar dos objetos, pelas quais a pessoa exerce controle sobre eles, passa a conhecê-los intimamente e investe neles parte de si.

No meio digital, esse sentimento fica ameaçado. Bens digitais acessados por meio de plataformas costumam ser apenas um direito de uso, e as pessoas sentem menos propriedade psicológica por livros, filmes e fotografias digitais do que por seus equivalentes físicos (MOREWEDGE et al., 2021). Mardon, Denegri-Knott e Molesworth (2023) mostram, porém, que o obstáculo não está apenas na natureza digital do objeto, e sim no desencontro entre o que a pessoa espera fazer com ele e o que o sistema permite, de modo que o mesmo objeto pode ser um obstáculo para uma pessoa e não para outra. Denegri-Knott et al. (2024) descrevem o trabalho de posse como selecionar, customizar, exibir e manter um objeto significativo, e relatam que uma atualização do Spotify limitou a capacidade dos usuários de criar, organizar e manter suas próprias bibliotecas de música.

O próprio meio digital oferece, por outro lado, caminhos para restaurar esse vínculo. Morewedge et al. (2021) apontam que os bens digitais podem ampliar o controle ao facilitar a personalização da experiência, o que pode gerar mais propriedade psicológica do que a sentida por bens físicos semelhantes, e recomendam oferecer meios de customizar a experiência e evitar organizar o conteúdo do usuário sem o seu consentimento. Denegri-Knott, Jenkins e Lindley (2022) mostram que o valor das posses digitais está no que elas permitem fazer, como compartilhar, editar e organizar em conjunto, e Denegri-Knott et al. (2024) observam que, nas

plataformas, o trabalho de posse passa a incluir ações como compartilhar, curtir e seguir. Odom et al. (2012) sugerem ainda reunir em um lugar delimitado tudo aquilo que a pessoa valoriza, como um inventário visual que possa ser percorrido e lhe devolva a noção do que tem. A plataforma proposta segue esse caminho, pois reúne a coleção em um único lugar, no qual o próprio usuário define a organização do acervo e a aparência de cada item.

2.1.4 Registro e Memória

Além do vínculo, o registro tem outra função, que é permitir revisitar a própria trajetória. Li, Dey e Forlizzi (2010) definem os sistemas de informática pessoal como aqueles que ajudam as pessoas a coletar informações relevantes sobre si mesmas para refletir e ganhar autoconhecimento, com a vantagem de não depender apenas da memória, que é limitada, e organizam o uso desses sistemas em cinco estágios, a preparação, a coleta, a integração, a reflexão e a ação. Ao mapear uma década de pesquisas na área, Epstein et al. (2020) retomam esse modelo como referência e mostram que os estudos se concentraram em saúde e bem-estar, enquanto outros domínios, como produtividade, finanças e aprendizagem, aparecem em número bem menor.

Registrar uma coleção pessoal segue a mesma lógica. Anotar a data em que um filme foi assistido e a nota dada a ele corresponde à coleta, e voltar a esses registros anos depois, comparando a opinião de antes com a de agora, corresponde à reflexão. Lyall (2024) aproxima o automonitoramento digital dos diários pessoais e mostra que as pessoas usam os próprios dados como apoio para construir histórias e afirmar sua identidade, de modo que os momentos registrados se tornam lembranças. O paralelo mais próximo de rever um histórico de leituras ou de filmes é o resumo anual do Spotify, estudado por Valiati, Lupinacci e Soares (2026), cujos participantes relatam perceber hábitos de escuta que passariam despercebidos e guardar os resumos anteriores para comparar o que mudou de um ano para outro. Hoven, Orth e Zijlema (2021) mostram ainda que posses, inclusive digitais, funcionam como gatilhos de memórias pessoais e que associar um objeto a lembranças é um dos principais motivos do apego a ele.

Esse uso do registro como memória está na origem deste trabalho, como

relatado na Justificativa, e por isso a plataforma permite guardar em cada item informações como datas e classificações, que o colecionador pode consultar depois. Para que esse registro reflita quem o mantém, importa também a forma como ele é apresentado, e é dessa forma que tratam as próximas seções.

2.2 Personalização de Interfaces

Personalizar uma interface é adaptá-la a quem a utiliza, seja no conteúdo exibido, seja na aparência, e essa adaptação pode partir do sistema ou do próprio usuário. Abrahão et al. (2021) chamam de adaptabilidade a capacidade de o usuário adaptar a interface e de adaptatividade a capacidade de o próprio sistema fazê-lo, e Sundar e Marathe (2010) distinguem, no mesmo sentido, a personalização, em que o usuário tem papel passivo e o sistema filtra o conteúdo por ele, da customização, em que o próprio usuário decide o que será exibido. No experimento desses autores, usuários mais experientes avaliaram melhor o conteúdo quando podiam customizar a interface, enquanto os demais preferiram o conteúdo personalizado pelo sistema, e Sundar (2020) retoma o estudo para observar que, à medida que aumentava a experiência do usuário, a sensação de controle aumentava na customização e diminuía na personalização. Customizar, porém, exige esforço. Alves et al. (2026) lembram que as pessoas muitas vezes deixam de aproveitar a personalização feita por elas mesmas pela incerteza sobre o que precisam e pelo esforço envolvido, e Alves et al. (2024) observam que as soluções existentes exigem habilidades técnicas, tempo e esforço que nem todos podem oferecer, enquanto as interfaces costumam permitir apenas ajustes mínimos, como o esquema de cores e o tamanho da fonte. Este trabalho se situa do lado da customização, e por isso ela precisa ser simples o bastante para que o usuário de fato a faça.

Se a escolha é do usuário, cabe perguntar por que ele escolhe personalizar a aparência. Alves et al. (2024) lembram que as pessoas querem que seus dispositivos tenham a aparência e o jeito de algo seu, e que a customização traz benefícios psicológicos ao reforçar o senso de controle, de identidade e de agência. O modo escuro é um exemplo cotidiano. Kim, Lee e Suk (2026) registram que ele se tornou um recurso padrão das interfaces digitais pelo conforto visual e pelo apelo estético,

e Gazit et al. (2025) reúnem entre os motivos de sua adoção a redução do cansaço visual, a usabilidade, a acessibilidade e a estética. A aparência também aproxima o usuário daquilo que o representa. No experimento de Rheu et al. (2022), jogar com um avatar customizado aumentou a identificação do jogador com ele e a sensação de estar no corpo do avatar. No caso de uma coleção, a aparência das fichas mostra quem a mantém e reforça o sentimento de posse discutido na Seção 2.1.

Identidade e controle aparecem juntos quando se estuda por que as pessoas customizam. Na revisão de Bennett et al. (2023), que cobre 161 artigos de IHC publicados ao longo de 32 anos, quase todas as discussões sobre agência e autonomia tratam do controle do usuário sobre as decisões e as ações, e cerca de um terço trata também da identidade, isto é, da coerência dos resultados com os valores e objetivos de quem usa e do efeito sobre a forma como a pessoa se constrói. Alves et al. (2026) observam que a personalização feita pelo próprio usuário dá maior senso de controle e de identidade do que a automática, e que quem a prefere se envolve mais com o processo em si, muitas vezes movido pelo desejo de agência. É essa ligação entre controle e identidade, apresentada brevemente na Justificativa, que faz da customização da aparência mais do que um ajuste estético.

Deixar a decisão com o usuário também se mostra vantajoso em termos de uso. Abrahão et al. (2021) apontam riscos da adaptação feita pelo sistema, como a imprevisibilidade, já que o usuário não sabe quando nem como a interface vai mudar, a falta de explicação sobre os motivos da mudança e a falta de envolvimento do usuário no processo. Todi et al. (2021) mostram que uma adaptação mal escolhida impõe custos ao usuário, por surpreendê-lo ou obrigá-lo a reaprender a interface, e Alves et al. (2026) observam que a personalização automática poupa esforço, mas reduz o senso de controle e aumenta a carga cognitiva ao romper os modelos mentais já formados. Em revisão brasileira sobre interfaces adaptativas, Silva et al. (2025) encontram estudos que optaram por adaptações semiautomáticas ou manuais justamente para preservar o controle e a autonomia do usuário, diante da sensação de perda de domínio que as mudanças automáticas podem gerar.

Além do controle, a aparência em si influencia a forma como o sistema é percebido. Tractinsky, Katz e Ikar (2000) encontraram forte correlação entre a beleza percebida de uma interface e sua usabilidade percebida, e o nível de estética

do sistema afetou as percepções de usabilidade depois do uso, enquanto a usabilidade real não teve esse efeito. Perrig et al. (2023), com 281 participantes, confirmaram que a versão mais estética de um aplicativo foi avaliada como significativamente mais usável após o uso, e lembram que a cor está entre os traços de design que mais chamam a atenção à primeira vista. Lima et al. (2024) reforçam que a estética visual é um dos primeiros aspectos percebidos em uma interface e contribui para sua usabilidade e sua credibilidade percebidas, e Haddad et al. (2024) observam que interfaces bem projetadas despertam experiências emocionais positivas, embora as respostas afetivas variem de uma pessoa para outra. Essa dimensão emocional é tratada pelo design emocional de Norman, cujos três níveis, o visceral, o comportamental e o reflexivo, são apresentados por Morisso e Ferreira (2021), que destacam ainda que os laços emocionais com um produto podem ser criados ao evocar memórias que refletem emoções e a autoimagem de quem o usa. Como os gostos variam, deixar a aparência nas mãos do usuário permite que cada coleção agrade a quem a mantém.

A liberdade de personalizar, porém, precisa preservar a legibilidade. Entre os fatores de usabilidade identificados por Nielsen (1994), o de flexibilidade e eficiência de uso reúne princípios como permitir que o usuário adapte a interface e torná-la customizável, inclusive em sua aparência. Por outro lado, o critério 1.4.3 das diretrizes de acessibilidade do W3C exige razão de contraste de pelo menos 4,5:1 para o texto (CAMPBELL et al., 2024), e o documento explicativo desse critério incentiva os autores a oferecer meios de ajustar as cores de frente e de fundo, desde que exista um mecanismo que devolva os valores exigidos (World Wide Web Consortium, 2026). O cuidado é necessário porque o baixo contraste de texto está entre os três tipos de problema de acessibilidade mais frequentes em aplicativos, em geral causado por escolhas de cores como texto cinza sobre fundo branco (CHEN et al., 2022), e Chen e Muhamad (2022) mostram que a razão de contraste explica 77,4% da variação na resolução visual quando há fundo colorido. Forçar tudo para preto e branco resolve a leitura, mas muda completamente o estilo visual da interface, e Zhang et al. (2023) mostram que é possível corrigir o contraste preservando a consistência desse estilo. Kim, Lee e Suk (2026) lembram ainda que cores pensadas para fundo claro podem perder contraste e visibilidade

sobre fundo escuro.

Personalizar a aparência das fichas responde, assim, ao problema apresentado no Capítulo 1, em que as ferramentas de campos fixos não permitem ao colecionador decidir como cada registro é apresentado. Para que essa personalização esteja ao alcance do usuário comum, é preciso uma forma de edição simples e direta, tema da próxima seção.

2.3 Manipulação Direta e Editores Visuais

Uma forma de dar ao usuário comum o poder de definir a aparência de algo é permitir que ele aja diretamente sobre a representação na tela, sem precisar de comandos ou código. Shneiderman (1983) reuniu sob o nome de manipulação direta as interfaces que despertavam entusiasmo em seus usuários, cujas ideias centrais eram a visibilidade do objeto de interesse, ações rápidas, reversíveis e incrementais e a substituição de comandos de sintaxe complexa pela ação direta sobre o objeto. O autor resume a abordagem em princípios como a representação contínua do objeto de interesse, o uso de ações físicas, como mover e selecionar com o mouse, no lugar de uma sintaxe complexa, e operações rápidas, incrementais e reversíveis cujo efeito sobre o objeto aparece imediatamente. Num editor de layouts, isso significa ver a ficha o tempo todo, arrastar um elemento para mudá-lo de lugar e enxergar o resultado no mesmo instante. Beaudouin-Lafon, Bødker e Mackay (2021) lembram que as interfaces gráficas atuais se baseiam nesse princípio, em que ações físicas sobre representações visuais substituem os comandos textuais, e Satyanarayan et al. (2020), ao analisar ferramentas de criação de gráficos, descrevem eixos e legendas reposicionados com o mouse e redimensionados por alças que aparecem sobre eles, além de relatar que, ao arrastar dados diretamente para a área do gráfico, os participantes se sentiram com mais controle do que ao usar painéis separados.

Hutchins, Hollan e Norman procuraram explicar por que essas interfaces parecem tão naturais. Para os autores, uma interface introduz distância na medida em que há golfos entre os objetivos da pessoa e o sistema, o golfo de execução, que diminui quando os comandos correspondem ao que o usuário pensa, e o golfo de avaliação, que diminui quando a saída apresenta o estado do sistema de forma

fácil de perceber e interpretar (HUTCHINS; HOLLAN; NORMAN, 1985). Quanto menor o esforço cognitivo para atravessar esses golfos, maior a sensação de interação direta, que se completa com o engajamento direto, isto é, a sensação de lidar com os próprios objetos e não com um intermediário, o que exige que o sistema responda sem atraso entre a ação e o resultado (HUTCHINS; HOLLAN; NORMAN, 1985). Querer uma imagem maior, puxar seu canto e vê-la crescer é um exemplo de golfos pequenos nos dois sentidos. Subramonyam et al. (2024) retomam esses conceitos e lembram que, na manipulação direta, o retorno costuma ser imediato, de modo que o usuário vê na hora se suas ações o aproximam do objetivo, e no estudo de Vaithilingam et al. (2024), com 24 participantes, a interface que oferecia controles visuais foi preferida pela facilidade de fazer novos ajustes e pela confiança trazida pelo retorno visual imediato.

Esse estilo de interação se distingue de outras formas de interagir com o computador, como instruir o sistema por comandos ou conversar com ele em linguagem natural. Beaudouin-Lafon, Bødker e Mackay (2021) contrastam a manipulação direta com os comandos textuais das interfaces anteriores, e Masson et al. (2024) observam que ajustar o resultado de um modelo de linguagem conversando com ele é lento, exige referências precisas aos elementos do resultado e depende de escolher bem as palavras. Vaithilingam et al. (2024) mostram que as interfaces apenas de linguagem natural perdem vantagens das interfaces gráficas, como explorar e repetir ajustes e ver o retorno visual na hora, e Srinivasan et al. (2020) propõem complementar a precisão do toque e da caneta com a liberdade de expressão da fala, em vez de abrir mão de uma delas. No editor de layouts, o usuário age diretamente sobre a ficha em vez de descrever a mudança desejada, o que o coloca do lado da manipulação.

Como a manipulação direta incentiva a experimentação, ela pede um meio de voltar atrás. Shneiderman (1983) já observava que um comando de desfazer, que devolve o conteúdo ao estado anterior, reduz a ansiedade do usuário quanto a errar, e que o usuário fica mais tranquilo porque as ações são facilmente reversíveis. Entre os princípios reunidos por Nielsen (1994) estão a possibilidade de desfazer o resultado de comandos anteriores e o suporte a desfazer e refazer, e a avaliação heurística baseada nesses princípios segue entre as técnicas de avaliação de usabilidade mais

conhecidas e usadas (ALMENARA et al., 2024). Sterman, Nicholas e Paulos (2022) mostram, em entrevistas com profissionais de diversas áreas criativas, que guardar versões anteriores dá confiança para explorar alternativas, e um dos entrevistados relata que perdeu o medo de experimentar por saber que podia desfazer e voltar a um estado anterior. Não por acaso, desfazer e refazer são recursos onipresentes em editores visuais de uso corrente, como o Figma, o PowerPoint e o Google Slides, nos quais Stewen e Kleppmann (2024) analisaram o comportamento dessas operações.

Para quem não é designer, porém, começar de uma tela em branco pode ser intimidador, pela dificuldade de conceber o desenho e harmonizá-lo com restrições como o layout e as cores, e no estudo de Lee et al. (2020) doze dos 16 participantes disseram precisar de um modelo com uma estrutura pronta cujas áreas pudessem ser substituídas. Os modelos prontos ajudam a começar, mas limitam bastante a flexibilidade e a diversidade do resultado (HSU et al., 2023). No estudo de Nouraei et al. (2024) com dez pessoas sem formação em design, os modelos ajudaram a chegar rapidamente a um resultado e pouparam o estresse da tela em branco, mas também levaram à fixação e à frustração quando a ferramenta não permitia uma edição flexível. O editor de layouts da plataforma aplica os princípios desta seção, pois o usuário posiciona e redimensiona os elementos da ficha diretamente sobre ela, vê o resultado no mesmo instante e pode desfazer e refazer cada alteração, de modo que montar o layout de uma ficha é uma tarefa de manipulação direta. Montada a ficha, resta mostrá-la a outras pessoas, o que leva às redes sociais.

2.4 Redes Sociais e Compartilhamento de Coleções

Exibir a coleção a outras pessoas faz parte do próprio ato de colecionar, e a internet ampliou essa exibição por meio das redes sociais. boyd e Ellison (2007) definem os sites de redes sociais como serviços na *web* que permitem ao indivíduo construir um perfil público ou semipúblico em um sistema delimitado, montar uma lista de outros usuários com quem tem conexão e ver e percorrer essa lista e as feitas por outros dentro do sistema, e observam que esses sites se organizam sobretudo em torno de pessoas, e não de interesses. Ao revisar 25 anos de definições, Aichner et al. (2021) mostram que o papel dessas plataformas como meio de interação e de

conexão entre usuários foi uma constante, mas que as definições posteriores a 2010 passaram a dar mais atenção ao conteúdo gerado pelos próprios usuários.

Com o tempo, o centro dessas plataformas passou a ser o fluxo de atualizações. Ellison e boyd (2013) revisaram a própria definição e passaram a descrever o site de rede social como uma plataforma de comunicação em rede em que os participantes têm perfis identificáveis, articulam publicamente conexões que podem ser vistas e percorridas por outros e podem consumir, produzir ou interagir com fluxos de conteúdo gerado por suas conexões. Poell, Nieborg e Dijck (2020) definem as plataformas como infraestruturas digitais reprogramáveis que facilitam e moldam interações personalizadas, organizadas pela coleta sistemática e pelo processamento algorítmico de dados, e Lemos e Pastor (2020) mostram que o Instagram declara usar as conexões, as preferências e as atividades dos usuários para personalizar o *feed*. Esse fluxo pode seguir critérios diferentes. Huszár et al. (2022) relatam um experimento em que parte dos usuários do Twitter recebeu um *feed* em ordem cronológica inversa, livre da personalização algorítmica que seleciona e ordena o conteúdo para os demais, e Bandy e Diakopoulos (2021) mostram que a curadoria algorítmica, comparada à cronológica, altera a diversidade de fontes e de temas a que o usuário é exposto. Na plataforma proposta, o *feed* reúne as atualizações das coleções que o usuário acompanha, em ordem cronológica, do evento mais recente ao mais antigo.

Seguir alguém e ser amigo de alguém, porém, não são o mesmo tipo de ligação. Granovetter (1973) define a força de um laço como uma combinação do tempo dedicado, da intensidade emocional, da intimidade e dos serviços recíprocos que o caracterizam. Recuero e Zago (2009) lembram que, nos sites de redes sociais tradicionais, as conexões costumam ser recíprocas e, ao adicionar alguém, é preciso que essa pessoa concorde com a conexão, enquanto no Twitter é possível formar uma rede de contatos em que nunca houve interação recíproca. Palomares et al. (2021) distinguem, nesse sentido, as redes simétricas, em que a conexão é bidirecional e precisa ser confirmada pelos dois lados, como no Facebook e no LinkedIn, das assimétricas, em que um usuário segue outro sem precisar de confirmação, como no Twitter e no Instagram. A plataforma proposta combina as duas formas, pois a amizade depende de solicitação e aceite, enquanto acompanhar uma coleção pública

não exige amizade com quem a mantém, e as coleções restritas a amigos só podem ser acompanhadas por quem tem amizade confirmada.

Decidir quem pode ver a coleção também faz parte do controle sobre ela. Odom et al. (2012) mostraram que, ao guardar algo na internet, as pessoas perdem a noção de quem tem acesso ao que é seu, e a própria definição de boyd e Ellison (2007) já previa perfis públicos ou semipúblicos. Charnsethikul et al. (2025), em pesquisa com 541 usuários de seis redes sociais, verificaram que 92% preferiam que ao menos uma das opções de privacidade fosse mais restrita do que o padrão da plataforma e que muitos tinham dificuldade de encontrar essas opções. Na plataforma proposta, a privacidade é definida em cada coleção, que pode ser pública, visível somente para amigos ou privada, de modo que o colecionador decide o que mostra e a quem.

Plataformas que combinam acervo pessoal e rede social já foram objeto de estudo. Thelwall e Kousha (2017) observam que o perfil de um usuário do Goodreads reúne, como em outras redes sociais, uma lista de amigos, além dos livros favoritos, dos que ele está lendo e dos que quer ler, e concluem que o site não é principalmente um site de livros nem principalmente uma rede social, mas um híbrido genuíno. Walsh e Antoniak (2021) mostram que, ao organizar seus livros em estantes com nomes criados por eles mesmos, os usuários do Goodreads categorizam coletivamente o acervo da própria plataforma, e Hider e Steele (2021) analisam o LibraryThing, plataforma de catalogação social, e mostram que as etiquetas atribuídas pelos usuários às obras oferecem pontos de acesso além dos cabeçalhos usados pelas bibliotecas. No Brasil, o Skoob reúne estante de livros, listas de livros lidos, abandonados e possuídos, linha do tempo e a possibilidade de adicionar contatos, funcionando ao mesmo tempo como rede social e como site de catalogação social (DAMASCENO; NEVES, 2022), e permite ainda marcar os livros que se está lendo e os que se pretende ler (MONTEIRO, 2020). O Letterboxd, por sua vez, se apresenta como uma rede social para discussão de filmes que funciona como diário para registrar e compartilhar opiniões, com uma lista de filmes a ver e a possibilidade de seguir amigos (LEITE; ALVES; MANDAJI, 2025). Reunir o registro de uma coleção e a interação entre pessoas é, portanto, um modelo reconhecido. Nos trabalhos citados, porém, essas plataformas aparecem dedicadas a um único tipo de acervo, livros ou filmes, e nenhum deles trata da definição, pelo

próprio usuário, da aparência de cada registro.

Reunir essas práticas em uma plataforma exige, antes de tudo, definir com precisão o que o sistema deve fazer, e é por aí que começam as seções seguintes, dedicadas à construção do sistema.

2.5 Engenharia de Requisitos

Todo sistema de software parte de uma definição do que ele deve fazer e das restrições sob as quais deve operar (VALENTE, 2020), e essa definição é objeto da engenharia de requisitos. Arora, Grundy e Abdelrazek (2024) a descrevem como uma fase crítica do desenvolvimento, que inclui a elicitação, a análise, a especificação e a validação dos requisitos, e Epifânio et al. (2023), em estudo com profissionais da indústria brasileira de software, apontam como seus objetivos especificar, documentar, validar, negociar e gerenciar os requisitos, garantindo sua qualidade. Valente (2020) resume essas atividades em descoberta, análise, especificação e manutenção, e Zhao et al. (2021), ao classificar a literatura da área, também a dividem em fases como elicitação, análise, modelagem e validação. Os autores descrevem, portanto, o mesmo processo com mais ou menos etapas. Entre elas, Franch et al. (2023) destacam a especificação como atividade central, pois é nela que se documentam os requisitos levantados e negociados nas etapas iniciais.

Entre os requisitos levantados, distinguem-se dois tipos. Os requisitos funcionais dizem o que o sistema deve fazer, como informar o saldo de uma conta bancária, e os não funcionais definem restrições ao seu funcionamento, como exigir que o sistema fique disponível 99,9% do tempo (VALENTE, 2020). Olsson, Sentilles e Papatheocharous (2022) explicam que os não funcionais, também chamados requisitos de qualidade, dizem respeito a quão bem o produto deve realizar a função pretendida, enquanto os funcionais expressam o que ele deve realizar, e citam como exemplos o tempo de inicialização, o controle de acesso e a facilidade de aprendizado. Werner et al. (2022) os definem como atributos ou restrições do sistema, como usabilidade, confiabilidade e manutenibilidade, e observam que eles influenciam fortemente a arquitetura do software. Além disso, um requisito não funcional pode valer para o sistema inteiro, para um componente ou para uma

funcionalidade específica (HABIBULLAH; GAY; HORKOFF, 2023), e, segundo Barbosa, Malcher e Santos (2023), tem efeito global, já que seu atendimento afeta vários componentes do sistema.

Para descrever como o usuário interage com essas funções, usam-se casos de uso. Segundo a especificação da UML, os casos de uso são um meio de capturar os requisitos de um sistema, isto é, o que ele deve fazer, e os usuários e demais sistemas que interagem com ele são representados como atores (OBJECT MANAGEMENT GROUP, 2017). Wang et al. (2022) definem o caso de uso como uma lista de ações ou passos que define as interações entre um ator e o sistema para atingir um objetivo, sendo o ator o papel de uma entidade externa ao sistema, e destacam que sua especificação textual documenta os requisitos funcionais. Santander et al. (2022) acrescentam que o caso de uso produz um resultado observável de valor para o ator e que, por não tratar de detalhes de implementação, torna mais fluida a comunicação com o usuário. Escritos na perspectiva de um ator que deseja usar o sistema com um objetivo (VALENTE, 2020), os casos de uso descrevem o sistema do ponto de vista de quem o usa, e Gupta et al. (2025), ao especificar uma aplicação *web*, observam que eles indicam as funcionalidades oferecidas da perspectiva do usuário e descrevem de forma abrangente o funcionamento de um sistema interativo. Na prática, Franch et al. (2023) mostram que, depois das frases em linguagem natural, os casos de uso e as histórias de usuário são os artefatos mais usados na especificação de requisitos.

Esses conceitos orientam a especificação dos requisitos e dos casos de uso da plataforma, apresentada no Capítulo 5. Definido o que o sistema faz, é preciso representar os conceitos sobre os quais ele opera.

2.6 Modelagem Conceitual com UML

Depois de definidos os requisitos, é preciso identificar os conceitos do domínio e as relações entre eles, e a UML oferece uma notação padronizada para isso. Segundo sua especificação, mantida pela OMG, o objetivo da UML é oferecer a arquitetos, engenheiros e desenvolvedores de software ferramentas para a análise, o projeto e a implementação de sistemas (OBJECT MANAGEMENT GROUP, 2017), e

Valente (2020) a descreve como uma notação gráfica para modelagem de software, usada sobretudo na forma de esboços que ajudam a comunicar ideias. Jolak et al. (2020) a classificam como uma linguagem de propósito geral, aplicável a qualquer domínio, com um conjunto de diagramas que descrevem o sistema por perspectivas diferentes, e destacam, com base em estudo anterior, que o diagrama de classes é usado muito mais do que os demais, o que Valente (2020) também afirma. Na indústria, Franch et al. (2023) observam que a UML é a linguagem dominante entre quem usa modelos na especificação de requisitos, e Júnior, Farias e Silva (2022), em pesquisa com profissionais brasileiros, lembram que os modelos UML servem para documentar decisões de projeto e melhorar a comunicação entre as equipes, e que a linguagem é ensinada nas universidades como padrão para modelar software. Neste trabalho, o diagrama de classes é usado para representar o domínio da plataforma.

No uso conceitual, o artefato central é o modelo de domínio. Storey, Lukyanenko e Castellanos (2023) definem a modelagem conceitual como a atividade de capturar, abstrair e representar aspectos relevantes da realidade para apoiar o entendimento, a comunicação, o projeto e a tomada de decisão, por meio de construtos como entidades, atributos e relacionamentos, e citam o diagrama de classes da UML entre as notações usadas para isso. Esses modelos estruturam a realidade omitindo aspectos irrelevantes para um propósito, o que reduz a complexidade do domínio representado (STOREY; LUKYANENKO; CASTELLANOS, 2023). Chaaben et al. (2026) definem a modelagem de domínio como uma representação explícita, estruturada e visual dos objetos do mundo real de um domínio e de suas ligações, e observam que as fases seguintes do projeto dependem fortemente desse modelo, de modo que um conceito omitido pode afetar a solução desenvolvida. Chou, Aydemir e Dalpiaz (2026) acrescentam que o modelo de domínio captura os conceitos-chave e as relações de um espaço de problema, e Wang et al. (2022) mostram que ele costuma ser feito como diagrama de classes e complementa os casos de uso, fixando a terminologia e os conceitos compartilhados por todos os envolvidos, o que ajuda a evitar mal-entendidos. Essa função de comunicação também é destacada por Antonelli et al. (2023), para quem especialistas e equipe de desenvolvimento pertencem a mundos diferentes e usam linguagens diferentes, cabendo ao modelo conceitual resumir o que foi especificado e dar a todos uma

visão geral do conhecimento reunido. Como o projeto de bancos de dados relacionais costuma ser precedido pela modelagem conceitual (STOREY; LUKYANENKO; CASTELLANOS, 2023), o modelo de domínio orienta as decisões tomadas depois dele.

Quando o domínio é extenso, o diagrama precisa ser organizado. A especificação da UML trata os pacotes como o principal recurso genérico de estruturação e organização da linguagem, usados para agrupar elementos e lhes oferecer um espaço de nomes (OBJECT MANAGEMENT GROUP, 2017), e Valente (2020) explica que o diagrama de pacotes serve a uma visão de mais alto nível do sistema, mostrando grupos de classes e as dependências entre eles. Gray e Rumpe (2021) defendem que módulos, classes e pacotes com fronteiras claras contêm a complexidade de sistemas grandes e permitem raciocinar sobre uma parte sem conhecer todos os detalhes das demais, e Sarhan et al. (2022), ao tratar da organização de sistemas em módulos, descrevem como objetivo decompor um sistema grande em partes menores, coesas e pouco acopladas, que reúnem elementos de características semelhantes. O modelo de domínio da plataforma, apresentado no Capítulo 5, é organizado em pacotes conceituais segundo esse princípio. O modelo conceitual orienta, assim, as decisões de arquitetura e de persistência discutidas a seguir.

2.7 Arquitetura de Aplicações *Web*

Aplicações *web* costumam ser organizadas em partes com responsabilidades distintas, o que facilita compreender, testar e evoluir cada uma delas. Hoppe e Souza (2023) descrevem essa separação em sistemas de informação *web* por meio de quatro modelos, o de entidades, com as classes do domínio e seu mapeamento para o banco de dados, o de persistência, com as classes que gravam e recuperam esses dados, o de aplicação, com a lógica de negócio, e o de navegação, que corresponde à camada de apresentação. Yang et al. (2020) mostram como uma requisição percorre essas partes em aplicações organizadas no padrão modelo-visão-controlador. O controlador recebe os parâmetros da requisição, a camada de mapeamento traduz as chamadas em consultas ao banco, e os resultados voltam como objetos que servem para gerar a página enviada ao usuário. Os mesmos autores observam que as regras

sobre os dados aparecem em mais de uma camada, na interface, na aplicação e no banco, e que a maior parte delas fica no banco de dados.

A comunicação entre a camada de apresentação e as demais segue, em muitas aplicações, o estilo REST. Fielding (2000) o define como um conjunto de restrições arquiteturais, que incluem a separação entre cliente e servidor, a comunicação sem estado, o uso de *cache*, a interface uniforme e o sistema em camadas, além do código sob demanda, que é opcional. Na comunicação sem estado, cada requisição do cliente deve conter toda a informação necessária para ser compreendida, e o estado da sessão fica inteiramente no cliente (FIELDING, 2000), de modo que o servidor não guarda contexto entre uma requisição e outra (NEUMANN; LARANJEIRO; BERNARDINO, 2021). Na interface uniforme, cada recurso é identificado por um endereço único, a URI, e manipulado por meio de representações (NEUMANN; LARANJEIRO; BERNARDINO, 2021). Golmohammadi, Zhang e Arcuri (2023) lembram que o REST não é um protocolo, e sim um conjunto de diretrizes para projetar APIs que acessam e manipulam recursos por HTTP, protocolo cujos métodos, códigos de *status* e cabeçalhos são definidos em sua especificação (FIELDING; NOTTINGHAM; RESCHKE, 2022). Como essas restrições descrevem o comportamento esperado em alto nível, sem instruções detalhadas de implementação, o REST deixa margem para interpretações diferentes (KOTSTEIN; BOGNER, 2021), e Neumann, Laranjeiro e Bernardino (2021), ao analisar 500 APIs públicas, encontraram apenas 0,8% delas seguindo rigorosamente todos os seus princípios.

Como cada requisição precisa identificar quem a faz, é comum usar *tokens*. O JSON Web Token (JWT) é um formato compacto para representar informações, chamadas de declarações, transmitidas entre duas partes e assinadas digitalmente ou protegidas por um código de autenticação, pensado para espaços restritos como o cabeçalho `Authorization` do HTTP (JONES; BRADLEY; SAKIMURA, 2015). Almeida e Canedo (2022) descrevem sua estrutura em três partes, o cabeçalho, o conteúdo e a assinatura, e destacam que a verificação pode ser feita diretamente pelo servidor que guarda o recurso, sem consultar o servidor de autenticação. Yang et al. (2026) explicam que, por ser assinado, quem o recebe confere sua autenticidade sem contatar quem o emitiu, o que permite a autenticação sem estado, e observam que foi a difusão das SPAs e das arquiteturas distribuídas que expôs os limites da sessão

guardada no servidor. Laurindo, Termos e Cardoso (2024) acrescentam que, como o próprio *token* contém os dados relevantes, o servidor não precisa registrar as sessões dos usuários. Esse tipo de autenticação é, assim, uma consequência natural da ausência de estado do REST. A plataforma proposta usa o JWT, mas não dispensa por completo o registro no servidor: como o encerramento da sessão precisa ter efeito imediato, a API guarda cada sessão e confere, a cada requisição, se ela não foi revogada, abrindo mão de parte da ausência de estado em troca dessa garantia, como detalhado no Capítulo 7.

Do lado do cliente, a interface também pode evitar recarregar a página a cada ação. Na aplicação de várias páginas, o estado fica no servidor e cada requisição recarrega a página, o que se torna impraticável em aplicações muito interativas (VEPSÄLÄINEN; HEVERY; VUORIMAA, 2024). A SPA, ao contrário, mantém o estado no navegador e atualiza a interface de forma dinâmica conforme a interação do usuário, economizando banda ao custo de um carregamento inicial maior, o que a torna adequada a aplicações complexas e de uso prolongado (VEPSÄLÄINEN; HEVERY; VUORIMAA, 2024). Komperla et al. (2022) explicam que a página inicial é carregada uma única vez, com seus recursos, e que depois só os componentes afetados são atualizados, e Ollila, Mäkitalo e Mikkonen (2022) mostram que, nos *frameworks* modernos, o código apenas descreve o estado desejado da interface e o *framework* gera as alterações necessárias na página. Hoppe e Souza (2023) acrescentam que, na SPA, a lógica de apresentação passa do servidor para o cliente, o que permite desenvolver a interface de forma independente do servidor, com o qual ela se comunica em geral por uma interface REST. Telas muito interativas, como um editor visual em que o usuário arrasta e redimensiona elementos, são o caso típico em que esse modelo se mostra vantajoso.

Do outro lado, os dados precisam ser guardados de forma consistente. Codd (1970) propôs um modelo baseado em relações, as tabelas, para que os usuários de grandes bancos de dados não precisassem saber como os dados estão organizados internamente, e introduziu os conceitos de chave primária e chave estrangeira. A chave primária identifica de forma única cada linha de uma tabela, e a chave estrangeira exige que os valores de uma coluna correspondam aos de uma linha de outra tabela, mantendo a integridade referencial entre tabelas relacionadas (THE

POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2025a). Jiang e Naumann (2020) lembram que essas são duas das restrições mais importantes dos bancos relacionais, responsáveis pela integridade de entidade e pela integridade referencial, e Döhmen et al. (2024), ao reunir mais de 221 mil esquemas de bancos publicados no GitHub, com mais de 700 mil chaves estrangeiras e 7 milhões de restrições de integridade, mostram quanto essas restrições estão presentes nos bancos usados na prática.

O modelo relacional pressupõe, porém, que a estrutura dos dados seja conhecida de antemão, o que nem sempre acontece. Quando cada registro pode ter atributos próprios, como uma coleção de livros com autor e número de páginas e outra de discos com gravadora, o esquema não pode ser fixado previamente. Huber et al. (2020) descrevem esse conflito no sistema AiiDA, em que os atributos de cada nó dependem do seu tipo e não podem ser definidos de forma estática, e relatam duas abordagens para ele. A primeira é o modelo entidade-atributo-valor (EAV), em que cada atributo é gravado como uma linha, solução que os autores consideram bem-sucedida por permitir guardar atributos arbitrários em um banco relacional, mas que, no volume de dados com que trabalhavam, aumentava o espaço ocupado e o custo de consultar e converter esses dados. A segunda são as colunas do tipo JSONB, que adotaram em seguida com ganho de desempenho (HUBER et al., 2020). A documentação do PostgreSQL explica que o JSONB guarda os dados em formato binário decomposto, mais rápido de processar e passível de indexação, e que representar dados em JSON é mais flexível que o modelo relacional tradicional, podendo as duas abordagens coexistir e se complementar na mesma aplicação (THE POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2025b).

A plataforma proposta combina as duas abordagens. As informações de cada ficha ficam nos elementos que a compõem, cada um gravado como uma linha com seu tipo e seu conteúdo, de forma próxima ao EAV, e a aparência de cada elemento fica em uma coluna JSONB, já que o banco apenas a guarda e a devolve. Como a ficha é carregada inteira para ser exibida, com todos os seus elementos de uma vez, e não montada atributo por atributo em consultas separadas, o custo de consulta apontado para o EAV pesa pouco nesse uso. O custo reaparece quando uma consulta depende de um atributo, como ordenar os itens pela classificação,

e por isso o conteúdo de cada elemento fica em uma coluna própria, que o banco pode ler, como detalhado no Capítulo 6.

Programas orientados a objetos, no entanto, não pensam em tabelas. Zhadko-Bazilevych (2025) explica que, nos bancos relacionais, os dados ficam em tabelas com campos definidos e as relações são feitas por chaves estrangeiras, enquanto as linguagens orientadas a objetos trabalham com objetos, atributos, métodos e ligações complexas entre classes, e que essa diferença dificulta a integração entre os dois modelos. As ferramentas de mapeamento objeto-relacional (ORM) automatizam a conversão entre os objetos do código e os registros do banco, reduzindo o tempo de desenvolvimento (ZHADKO-BAZILEVYCH, 2025). Com elas, cada modelo do código é mapeado para uma tabela e as chamadas feitas no programa são traduzidas em consultas SQL (YANG et al., 2020), e Hoppe e Souza (2023) incluem esse mapeamento no próprio modelo de entidades das aplicações *web*. A conveniência tem custo no desempenho. Shao et al. (2020) catalogam antipadrões de acesso ao banco em aplicações *web*, muitos dos quais correspondem ao uso inadequado das APIs de ORM, e Zhadko-Bazilevych (2025), comparando ferramentas de ORM para Node.js com PostgreSQL, encontra diferenças de desempenho entre elas conforme o tipo de consulta e a carga.

A plataforma proposta segue essa organização, com uma interface do tipo SPA que se comunica por uma API REST com o servidor, o qual acessa um banco de dados relacional por meio de uma ferramenta de ORM, e as tecnologias usadas em cada parte são apresentadas no Capítulo 3.

2.8 Considerações sobre o Capítulo

As três práticas que dão sentido a uma coleção, apresentadas na Seção 2.1, encontram sua base neste capítulo. A curadoria, que organiza e dá forma ao acervo, apoia-se na personalização de interfaces e na manipulação direta das Seções 2.2 e 2.3, e a exibição apoia-se nas redes sociais e no compartilhamento da Seção 2.4. A aquisição, isto é, o cadastro de itens de coleções de qualquer natureza, apoia-se na engenharia de requisitos, na modelagem conceitual e na arquitetura das Seções 2.5 a 2.7, inclusive no armazenamento de atributos que variam de uma coleção para

outra.

Nos trabalhos revisados, porém, essas práticas aparecem separadas. As aplicações discutidas por Watkins, Sellen e Lindley (2015) impõem os próprios esquemas de organização, a colecionadora relatada por Mardon, Denegri-Knott e Molesworth (2023) não conseguia organizar nem exibir a coleção digital como queria, as plataformas para colecionadores analisadas por Razali e Ishak (2025) costumam ser voltadas a domínios específicos, oferecem pouca personalização e deixam de lado o compartilhamento, e as interfaces em geral permitem apenas ajustes mínimos de aparência (ALVES et al., 2024). Nenhum dos trabalhos revisados reúne, em uma mesma solução, coleções de qualquer natureza, aparência das fichas definida pelo próprio usuário por manipulação direta e compartilhamento em rede social, e é essa articulação que a plataforma proposta neste trabalho oferece.

3 MATERIAIS E MÉTODOS

Este capítulo descreve os procedimentos metodológicos, os recursos materiais e as ferramentas computacionais empregadas no desenvolvimento da plataforma proposta. A seleção das abordagens levou em consideração tanto a natureza exploratória do problema abordado, que é a carência de soluções capazes de combinar a organização de coleções pessoais com a personalização aprofundada da interface, quanto os objetivos práticos deste trabalho, voltados à construção de um protótipo funcional.

3.1 Metodologia da Pesquisa

Esta seção caracteriza a pesquisa quanto aos procedimentos adotados para a construção do referencial teórico e quanto à sua natureza aplicada.

3.1.1 Pesquisa Bibliográfica

A construção do referencial teórico deste trabalho apoiou-se na revisão de literatura, recurso metodológico que, conforme (FONTELLES et al., 2009), abrange a análise de materiais previamente publicados em diversos suportes, incluindo livros, periódicos científicos, teses, documentos eletrônicos e demais fontes de conhecimento consolidado. Essa fundamentação é essencial para situar a pesquisa no estado da arte e identificar lacunas que justifiquem a proposta desenvolvida.

O processo de revisão foi conduzido em três momentos. No primeiro, delimitaram-se os eixos temáticos que sustentam o trabalho, os mesmos das seções do Capítulo 2: colecionismo digital, personalização de interfaces, manipulação direta e editores visuais, redes sociais e compartilhamento de coleções, engenharia de requisitos, modelagem conceitual com UML e arquitetura de aplicações *web*, incluída a persistência relacional. A escolha desses eixos refletiu tanto as práticas do colecionador que a plataforma pretende atender quanto as decisões técnicas que precisariam ser tomadas ao longo do projeto.

No segundo momento, procedeu-se à busca de materiais, utilizando-se como bases primárias o Google Scholar e o Portal de Periódicos da CAPES, este último acessado via sistema CAFe. Complementarmente, foram realizadas consultas em documentações oficiais de frameworks e em publicações de conferências da área de engenharia de *software*. Privilegiaram-se produções dos últimos anos, sem, contudo, excluir obras de referência como as de (SOMMERVILLE, 2011), (PRESSMAN; MAXIM, 2016) e (LARMAN, 2007), que orientam a especificação dos requisitos e do modelo de domínio no Capítulo 5.

No terceiro momento, os materiais coletados passaram por triagem a partir da leitura dos resumos, seguida de leitura integral dos trabalhos considerados pertinentes. As informações extraídas foram organizadas em sínteses temáticas, revisadas em duas rodadas para assegurar precisão nas citações e consistência na argumentação desenvolvida ao longo do texto.

3.1.2 Pesquisa Aplicada

O trabalho adota também a perspectiva da pesquisa aplicada, caracterizada por (GERHARDT; SILVEIRA, 2009) como aquela cujo propósito é gerar conhecimento voltado à resolução de problemas concretos. No contexto desta pesquisa, tal abordagem materializa-se na transposição dos fundamentos teóricos levantados para decisões concretas de projeto e implementação do sistema.

Dessa articulação resultam três contribuições práticas principais: a especificação de requisitos fundamentada em (SOMMERVILLE, 2011) e (PRESSMAN; MAXIM, 2016); a elaboração do modelo conceitual baseada na abordagem de (LARMAN, 2007); e a definição da arquitetura do sistema em camadas, inspirada em (FOWLER, 2002), com comunicação entre *frontend* e *backend* pautada pelo estilo REST (FIELDING, 2000).

3.2 Materiais

3.2.1 Equipamentos

O desenvolvimento foi conduzido em um computador pessoal cuja configuração atende aos requisitos de execução simultânea do ambiente de desenvolvimento, do contêiner de banco de dados, do servidor de aplicação e dos navegadores utilizados para validação da interface. O Quadro 1 sintetiza as especificações do equipamento.

Quadro 1 – Configuração do equipamento utilizado no desenvolvimento

Componente	**Especificação**
Processador	AMD Ryzen 5 5500 (6 núcleos, 12 *threads*, 3,6 GHz, *turbo* de 4,2 GHz)
Placa-mãe	MSI A520M-A PRO (soquete AM4, *chipset* AMD A520, M-ATX)
Memória RAM	16 GB DDR4 (2×8 GB Kingston Fury Beast, 3200 MHz, CL16)
Placa de vídeo	ASRock Radeon RX 6600 Challenger D (8 GB GDDR6, 128-bit)
Armazenamento	SSD Kingston NV3 de 1 TB (M.2 2280, PCIe NVMe)
Gabinete	Aigo Darkflash E330M (*mid-tower*, lateral de vidro)
Sistema operacional	Windows 11

Fonte: Elaborada pelo autor

3.2.2 Softwares

As ferramentas de *software* empregadas neste trabalho foram escolhidas considerando três critérios: aderência técnica ao escopo do projeto, disponibilidade gratuita ou licenciamento de código aberto, e existência de documentação consolidada que facilitasse a curva de aprendizado. As subseções da Seção 3.3 descrevem cada uma delas, contextualizando seu uso específico na construção da plataforma proposta.

3.3 Ferramentas Utilizadas

3.3.1 Visual Studio Code

O Visual Studio Code (VS Code) é um editor de código multiplataforma mantido pela Microsoft e distribuído gratuitamente. Sua arquitetura baseada em extensões permite estender suas capacidades para atender a diferentes ecossistemas de desenvolvimento, aproximando-o da funcionalidade de um ambiente de desenvolvimento integrado (IDE) tradicional, porém com consumo de recursos mais enxuto.

Neste trabalho, a ferramenta concentrou três frentes distintas de uso: a edição do código *backend* em TypeScript, a edição do código *frontend* em React e a escrita do presente documento em LaTeX, viabilizada pela extensão *LaTeX Workshop*. A centralização dessas atividades em um único editor simplificou o fluxo de trabalho e reduziu o tempo dedicado a alternância entre ambientes.

3.3.2 Node.js e NestJS

O ecossistema Node.js consiste em um ambiente de execução JavaScript construído sobre o motor V8 do Chrome, que habilita a execução desta linguagem fora do navegador, particularmente em contextos de servidor. Por cima dele, o NestJS oferece um *framework* opinativo inspirado em conceitos do Angular, organizando a aplicação em módulos coesos compostos por *controllers*, *providers* e *services*. Essa organização modular alinha-se aos preceitos de separação de responsabilidades defendidos por (FOWLER, 2002).

No sistema proposto, essa dupla compõe a camada responsável pela lógica de negócio. Suas principais atribuições incluem a autenticação de usuários e a emissão de *tokens* de sessão, o processamento das operações sobre categorias, coleções, itens, *layouts* e Elementos, a aplicação das regras de privacidade configuradas no nível das coleções, o gerenciamento dos vínculos sociais (amizades e acompanhamentos) e o disparo de notificações decorrentes dessas interações.

Complementam o *backend* algumas bibliotecas do próprio ecossistema: o Passport e o módulo JWT do NestJS, responsáveis pela emissão e validação dos

tokens de sessão; o bcrypt, utilizado no armazenamento das senhas em forma de *hash*, conforme exigido pelo RNF-03; o Nodemailer, encarregado do envio das mensagens de confirmação do cadastro e de recuperação de senha; e um mecanismo de limitação de requisições, que restringe as tentativas de autenticação por endereço de origem.

3.3.3 PostgreSQL e Docker

O PostgreSQL, em sua versão 16, é um sistema gerenciador de banco de dados relacional maduro, de código aberto, reconhecido por sua fidelidade ao padrão SQL e por seu suporte robusto a transações. Sua integração com aplicações Node.js é bem consolidada no mercado, o que pesou em sua escolha como camada de persistência deste projeto.

A modelagem do banco espelha a organização do modelo de domínio, contemplando as entidades de usuários, categorias, coleções, itens, *layouts* e Elementos, sendo que o conteúdo e o estilo exibidos nas fichas residem nos próprios Elementos. Estruturas relacionadas à dimensão social (solicitações de amizade, acompanhamento de coleções, eventos de atualização e notificações), às preferências visuais globais e ao controle de sessões e de redefinição de senha também residem nessa camada.

A instalação do PostgreSQL foi realizada por meio do Docker, plataforma que abstrai o processo de provisionamento da instância em um contêiner isolado. Essa decisão trouxe dois benefícios imediatos: a reprodutibilidade do ambiente entre máquinas distintas e a facilidade de resetar a base durante os ciclos de teste, sem comprometer o sistema operacional hospedeiro.

3.3.4 Prisma

O Prisma é a ferramenta de mapeamento objeto-relacional adotada para intermediar o acesso do *backend* ao PostgreSQL. A partir de um arquivo de esquema, no qual as entidades são declaradas em uma linguagem própria, o Prisma gera um cliente tipado em TypeScript, de modo que consultas e escritas passam a ser

verificadas pelo compilador antes da execução, um ganho relevante em um domínio com muitas associações entre entidades.

Duas características pesaram na escolha. A primeira é o mecanismo de migrações: cada alteração no esquema gera um arquivo versionado com o SQL correspondente, o que preserva o histórico da evolução do banco e permite reconstruí-lo do zero de forma determinística. A segunda é a possibilidade de dividir o esquema em vários arquivos, recurso empregado neste trabalho para separá-lo por assunto, acompanhando os pacotes do modelo de domínio apresentados no Capítulo 5. O detalhamento do mapeamento entre o modelo de objetos e o modelo relacional é apresentado no Capítulo 6.

3.3.5 DBeaver

O DBeaver é um cliente gráfico multiplataforma voltado à administração de bancos de dados relacionais, compatível com os principais SGBDs disponíveis no mercado, entre eles o PostgreSQL. Por meio de sua interface visual, a ferramenta permite inspecionar estruturas, executar consultas SQL diretamente no banco e exportar resultados em diferentes formatos.

Ao longo do desenvolvimento, o DBeaver complementou o fluxo de trabalho ao oferecer uma alternativa visual ao terminal interativo do PostgreSQL. Foi particularmente útil para verificar a integridade das associações entre entidades, confirmando, por exemplo, que um item estava vinculado à coleção correta e utilizava os Elementos definidos pelo *layout* aplicável, e para explorar os dados inseridos durante as sessões de teste manual.

3.3.6 React e Vite

O React é uma biblioteca para construção de interfaces a partir de componentes, na qual a tela é descrita como função do estado da aplicação. O Vite, por sua vez, é a ferramenta de construção e o servidor de desenvolvimento adotados para executá-la: durante o desenvolvimento, serve os módulos diretamente ao navegador, o que torna a atualização em tela praticamente instantânea; para produção, gera

um pacote otimizado. A aplicação resultante é uma *single-page application* (SPA), renderizada inteiramente no navegador.

A adoção do Vite em lugar do Next.js, alternativa natural no ecossistema React, foi deliberada, e apoia-se em três observações sobre a natureza deste sistema. Em primeiro lugar, os recursos de renderização no servidor e de geração estática que justificam o Next.js visam sobretudo a indexação por mecanismos de busca, e todo o conteúdo da plataforma fica atrás de autenticação. A única página pública, a de apresentação, é gerada como HTML pronto durante a construção (Seção 7.5), sem exigir um servidor de renderização. Em segundo lugar, o modelo de autenticação do *backend* baseia-se no envio do *token* no cabeçalho `Authorization`, ao passo que as abordagens mais recentes do Next.js pressupõem a leitura de *cookies* no servidor antes da renderização, o que exigiria trabalhar contra o desenho do *framework*. Por fim, a tela mais complexa do sistema, o editor de *layouts*, é integralmente interativa e depende do estado mantido no navegador, de modo que a renderização prévia no servidor não traria ganho. Manteve-se, assim, uma configuração mais simples e diretamente alinhada ao problema.

A camada de apresentação foi construída com essa combinação, complementada pelo React Router para o roteamento entre telas, pelo TanStack Query para a busca e o *cache* dos dados vindos da API e pelo Axios para as requisições HTTP. As telas contemplam o cadastro e a autenticação de usuários, a recuperação e a redefinição de senha, o *dashboard* que lista as categorias do usuário, a navegação entre categorias, coleções e itens, o editor visual de *layouts* (com funcionalidade de *drag-and-drop* de Elementos), a exibição e a edição de fichas individuais de itens, o painel de personalização visual global, as telas de interação social e o *feed* de atualizações. A renderização dinâmica das fichas, que varia conforme os Elementos configurados em cada *layout*, é um dos pontos em que as capacidades de composição do React foram mais intensamente exploradas.

3.3.7 Figma

O Figma é uma ferramenta colaborativa de *design* de interfaces executada diretamente no navegador, sem necessidade de instalação local. Seus recursos incluem a criação de componentes reutilizáveis, a definição de sistemas de *design*

(tipografia, cores, espaçamentos) e a prototipação de fluxos interativos entre telas.

No presente trabalho, o Figma foi utilizado na fase de concepção visual da plataforma, antes da implementação efetiva. Essa etapa permitiu avaliar a disposição dos elementos, a hierarquia visual e a consistência entre telas, reduzindo a necessidade de refatorações estéticas posteriores no código. Foram prototipadas, entre outras, a tela inicial, as telas de coleções e de itens, a biblioteca e o editor de *layouts*, as telas sociais e a de aparência, apresentadas na Seção 6.1.

3.3.8 Astah Community

O Astah Community, desenvolvido pela empresa japonesa Change Vision, é uma ferramenta voltada à modelagem UML que disponibiliza, em sua versão gratuita para uso acadêmico, recursos para a construção dos principais tipos de diagrama previstos pelo padrão.

Sua utilização neste trabalho concentrou-se na produção dos artefatos de análise: os diagramas de casos de uso, que formalizam as interações dos atores com o sistema, e o modelo de domínio, que evidencia as classes conceituais e suas associações. A organização dos diagramas em pacotes, facilitada pelo Astah, foi especialmente útil para representar visualmente os sete agrupamentos conceituais do modelo de domínio deste projeto (Usuário, Sistema, Coleções, Editor de Layout, Aparência, Interação Social e Notificações).

3.4 Métodos de Desenvolvimento

3.4.1 Elicitação e especificação de requisitos

A primeira etapa metodológica consistiu em identificar, analisar e especificar as funcionalidades que a plataforma precisaria oferecer aos seus usuários. Adotou-se para isso a classificação tradicional proposta por (SOMMERVILLE, 2011), que distingue requisitos funcionais, descrições do que o sistema deve fazer, de requisitos não funcionais, que tratam de propriedades como desempenho, segurança e usabilidade.

O levantamento partiu da experiência da autora com os meios de registro relatados na Justificativa, como planilhas e modelos de acompanhamento, e das limitações descritas pelos estudos do Capítulo 2, especialmente a rigidez quanto à estrutura de dados e à aparência dos registros. Essa reflexão orientou a formulação de requisitos que valorizassem a autonomia do usuário sobre a organização e a apresentação de seu acervo. Cada requisito funcional foi descrito segundo um formato padronizado contendo ação, objeto, atributos, exemplos e restrições, com o verbo *gerenciar* adotado de forma recorrente para representar operações CRUD (cadastrar, editar e excluir). A relação completa dos requisitos é apresentada no Capítulo 5.

3.4.2 Modelagem conceitual

A partir dos requisitos elicitados, procedeu-se à modelagem conceitual do sistema, etapa na qual as entidades do mundo real relevantes ao domínio são representadas, juntamente com seus atributos e relações. Tomou-se como base a proposta de (LARMAN, 2007), para quem o modelo de domínio é uma ferramenta visual que evidencia os conceitos significativos antes de qualquer decisão de implementação.

Diante da quantidade de entidades identificadas, adotou-se a organização do diagrama em sete pacotes conceituais: Usuário, Sistema, Coleções, Editor de Layout, Aparência, Interação Social e Notificações. Esse agrupamento, além de reduzir a sobrecarga visual do diagrama, evidencia a separação de responsabilidades que seria posteriormente materializada tanto na divisão do esquema do banco em arquivos por assunto quanto na estrutura modular do *backend* implementado com NestJS.

3.4.3 Definição arquitetural

A arquitetura do sistema seguiu o modelo em camadas descrito por (FOWLER, 2002), com separação clara entre apresentação, lógica de negócio e persistência. A camada de apresentação é implementada em React, com Vite, e comunica-se com o *backend* em NestJS por meio de uma API baseada no estilo REST (FIEL-

DING, 2000), enquanto este último acessa o PostgreSQL por meio do Prisma, que mapeia objetos para tabelas relacionais.

Essa separação em camadas foi uma decisão consciente para permitir que cada responsabilidade fosse testada e evoluída de forma independente. A adoção do estilo REST, por sua vez, ofereceu um vocabulário padronizado para a API (métodos HTTP, códigos de *status* e representações em JSON) que facilita tanto a documentação quanto eventuais evoluções futuras.

3.4.4 Prototipação

Antes da codificação, as telas centrais do sistema foram prototipadas no Figma, o que permitiu antecipar discussões sobre disposição de elementos, nomenclatura de botões e organização de conteúdo em cada tela. Particular atenção foi dada ao editor de *layouts*, já que a qualidade da interação com essa tela influencia diretamente a percepção que o usuário terá sobre a personalização oferecida pela plataforma.

A prototipação também serviu para definir o sistema de *design* de base, incluindo paleta de cores, tipografia e componentes reutilizáveis, que posteriormente orientaram a implementação visual em React.

3.4.5 Implementação

A codificação foi conduzida de forma iterativa e incremental, da camada de persistência em direção à interface. Iniciou-se pela modelagem do esquema do banco, estabelecendo as tabelas e relacionamentos derivados do modelo conceitual. Em seguida, foi implementada a camada de lógica de negócio, organizada em módulos NestJS que correspondem aos pacotes do modelo de domínio. Por fim, foi desenvolvida a camada de apresentação em React, consumindo as APIs disponibilizadas pelo *backend*.

A escolha pela ordem *backend*-para-*frontend* decorre do fato de que as telas dependem dos dados e regras já disponíveis na API. Durante toda a implementação, ajustes pontuais nos requisitos ou na modelagem foram feitos quando dificuldades práticas apontavam para oportunidades de refinamento.

3.4.6 Gestão do projeto

O planejamento do projeto foi estruturado em ciclos quinzenais, nos quais as tarefas eram definidas, priorizadas e acompanhadas ao longo de cada período. Essa abordagem possibilitou melhor controle do tempo, organização das entregas e adaptação contínua às necessidades do projeto.

O versionamento do código e do próprio documento foi feito com o Git, em repositórios separados para o *backend*, o *frontend* e a monografia. Além de preservar o histórico das alterações, essa prática serviu ao registro das decisões de projeto: cada mudança relevante de rumo ficou associada ao momento em que foi tomada, o que facilitou recuperar as justificativas na redação deste texto.

3.4.7 Estratégia de Testes

A verificação do sistema apoiou-se em três frentes complementares. A primeira é a de testes automatizados no *backend*, escritos com o Jest e organizados junto ao código que verificam: os serviços, os controladores e os DTOs dos módulos têm arquivos de teste próprios, nos quais as dependências externas são substituídas por implementações simuladas, de modo que a regra de negócio seja exercitada de forma isolada. Essa estratégia é particularmente relevante nas regras de privacidade e de acompanhamento de coleções, cujas combinações de vínculo de amizade e nível de visibilidade seriam trabalhosas de cobrir manualmente. Complementam-nos testes de ponta a ponta da API, executados sobre um banco de dados PostgreSQL real, em contêiner, que verificam em conjunto a autenticação, a validação dos dados, as regras de negócio e as restrições do banco. A segunda frente é a de testes manuais exploratórios, conduzidos sobre a interface a cada funcionalidade concluída, com apoio do DBeaver para inspecionar o estado do banco após cada operação. Essa verificação direta foi o meio adotado para validar aspectos visuais e de interação que os testes automatizados não alcançam, como o comportamento do editor de *layouts* durante o arrastar e soltar de Elementos.

A terceira frente é a das medições de desempenho, destinadas aos requisitos não funcionais que fixam limites de tempo. *Scripts* mantidos no repositório da API criam, pela própria API, uma conta de teste e um acervo sintético, medem o

tempo de resposta das rotas principais e, no navegador, o carregamento das telas, a movimentação de Itens e a resposta do editor de *layouts*, gravando cada amostra para que os números possam ser conferidos e a medição, repetida. Os procedimentos das três frentes são detalhados no Capítulo 8, e os resultados das medições, com o juízo sobre cada requisito, no Capítulo 10.

4 DESCRIÇÃO GERAL DO PRODUTO

Este capítulo apresenta uma visão panorâmica da plataforma proposta, descrevendo seu propósito, os perfis de usuários atendidos e as fronteiras que delimitam o escopo do desenvolvimento. A descrição aqui contida serve como base para o detalhamento técnico e a especificação de requisitos apresentados nos capítulos subsequentes.

4.1 Perspectiva do Produto

O produto consiste em uma plataforma web de gerenciamento e catalogação de acervos pessoais. O sistema surge como uma solução para o problema da rigidez em softwares de inventário tradicionais, que geralmente oferecem campos fixos que não se adaptam adequadamente a diferentes tipos de itens.

A perspectiva central do produto é a **flexibilidade estrutural**. Por meio de uma arquitetura baseada em **Elementos** visuais reutilizáveis, o usuário deixa de ser apenas um consumidor de formulários prontos para se tornar o arquiteto de sua própria organização. O sistema permite que colecionadores de diferentes nichos (como livros, moedas ou jogos) utilizem a mesma plataforma, mas com experiências de preenchimento e visualização completamente distintas e adequadas a cada domínio.

4.2 Funções do Produto

As funcionalidades do sistema estão organizadas em cinco eixos principais que sustentam a proposta de valor da plataforma:

- **Organização Multinível:** Estruturação lógica dos dados em **Categorias** (agrupadores temáticos), **Coleções** (subdivisões onde os itens residem) e **Itens** (a unidade individual de conteúdo). A Coleção é o nível em que se

configuram a privacidade e o layout, de modo que um mesmo acervo pode reunir partes públicas e partes reservadas sob uma única Categoria.

- **Customização Estrutural (Editor de Layout):** Motor que permite criar e editar modelos (*templates*) visuais. O usuário pode adicionar, reposicionar e redimensionar **Elementos** dos seis tipos oferecidos pelo editor: texto, imagem, forma, ícone, classificação e data.
- **Interação Social:** Funcionalidades que permitem adicionar amigos, seguir coleções visíveis de outros usuários e visualizar atualizações de acervos em um *feed* cronológico.
- **Personalização Visual Global:** Controle sobre a estética da interface por meio de uma biblioteca de paletas de cores e tipografia, com estilo claro, escuro ou automático, garantindo que o ambiente digital reflita o estilo do colecionador.
- **Notificação:** Comunicação ao usuário dos eventos que o envolvem, como solicitações de amizade e atualizações nas coleções que ele acompanha, reunidas em uma área dedicada dentro da plataforma.

4.3 Características dos Usuários

O produto foi concebido para atender a dois perfis principais de interação. Eles são hipóteses de projeto, definidas na concepção a partir da motivação relatada na Justificativa e da distinção, discutida na Seção 2.2, entre quem prefere ajustes mínimos e quem se envolve com a personalização, e não resultado de uma pesquisa com usuários:

- **Usuário Organizador (Casual):** Utiliza a plataforma para manter listas simples de seus pertences. Prefere associar às suas coleções os *layouts* padrão oferecidos pelo sistema, focando na velocidade e praticidade do cadastro.
- **Usuário Curador (Entusiasta):** Possui um envolvimento profundo com seus acervos. Este perfil dedica tempo ao **Editor de Layout** para criar fichas

técnicas detalhadas e esteticamente personalizadas, buscando uma experiência de catalogação profissional.

4.4 Restrições Gerais

O desenvolvimento e a operação do sistema estão condicionados aos seguintes fatores limitadores:

- **Conectividade:** Por se tratar de uma aplicação *web* cliente-servidor, com os dados mantidos no servidor, o acesso ao acervo e às funcionalidades sociais exige conexão estável com a internet.
- **Compatibilidade de Interface:** O sistema é otimizado para navegadores modernos que suportam tecnologias de renderização dinâmica. Embora seja responsivo, a experiência avançada do **Editor de Layout** é otimizada para dispositivos de médio e grande porte, como *tablets* e *desktops*.
- **Escopo de Mídia:** O armazenamento de arquivos (imagens) está sujeito a limites de tamanho definidos para garantir a performance de carregamento e a viabilidade da infraestrutura do servidor.

4.5 Diferenciais do Produto

Diferente de planilhas eletrônicas ou aplicativos de listas estáticas, o grande diferencial desta plataforma é o **acoplamento entre dado e forma**. Enquanto outras ferramentas tratam o registro de um item como uma simples linha em uma tabela, este produto trata cada item como uma experiência visual única. O usuário define não apenas *o que* guardar, mas *como* aquela informação deve ser apresentada, proporcionando uma ferramenta de curadoria que evolui junto com a coleção.

5 ELICITAÇÃO DE REQUISITOS E ANÁLISE

A engenharia de requisitos é uma etapa fundamental no processo de desenvolvimento de software, sendo responsável por definir o que o sistema deve fazer e sob quais restrições ele deve operar (VALENTE, 2020). Segundo (SOMMERVILLE, 2011), essa disciplina engloba as atividades de descoberta, análise, documentação e validação dos requisitos, constituindo a base sobre a qual todo o projeto será construído. (PRESSMAN; MAXIM, 2016) complementam essa visão ao afirmar que a engenharia de requisitos envolve a coleta de informações sobre o sistema a ser desenvolvido, a análise do problema e a descrição, classificação e priorização dos requisitos identificados.

O processo de elicitação de requisitos, também chamado de levantamento ou descoberta de requisitos, consiste em reunir informações junto aos stakeholders para compreender suas necessidades e expectativas em relação ao sistema (SOMMERVILLE, 2011). Diversas técnicas podem ser empregadas nessa etapa, como entrevistas, questionários, workshops, prototipação e análise de cenários de uso (VALENTE, 2020)

Este capítulo apresenta os requisitos identificados para o sistema proposto, organizados em requisitos do usuário e requisitos do sistema, sendo estes últimos subdivididos em requisitos funcionais e não funcionais.

5.1 Requisitos do Usuário

Requisitos de usuário são declarações, em linguagem natural com diagramas, dos serviços que o sistema deverá fornecer aos seus usuários e das restrições com as quais ele deve operar (SOMMERVILLE, 2011). Pressman e Maxim (2016) destacam que um documento de requisitos de sistema precisa ser claro, consistente e completo, pois servirá de referência para desenvolvedores, gerentes de projeto e engenheiros

de testes ao longo de todo o ciclo de vida do software.

Antes de apresentar os requisitos propriamente ditos, é importante compreender os **quatro** conceitos centrais que estruturam a modelagem de dados da plataforma proposta: **Elemento**, **TipoElemento**, **Layout** e **Item**. Para facilitar a compreensão do papel de cada um no sistema, vale partir de uma analogia com ferramentas de edição visual consolidadas no mercado, como o *Canva*. Nesses editores, o usuário monta uma tela de forma livre, acrescentando componentes gráficos a partir de uma paleta, como caixas de texto, imagens e ícones, e ajustando em seguida a posição e o tamanho de cada um. A plataforma proposta adota essa mesma dinâmica de montagem visual, mas com um diferencial fundamental: a tela montada pelo usuário não é uma arte gráfica isolada, e sim um **modelo de dados reutilizável**, que pode ser compartilhado por dezenas ou centenas de itens de uma coleção, preservando a individualidade das informações de cada um.

A partir dessa analogia, os quatro conceitos podem ser entendidos na mesma ordem. O **Elemento** é cada peça que o usuário acrescenta à tela, como uma caixa de texto, uma imagem ou uma classificação por estrelas. É nele que residem tanto o conteúdo exibido quanto os atributos que determinam onde ele aparece: posição, largura, altura e camada de sobreposição. O modo como ele aparece fica a cargo do **EstiloDeElemento**, conjunto de propriedades visuais associado a cada Elemento, tratado adiante. O **TipoElemento** é a classificação desse Elemento dentro do conjunto fixo de tipos que o editor oferece. Trata-se de uma enumeração, e não de uma entidade com atributos próprios, de modo que dois Elementos do mesmo tipo, como “Autor” e “Gênero”, ambos de texto, são registros distintos, cada um com suas próprias coordenadas. O **Layout** é a tela montada, isto é, o conjunto de Elementos organizados visualmente, funcionando como um modelo reutilizável associado a uma coleção. E o **Item** é uma unidade concreta do acervo do usuário, por exemplo o livro “Duna”, que parte desse modelo e pode manter um layout próprio. O conteúdo exibido pode estar diretamente nos Elementos textuais, como em “Autor:” ou “Autor: Frank Herbert”, e o usuário pode separar rótulos e valores criando Elementos independentes quando desejar.

Uma analogia complementar, mais próxima da estrutura de dados, é a de um **molde reutilizável**. O *Layout* funciona como um molde já desenhado, que

define quais Elementos existem, sua posição e seu tamanho, mas não o que cada um exibe; os Elementos são as peças desenhadas nesse molde; e o *Item* é a unidade que parte dele dentro de uma coleção. Para receber valores próprios, o item ganha uma cópia do molde, o seu layout próprio, que pode ser alterado sem afetar o layout da coleção nem os demais itens.

Em termos práticos, considere um exemplo concreto. Suponha que o usuário crie um Layout chamado *"Card de Livro"* contendo quatro elementos: um de *Imagem* para a capa, um de *Texto* para o autor, um elemento de *Classificação* de um a cinco estrelas e outro de *Texto* para notas pessoais. Cada elemento possui posição e tamanho próprios dentro da área de edição do layout. O usuário cadastra o item *"Duna"* e, em seguida, abre a sua ficha para preenchê-la. Nesse momento, o sistema copia para o item os elementos do layout, e é nessas cópias que ficam os valores do item: para o elemento *Imagem*, a capa; para o primeiro *Texto*, o autor *"Frank Herbert"*; para o elemento *Classificação*, o valor *5*; e para o elemento *Texto*, a nota *"Obra-prima do gênero"*. Desse modo, o Layout e seus elementos permanecem reutilizáveis por diferentes itens da coleção, enquanto cada item preserva sua própria ficha visual.

Essa separação traz três benefícios diretos para o projeto:

(i) permite que o usuário reutilize o mesmo layout para múltiplos itens sem duplicação de estrutura;

(ii) viabiliza que cada item preserve valores distintos, respeitando a individualidade de cada obra cadastrada; e

(iii) permite modificações individuais em um item específico, como adicionar um Elemento extra apenas a um único livro, sem afetar o layout original nem os demais itens (RF-17).

O Quadro 2 apresenta, de forma resumida, a definição dos quatro conceitos centrais e de dois termos que deles derivam e são usados ao longo do texto: o EstiloDeElemento, que reúne as propriedades visuais de um Elemento, e o layout padrão, oferecido pelo sistema.

Cabe esclarecer dois pontos antes de apresentar os requisitos. Em primeiro

lugar, o conjunto de tipos de elemento é **fixo** e definido por uma enumeração (TipoElemento), e não um cadastro gerido em tempo de execução. Essa decisão segue o modelo de editores visuais consolidados, como o Canva, nos quais os *tipos* de elemento (texto, imagem, formas, entre outros) fazem parte das capacidades nativas da ferramenta, enquanto o que o usuário cria livremente são *instâncias* desses tipos: quantos elementos quiser, acrescentando-os pela paleta e depois posicionando-os e redimensionando-os na tela. A eventual inclusão de um tipo inteiramente novo (por exemplo, vídeo incorporado) caracteriza uma evolução do próprio editor, e não um dado a ser cadastrado pelo usuário final.

Em segundo lugar, cabe registrar onde reside o conteúdo de cada item, pois é disso que depende a relação entre o Layout e o Item. O conteúdo não é guardado em uma entidade à parte: fica no próprio Elemento, ao lado da posição, do tamanho e das propriedades visuais. O Layout associado a uma coleção define apenas quais Elementos existem e como se organizam, sem guardar o que qualquer item exibe. A individualização ocorre em outro nível: na primeira vez em que a ficha do item é aberta para edição, o Item recebe um **layout próprio**, copiado do layout da coleção quando ela possui um, ou vazio quando não possui (RF-17). A interface abre essa ficha logo depois do cadastro, de modo que, no uso normal, todo item preenchido tem seus próprios Elementos, nos quais conteúdo, posição e estilo permanecem indissociáveis. Chama-se **layout efetivo** de um item o seu layout próprio, quando existe, ou, na falta dele, o layout da coleção; esse segundo caso corresponde ao item cuja ficha ainda não foi aberta, que exibe o layout da coleção tal como ele está.

5.1.1 Organização em Pacotes

Devido à quantidade de classes conceituais envolvidas, o modelo de domínio foi organizado em sete pacotes, de modo a agrupar entidades que compartilham propósito semântico e reduzir a complexidade visual do diagrama. Essa segmentação segue a recomendação de (LARMAN, 2007), segundo o qual o agrupamento de classes em pacotes facilita a leitura do modelo, evidencia suas fronteiras conceituais e favorece o raciocínio modular sobre o sistema. Os pacotes adotados neste trabalho são:

- **Usuário:** concentra a entidade central da plataforma, responsável pelos dados de identificação e pelo vínculo com todas as demais entidades do sistema. Foi isolada em um pacote próprio por atuar como ponto de referência para os demais agrupamentos.
- **Coleções:** reúne as entidades que compõem a estrutura de organização do acervo em três níveis hierárquicos: Categoria, Coleção e Item. Inclui também a enumeração de privacidade. Representa o núcleo de domínio da aplicação, pois é nele que o usuário efetivamente cadastra e organiza seu conteúdo.
- **Editor de Layout:** agrupa as entidades responsáveis pela personalização visual das fichas de itens, incluindo Layout, Elemento, o EstiloDeElemento, que reúne as propriedades visuais de cada Elemento, e a enumeração TipoElemento, que define o conjunto fixo de tipos de elemento disponíveis no editor (texto, imagem, forma, ícone, classificação e data). É este pacote que sustenta o diferencial de customização da plataforma.
- **Interação Social:** concentra as entidades ligadas aos vínculos entre usuários, como as solicitações de amizade (Amizade e StatusAmizade) e o acompanhamento de coleções de outros usuários (Seguir Coleção).
- **Notificações:** reúne as entidades responsáveis pelo registro e pela exibição de eventos relevantes ao usuário: a própria entidade Notificação, a enumeração TipoNotificacao, a entidade AtualizaçãoEvento e a enumeração TipoEvento. A separação deste pacote em relação ao de Interação Social reflete a distinção entre o vínculo social em si e o mecanismo de comunicação dessas ocorrências ao usuário, além de isolar a lógica de geração e exibição de notificações para facilitar futuras extensões do sistema.
- **Aparência:** reúne as entidades que definem a configuração visual global da interface: a biblioteca de paletas do usuário, cada uma com suas cores, sua tipografia e seu estilo claro ou escuro.
- **Sistema:** reúne as entidades de apoio à operação da plataforma, que não pertencem ao domínio do acervo nem às interações entre usuários: a sessão de usuário, que permite revogar o acesso no momento do logout, o token de redefinição de senha, de uso único e com prazo de validade, e o código de

confirmação do e-mail, exigido antes do primeiro acesso. Foi isolado em pacote próprio por sustentar os requisitos de acesso sem participar das regras de negócio das coleções.

Essa organização reflete a separação de responsabilidades identificada durante a elicitação de requisitos: a entidade central do pacote *Usuário* sustenta o cadastro e o perfil (RF-01 e RF-05), enquanto as do pacote *Sistema* viabilizam a confirmação do e-mail no cadastro (RF-01), a autenticação, a alteração e a recuperação de senha (RF-02 a RF-04); as classes do pacote *Coleções* sustentam os requisitos funcionais de gestão de conteúdo (RF-06 a RF-13); as do pacote *Editor de Layout* suportam os requisitos de personalização estrutural (RF-14 a RF-17); as do pacote *Aparência* viabilizam a biblioteca de paletas oferecida no cadastro e gerenciada pelo usuário (RF-18 e RF-19); as do pacote *Interação Social* dão suporte às funcionalidades de rede entre usuários (RF-20 a RF-23); e as do pacote *Notificações* sustentam o mecanismo de comunicação de eventos do sistema ao usuário (RF-24).

Cabe observar que os grupos de requisitos e os pacotes conceituais são recortes distintos e não coincidem termo a termo. O grupo *Acesso ao Sistema* (RF-01 a RF-05), por exemplo, distribui-se entre dois pacotes: os dados de identificação do usuário residem em *Usuário*, ao passo que as estruturas que sustentam sessão e redefinição de senha residem em *Sistema*. O agrupamento em pacotes obedece à afinidade conceitual entre as classes, e não à organização adotada na especificação dos requisitos.

O modelo de domínio resultante é apresentado na Seção 5.4, ao final deste capítulo, depois de detalhados os requisitos e os casos de uso que o originaram.

5.2 Requisitos do Sistema

Sommerville (2011) afirma que os requisitos do sistema são tradicionalmente classificados em requisitos **funcionais**, que descrevem as funcionalidades que o sistema deve oferecer, e requisitos **não funcionais**, que definem restrições de qualidade como desempenho, segurança e usabilidade. Eles constituem o núcleo

da especificação do sistema, pois delimitam o escopo das funcionalidades a serem implementadas, e sua classificação permite uma organização sistemática das especificações, facilitando tanto o desenvolvimento quanto a validação do sistema.

Assim, esta seção apresenta os requisitos identificados neste trabalho, organizados nas subseções a seguir.

5.2.1 Requisitos Funcionais

Segundo Sommerville (2011), requisitos funcionais descrevem as funções ou serviços que o sistema deve fornecer, ou seja, o que o sistema deve fazer. Eles especificam as ações que o sistema deve ser capaz de executar em resposta a entradas específicas ou em determinadas condições.

Os atores que interagem com o sistema são: **Usuário**, que representa a pessoa cadastrada na plataforma e responsável por criar, editar e gerenciar suas categorias, coleções, itens e interações sociais; e o **Sistema**, que atua de forma autônoma em funcionalidades como exibição de fichas de itens, envio de notificações, registro dos eventos que alimentam o feed e organização visual dos dados.

Para padronizar a especificação dos requisitos, adotou-se o verbo **gerenciar** como forma genérica de representar operações do tipo CRUD (cadastrar, editar e excluir), sempre que aplicável. Essa padronização reduz a repetição textual e torna a descrição dos requisitos mais objetiva.

Os requisitos funcionais estão organizados em seis grupos, apresentados de forma sequencial e seguindo uma ordem lógica de dependência entre funcionalidades, alinhada à jornada natural do usuário no sistema: iniciam pelo **Acesso ao Sistema** (cadastro, autenticação, recuperação e alteração de senha e gerenciamento de perfil); prosseguem com a **Coleção**, núcleo do sistema (categorias, coleções, privacidade, itens, suas fichas e formas de visualização); avançam para o **Editor de Layout** (criação de layouts, associação a coleções, compartilhamento entre usuários e personalização individual); seguem para a **Aparência**, camada de personalização visual que se aplica sobre o conteúdo já organizado (paletas de cores e tipografia, com estilo claro ou escuro); contemplam a **Interação Social** (amizade, acompanhamento de coleções e visualização de perfis); e culminam na **Notificação**, mecanismo de

comunicação de eventos ao usuário. Os quadros 3 a 7 apresentam a definição dos campos e das classificações utilizadas na especificação, e os requisitos funcionais identificados para este sistema estão apresentados nos Quadros 8 a 31.

Quadro 2 – Conceitos centrais do modelo de domínio

Conceito	**Descrição**
Elemento	Componente visual reutilizável utilizado para compor layouts, como textos, imagens, classificações, datas e ícones. Cada elemento possui nome, tipo, conteúdo opcional, posição, largura, altura e camada de sobreposição, além do seu EstiloDeElemento.
EstiloDeElemento	Conjunto de propriedades visuais que define a aparência de um Elemento, sem interferir na sua posição nem no seu conteúdo. Reúne, para o texto, fonte, tamanho, peso, itálico, sublinhado, riscado, alinhamento horizontal e vertical, nível de título, entrelinha, espaçamento entre letras e apresentação em lista; para a forma e a imagem, forma do elemento, arredondamento e borda (cor, espessura e traço); para a forma, ainda, curva e ponta da linha; para a imagem, ajuste; para a classificação, nota máxima e ícone; para a data, formato e estética; e, para qualquer tipo, cor, cor de fundo, opacidade e rotação. É um conjunto extensível: novas propriedades acompanham a evolução do editor, sem alterar os demais conceitos.
TipoElemento	Conjunto fixo e predefinido de categorias de elemento que o usuário pode acrescentar no editor, à semelhança dos elementos básicos de ferramentas como o Canva. Compreende: texto, imagem, forma, ícone, classificação e data. É representado por uma enumeração, e não por uma entidade com atributos próprios, pois constitui um conjunto estável que faz parte das capacidades do próprio editor.
Layout	Estrutura personalizada que define quais elementos compõem a ficha de um item e como eles se organizam visualmente, sem guardar o conteúdo de nenhum item em particular. Funciona como um *template* reutilizável, que pode ser associado a uma coleção; um mesmo layout pode ser reutilizado por diferentes coleções. A associação é opcional: os itens de uma coleção que possui layout partem dele, e os de uma coleção sem layout partem de uma ficha vazia.
Layout padrão	Layout já montado, oferecido pelo sistema na biblioteca de layouts do usuário desde o cadastro (RF-01). Serve de facilitador para quem não deseja compor uma ficha do zero: pode ser associado a uma coleção como qualquer outro layout, e igualmente editado ou duplicado (RF-14 e RF-15). Não é aplicado automaticamente.
Item	Unidade individual de conteúdo cadastrada pelo usuário (por exemplo, um livro, filme ou jogo específico). Todo item pertence a uma coleção e parte do layout associado a ela, quando existe, podendo receber um layout próprio conforme RF-17.

Fonte: Elaborada pelo autor

Quadro 3 – Descrição dos Campos dos Requisitos Funcionais

Campo	**Descrição**
Código	Identificador único do requisito funcional (ex.: RF-01).
Grupo	Categoria à qual o requisito pertence, organizando funcionalidades semelhantes (ex.: Acesso ao Sistema, Coleção, etc.).
Descrição	Detalhamento do requisito estruturado em ação, objeto, atributos, exemplos e restrições.
Prioridade	Define o nível de importância do requisito (Essencial, Importante ou Desejável).
Operação	Classifica o tipo de operação (Entrada, Processamento ou Saída).
Ator	Entidade responsável pela execução ou interação (Usuário ou Sistema).

Fonte: Elaborada pelo autor

Quadro 4 – Grupos de Requisitos Funcionais

Grupo	**Descrição**
Acesso ao Sistema	Cadastro, autenticação, recuperação e alteração de senha e gerenciamento do perfil do usuário.
Coleção	Gerenciamento de categorias, coleções, privacidade, itens e respectivas formas de visualização.
Editor de Layout	Criação, associação, compartilhamento e personalização de layouts e Elementos.
Aparência	Configuração visual global da interface por meio de paletas de cores e tipografia, com estilo claro ou escuro.
Interação Social	Vínculos de amizade, acompanhamento de coleções e visualização de perfis de outros usuários.
Notificação	Geração, exibição e gerenciamento das notificações enviadas ao usuário.

Fonte: Elaborada pelo autor

Quadro 5 – Classificação de Prioridade dos Requisitos

Prioridade	**Descrição**
Essencial	Requisito indispensável para o funcionamento do sistema.
Importante	Requisito necessário para o funcionamento adequado, mas não crítico.
Desejável	Requisito opcional, podendo ser implementado posteriormente.

Fonte: Adaptado do modelo de requisitos

Quadro 6 – Classificação de Operação dos Requisitos

Operação	**Descrição**
Entrada	Dados fornecidos pelo usuário ao sistema.
Processamento	Tratamento e validação realizados pelo sistema.
Saída	Exibição ou retorno de informações ao usuário.

Fonte: Adaptado do modelo de requisitos

Quadro 7 – Atores do Sistema

Ator	**Descrição**
Usuário	Pessoa que utiliza o sistema para gerenciar suas coleções e interações.
Sistema	Responsável por execuções automáticas e processamento de dados.

Fonte: Elaborada pelo autor

Quadro 8 – RF-01 – Cadastrar Usuário

Código	RF-01
Grupo	Acesso ao Sistema
Descrição	**Ação:** Cadastrar **Objeto:** Usuário **Atributos:** nome (100 caracteres), email (150 caracteres), senha (mínimo 8 caracteres), foto de perfil (JPEG/PNG, máx. 5 MB), descrição (500 caracteres), código de usuário (gerado automaticamente pelo sistema, único e imutável), código de confirmação do e-mail (6 dígitos, gerado pelo sistema), data de confirmação do e-mail. **Exemplos:** Usuário preenche nome 'Camila Albieri Mattos', email 'camilamattos.mila@gmail.com' e senha 'Camila@2024'; o sistema gera automaticamente o código de usuário 'camila#4827' e envia ao e-mail informado um código de 6 dígitos, que Camila digita para concluir o cadastro e entrar na plataforma. Usuário tenta cadastrar com o email 'camilamattos.mila@gmail.com', que já existe, e recebe mensagem de erro informando que o email já está em uso. **Restrições:** 1. Nome, email e senha são obrigatórios; foto de perfil e descrição são opcionais. 2. O email deve estar em formato válido (ex.: usuario@dominio.com). 3. Não podem ser registrados dois ou mais usuários com o mesmo email. 4. A senha deve ter no mínimo 8 caracteres. 5. A senha deve ser armazenada de forma criptografada (RNF-03). 6. O código de usuário é gerado automaticamente pelo sistema no momento do cadastro, deve ser único em toda a plataforma e não pode ser alterado posteriormente. 7. O código de usuário é o identificador público utilizado para buscas e envio de solicitações de amizade. 8. Ao concluir o cadastro, o sistema disponibiliza na biblioteca de aparência do usuário as paletas predefinidas que oferece, três claras e três escuras, deixando em uso a paleta 'Claro' ou a paleta 'Escuro', conforme o tema do sistema operacional no momento do cadastro (RF-18).

Continua na próxima página

Quadro 8 – Continuação da página anterior

	9. Ao concluir o cadastro, o sistema também disponibiliza na biblioteca de layouts do usuário os layouts padrão que oferece, prontos para serem associados a coleções, editados ou duplicados (RF-14 e RF-15). 10. A conta só concede acesso depois que o e-mail é confirmado. O sistema envia ao endereço cadastrado um código de 6 dígitos, válido por 15 minutos, guardado apenas na forma de *hash* e aceito em no máximo cinco tentativas. 11. O usuário pode pedir um novo código, com intervalo mínimo de um minuto entre pedidos e no máximo cinco códigos por hora; o código novo invalida o anterior.
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 9 – RF-02 – Autenticar Usuário (Login e Logout)

Código	RF-02
Grupo	Acesso ao Sistema
Descrição	**Ação:** Autenticar (login e logout) **Objeto:** Usuário **Atributos:** email (150 caracteres), senha (mínimo 8 caracteres), token de sessão (gerado automaticamente pelo sistema). **Exemplos:** Usuário informa 'camilamattos.mila@gmail.com' e 'Camila@2024', o sistema valida as credenciais e concede acesso à área restrita. Usuário informa email não cadastrado e recebe a mensagem 'Credenciais de login inválidas'. Usuário clica em 'Sair' e tem sua sessão encerrada imediatamente, sendo redirecionado para a tela de login. **Restrições:** 1. Email e senha são obrigatórios para realizar o login. 2. O sistema deve verificar se o email está cadastrado e se a senha confere com o hash armazenado. 3. Não conferindo, o sistema deve exibir 'Credenciais de login inválidas' sem especificar qual campo está errado.

Continua na próxima página

Quadro 9 – Continuação da página anterior

	4. A cada autenticação bem-sucedida o sistema deve registrar uma sessão própria e vinculá-la ao token emitido. 5. O token de sessão deve ter prazo de validade definido, expirando automaticamente ao final desse prazo. 6. A validade da sessão deve ser verificada pelo sistema a cada requisição autenticada, de modo que uma sessão revogada deixe de conceder acesso imediatamente, mesmo que o token ainda não tenha expirado. 7. O logout deve revogar imediatamente a sessão ativa, tornando o token inutilizável a partir desse momento.
Prioridade	Essencial
Operação	Processamento
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 10 – RF-03 – Alterar Senha na Área Restrita

Código	RF-03
Grupo	Acesso ao Sistema
Descrição	**Ação:** Alterar senha **Objeto:** Usuário (autenticado) **Atributos:** senha atual (para confirmação), nova senha (mínimo 8 caracteres), confirmação da nova senha. **Exemplos:** Camila, já autenticada, acessa 'Configurações de Conta', informa a senha atual, define a nova senha 'NovaSenha@2025', confirma e o sistema atualiza a credencial. Camila informa a senha atual incorreta e recebe a mensagem 'Senha atual inválida'. Camila digita a nova senha e a confirmação divergentes e recebe aviso de que os campos não coincidem. **Restrições:** 1. Esta operação só está disponível para o usuário autenticado, dentro da área restrita. 2. O usuário deve informar corretamente a senha atual antes de definir a nova senha. 3. A nova senha deve ter no mínimo 8 caracteres e ser diferente da senha atual.

Continua na próxima página

Quadro 10 – Continuação da página anterior

	4. A nova senha e sua confirmação devem coincidir. 5. A nova senha deve ser armazenada de forma criptografada (RNF-03). 6. Após a alteração, as demais sessões ativas do usuário devem ser invalidadas por segurança.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 11 – RF-04 – Recuperar Senha via E-mail

Código	RF-04
Grupo	Acesso ao Sistema
Descrição	**Ação:** Recuperar senha **Objeto:** Usuário (não autenticado) **Atributos:** email cadastrado, token de redefinição (gerado automaticamente, único e temporário), nova senha (mínimo 8 caracteres), confirmação da nova senha. **Exemplos:** Camila, sem estar logada, clica em 'Esqueci minha senha', informa seu email e recebe uma mensagem com link de redefinição. Camila abre o link recebido, define a nova senha e passa a conseguir autenticar-se com a nova credencial. Camila tenta usar um link de redefinição expirado e recebe a mensagem de que o link não é mais válido, com opção de solicitar um novo. **Restrições:** 1. A recuperação é iniciada por usuário não autenticado, a partir da tela de login. 2. O sistema solicita o email cadastrado e, se ele existir, envia um link de redefinição contendo um token único e temporário (RF-24). 3. Por segurança, a mensagem exibida ao solicitante deve ser a mesma independentemente de o email existir ou não, para não revelar quais emails estão cadastrados. 4. O token de redefinição deve expirar em uma hora e ser de uso único.

Continua na próxima página

Quadro 11 – Continuação da página anterior

	5. Ao acessar o link, o usuário define a nova senha e sua confirmação, que devem coincidir e ter no mínimo 8 caracteres. 6. A nova senha deve ser armazenada de forma criptografada (RNF-03). 7. Concluída a redefinição, o token é invalidado e as sessões ativas anteriores são encerradas.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 12 – RF-05 – Gerenciar Perfil do Usuário

Código	RF-05
Grupo	Acesso ao Sistema
Descrição	**Ação:** Editar **Objeto:** Usuário **Atributos:** nome de exibição (100 caracteres), foto de perfil (JPEG/PNG, máx. 5 MB), imagem de capa do perfil (JPEG/PNG, opcional; enviada com até 10 MB e enquadrada pelo usuário, e o recorte gravado tem até 2 MB), imagem original da foto e da capa (guardadas apenas para permitir o reenquadramento), área do recorte de cada uma, descrição/bio (500 caracteres). **Exemplos:** Usuário acessa 'Editar Perfil', altera o nome de exibição para 'Cami Mattos', adiciona bio 'Leitora e colecionadora de histórias' e salva as alterações. Usuário envia uma foto de capa, enquadra a faixa exibida no topo do perfil e, dias depois, reabre o enquadramento para ajustá-lo sem enviar a imagem de novo. **Restrições:** 1. O nome de exibição deve ter entre 3 e 100 caracteres e é obrigatório. 2. A foto de perfil deve estar em formato JPEG ou PNG com tamanho máximo de 5 MB. 3. A descrição/bio é opcional e pode ser deixada em branco. 4. Alterações só têm efeito após o usuário salvar explicitamente.

Continua na próxima página

Quadro 12 – Continuação da página anterior

	5. A foto e a capa são enquadradas pelo usuário antes de serem gravadas. O sistema guarda o arquivo original junto com a área recortada, de modo que o enquadramento possa ser refeito sem novo envio; remover a imagem apaga as duas versões. 6. O e-mail cadastrado não é alterado por este requisito; a troca de endereço, com verificação do novo e-mail, consta dos requisitos adiados (Subseção 5.2.4). 7. A alteração de senha não é feita por este requisito, e sim pelo RF-03 (área restrita). 8. O código de usuário (RF-01) não pode ser alterado por este caso de uso.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 13 – RF-06 – Gerenciar Categoria

Código	RF-06
Grupo	Coleção
Descrição	**Ação:** Gerenciar **Objeto:** Categoria (agrupador temático de alto nível que reúne coleções relacionadas) **Atributos:** nome (100 caracteres), tipo/tema (texto livre de até 50 caracteres, com 'livros', 'filmes', 'séries', 'jogos' e 'músicas' oferecidos como sugestão; o usuário pode definir o seu próprio), ícone (emoji Unicode, opcional), imagem de capa (JPEG/PNG, opcional; enviada com até 10 MB e enquadrada pelo usuário na proporção do banner, e o recorte gravado tem até 2 MB), imagem original da capa (guardada apenas para permitir o reenquadramento, nunca exibida), área do recorte (posição e tamanho em pixels do original), cor de fundo do cartão (hexadecimal, opcional), descrição (500 caracteres), data de criação (gerada automaticamente).

Continua na próxima página

Quadro 13 – Continuação da página anterior

	Exemplos: Usuário Camila cria a categoria 'Livros' com ícone personalizado e descrição 'Tudo que leio ou pretendo ler'; a categoria é criada vazia, pronta para receber coleções. Camila edita a categoria, renomeando para 'Biblioteca Pessoal'. Camila exclui a categoria 'Jogos Antigos' e confirma a exclusão permanente no diálogo de confirmação, que alerta sobre a remoção de todas as coleções e itens vinculados. **Restrições:** 1. O nome da categoria é obrigatório; ícone, imagem de capa e descrição são opcionais. 2. Não podem existir duas categorias com o mesmo nome para o mesmo usuário. 3. A categoria, ao ser criada, permanece vazia e não contém itens diretamente; os itens só podem ser cadastrados dentro de suas coleções (RF-07 e RF-09). 4. A categoria serve apenas como agrupador temático; a definição de layout ocorre no nível da coleção, e não no da categoria (RF-07 e RF-15). 5. A categoria não possui nível de privacidade próprio; a privacidade é configurada individualmente em cada coleção (RF-08). 6. A exclusão deve exibir diálogo de confirmação alertando que todas as coleções e itens vinculados serão removidos permanentemente. 7. A data de criação é gerada automaticamente pelo sistema e não pode ser editada. 8. O tema é gravado em letras minúsculas, sem espaços nas extremidades e sem acentuação, de modo que grafias diferentes da mesma palavra sejam tratadas como o mesmo tema. A exibição apresenta a primeira letra em maiúscula. 9. O ícone e a imagem de capa coexistem e são escolhidos separadamente: a capa ocupa o banner do cartão, e o emoji aparece sobre ela e no modo lista. Difere da coleção (RF-07), onde as duas formas são alternativas entre si. 10. Categoria sem ícone e sem capa é exibida apenas com a cor de fundo, sem símbolo padrão.

Continua na próxima página

Quadro 13 – Continuação da página anterior

	11. A imagem de capa é enquadrada pelo usuário na proporção do banner antes de ser gravada. O sistema guarda o arquivo original junto com a área recortada, de modo que o enquadramento possa ser refeito depois sem novo envio; remover a capa apaga as duas imagens. 12. O nome da categoria é exibido sobre a capa, e a legibilidade sobre qualquer imagem é responsabilidade da interface, que aplica escurecimento e sombra ao texto (RNF-05).
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 14 – RF-07 – Gerenciar Coleção

Código	RF-07
Grupo	Coleção
Descrição	**Ação:** Gerenciar **Objeto:** Coleção (subdivisão estruturada que pertence a uma categoria e agrupa itens) **Atributos:** nome (100 caracteres), categoria pai (referência obrigatória), ícone (emoji Unicode ou imagem JPEG/PNG, opcional; a imagem é enviada com até 10 MB e enquadrada pelo usuário, e o recorte gravado tem até 2 MB), imagem original do ícone (guardada apenas para permitir o reenquadramento, nunca exibida), área do recorte (posição e tamanho em pixels do original), cor de fundo do cartão (hexadecimal, opcional), descrição (500 caracteres), layout associado (referência opcional a um layout do usuário), nível de privacidade (pública, somente amigos ou privada), tags (de nenhuma a dez, texto livre de até 30 caracteres cada, opcionais), data de criação (gerada automaticamente).

Continua na próxima página

Quadro 14 – Continuação da página anterior

	Exemplos: Usuário cria a coleção ‘Lidos’ dentro da categoria ‘Livros’, definindo a privacidade como ‘pública’; a coleção passa a aceitar itens. Camila edita a coleção ‘Lidos’ para associar o layout ‘Card de Mangá’, que passa a valer para os itens dessa coleção. Camila exclui a coleção ‘Rascunhos’ e confirma no diálogo a remoção de todos os itens vinculados. Camila marca a coleção ‘Lidos’ com a tag ‘fantasia’ e, clicando nela, vê as coleções de outras pessoas marcadas com a mesma tag que a privacidade lhe permite ver, e quem as mantém. Ao escrever uma tag no formulário, Camila recebe como sugestão ‘ficcao cientifica’, palavra já utilizada por outro usuário do sistema. **Restrições:** 1. O nome e a referência à categoria pai são obrigatórios. 2. Não podem existir duas coleções com o mesmo nome dentro da mesma categoria para o mesmo usuário. 3. Toda coleção pertence a exatamente uma categoria. 4. Por padrão, a privacidade de uma nova coleção é ‘pública’, podendo ser alterada conforme RF-08. 5. Toda coleção pode ter um layout associado, que determina os elementos exibidos e preenchíveis nos itens dessa coleção; um mesmo layout pode ser reutilizado por diferentes coleções (RF-15). 6. A exclusão deve exibir diálogo de confirmação alertando que todos os itens vinculados serão removidos permanentemente. 7. A data de criação é gerada automaticamente pelo sistema e não pode ser editada. 8. O usuário deve poder alternar entre os modos de visualização ‘grade’ e ‘lista’ a qualquer momento; o modo padrão é ‘grade’ e a escolha deve ser persistida entre sessões. 9. O usuário deve poder ordenar as coleções por pelo menos quatro critérios: nome crescente, nome decrescente, data de criação crescente e data de criação decrescente. 10. A ordenação padrão é por data de criação decrescente (mais recentes primeiro). 11. O usuário deve poder filtrar as coleções por nível de privacidade (RF-08); a ordenação e o filtro aplicados devem ser persistidos entre sessões.

Continua na próxima página

Quadro 14 – Continuação da página anterior

	12. A coleção pode receber até dez tags, opcionais, de texto livre com até 30 caracteres cada. As tags são gravadas em forma canônica (sem espaços nas pontas, em minúsculas e sem acentuação), de modo que grafias diferentes da mesma palavra correspondam sempre à mesma tag; tags repetidas na mesma coleção são descartadas. 13. O vocabulário de tags é compartilhado por todos os usuários: cada palavra existe uma única vez no sistema, e o formulário sugere as tags já utilizadas por qualquer conta. O compartilhamento alcança apenas o rótulo: as coleções que o utilizam continuam sujeitas às regras de privacidade do RF-08, e uma tag pode existir no vocabulário sem que o usuário possua qualquer coleção que a use. 14. Cada tag exibida é um vínculo para a tela Explorar, que reúne as coleções de outros usuários marcadas com todas as tags escolhidas, respeitando a privacidade de cada uma (RF-08): públicas para qualquer usuário, somente amigos apenas para os amigos do dono, e privadas nunca. As coleções do próprio usuário ficam de fora, a resposta traz no máximo 60 coleções, e a tela mostra também as pessoas que mantêm as coleções encontradas, a partir das quais se chega ao perfil (RF-23). 15. A tag sobrevive à exclusão das coleções que a utilizavam: o vínculo é desfeito, e a palavra permanece disponível no vocabulário. 16. O ícone aceita uma única forma por vez: escolher uma imagem substitui o emoji, e escolher um emoji descarta a imagem. Difere da categoria (RF-06), onde o emoji e a capa coexistem. 17. A imagem do ícone é enquadrada pelo usuário em proporção quadrada antes de ser gravada. O sistema guarda o arquivo original junto com a área recortada, de modo que o enquadramento possa ser refeito depois sem novo envio; remover o ícone apaga as duas imagens.
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 15 – RF-08 – Gerenciar Privacidade da Coleção

Código	RF-08
Grupo	Coleção
Descrição	**Ação:** Configurar **Objeto:** Nível de visibilidade da coleção **Atributos:** nível de privacidade (privada, somente amigos, pública), coleção alvo (referência). **Exemplos:** Camila define a coleção 'Lidos' como 'pública', tornando-a visível a qualquer usuário. Camila altera a coleção 'Diário' para 'privada'; ela deixa de aparecer para qualquer outro usuário. Camila define 'Favoritos' como 'somente amigos'; apenas amigos confirmados passam a visualizá-la. **Restrições:** 1. A privacidade é configurada no nível da coleção; a categoria não possui privacidade própria (RF-06). 2. Coleções 'privadas' são visíveis unicamente pelo proprietário. 3. Coleções 'somente amigos' liberam visualização e acompanhamento apenas para usuários com vínculo de amizade aceito (RF-20). 4. Coleções 'públicas' ficam acessíveis a qualquer usuário cadastrado. 5. Por padrão, toda coleção recém-criada nasce como 'pública' (RF-07). 6. Se uma coleção 'pública' ou 'somente amigos' for alterada para 'privada', o sistema deve encerrar automaticamente o acompanhamento de todos os seguidores e remover seus eventos do feed (RF-21 e RF-22). 7. Se uma coleção 'pública' for alterada para 'somente amigos', o sistema deve manter o acompanhamento dos amigos confirmados e encerrar o dos demais seguidores (RF-20 e RF-21).
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 16 – RF-09 – Gerenciar Item

Código	RF-09
Grupo	Coleção
Descrição	**Ação:** Gerenciar **Objeto:** Item dentro de uma coleção **Atributos:** nome (200 caracteres), subtítulo (150 caracteres, opcional, exibido sob o nome no cartão), coleção pai (referência obrigatória), imagem de capa do cartão (JPEG/PNG, opcional, enquadrada pelo usuário, com o original guardado para reenquadramento), cor de fundo do cartão (hexadecimal, opcional), layout visual efetivo do item, Elementos herdados ou personalizados (Texto, Imagem, Forma, Ícone, Classificação e Data), conteúdo e estilo armazenados nos próprios Elementos, data de criação (gerada automaticamente). **Exemplos:** Camila cria o item 'Duna' na coleção 'Lidos' (dentro da categoria 'Livros'); ao concluir o cadastro, a ficha do item se abre com o layout aplicável e Camila edita visualmente os textos, imagens, datas e a classificação que aparecem nela, como faria em um canvas. Camila adiciona individualmente um Elemento de Imagem e um texto extra 'Personagem Favorito: Paul Atreides', sem afetar os demais itens nem o layout da coleção. Posteriormente edita a classificação de 5 para 4 estrelas e, por fim, exclui o item 'Duna – Rascunho' confirmando no diálogo de confirmação. **Restrições:** 1. O nome do item é obrigatório; todos os demais campos são opcionais. 2. Todo item deve pertencer a uma coleção existente (RF-07); não é possível criar itens diretamente em uma categoria. 3. O item exibe a estrutura visual do layout associado à sua coleção (RF-15) até que sua ficha seja aberta para edição, o que a interface faz logo após o cadastro; nesse momento, o item recebe uma cópia própria desse layout, ou uma ficha vazia quando a coleção não tem layout, e o próprio usuário a monta (RF-17). O sistema não desenha uma ficha padrão: o que ele oferece são os layouts padrão da biblioteca, que o usuário pode associar à coleção quando não quiser compor do zero (RF-15).

Continua na próxima página

Quadro 16 – Continuação da página anterior

	4. O usuário pode adicionar, remover ou reposicionar Elementos individualmente no item sem afetar o layout da coleção nem os demais itens (RF-17). 5. Não podem existir dois itens com o mesmo nome dentro da mesma coleção. 6. Imagens enviadas devem estar em formato JPEG ou PNG com tamanho máximo de 5 MB por arquivo. 7. A exclusão deve exibir diálogo de confirmação informando que a ação é irreversível. 8. A data de criação é gerada automaticamente e não pode ser editada pelo usuário.
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 17 – RF-10 – Mover Item entre Coleções

Código	RF-10
Grupo	Coleção
Descrição	**Ação:** Mover **Objeto:** Item entre coleções, dentro da mesma categoria ou entre categorias distintas **Atributos:** item de origem (nome, categoria e coleção atuais), destino (coleção existente, podendo pertencer a uma categoria diferente). **Exemplos:** Camila abre o menu do item 'A Metamorfose' na coleção 'Para Ler', escolhe 'Mover', seleciona a coleção 'Lidos', da mesma categoria 'Livros', e confirma; o item sai da origem e aparece no destino imediatamente. Camila move 'O Hobbit' da coleção 'Lidos' para a coleção 'Releituras Planejadas', de outra categoria, escolhendo primeiro a categoria e depois a coleção de destino. Logo após mover um item, Camila percebe o engano e clica em 'Desfazer'; o item volta à coleção de origem. **Restrições:** 1. Um item pode pertencer a apenas uma coleção por vez.

Continua na próxima página

Quadro 17 – Continuação da página anterior

	2. A movimentação deve ser concluída com no máximo três comandos (abrir a movimentação, escolher o destino e confirmar). 3. Se a coleção de destino usa o mesmo layout da coleção de origem, todos os dados e a estrutura de elementos são preservados automaticamente. 4. Se o item ainda usa o layout da coleção, ao ser movido ele passa a seguir o layout da coleção de destino; se já possui layout próprio, esse layout é preservado (RF-17). 5. A operação deve ser reversível por um botão de desfazer, oferecido durante 30 segundos após a confirmação. 6. As contagens de itens das coleções de origem e destino devem ser atualizadas imediatamente.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 18 – RF-11 – Exibir Ficha de Item em Modo Leitura

Código	RF-11
Grupo	Coleção
Descrição	**Ação:** Exibir em modo leitura **Objeto:** Ficha individual de item **Atributos:** layout efetivo do item, Elementos com conteúdo e estilo, data de criação do item, modo de acesso (dono, com acesso à edição do layout do item; visitante, somente leitura). **Exemplos:** Camila acessa a ficha do item 'Duna' da coleção 'Lidos' e visualiza todos os Elementos visuais, com a opção de editar o layout do item disponível. Um amigo visita o perfil de Camila, entra na categoria 'Livros', abre a coleção pública 'Lidos', acessa a ficha do item 'Duna' e visualiza os mesmos Elementos, porém sem nenhuma opção de edição. Elementos não preenchidos continuam visíveis para visitantes com indicação visual de que estão vazios, sem permitir edição.

Continua na próxima página

Quadro 18 – Continuação da página anterior

	Restrições: 1. A ficha exibe os Elementos do layout efetivo do item; quando existir layout próprio do item, ele prevalece sobre o layout da coleção (RF-17). 2. Elementos sem conteúdo devem ser exibidos para dono e visitantes com indicação visual de que estão vazios; isso não concede permissão de edição ao visitante. 3. Para o dono do item, a ficha deve oferecer acesso direto à edição da própria ficha, isto é, do layout do item (RF-17). As ações de alterar os dados do item e de excluí-lo não ficam na ficha: elas pertencem ao menu de ações do item na listagem da coleção, de onde valem para qualquer item sem precisar abri-lo. 4. Para visitantes, a ficha é estritamente somente leitura, sem nenhuma opção de edição visível. 5. A ficha deve respeitar a aparência (tema e cores) configurada pelo dono (RF-18). 6. O sistema deve fornecer ao *front-end* as informações necessárias para exibir a ficha final do item em modo leitura, diferenciando visualmente o acesso do dono e o de visitantes.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 19 – RF-12 – Visualizar Itens de uma Coleção

Código	RF-12
Grupo	Coleção
Descrição	**Ação:** Visualizar **Objeto:** Itens de uma coleção **Atributos:** lista de itens da coleção selecionada, critérios de ordenação (nome A–Z, nome Z–A, data de criação crescente, data de criação decrescente), termo de busca (aplicado ao nome e ao subtítulo de cada item).

Continua na próxima página

Quadro 19 – Continuação da página anterior

	Exemplos: Camila abre a coleção 'Lidos' da categoria 'Livros' e visualiza todos os 20 itens ordenados por data de criação decrescente. Camila ordena por nome de A a Z e escreve 'duna' na busca, vendo apenas os itens cujo nome ou subtítulo contém a palavra. Camila abre a coleção 'Para Ler'; o sistema mantém a ordenação escolhida, mas a busca começa vazia. **Restrições:** 1. A visualização deve exibir todos os itens da coleção selecionada. 2. A ordenação padrão é por data de criação decrescente (mais recentes primeiro). 3. A busca compara o termo com o nome e o subtítulo de cada item, sem distinguir maiúsculas, minúsculas e acentos. 4. A ordenação aplicada deve ser guardada no navegador do usuário e valer para todas as coleções até ser alterada. 5. A busca vale apenas enquanto o usuário permanece na coleção e é reiniciada ao sair dela. 6. A ordenação pela classificação e os filtros pelo conteúdo dos Elementos não fazem parte desta versão (Subseção 5.2.4). 7. A tag não é critério de filtro nesta tela. A tag pertence à coleção (RF-07), e não ao item: como todos os itens de uma mesma coleção compartilhariam as mesmas tags, o filtro não separaria nada. A busca por tag reúne coleções e está descrita no RF-07.
Prioridade	Essencial
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 20 – RF-13 – Exibir Tela Principal de Categorias

Código	RF-13
Grupo	Coleção
Descrição	**Ação:** Exibir **Objeto:** Tela principal (dashboard) de categorias do usuário **Atributos:** lista de categorias do usuário autenticado (nome, ícone, imagem de capa, cor de fundo do cartão, contagem total de coleções, contagem total de itens somados entre todas as coleções), modo de exibição (grade ou lista), critérios de ordenação (nome A–Z, nome Z–A, data de criação crescente, data de criação decrescente), critérios de filtro (tipo/tema da categoria), atalhos de navegação para coleções e itens. **Exemplos:** Ao fazer login, Camila vê na tela inicial os cartões 'Livros (3 coleções, 52 itens)', 'Filmes (2 coleções, 30 itens)' e 'Cafés (1 coleção, 7 itens)' em modo grade. Camila clica em 'Livros' e visualiza as coleções 'Lidos (20, pública)', 'Para Ler (30, somente amigos)' e 'Diário (2, privada)', podendo navegar para os itens de cada uma. Camila alterna para o modo lista e ordena as categorias por data de criação decrescente; a preferência é mantida na próxima sessão. Camila, em seu primeiro acesso, vê mensagem de boas-vindas e botão 'Criar primeira categoria'. **Restrições:** 1. A tela deve exibir todas as categorias do usuário autenticado em uma única tela dedicada. 2. Cada categoria deve exibir nome, ícone e as contagens de coleções e de itens. Em modo grade, a imagem de capa (RF-06) ocupa o topo do cartão, com o nome sobre ela. 3. A categoria não possui nível de privacidade próprio; o indicador individual de privacidade aparece no *card* de cada coleção dentro da categoria, e não no *card* da categoria (RF-08). 4. A navegação até as coleções de uma categoria deve ser acessível com um único clique ou toque; a navegação até um item, com no máximo três. 5. Usuários sem categorias devem ver mensagem de boas-vindas com atalho para criação da primeira categoria.

Continua na próxima página

Quadro 20 – Continuação da página anterior

	6. Ao acessar uma categoria vazia (sem coleções), o sistema deve exibir mensagem informativa e atalho para criação da primeira coleção. 7. O usuário deve poder alternar entre os modos de visualização 'grade' e 'lista' a qualquer momento. 8. O modo de visualização padrão é 'grade'; a escolha do usuário deve ser persistida entre sessões. 9. O usuário deve poder ordenar as categorias por pelo menos quatro critérios: nome crescente, nome decrescente, data de criação crescente e data de criação decrescente. 10. A ordenação padrão é por data de criação decrescente (mais recentes primeiro). 11. Os filtros e a ordenação aplicados devem ser persistidos entre sessões do mesmo usuário. 12. O filtro por tema opera sobre o texto livre definido pelo usuário (RF-06), comparando os valores já normalizados, de modo que grafias diferentes da mesma palavra sejam tratadas como o mesmo tema.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 21 – RF-14 – Gerenciar Layout Personalizado

Código	RF-14
Grupo	Editor de Layout
Descrição	**Ação:** Gerenciar e duplicar **Objeto:** Layout personalizado (template) **Atributos:** nome do layout (100 caracteres), descrição (300 caracteres, opcional), fundo da ficha (uma cor hexadecimal, ou um degradê de duas cores com um ângulo e um ponto de início), Elementos disponíveis (conforme os tipos fixos definidos pela enumeração TipoElemento: texto, imagem, forma, ícone, classificação e data), conteúdo textual opcional do Elemento, EstiloDeElemento de cada Elemento (fonte, tamanho, peso, itálico, sublinhado, riscado, alinhamento horizontal e vertical, nível de título, entrelinha, espaçamento entre letras, cor, cor de fundo, arredondamento, opacidade, rotação, borda (cor, espessura e traço), forma do elemento, curva e ponta da linha, ajuste de imagem, apresentação em lista, nota máxima e ícone da classificação, formato e estética da data), camada de sobreposição, posição e tamanho de cada Elemento na área de edição, layout de origem (referência opcional, preenchida apenas na operação de duplicação). **Exemplos:** Camila cria o layout 'Card de Livro' com Elemento de Imagem para capa, textos livres como 'Autor: Frank Herbert', Classificação de 1 a 5 estrelas, dois Elementos de Data (início e fim de leitura) e área de Texto para notas. Camila reposiciona o Elemento de Classificação via *drag-and-drop* e visualiza o resultado em tempo real na prévia. Camila seleciona o layout 'Card de Livro' e clica em 'Duplicar'; o sistema cria 'Card de Livro (cópia)' com a mesma estrutura, que Camila renomeia para 'Card de Livro – Edição Especial' e ajusta sem afetar o original. Camila exclui o layout 'Card Antigo' que não está associado a nenhuma coleção. Para excluir o layout 'Card de Série', ainda usado pela coleção 'Assistidas', Camila primeiro o desassocia das coleções; os itens que o exibiam sem ficha própria recebem uma cópia dele, e a aparência desses itens não muda. **Restrições:** 1. O nome do layout é obrigatório e deve ser único por usuário.

Continua na próxima página

Quadro 21 – Continuação da página anterior

	2. A biblioteca de layouts do usuário reúne três origens: os layouts que ele mesmo cria, os layouts padrão oferecidos pelo sistema desde o cadastro (RF-01) e os layouts importados de outros usuários (RF-16). Todos podem ser igualmente editados, duplicados e associados a coleções. 3. Um layout pode ser gravado sem Elementos, pois a gravação é automática e acompanha cada alteração; o layout vazio pode ser associado a uma coleção e preenchido depois. 4. Os tipos de Elemento disponíveis para inclusão são os definidos pela enumeração TipoElemento (texto, imagem, forma, ícone, classificação e data), à semelhança dos elementos fixos oferecidos por editores visuais como o Canva. 5. Elementos podem armazenar conteúdo visual próprio, como 'Autor:' ou 'Autor: Frank Herbert', além de propriedades de fonte, cor e alinhamento. 6. Elementos podem ser reposicionados e redimensionados via arrastar e soltar (*drag-and-drop*). 7. O fundo da ficha é escolhido pelo usuário, como cor única ou como degradê entre duas cores, e vale para o layout inteiro, e não para um Elemento em particular. 8. A tinta padrão dos Elementos acompanha o fundo escolhido, por razão de contraste: sobre um fundo escuro, o Elemento que não define cor própria é desenhado em tom claro, e vice-versa. A escolha explícita de cor em um Elemento prevalece sobre esse padrão. 9. Alterações devem ser refletidas em tempo real na prévia do editor, sem recarregar a página. 10. A duplicação deve copiar integralmente a estrutura de Elementos (tipo, conteúdo, estilo, posição, tamanho e propriedades) do layout de origem. 11. Na duplicação, o sistema sugere automaticamente o nome original acrescido de '(cópia)', podendo ser editado antes de salvar. 12. Alterações no layout duplicado não devem refletir no layout de origem.

Continua na próxima página

Quadro 21 – Continuação da página anterior

	13. O usuário pode excluir um layout somente se ele não estiver associado a nenhuma coleção nem tiver originado fichas próprias de itens (RF-15 e RF-17). Para os layouts em uso, o sistema oferece a desassociação de todas as coleções que o utilizam: cada item que o exibia sem ficha própria recebe antes uma cópia dele, de modo que nenhuma ficha muda de aparência, e as fichas próprias que dele derivaram deixam de referenciá-lo, sem perder nenhum Elemento. Feita a desassociação, o layout pode ser excluído. 14. A data de criação do layout duplicado é a data da operação, e não a do layout de origem.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 22 – RF-15 – Associar Layout a Coleção

Código	RF-15
Grupo	Editor de Layout
Descrição	**Ação:** Associar **Objeto:** Layout (template) a uma coleção **Atributos:** coleção de destino (existente, pertencente ao usuário), layout selecionado na biblioteca do usuário (criado por ele, padrão do sistema ou importado de outro usuário, podendo já estar associado a outras coleções). **Exemplos:** Camila associa o layout 'Card de Livro' à coleção 'Lidos'; os itens dessa coleção que ainda não têm ficha própria passam a exibir esse layout, e os novos itens partem dele. Camila associa o mesmo layout 'Card de Livro' também à coleção 'Para Ler'; as duas coleções reutilizam o mesmo template, de forma independente. Camila associa à coleção 'Mangás' um layout padrão oferecido pelo sistema, por não desejar compor a ficha do zero. Camila troca o layout da coleção 'Mangás' para 'Card de Mangá'; antes de confirmar, vê a prévia das fichas próprias levadas ao novo layout e escolhe quais levar; as que não escolheu mantêm sua ficha como estava (RF-17).

Continua na próxima página

Quadro 22 – Continuação da página anterior

	Restrições: 1. Um layout só pode ser associado a coleções existentes e pertencentes ao mesmo usuário. 2. O layout é associado no nível da coleção; a categoria não recebe layout (RF-06). 3. A associação é opcional. O usuário pode escolher qualquer layout de sua biblioteca, seja criado por ele, padrão do sistema ou importado de outro usuário (RF-14 e RF-16); nenhum layout é aplicado automaticamente. 4. Uma coleção pode ter, no máximo, um layout ativo por vez. 5. Um mesmo layout pode ser associado a várias coleções simultaneamente, sendo reutilizado como template. 6. O layout aplicável a um item é o da sua coleção; na ausência de layout associado, a ficha do item nasce vazia e ele pode receber um layout próprio, montado ali mesmo. Modificações individuais por item são tratadas como *override* (RF-17). 7. A troca ou remoção do layout da coleção não altera, por si só, os dados nem os Elementos individuais (*override*) dos itens já cadastrados (RF-17). Os itens sem ficha própria passam a exibir o layout novo. 8. Na troca, o sistema oferece a mescla das fichas próprias com o layout novo, apenas para os itens que o usuário escolher. Cada Elemento da ficha é associado a um Elemento do layout novo pela mesma origem, depois pelo mesmo tipo e nome e, por fim, pelo mesmo tipo na ordem de leitura; do Elemento antigo vem apenas o valor, e a posição, o tamanho e o estilo vêm do layout novo. Os Elementos com valor que não encontram correspondência permanecem na ficha; formas e ícones, que são desenho do molde e não valor do item, não são transferidos. A mescla também pode ser pedida a qualquer momento para um único item, a partir do editor da sua ficha. 9. Antes de confirmar a troca, o sistema exibe a prévia de cada ficha própria levada ao layout novo. 10. O usuário pode remover a associação de layout da coleção a qualquer momento.
Prioridade	Importante

Continua na próxima página

Quadro 22 – Continuação da página anterior

Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 23 – RF-16 – Compartilhar Layout

Código	RF-16
Grupo	Editor de Layout
Descrição	**Ação:** Compartilhar e importar **Objeto:** Layout (template) compartilhado entre usuários **Atributos:** layout de origem (pertencente ao usuário que compartilha), identificador do layout compartilhado (copiado pelo dono e colado pelo importador), usuário importador, layout importado (cópia gerada na biblioteca do importador), finalidade interna do layout (*template* importado), que registra que a cópia nasceu de uma importação. **Exemplos:** Camila cria o layout 'Card de Livro', copia o identificador desse layout na biblioteca e o envia a uma amiga por mensagem. A amiga cola o identificador na opção de importar da biblioteca de layouts, vê a prévia e confirma; o sistema cria uma cópia na conta dela, que pode ser usada e editada livremente como se tivesse sido criada por ela. Camila altera o layout original depois do compartilhamento; as cópias já importadas por outros usuários permanecem intactas. **Restrições:** 1. O compartilhamento usa o próprio identificador do layout, que o dono copia na biblioteca e envia pelo meio que preferir; não há código separado, ativação nem cancelamento de compartilhamento neste fluxo. 2. A importação cria uma cópia independente do layout na biblioteca do usuário importador; alterações na cópia não afetam o layout de origem, e vice-versa. 3. Layouts importados devem aparecer na biblioteca do usuário como *templates* utilizáveis, mas ficam marcados internamente com a finalidade de *template* importado.

Continua na próxima página

Quadro 23 – Continuação da página anterior

	4. O compartilhamento abrange apenas a estrutura visual do layout, não os itens do dono nem as personalizações individuais feitas em itens. 5. A importação está disponível a partir do identificador, independentemente de haver vínculo de amizade entre os usuários, e é precedida de uma prévia do layout. 6. O usuário não deve importar um layout da própria biblioteca por este fluxo; para isso deve usar a duplicação (RF-14).
Prioridade	Desejável
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 24 – RF-17 – Modificar Layout de Item Individualmente

Código	RF-17
Grupo	Editor de Layout
Descrição	**Ação:** Modificar individualmente **Objeto:** Layout herdado em um item específico **Atributos:** Elementos herdados do layout aplicável ao item (base de partida), conteúdo e estilo dos Elementos, Elementos adicionados pelo usuário no item (*override*), Elementos removidos pelo usuário no item (*override*), Elementos reposicionados ou redimensionados no item (*override*). **Exemplos:** Camila cria o item 'Harry Potter e a Pedra Filosofal' na coleção 'Lidos' (categoria 'Livros'); o item abre com o layout aplicável herdado; Camila adiciona Elemento extra de texto 'Data de Início de Leitura' apenas nesse item. Camila remove o Elemento de Forma de um item específico pois não se aplica, sem afetar o layout da coleção nem os outros itens. Depois da personalização, o item mantém seu próprio layout, sem alterar o layout usado pela coleção. **Restrições:** 1. Todo item parte do layout da sua coleção (RF-09 e RF-15). Caso a coleção não possua layout, o item começa sem layout, e o usuário o monta a partir da própria ficha.

Continua na próxima página

Quadro 24 – Continuação da página anterior

	2. O usuário pode adicionar, remover ou reposicionar Elementos no item sem afetar o layout da coleção ou os demais itens. 3. O layout aplicável deve ser exibido como referência visual durante a edição individual do item. 4. As modificações individuais são exclusivas do item; nenhuma alteração feita no item reflete no layout da coleção. 5. Ao abrir a ficha do item para edição pela primeira vez, o que a interface faz logo após o cadastro, o sistema deve criar uma cópia própria do layout da coleção para aquele item, preservando Elementos, conteúdo e estilos como ponto de partida. Não havendo layout na coleção, a cópia é criada vazia: a existência de um layout na coleção não é pré-requisito para personalizar a ficha de um item. 6. Após a criação da cópia individual, alterações de conteúdo, estilo, posição, tamanho, adição ou remoção de Elementos devem afetar apenas aquele item, sem modificar o layout da coleção nem os demais itens. 7. Depois de personalizado, o item mantém seu layout próprio, mesmo que o layout da coleção seja alterado posteriormente; a ficha só é levada ao layout novo quando o usuário pede a mescla (RF-15).
Prioridade	Desejável
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 25 – RF-18 – Gerenciar Aparência (Tema e Cores)

Código	RF-18
Grupo	Aparência
Descrição	**Ação:** Gerenciar aparência **Objeto:** Paletas de aparência do usuário e configuração visual global da interface **Atributos:** cor primária (hex), subcores (12 valores hex: fundo da página, cartões, títulos, barra lateral, cabeçalho, modais e um texto para cada superfície, isto é, texto no fundo, nos cartões, na barra lateral, no cabeçalho, nos modais e dentro dos botões de destaque), paletas predefinidas (lista do sistema), paletas personalizadas (nome de até 50 caracteres), paleta em uso, estilo global (claro / escuro / automático conforme sistema operacional). **Exemplos:** No momento do cadastro (RF-01), o sistema disponibiliza ao usuário seis paletas predefinidas (‘Claro’, ‘Escuro’, ‘Outono’, ‘Oceano’, ‘Floresta’ e ‘Lavanda’); como o sistema operacional de Camila está no tema escuro, a paleta ‘Escuro’ já nasce em uso. Camila seleciona a paleta predefinida ‘Oceano’, de tons azulados e estilo escuro; toda a interface é atualizada imediatamente. Camila cria a paleta personalizada ‘Meu Rosa’ a partir da paleta em uso, altera a cor primária para #E91E8C, confere o resultado na área de prévia e salva. Com a paleta clara ‘Meu Rosa’ em uso, Camila muda o estilo global para escuro; as cores da paleta passam a ser exibidas com a claridade invertida, preservando os tons. **Restrições:** 1. Todo usuário recém-cadastrado recebe as paletas predefinidas do sistema (RF-01). Nasce em uso a paleta ‘Escuro’ se o sistema operacional estiver no tema escuro no momento do cadastro, e a paleta ‘Claro’ nos demais casos; mudanças posteriores no sistema operacional não trocam a paleta em uso. 2. O usuário pode criar, editar, duplicar, renomear e excluir paletas, e selecionar a paleta em uso a qualquer momento. As paletas predefinidas pertencem ao usuário e podem ser editadas e excluídas como as personalizadas. O nome de uma paleta não pode se repetir entre as paletas do mesmo usuário, mas pode coincidir com o de paletas de outros usuários. 3. A paleta em uso deve ser aplicada globalmente a todas as telas do sistema de forma imediata.

Continua na próxima página

Quadro 25 – Continuação da página anterior

	4. A paleta em uso não pode ser excluída, o que garante que o usuário tenha sempre ao menos uma paleta. 5. Uma paleta é sempre completa: a cor primária e todas as subcores devem estar preenchidas. Cada superfície (fundo da página, cartões, barra lateral, cabeçalho e modais) tem sua própria cor de texto, escolhida pelo usuário; no estilo em que a paleta foi desenhada, o sistema exibe exatamente a cor escolhida, sem substituí-la, e a única transformação aplicada é a inversão de claridade descrita na restrição 6. 6. O sistema classifica cada paleta como clara ou escura a partir da cor de fundo. Quando o estilo global em vigor é o oposto ao da paleta em uso, as cores são exibidas com a claridade invertida, preservando os tons. 7. As paletas e a paleta em uso devem ser salvas no perfil e persistidas entre sessões e dispositivos. 8. Enquanto o usuário edita uma paleta, a interface exibe a própria paleta em edição, para que ele veja o resultado exato antes de confirmar. Nada é gravado até que ele salve, e cancelar ou sair devolve a interface à paleta em uso. A paleta é exibida no estilo em que foi desenhada; a inversão para o estilo oposto acontece apenas quando ela está em uso (restrição 6). 9. O sistema deve alertar o usuário quando uma combinação de cores não atender ao contraste mínimo (RNF-09), sem impedir a escolha. 10. A biblioteca de paletas pode ser exibida em grade ou em lista e filtrada por paletas claras ou escuras.
Prioridade	Desejável
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 26 – RF-19 – Gerenciar Tipografia Global

Código	RF-19
Grupo	Aparência
Descrição	**Ação:** Alterar **Objeto:** Tipografia e aparência textual global **Atributos:** família de fonte (lista predefinida com no mínimo 5 opções), tamanho base em px (mínimo 12 px, máximo 24 px), tamanho de título em px (mínimo 16 px, máximo 48 px), espaço entre linhas (1.0 a 2.0), espaço entre letras, peso da fonte (300 a 800), itálico. **Exemplos:** Camila edita a paleta 'Meu Rosa' e altera a fonte para 'Georgia', o tamanho base para 16 px e o espaço de linha para 1.6; a prévia reflete as mudanças antes da confirmação. **Restrições:** 1. As fontes disponíveis devem incluir ao menos cinco opções tipográficas distintas (ex.: serif, sans-serif, monospace). 2. As alterações devem ser exibidas em área de prévia antes de o usuário confirmar a aplicação. 3. A tipografia faz parte de cada paleta (RF-18): é aplicada globalmente junto com a paleta em uso e persistida entre sessões e dispositivos.
Prioridade	Desejável
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 27 – RF-20 – Gerenciar Amizade

Código	RF-20
Grupo	Interação Social
Descrição	**Ação:** Adicionar, aceitar e visualizar **Objeto:** Amizade entre usuários **Atributos:** usuário solicitante (nome, código de usuário e email), usuário receptor (nome, código de usuário e email), critério de busca (código de usuário ou nome), status da solicitação (pendente / aceita / recusada / cancelada), data da solicitação e data da resposta.

Continua na próxima página

Quadro 27 – Continuação da página anterior

	Exemplos: Camila informa o código de usuário 'joao#1203' no campo de busca e envia a solicitação de amizade; João aceita e ambos aparecem mutuamente na lista de amigos. Camila busca por 'João Silva' pelo nome e o sistema exibe a lista de usuários correspondentes; Camila seleciona o usuário correto e envia a solicitação. Camila recusa a solicitação de um usuário desconhecido; o status é atualizado para 'recusada' sem gerar notificação ao solicitante. **Restrições:** 1. A busca por usuários pode ser realizada tanto pelo código de usuário (busca exata) quanto pelo nome (busca parcial). 2. A busca por código de usuário retorna no máximo um resultado, pois o código é único. 3. A busca por nome pode retornar múltiplos resultados; o sistema exibe nome, foto e código de cada usuário para desambiguação. 4. Não é possível enviar solicitação de amizade para si mesmo. 5. Uma solicitação pendente não pode ser reenviada enquanto aguarda resposta. 6. O remetente pode cancelar uma solicitação ainda pendente, que passa ao status 'cancelada' e deixa de aparecer para o destinatário. 7. A lista de amigos exibe apenas amizades com status 'aceita'. 8. O usuário pode desfazer uma amizade aceita a qualquer momento, removendo o vínculo para ambos os lados. 9. Envio, aceite e desfazer amizade devem gerar notificação ao usuário envolvido, conforme RF-24; a recusa é silenciosa.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 28 – RF-21 – Seguir Coleção Visível

Código	RF-21
Grupo	Interação Social
Descrição	**Ação:** Seguir **Objeto:** Coleção visível de outro usuário **Atributos:** coleção seguida (com privacidade pública ou 'somente amigos'), categoria pai da coleção (referência), usuário seguidor, data de início do acompanhamento. **Exemplos:** Camila visita o perfil de 'João Silva', com quem não possui amizade, vê a coleção pública 'Filmes Assistidos' (dentro da categoria 'Filmes') e clica em 'Seguir'; as atualizações dessa coleção passam a aparecer no feed de Camila, mas as demais coleções da categoria 'Filmes' (como 'Quero Ver') não, a menos que sejam seguidas individualmente. Após João aceitar a amizade de Camila, ela também passa a visualizar e pode seguir a coleção 'Favoritos', configurada como 'somente amigos'. Camila tenta acessar uma coleção privada de João e não encontra a opção de acompanhamento, pois a coleção sequer aparece no perfil visitado. **Restrições:** 1. Qualquer usuário autenticado pode seguir uma coleção 'pública', independentemente de amizade com o proprietário. 2. Coleções 'somente amigos' podem ser seguidas apenas enquanto existir vínculo de amizade confirmado (RF-20). 3. Coleções 'privadas' não aparecem e não podem ser seguidas por outros usuários, nem mesmo por amigos. 4. O proprietário não pode seguir a própria coleção. 5. O acompanhamento ocorre no nível da coleção: o usuário pode seguir múltiplas coleções da mesma categoria, de forma independente, sem que isso implique seguir as demais. 6. O usuário pode deixar de seguir uma coleção a qualquer momento, sem restrições. 7. Se a coleção seguida for alterada para 'privada', todos os acompanhamentos devem ser encerrados automaticamente (RF-08). 8. Se uma coleção 'pública' for alterada para 'somente amigos', acompanhamentos de usuários sem amizade confirmada devem ser encerrados; os de amigos permanecem ativos.

Continua na próxima página

Quadro 28 – Continuação da página anterior

	9. Ao desfazer uma amizade, acompanhamentos de coleções 'públicas' permanecem ativos e acompanhamentos de coleções 'somente amigos' entre os dois usuários são encerrados.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 29 – RF-22 – Exibir Feed de Atualizações

Código	RF-22
Grupo	Interação Social
Descrição	**Ação:** Exibir **Objeto:** Feed de atualizações das coleções seguidas **Atributos:** tipo de evento (adição, edição ou remoção de item em coleção seguida), item afetado ou registro mínimo do item quando ele já tiver sido removido, coleção seguida, categoria pai, usuário autor da ação, data e hora do evento e descrição textual auxiliar. **Exemplos:** Feed de Camila exibe 'João adicionou Oppenheimer na coleção Filmes Assistidos (categoria Filmes) – há 2 horas'. Feed exibe 'Ana avaliou o item Breaking Bad na coleção Séries Concluídas – há 5 minutos'. Camila rola o feed e carrega eventos mais antigos progressivamente. **Restrições:** 1. O feed deve exibir apenas eventos de coleções que o usuário segue ativamente (RF-21). 2. Os eventos devem ser ordenados cronologicamente, do mais recente ao mais antigo. 3. O feed deve ser atualizado automaticamente, sem necessidade de recarregar a página, por consulta periódica à API a cada minuto e sempre que a janela volta a ter foco; a entrega imediata pelo servidor consta dos requisitos adiados (Subseção 5.2.4). 4. Se a privacidade de uma coleção seguida for alterada para 'privada', os eventos anteriores dessa coleção não devem mais ser exibidos, e o acompanhamento é encerrado automaticamente (RF-21).

Continua na próxima página

Quadro 29 – Continuação da página anterior

	5. Eventos de criação ou exclusão de coleção em si não são exibidos no feed; a descoberta de novas coleções ocorre por meio da visita a perfis de usuários (RF-23). 6. O feed deve suportar carregamento progressivo (paginação) para históricos longos. 7. O registro de evento deve usar campos estruturados (coleção, item quando aplicável e usuário autor) como fonte principal dos dados; a descrição textual é apenas informação auxiliar. 8. Eventos de remoção de item devem preservar um registro mínimo do item afetado, de modo que o histórico continue compreensível após a exclusão do item.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 30 – RF-23 – Visualizar Perfis e Coleções de Usuários

Código	RF-23
Grupo	Interação Social
Descrição	**Ação:** Visualizar **Objeto:** Perfis, categorias, coleções acessíveis e itens de outros usuários **Atributos:** perfil do usuário visitado (nome, foto, bio), coleções acessíveis (com privacidade 'pública' ou 'somente amigos'), categorias agrupadoras dessas coleções, itens pertencentes a cada coleção visível. **Exemplos:** Camila visita o perfil de 'João Silva' e visualiza, agrupadas pelas respectivas categorias, as coleções públicas e somente para amigos de João – por exemplo, 'Filmes / Assistidos' (pública), 'Filmes / Favoritos' (somente amigos) e 'Séries / Concluídas' (pública); dentro de cada coleção, vê os itens cadastrados. Camila não encontra a coleção 'Diário Pessoal' de João (categoria 'Diários'), pois ela é privada; como é a única coleção da categoria, a categoria 'Diários' inteira não aparece para Camila. A categoria 'Jogos' de João aparece para Camila apenas com as coleções não-privadas; coleções privadas dessa mesma categoria continuam ocultas.

Continua na próxima página

Quadro 30 – Continuação da página anterior

	Restrições: 1. Apenas coleções com privacidade 'pública' ou 'somente amigos' são exibidas no perfil visitado, respeitando o vínculo de amizade entre os usuários (coleções 'somente amigos' só aparecem para amigos confirmados). 2. Coleções 'privadas' e seus itens não devem ser listados nem acessíveis, mesmo que o amigo compartilhe um link direto. 3. Categorias servem apenas como agrupadores visuais no perfil visitado e não possuem privacidade própria. 4. Categorias que, para o visitante, não tenham nenhuma coleção visível devem ser ocultadas integralmente do perfil, de modo que o visitante não tenha conhecimento de sua existência. 5. Todos os itens dentro de uma coleção visível são acessíveis ao visitante; não há privacidade individual por item. 6. A navegação nas categorias, coleções e itens de outros usuários é somente leitura; não é possível editar ou excluir dados.
Prioridade	Importante
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 31 – RF-24 – Gerenciar Notificações do Sistema

Código	RF-24
Grupo	Notificação
Descrição	**Ação:** Gerar, exibir e gerenciar **Objeto:** Notificações do sistema **Atributos:** tipo de notificação (solicitação de amizade recebida, solicitação de amizade aceita, amizade desfeita, atualização em coleção seguida), usuário destinatário, usuário autor (quando aplicável), mensagem (texto descritivo gerado pelo sistema), evento estruturado vinculado quando for atualização de coleção seguida (com coleção, categoria, item e registro mínimo), data e hora de geração, status (não lida / lida), data e hora de leitura quando aplicável, canal de entrega (in-app). As mensagens de confirmação do cadastro (RF-01) e de recuperação de senha (RF-04) seguem por e-mail.

Continua na próxima página

Quadro 31 – Continuação da página anterior

	Exemplos: João envia solicitação de amizade para Camila; o sistema gera notificação in-app para Camila com o texto 'João Silva (joao#1203) enviou uma solicitação de amizade' e exibe indicador visual de notificação não lida. Camila aceita a solicitação; o sistema gera notificação in-app para João com o texto 'Camila Mattos aceitou sua solicitação de amizade'. Camila solicita recuperação de senha; o sistema envia notificação via e-mail com link de redefinição (conforme RF-04). **Restrições:** 1. Envio, aceite e desfazimento de amizade devem gerar notificação in-app ao usuário envolvido; recusas são silenciosas (RF-20). 2. As notificações in-app devem ser exibidas em uma área dedicada (ex.: ícone de sino no menu principal) com indicador visual de quantidade de notificações não lidas. 3. As notificações devem ser ordenadas cronologicamente, da mais recente à mais antiga. 4. O usuário pode marcar uma notificação como lida, marcar todas como lidas ou limpar sua área de notificações sem excluir os registros do sistema: um campo indica o estado atual de leitura, outro registra quando a leitura ocorreu, e a limpeza apenas oculta as notificações da listagem do usuário. 5. A confirmação do cadastro (RF-01) e a recuperação de senha (RF-04) são comunicadas por e-mail, com um código ou um link de uso único e prazo de expiração, e não geram notificação in-app. 6. Notificações referentes a coleções cuja privacidade tenha sido alterada para 'privada' devem ser ocultadas automaticamente do destinatário (RF-08). 7. As notificações in-app devem ser atualizadas automaticamente, sem necessidade de recarregar a página, pela mesma consulta periódica do feed (RF-22). 8. Notificações lidas devem ser mantidas por no mínimo 30 dias antes de qualquer limpeza automática.
Prioridade	Importante
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

5.2.2 Requisitos Não-Funcionais

Segundo Sommerville (2011), os requisitos não funcionais são requisitos que não estão diretamente relacionados com os serviços específicos oferecidos pelo sistema a seus usuários. Logo, não dizem respeito diretamente às funcionalidades fornecidas pelo sistema, mas sim às suas propriedades e restrições de qualidade. Pressman e Maxim (2016), dizem que esses requisitos especificam atributos como desempenho, usabilidade, segurança, confiabilidade e portabilidade, sendo fundamentais para garantir que o sistema atenda aos padrões de qualidade esperados. Assim, esta seção apresenta os requisitos não-funcionais identificados neste trabalho, consolidados nos Quadros de 32 a 41.

Quadro 32 – RNF-01 – Usabilidade

Código	RNF-01
Categoria	Usabilidade
Descrição	**Descrição:** A interface deve ser intuitiva, facilitando o uso do editor de layout e das funcionalidades de personalização visual. **Métrica:** Usuários iniciantes devem ser capazes de criar uma coleção e adicionar um item em até 5 minutos sem consultar documentação externa. **Exemplos:** Um novo usuário acessa o sistema pela primeira vez e consegue criar a coleção 'Meus Filmes', associar um layout e cadastrar o item 'Interestelar' seguindo apenas as indicações visuais da interface. Um usuário utiliza o editor de layout pela primeira vez e compreende como arrastar e posicionar elementos sem necessitar de tutorial. **Restrições:** 1. Todas as ações principais devem ser acessíveis em no máximo três cliques a partir da tela inicial. 2. Elementos interativos devem possuir rótulos descritivos e tooltips explicativos. 3. Mensagens de erro devem ser claras, indicando o problema e a ação corretiva. 4. O sistema deve utilizar ícones e terminologia consistentes em todas as telas. 5. O sistema deve oferecer um guia de primeiros passos: um tour no primeiro acesso, que pode ser pulado e revisto, uma lista de passos iniciais com o progresso calculado a partir da conta e dicas na primeira visita a cada tela. A página inicial pública deve explicar, antes do cadastro, a relação entre o *layout*, a coleção e as fichas.
Prioridade	Essencial
Operação	Saída

Fonte: Elaborada pelo autor

Quadro 33 – RNF-02 – Desempenho de Carregamento

Código	RNF-02
Categoria	Desempenho
Descrição	**Descrição:** O sistema deve carregar categorias, coleções e itens em tempo adequado sob condições normais de uso. **Métrica:** O tempo de carregamento de qualquer tela principal (categorias, coleções, lista de itens) não deve exceder 3 segundos em conexão de banda larga padrão (10 Mbps). **Exemplos:** Camila acessa sua coleção 'Para Ler', com 30 itens, e a listagem completa é exibida em menos de 2 segundos. Camila abre o feed de atualizações com 50 eventos recentes e o carregamento inicial ocorre em menos de 3 segundos. **Restrições:** 1. Imagens de capa e perfil devem ser otimizadas automaticamente pelo sistema para reduzir o tempo de carregamento. 2. Listas com mais de 20 itens devem utilizar carregamento progressivo (paginação ou scroll infinito). 3. Requisições à API devem retornar resposta em no máximo 2 segundos para operações de leitura. 4. Operações de escrita (cadastro, edição, exclusão) devem ser concluídas em no máximo 3 segundos.
Prioridade	Essencial
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 34 – RNF-03 – Segurança de Senhas e Dados Pessoais

Código	RNF-03
Categoria	Segurança
Descrição	**Descrição:** O sistema deve criptografar senhas e proteger informações pessoais dos usuários de acordo com boas práticas de segurança. **Métrica:** Todas as senhas devem ser armazenadas utilizando algoritmo de hash seguro (bcrypt ou argon2) com salt único por usuário. **Exemplos:** A senha 'Camila@2026' é armazenada como hash irreversível no banco de dados; mesmo com acesso direto à base, não é possível recuperar a senha original. Uma tentativa de login com credenciais inválidas retorna mensagem genérica sem revelar se o erro é no email ou na senha. **Restrições:** 1. Senhas nunca devem ser armazenadas em texto puro ou com criptografia reversível. 2. Toda comunicação entre cliente e servidor deve utilizar protocolo HTTPS/TLS. 3. Tokens de sessão devem ser gerados com entropia suficiente, ter prazo de validade definido e poder ser revogados a qualquer momento pelo servidor, independentemente da expiração. 4. Dados sensíveis (email, senha) não devem ser expostos em logs do sistema. 5. O sistema deve limitar tentativas de login a 5 por minuto por endereço IP para mitigar ataques de força bruta.
Prioridade	Essencial
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 35 – RNF-04 – Compatibilidade e Responsividade

Código	RNF-04
Categoria	Compatibilidade
Descrição	**Descrição:** O sistema deve oferecer compatibilidade com navegadores modernos e dispositivos móveis, mantendo responsividade. **Métrica:** O sistema deve funcionar corretamente nas duas versões mais recentes dos navegadores Google Chrome, Mozilla Firefox, Safari e Microsoft Edge, além de dispositivos com tela a partir de 320 px de largura. **Exemplos:** Camila acessa o sistema pelo navegador Chrome no desktop e todas as funcionalidades operam normalmente. Camila acessa o mesmo sistema pelo Safari no iPhone e consegue navegar, criar coleções e adicionar itens com a interface adaptada ao tamanho da tela. **Restrições:** 1. O layout deve ser responsivo, adaptando-se automaticamente a resoluções de 320 px até 2560 px de largura. 2. Funcionalidades de arrastar e soltar (drag-and-drop) devem possuir alternativa por toque em dispositivos móveis. 3. O editor de layout deve ser funcional tanto em dispositivos desktop quanto em tablets. 4. Nenhuma funcionalidade essencial deve depender de plugins ou extensões de navegador.
Prioridade	Essencial
Operação	Saída

Fonte: Elaborada pelo autor

Quadro 36 – RNF-05 – Personalização Visual sem Quebras de Layout

Código	RNF-05
Categoria	Personalização
Descrição	**Descrição:** O sistema deve permitir que o usuário personalize temas, cores, subcores e fontes sem comprometer o funcionamento das telas. **Métrica:** Qualquer combinação de tema e tipografia escolhida pelo usuário deve manter todas as funcionalidades do sistema operacionais, sem quebras de layout ou sobreposição de elementos. **Exemplos:** Camila aplica a paleta personalizada 'Meu Rosa' com fonte 'Georgia' 18 px e todas as telas do sistema permanecem funcionais e legíveis. Camila alterna entre os estilos claro e escuro e nenhum texto fica ilegível ou botão fica inacessível. **Restrições:** 1. As personalizações visuais devem ser aplicadas exclusivamente via variáveis CSS, sem alterar a estrutura HTML ou a lógica do sistema. 2. O sistema deve validar combinações de cores antes de aplicá-las, alertando sobre problemas de contraste. 3. Personalizações devem ser aplicadas em tempo real, sem necessidade de recarregar a página. 4. As configurações devem ser persistidas no perfil do usuário e sincronizadas entre dispositivos.
Prioridade	Importante
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 37 – RNF-06 – Desempenho na Movimentação de Itens

Código	RNF-06
Categoria	Desempenho
Descrição	**Descrição:** A movimentação de itens entre coleções deve ocorrer com no máximo três comandos e sem atraso perceptível. **Métrica:** A operação de mover um item deve ser concluída em no máximo 500 milissegundos desde a ação do usuário até a atualização visual na interface. **Exemplos:** Camila move o item 'Duna' da coleção 'Para Ler' para 'Lidos' e, ao confirmar, o cartão sai da lista instantaneamente, sem indicador de carregamento. Camila move 'O Hobbit' para outra coleção via menu de contexto (selecionar destino e confirmar) e a contagem de itens é atualizada imediatamente em ambas as coleções. **Restrições:** 1. A interface deve aplicar a movimentação de forma otimista (atualizar visualmente antes da confirmação do servidor). 2. Em caso de falha na comunicação com o servidor, o sistema deve reverter a movimentação e informar o usuário. 3. O botão de desfazer deve estar disponível por 30 segundos após a movimentação, conforme RF-10; um atalho de teclado para a mesma ação não faz parte desta versão. 4. As contagens de itens nas coleções de origem e destino devem ser recalculadas imediatamente.
Prioridade	Importante
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 38 – RNF-07 – Desempenho do Editor de Layout

Código	RNF-07
Categoria	Desempenho
Descrição	**Descrição:** O editor de layout deve refletir alterações de elementos em tempo real, sem travamentos ou recarregamentos completos da página. **Métrica:** Qualquer operação no editor (adicionar, remover, reposicionar ou redimensionar elemento) deve ser refletida na prévia em no máximo 200 milissegundos. **Exemplos:** Camila arrasta um elemento de Imagem para uma nova posição no editor e a prévia atualiza instantaneamente sem piscar ou travar. Camila adiciona cinco elementos consecutivos ao layout e o editor mantém fluidez durante todas as operações. **Restrições:** 1. O editor deve utilizar renderização parcial, atualizando apenas os componentes afetados pela alteração. 2. O salvamento do layout deve ocorrer de forma assíncrona, sem bloquear a interação do usuário com o editor. 3. A prévia deve manter fidelidade visual com o resultado final exibido na ficha do item. 4. O editor deve suportar pelo menos 15 elementos simultâneos sem degradação perceptível de desempenho.
Prioridade	Essencial
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 39 – RNF-08 – Disponibilidade do Sistema

Código	RNF-08
Categoria	Disponibilidade
Descrição	**Descrição:** O sistema deve manter alta disponibilidade para que atualizações de amigos e sincronização de dados ocorram continuamente. **Métrica:** O sistema deve garantir disponibilidade mínima de 99% do tempo (uptime), excluindo janelas de manutenção programada comunicadas com antecedência mínima de 24 horas. **Exemplos:** Camila acessa o sistema às 23h de um domingo e consegue visualizar o feed de atualizações dos amigos normalmente. Durante uma manutenção programada, o sistema exibe página informativa com previsão de retorno, comunicada previamente por email. **Restrições:** 1. O sistema deve possuir mecanismo de recuperação automática em caso de falha de serviço. 2. Manutenções programadas devem ser realizadas em horários de menor uso (madrugada) e comunicadas com antecedência. 3. O feed de atualizações deve funcionar com conexão intermitente, exibindo dados em cache quando offline. 4. Dados não sincronizados devem ser enviados automaticamente quando a conexão for restabelecida.
Prioridade	Importante
Operação	Processamento

Fonte: Elaborada pelo autor

Quadro 40 – RNF-09 – Acessibilidade

Código	RNF-09
Categoria	Acessibilidade
Descrição	**Descrição:** O sistema deve adotar padrões de acessibilidade, como contraste adequado, textos legíveis e navegação clara. **Métrica:** O sistema deve atender aos critérios de conformidade WCAG 2.1 nível AA, incluindo relação de contraste mínima de 4,5:1 para texto normal e 3:1 para texto grande. **Exemplos:** Camila, que possui baixa visão, escolhe uma paleta cujas combinações atendem ao contraste mínimo e consegue ler todos os textos e identificar todos os botões da interface. Um usuário utiliza o sistema exclusivamente via teclado (navegação por Tab e Enter) e consegue acessar todas as funcionalidades principais. **Restrições:** 1. Todos os elementos interativos devem ser acessíveis via navegação por teclado. 2. Imagens informativas devem possuir texto alternativo (alt text) descritivo. 3. O sistema deve alertar o usuário ao selecionar combinações de cores que não atendam ao contraste mínimo. 4. Fontes devem possuir tamanho mínimo de 12 px e ser escaláveis sem perda de funcionalidade até 200% de zoom.
Prioridade	Importante
Operação	Saída

Fonte: Elaborada pelo autor

Quadro 41 – RNF-10 – Segurança e Integridade dos Dados

Código	RNF-10
Categoria	Segurança
Descrição	**Descrição:** O sistema deve armazenar dados em servidor seguro e garantir integridade das coleções criadas pelo usuário. **Métrica:** O sistema deve realizar backups automáticos diários do banco de dados, com retenção mínima de 30 dias, e garantir que nenhum dado de coleção seja corrompido ou perdido sem ação explícita do usuário. **Exemplos:** Uma falha no servidor é corrigida e todos os dados de coleções, itens e configurações de Camila são restaurados integralmente a partir do backup mais recente. Camila exclui acidentalmente uma coleção; o sistema solicita confirmação antes de executar e, uma vez confirmada, a exclusão é permanente e registrada em log de auditoria. **Restrições:** 1. O banco de dados deve ser hospedado em servidor com certificação de segurança e acesso restrito. 2. Backups automáticos devem ser realizados diariamente e armazenados em localização separada do servidor principal. 3. Operações de exclusão devem ser registradas em log de auditoria com identificação do usuário, data e hora. 4. O sistema deve utilizar transações atômicas para garantir que operações de escrita não resultem em dados inconsistentes. 5. Arquivos de mídia (imagens) devem ser armazenados com redundância para evitar perda.
Prioridade	Essencial
Operação	Processamento

Fonte: Elaborada pelo autor

5.2.3 Restrições, Suposições e Dependências

Esta seção apresenta os fatores que podem limitar decisões de desenvolvimento ou influenciar diretamente a implementação dos requisitos definidos.

- O sistema depende de conexão com a internet para sincronizar dados e exibir atualizações de amigos.

- O desempenho do editor de layout está condicionado à capacidade do dispositivo do usuário.
- A troca de dados entre usuários exige um servidor ativo e funcional.
- A expansão das funcionalidades sociais depende da adoção do sistema por múltiplos usuários.

5.2.4 Requisitos Adiados

Alguns requisitos foram identificados durante a elicitação, porém não serão implementados nesta primeira versão do sistema. Eles poderão ser considerados em iterações futuras.

- Implementação de chat privado entre usuários.
- Recomendações automáticas baseadas nas coleções dos amigos.
- Exportação de coleções em PDF ou outros formatos.
- Mecanismos avançados de descoberta pública, como ranking e tendências. A descoberta simples, que reúne coleções de outras pessoas pelas tags, faz parte do RF-07.

Outros três comportamentos chegaram a ser especificados, mas foram retirados dos requisitos desta versão e ficam registrados aqui, para que a especificação descreva apenas o que o sistema entregue faz:

- **Alteração do e-mail cadastrado** (RF-05). A troca do endereço, com envio de um link de verificação ao novo e-mail e manutenção do antigo até a confirmação, não está disponível. O envio de mensagens já existe na plataforma, pois é usado na confirmação do cadastro (RF-01) e na recuperação de senha (RF-04), e seria o ponto de partida desse fluxo.
- **Entrega imediata de eventos ao navegador** (RF-22 e RF-24). O *feed* e as notificações são atualizados por consulta periódica à API, sem recarregar

a página. A entrega iniciada pelo servidor no instante em que o evento ocorre, por um canal contínuo de comunicação, fica para uma etapa posterior.

- **Ordenação dos Itens pela classificação e filtros pelo conteúdo dos Elementos** (RF-12). Como a classificação é um Elemento da ficha, e não um campo do Item, ordenar e filtrar por ela exige consultar o conteúdo das fichas, o que não foi implementado; a listagem oferece as ordenações por nome e por data e a busca pelo nome e pelo subtítulo.

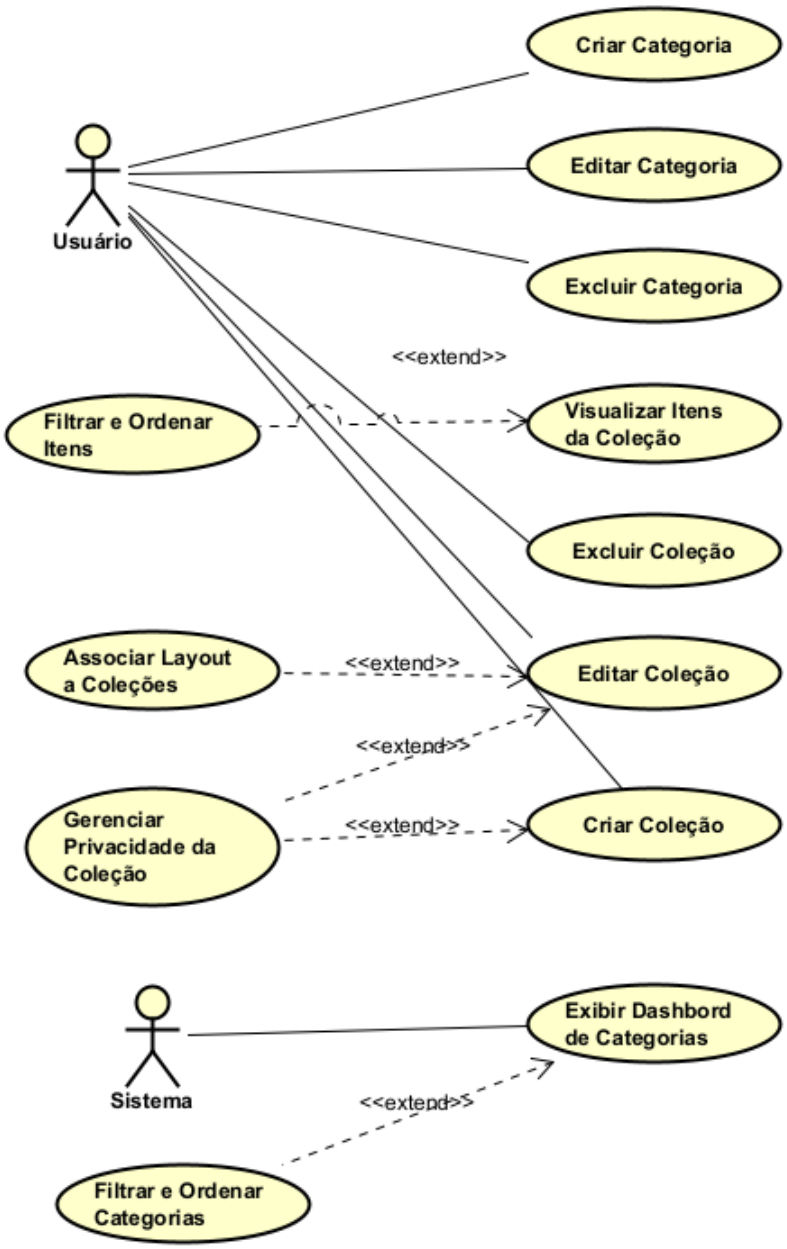
5.3 Casos de Uso

Conforme Valente (2020), um caso de uso descreve a sequência de interações realizadas por um ator com um sistema para atingir um objetivo específico, sendo esse ator, em geral, uma entidade externa, como um usuário humano ou outro sistema. Logo, um caso de uso é uma descrição de um cenário específico de interação entre um ator e o sistema, detalhando as ações realizadas e os resultados esperados. Ele é utilizado para capturar os requisitos funcionais do sistema, fornecendo uma visão clara e estruturada das funcionalidades que o sistema deve oferecer.

5.3.1 Diagrama de Casos de Uso

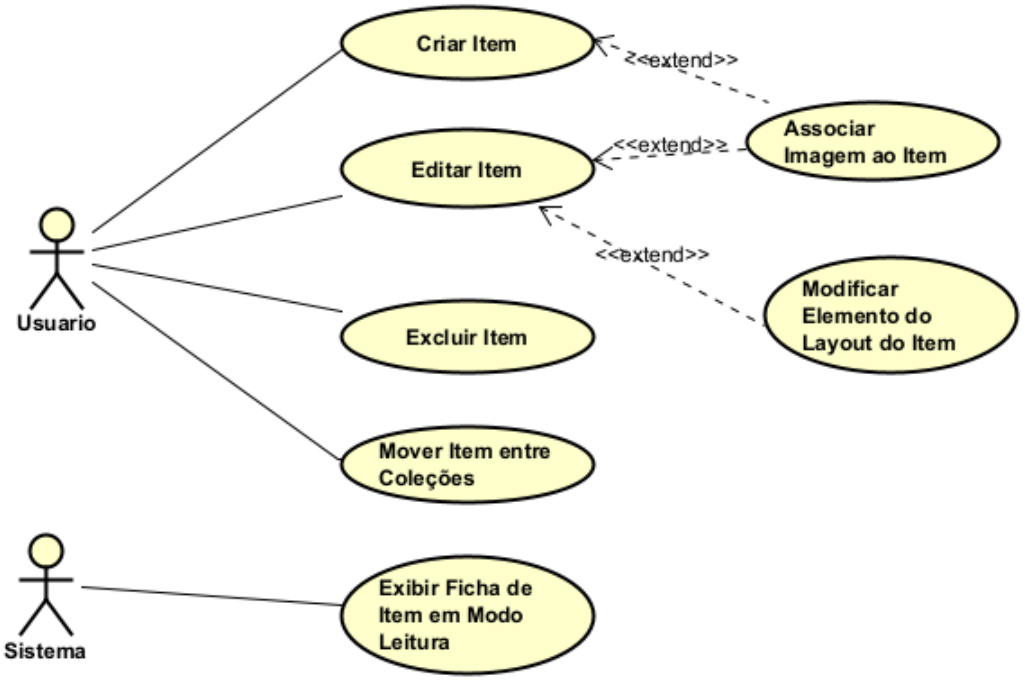
O diagrama de casos de uso elaborado neste trabalho descreve as principais interações do usuário com a plataforma personalizável para criação e compartilhamento de coleções digitais, representando ações como o gerenciamento de categorias, coleções e itens, a montagem de layouts no editor visual, a configuração da aparência da interface e as interações sociais entre usuários. Essas interações são essenciais para garantir que a plataforma atenda às necessidades dos usuários, proporcionando uma experiência eficaz e acessível. Dada a quantidade de casos de uso identificados, o diagrama foi dividido por área funcional e está apresentado nas Figuras de 1 a 7.

Figura 1 – Gerenciamento de Coleções



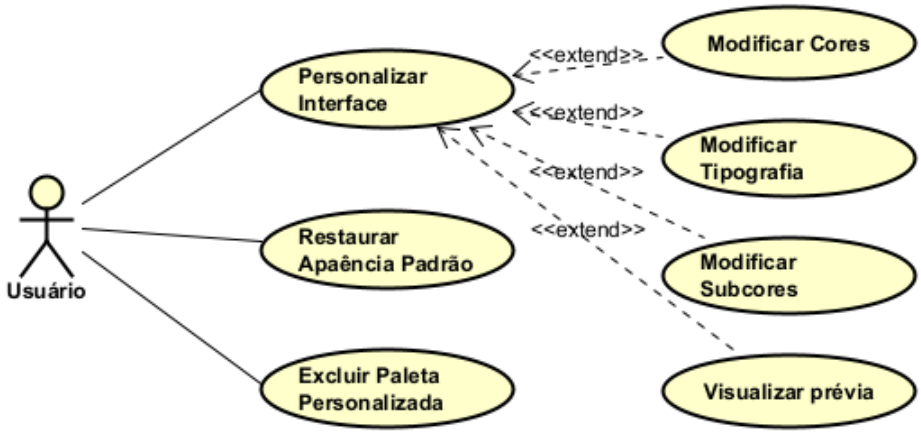
Fonte: Elaborada pelo autor

Figura 2 – Interações Com Itens da Coleção



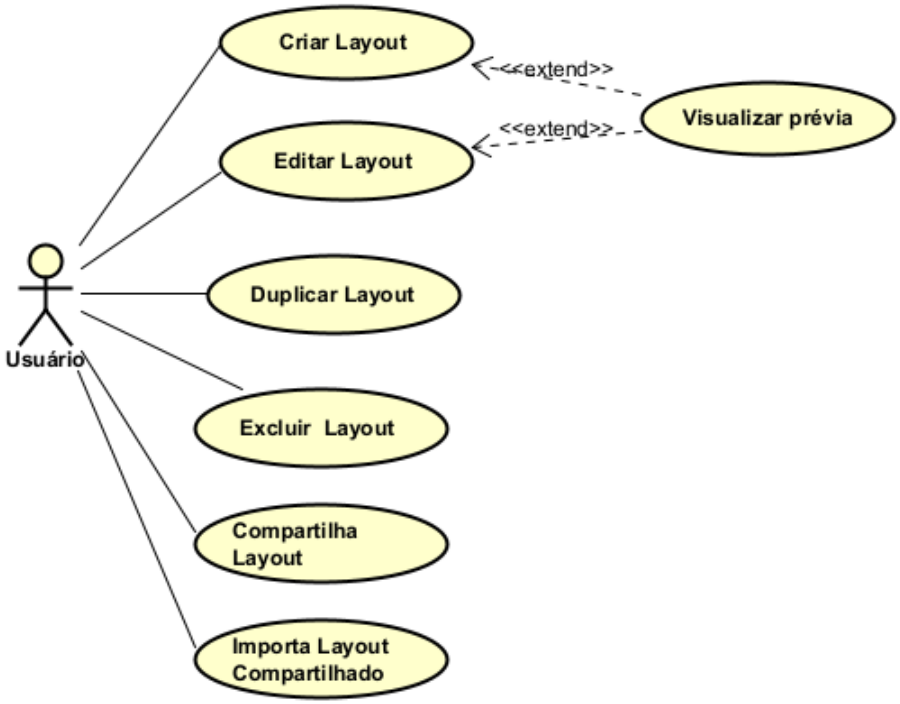
Fonte: Elaborada pelo autor

Figura 3 – Personalização da Plataforma



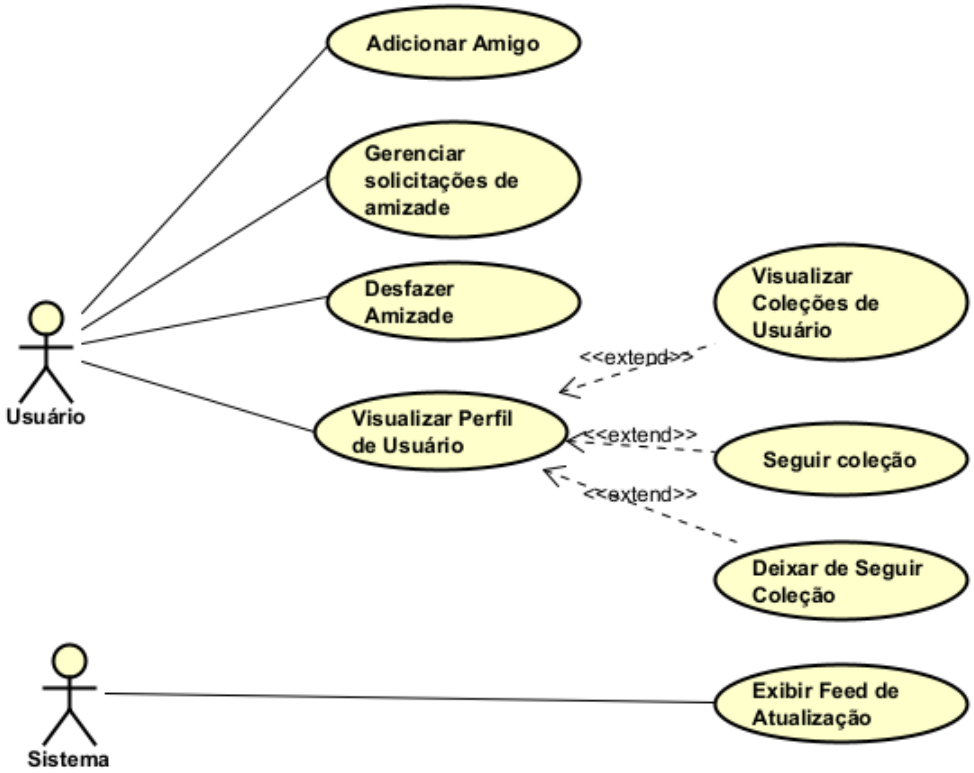
Fonte: Elaborada pelo autor

Figura 4 – Layout e Templates



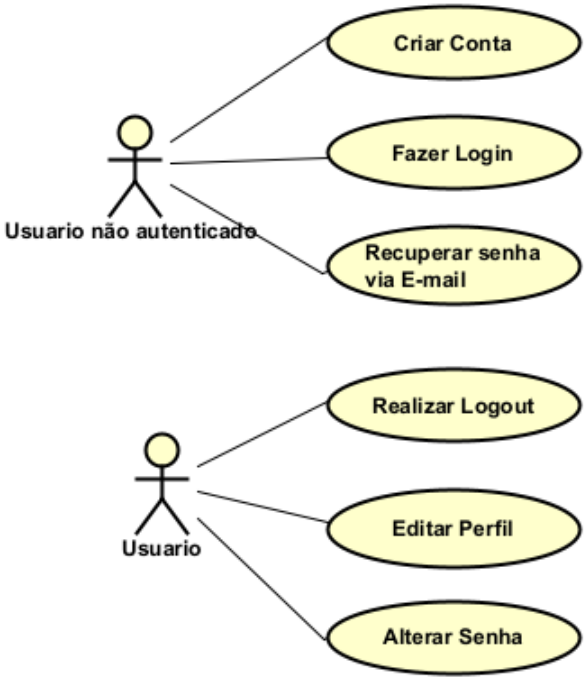
Fonte: Elaborada pelo autor

Figura 5 – Perfil e Social



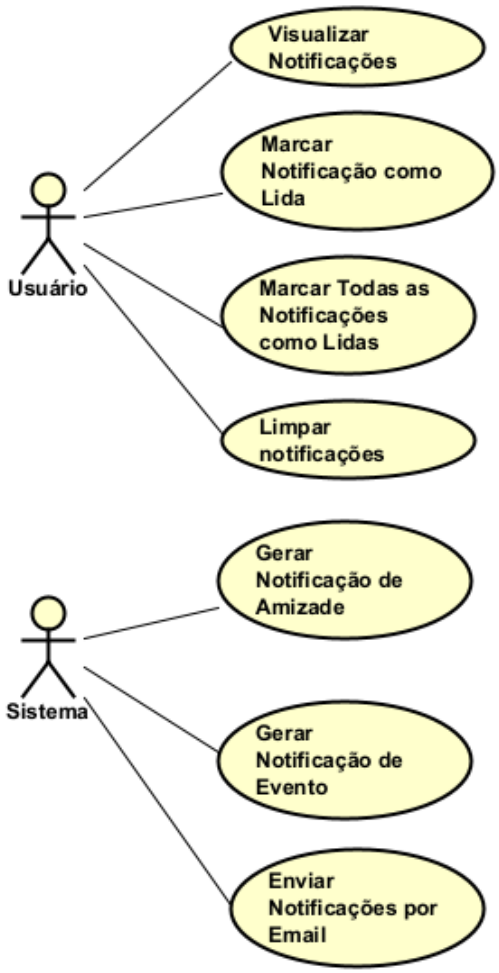
Fonte: Elaborada pelo autor

Figura 6 – Autenticação



Fonte: Elaborada pelo autor

Figura 7 – Notificações



Fonte: Elaborada pelo autor

5.3.2 Especificação dos Casos de Uso

Uma especificação de caso de uso fornece a descrição textual detalhada do comportamento do sistema a partir da perspectiva do usuário, permitindo compreender como as funcionalidades são executadas e quais resultados são esperados. As especificações apresentam os fluxos principais, fluxos alternativos, condições prévias e pós-condições associadas a cada interação.

Neste trabalho, foram elaboradas dezenove especificações textuais, cobrindo os casos de uso que possuem fluxo próprio e alternativas relevantes. Os demais casos

de uso presentes nos diagramas não receberam especificação por representarem variações, detalhamentos ou funcionalidades já incluídas nos fluxos básicos aqui descritos, como é o caso das operações de edição e exclusão, cujo fluxo espelha o de criação com a diferença do registro previamente selecionado. O modelo adotado para a especificação dos casos de uso é apresentado nos Quadros 42 a 60, adaptado da IBM.

Cabe destacar a estrutura hierárquica adotada pelo sistema, que organiza os dados em três níveis: **Categoria** (agrupador temático de alto nível, ex.: "Livros"), **Coleção** (subdivisão de uma categoria onde os itens são efetivamente cadastrados e onde o layout é associado, ex.: "Lidos", "Para Ler") e **Item** (unidade individual de conteúdo, ex.: o livro "Duna"). Essa separação permite que o usuário organize seu acervo em duas camadas antes de cadastrar qualquer item, facilitando a categorização e a personalização por meio do layout definido em cada coleção.

A seguir, são apresentadas as especificações detalhadas dos casos de uso principais identificados no sistema.

5.3.2.1 Fazer Login

Quadro 42 – Especificação do Caso de Uso: Fazer Login

Item	**Descrição**
Nome do Caso de Uso	Fazer Login
Descrição Resumida	Este caso de uso permite que o usuário já cadastrado se autentique no sistema informando seu e-mail e senha, obtendo acesso às funcionalidades restritas da plataforma por meio de um token de sessão gerado automaticamente (RF-02).

Continua na próxima página

Quadro 42 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Autenticação de usuário 1. O usuário acessa a tela de login do sistema. 2. O usuário insere seu e-mail e senha. 3. O usuário confirma clicando em “Entrar”. 4. O sistema valida o formato do e-mail inserido [include: Validar Formato de E-mail]. 5. O sistema verifica se o e-mail está cadastrado na base de dados. 6. O sistema compara a senha informada com o hash armazenado. 7. O sistema gera um token de sessão [include: Gerar Token de Sessão]. 8. O sistema registra a sessão ativa do usuário. 9. O sistema redireciona o usuário para o dashboard de categorias.

Continua na próxima página

Quadro 42 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Credenciais inválidas 1. Nos passos 5 ou 6 do FB01, o sistema identifica que o e-mail não está cadastrado ou a senha não corresponde ao hash armazenado. 2. O sistema exibe a mensagem genérica: "Credenciais de login inválidas", sem especificar qual campo está incorreto. 3. O sistema retorna ao passo 2 do FB01. **FA02:** Campos não preenchidos 1. No passo 3 do FB01, o sistema identifica que um ou ambos os campos estão vazios. 2. O sistema exibe mensagem de erro indicando os campos obrigatórios. 3. O sistema retorna ao passo 2 do FB01. **FA03:** Sessão expirada ou revogada durante o uso 1. O token de sessão atinge o fim de seu prazo de validade, ou a sessão correspondente é revogada pelo sistema, por exemplo após um logout ou uma alteração de senha (RF-03). 2. Na próxima ação do usuário, o sistema verifica a sessão vinculada ao token e a identifica como inválida. 3. O sistema exibe mensagem: "Sua sessão expirou. Por favor, faça login novamente." 4. O sistema redireciona o usuário para a tela de login. **FA04:** E-mail ainda não confirmado 1. No passo 6 do FB01, a senha confere, mas o e-mail da conta ainda não foi confirmado (RF-01). 2. O sistema envia um código de confirmação ao e-mail, se não houver um válido, e não gera o token de sessão. 3. O sistema leva o usuário à tela de confirmação, onde ele digita o código de 6 dígitos. 4. Confirmado o e-mail, o fluxo continua no passo 7 do FB01.

Continua na próxima página

Quadro 42 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-02 - A autenticação deve ser concluída em até 3 segundos. RNF-03 - A senha nunca deve ser trafegada ou armazenada em texto plano. RNF-03 - O token de sessão deve ter prazo de validade definido e poder ser revogado pelo servidor a qualquer momento.
Condições Prévias	O usuário deve estar cadastrado no sistema (RF-01). O mesmo usuário pode manter sessões ativas em outros dispositivos; cada login cria uma sessão própria (RF-02).
Pós-Condições	O usuário está autenticado e possui sessão ativa. Um token de sessão válido é gerado e armazenado. O usuário tem acesso às funcionalidades restritas da plataforma.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

5.3.2.2 Realizar Logout

Quadro 43 – Especificação do Caso de Uso: Realizar Logout

Item	**Descrição**
Nome do Caso de Uso	Realizar Logout
Descrição Resumida	Este caso de uso permite que o usuário autenticado encerre sua sessão ativa na plataforma, invalidando o token de sessão e impedindo acesso não autorizado posterior (RF-02).

Continua na próxima página

Quadro 43 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Encerramento de sessão 1. O usuário clica na opção “Sair” disponível no menu do sistema. 2. O sistema invalida imediatamente o token de sessão ativo [include: Invalidar Token de Sessão]. 3. O sistema encerra a sessão do usuário no servidor. 4. O sistema redireciona o usuário para a tela de login. 5. O sistema exibe mensagem informativa: “Você saiu com sucesso.”
Fluxos Alternativos	**FA01:** Erro ao encerrar sessão 1. No passo 3 do FB01, ocorre falha na comunicação com o servidor. 2. O sistema exibe mensagem de erro: “Não foi possível encerrar sua sessão. Tente novamente.” 3. O sistema mantém o usuário na tela atual.
Requisitos Especiais	RNF-02 - O logout deve ser concluído em até 2 segundos. RNF-03 - O token invalidado não deve ser reutilizável em nenhuma hipótese.
Condições Prévias	O usuário deve estar autenticado com sessão ativa.
Pós-Condições	A sessão do usuário é encerrada. O token de sessão é invalidado e não pode ser reutilizado. O usuário é redirecionado para a tela de login.

Continua na próxima página

Quadro 43 – Continuação da página anterior

Item	**Descrição**
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

5.3.2.3 Alterar Senha

Quadro 44 – Especificação do Caso de Uso: Alterar Senha

Item	**Descrição**
Nome do Caso de Uso	Alterar Senha
Descrição Resumida	Este caso de uso permite que o usuário autenticado troque sua senha dentro da área restrita, informando a senha atual para confirmação de identidade antes de definir a nova credencial (RF-03).

Continua na próxima página

Quadro 44 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Alteração de senha 1. O usuário acessa as configurações de conta. 2. O usuário seleciona a opção “Alterar Senha”. 3. O sistema exibe o formulário com os campos de senha atual, nova senha e confirmação. 4. O usuário informa a senha atual. 5. O usuário define a nova senha e a repete no campo de confirmação. 6. O usuário confirma clicando em “Salvar”. 7. O sistema compara a senha atual informada com o hash armazenado. 8. O sistema valida a nova senha quanto ao comprimento mínimo e à divergência em relação à atual. 9. O sistema armazena a nova senha de forma criptografada. 10. O sistema revoga as demais sessões ativas do usuário. 11. O sistema exibe mensagem de sucesso: “Senha alterada com sucesso.”

Continua na próxima página

Quadro 44 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Senha atual incorreta 1. No passo 7 do FB01, o sistema identifica que a senha atual não corresponde ao hash armazenado. 2. O sistema exibe a mensagem: “Senha atual inválida”. 3. O sistema retorna ao passo 4 do FB01. **FA02:** Nova senha e confirmação divergentes 1. No passo 8 do FB01, o sistema identifica que os dois campos não coincidem. 2. O sistema exibe aviso de que a nova senha e sua confirmação devem ser iguais. 3. O sistema retorna ao passo 5 do FB01. **FA03:** Nova senha inválida 1. No passo 8 do FB01, o sistema identifica que a nova senha tem menos de 8 caracteres ou é igual à senha atual. 2. O sistema exibe mensagem indicando a regra não atendida. 3. O sistema retorna ao passo 5 do FB01.
Requisitos Especiais	RNF-03 - A senha deve ser armazenada com algoritmo de hash seguro e nunca trafegar em texto plano. RNF-02 - A operação deve ser concluída em até 3 segundos.
Condições Prévias	O usuário deve estar autenticado com sessão ativa.
Pós-Condições	A senha é atualizada no banco de dados em forma criptografada. As demais sessões ativas do usuário são revogadas, permanecendo válida apenas a sessão corrente.

Continua na próxima página

Quadro 44 – Continuação da página anterior

Item	**Descrição**
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

5.3.2.4 Recuperar Senha via E-mail

Quadro 45 – Especificação do Caso de Uso: Recuperar Senha via E-mail

Item	**Descrição**
Nome do Caso de Uso	Recuperar Senha via E-mail
Descrição Resumida	Este caso de uso permite que um usuário não autenticado, que tenha esquecido sua senha, redefina a credencial por meio de um link enviado ao e-mail cadastrado, contendo um token único e temporário (RF-04 e RF-24).

Continua na próxima página

Quadro 45 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Redefinição de senha por e-mail 1. O usuário acessa a tela de login e seleciona “Esqueci minha senha”. 2. O usuário informa o e-mail cadastrado. 3. O sistema exibe mensagem informando que, se o e-mail estiver cadastrado, um link de redefinição será enviado. 4. O sistema verifica se o e-mail corresponde a uma conta existente. 5. O sistema gera um token único e temporário e o armazena de forma protegida, associado à conta [include: Gerenciar Notificações do Sistema]. 6. O sistema envia ao endereço cadastrado uma mensagem contendo o link de redefinição. 7. O usuário acessa o link recebido. 8. O sistema valida o token quanto à existência, ao prazo de validade e ao uso prévio. 9. O usuário define a nova senha e a repete no campo de confirmação. 10. O sistema valida a nova senha e a armazena de forma criptografada. 11. O sistema invalida o token e encerra as sessões ativas anteriores do usuário. 12. O sistema redireciona o usuário para a tela de login com mensagem de sucesso.

Continua na próxima página

Quadro 45 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** E-mail não cadastrado 1. No passo 4 do FB01, o sistema identifica que o e-mail informado não corresponde a nenhuma conta. 2. O sistema não envia mensagem alguma, mas mantém a mesma resposta exibida no passo 3, de modo a não revelar quais endereços estão cadastrados. 3. O caso de uso é encerrado. **FA02:** Token expirado 1. No passo 8 do FB01, o sistema identifica que o token ultrapassou o prazo de validade de uma hora. 2. O sistema exibe mensagem informando que o link não é mais válido. 3. O sistema oferece a opção de solicitar um novo link, retornando ao passo 2 do FB01. **FA03:** Token já utilizado 1. No passo 8 do FB01, o sistema identifica que o token já foi usado em uma redefinição anterior. 2. O sistema exibe mensagem informando que o link não é mais válido. 3. O sistema oferece a opção de solicitar um novo link, retornando ao passo 2 do FB01. **FA04:** Nova senha inválida ou divergente 1. No passo 10 do FB01, o sistema identifica que a nova senha tem menos de 8 caracteres ou não coincide com a confirmação. 2. O sistema exibe mensagem indicando a regra não atendida. 3. O sistema retorna ao passo 9 do FB01, mantendo o token válido.

Continua na próxima página

Quadro 45 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-03 - O token deve ser gerado com entropia suficiente e armazenado de forma protegida; a nova senha nunca deve trafegar em texto plano. RNF-02 - O envio da mensagem deve ser disparado em até 3 segundos.
Condições Prévias	O usuário não deve estar autenticado. O usuário deve possuir uma conta cadastrada (RF-01) e acesso ao e-mail informado no cadastro.
Pós-Condições	A senha é atualizada em forma criptografada. O token utilizado é invalidado e não pode ser reaproveitado. As sessões ativas anteriores são encerradas, exigindo nova autenticação.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

5.3.2.5 Editar Perfil

Quadro 46 – Especificação do Caso de Uso: Editar Perfil

Item	**Descrição**
Nome do Caso de Uso	Editar Perfil

Continua na próxima página

Quadro 46 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário visualize e edite as informações de seu perfil: o nome de exibição, a descrição e as imagens de foto e de capa, cada uma enquadrada por ele (RF-05). O e-mail cadastrado não é alterado por este caso de uso, e a senha é alterada pelo caso de uso Alterar Senha (RF-03).
Fluxo Básico	**FB01:** Edição de perfil 1. O usuário acessa o próprio perfil pelo menu da conta. 2. O usuário seleciona a opção "Editar Perfil". 3. O sistema exibe o formulário de edição com as informações atuais do usuário. 4. O usuário modifica os campos desejados (nome de exibição, descrição, foto e capa) [E01]. 5. O usuário confirma as alterações clicando em "Salvar". 6. O sistema valida os dados inseridos. 7. O sistema atualiza as informações no banco de dados. 8. O sistema exibe mensagem de sucesso: "Perfil atualizado com sucesso." 9. O sistema retorna à visualização do perfil com as informações atualizadas.

Continua na próxima página

Quadro 46 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Imagem com formato não suportado 1. Durante o passo 4 do FB01, o usuário tenta enviar uma foto ou capa com formato não suportado. 2. O sistema exibe mensagem de erro: “Formato de imagem não suportado. Use PNG, JPG ou JPEG.” 3. O sistema retorna ao passo 4 do FB01. **FA02:** Imagem muito grande 1. Durante o passo 4 do FB01, o usuário tenta enviar uma foto maior que 5 MB ou uma capa maior que 10 MB. 2. O sistema exibe mensagem de erro informando o tamanho máximo aceito. 3. O sistema retorna ao passo 4 do FB01. **FA03:** Nome de exibição inválido 1. No passo 6 do FB01, o sistema identifica que o nome de exibição tem menos de 3 ou mais de 100 caracteres. 2. O sistema exibe mensagem indicando a regra não atendida. 3. O sistema retorna ao passo 4 do FB01. **FA04:** Cancelamento da operação 1. Em qualquer momento entre os passos 4 e 5 do FB01, o usuário pode selecionar “Cancelar”. 2. O sistema exibe mensagem de confirmação se houver alterações não salvas. 3. Se o usuário confirmar, o sistema descarta as alterações e retorna à visualização do perfil. 4. Se o usuário cancelar a confirmação, retorna ao formulário de edição.

Continua na próxima página

Quadro 46 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-01 - A interface deve ser intuitiva. RNF-02 - As alterações devem ser salvas em até 3 segundos. RNF-03 - O sistema deve proteger informações pessoais. RNF-04 - Compatibilidade com dispositivos móveis.
Condições Prévias	O usuário deve estar autenticado no sistema.
Pós-Condições	As informações do perfil são atualizadas no banco de dados. As alterações são refletidas em todas as áreas do sistema que exibem informações do usuário. O e-mail e o código de usuário (RF-01) permanecem inalterados.
Pontos de Extensão	E01 - Enquadrar Imagem (estende o passo 4 do FB01: o usuário recorta a foto ou a capa, e o sistema guarda o original para um novo enquadramento).

Fonte: Elaborada pelo autor

5.3.2.6 Personalizar Interface

Quadro 47 – Especificação do Caso de Uso: Personalizar Interface

Item	**Descrição**
Nome do Caso de Uso	Personalizar Interface

Continua na próxima página

Quadro 47 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário gerencie a aparência visual global da plataforma por meio de paletas: selecionar a paleta em uso e criar, editar, duplicar, renomear e excluir paletas, cada uma com cor primária, subcores e tipografia. O estilo global (claro, escuro ou automático) é escolhido no menu da conta. Enquanto uma paleta é editada, a interface passa a exibi-la, para que o usuário veja o resultado exato antes de confirmar; nada é gravado até salvar, e tudo persiste entre sessões e dispositivos (RF-18 e RF-19).
Fluxo Básico	**FB01:** Seleção da paleta em uso 1. O usuário seleciona a opção “Aparência” no menu da conta. 2. O sistema exibe a biblioteca de paletas do usuário, com a paleta em uso destacada. 3. O usuário seleciona uma paleta da biblioteca. 4. O sistema define a paleta selecionada como paleta em uso, adotando o estilo global associado a ela. 5. O sistema aplica a paleta globalmente a todas as telas do sistema de forma imediata.

Continua na próxima página

Quadro 47 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Criação de paleta 1. No passo 3 do FB01, o usuário seleciona “Nova paleta”. 2. O sistema exibe o editor de paleta com as cores e a tipografia da paleta em uso, e passa a exibir a interface com a paleta em edição. 3. O usuário informa o nome e modifica cores e tipografia [E01][E02]; a cada alteração o sistema atualiza a interface exibida, sem gravar. 4. O usuário clica em “Salvar”. 5. O sistema classifica a paleta como clara ou escura pela cor de fundo, salva-a na biblioteca e retorna ao passo 2 do FB01. **FA02:** Edição de paleta 1. No passo 3 do FB01, o usuário seleciona “Editar” no menu de uma paleta. 2. O sistema exibe o editor de paleta com os valores atuais da paleta, e passa a exibir a interface com a paleta em edição. 3. O usuário modifica nome, cores e tipografia [E01][E02] e clica em “Salvar”. 4. O sistema salva a paleta; se ela for a paleta em uso, as alterações são aplicadas globalmente de forma imediata. 5. O fluxo retorna ao passo 2 do FB01. **FA03:** Cancelamento da edição 1. Em qualquer momento no editor de paleta, antes de salvar, o usuário seleciona “Cancelar”. 2. Havendo alterações não salvas, o sistema exibe mensagem de confirmação: “Sair sem salvar?” 3. Se o usuário confirmar, o sistema descarta as alterações, devolve a interface à paleta em uso e retorna ao passo 2 do FB01; caso contrário, permanece no editor. **FA04:** Duplicação de paleta 1. No passo 3 do FB01, o usuário seleciona “Duplicar” no menu de uma paleta. 2. O sistema cria uma cópia da paleta, com “(cópia)” no nome, e a exibe na biblioteca. **FA05:** Renomeação de paleta

Quadro 47 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-01 - A biblioteca e o editor de paletas devem ser intuitivos e acessíveis. RNF-05 - As alterações devem ser aplicadas imediatamente, sem recarregar a página, e as paletas devem ser persistidas no perfil e sincronizadas entre dispositivos.
Condições Prévias	O usuário deve estar autenticado no sistema.
Pós-Condições	As paletas e a paleta em uso são salvas no perfil do usuário. A paleta em uso é aplicada globalmente e imediatamente a todas as telas, e persiste em futuras sessões e em outros dispositivos.
Pontos de Extensão	E01 - Modificar Cores (estende o FA01 e o FA02, conforme RF-18). E02 - Modificar Fontes (estende o FA01 e o FA02, conforme RF-19).

Fonte: Elaborada pelo autor

5.3.2.7 Criar Categoria

Quadro 48 – Especificação do Caso de Uso: Criar Categoria

Item	**Descrição**
Nome do Caso de Uso	Criar Categoria

Continua na próxima página

Quadro 48 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário crie uma nova categoria (agrupador temático de alto nível) para organizar coleções e itens relacionados a um tema, como livros, filmes, séries ou jogos. A categoria é criada vazia e servirá como container para coleções que serão criadas posteriormente. A categoria não possui nível de privacidade próprio; a privacidade é configurada individualmente em cada coleção (RF-06 e RF-08).

Continua na próxima página

Quadro 48 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Criação de categoria 1. O usuário acessa a funcionalidade de gerenciamento de categorias no dashboard. 2. O usuário seleciona a opção “Criar Categoria”. 3. O sistema exibe o formulário de criação de categoria. 4. O usuário insere o nome da categoria. 5. O usuário seleciona um ícone para representar a categoria (opcional). 6. O usuário insere uma descrição para a categoria (opcional). 7. O usuário informa o tipo/tema (opcional), podendo escolher uma das sugestões oferecidas ou digitar o seu próprio. 8. O usuário confirma a criação da categoria. 9. O sistema valida os dados inseridos. 10. O sistema cria a categoria vazia no banco de dados. 11. O sistema exibe mensagem de sucesso. 12. O sistema exibe a categoria recém-criada com atalho destacado para “Criar primeira coleção”.

Continua na próxima página

Quadro 48 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Nome da categoria já existe 1. No passo 9 do FB01, o sistema identifica que já existe uma categoria com o mesmo nome para o usuário. 2. O sistema exibe mensagem de erro: "Já existe uma categoria com este nome. Por favor, escolha outro nome." 3. O sistema retorna ao passo 4 do FB01. **FA02:** Campos obrigatórios não preenchidos 1. No passo 9 do FB01, o sistema identifica que o campo nome não foi preenchido. 2. O sistema exibe mensagem de erro: "O nome da categoria é obrigatório." 3. O sistema retorna ao passo 4 do FB01. **FA03:** Cancelamento da operação 1. Em qualquer momento entre os passos 4 e 8 do FB01, o usuário pode selecionar "Cancelar". 2. O sistema descarta as informações inseridas. 3. O sistema retorna ao dashboard de categorias. 4. O caso de uso é encerrado.
Requisitos Especiais	RNF-01 - A interface deve ser intuitiva. RNF-02 - O sistema deve criar a categoria em até 3 segundos. RNF-04 - Compatibilidade com navegadores modernos e dispositivos móveis.
Condições Prévias	O usuário deve estar autenticado no sistema.

Continua na próxima página

Quadro 48 – Continuação da página anterior

Item	**Descrição**
Pós-Condições	Uma nova categoria é criada e armazenada no banco de dados. A categoria aparece no dashboard do usuário. A categoria está pronta para receber coleções, que terão sua própria privacidade individual (padrão pública, conforme RF-08). Nenhum item pode ser cadastrado diretamente na categoria; é necessário criar ao menos uma coleção antes.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

5.3.2.8 Criar Coleção

Quadro 49 – Especificação do Caso de Uso: Criar Coleção

Item	**Descrição**
Nome do Caso de Uso	Criar Coleção
Descrição Resumida	Este caso de uso permite que o usuário crie uma nova coleção dentro de uma categoria existente. A coleção é onde os itens serão efetivamente cadastrados e é também a unidade onde a privacidade é configurada (pública, somente amigos ou privada) e onde o layout é associado (RF-07, RF-15 e RF-08).

Continua na próxima página

Quadro 49 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Criação de coleção 1. O usuário acessa uma categoria específica no dashboard. 2. O usuário seleciona a opção “Criar Coleção”. 3. O sistema exibe o formulário de criação de coleção, com privacidade preenchida como “pública” por padrão. 4. O usuário insere o nome da coleção. 5. O usuário seleciona um ícone (opcional). 6. O usuário insere uma descrição (opcional). 7. O usuário associa um layout à coleção (opcional) [E01]. 8. O usuário configura a privacidade da coleção (pública, somente amigos ou privada) [E02]. 9. O usuário escreve as tags da coleção (opcional), recebendo como sugestão o vocabulário já existente no sistema (RF-07). 10. O usuário confirma a criação da coleção. 11. O sistema valida os dados inseridos. 12. O sistema cria a coleção vinculada à categoria pai, com a privacidade configurada, e vincula as tags informadas ao vocabulário, criando as que ainda não existirem. 13. O sistema exibe mensagem de sucesso. 14. O sistema redireciona para a visualização da coleção, pronta para receber itens.

Continua na próxima página

Quadro 49 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Nome da coleção já existe na categoria 1. No passo 11 do FB01, o sistema identifica que já existe uma coleção com o mesmo nome na categoria pai. 2. O sistema exibe mensagem de erro: "Já existe uma coleção com este nome nesta categoria. Por favor, escolha outro nome." 3. O sistema retorna ao passo 4 do FB01. **FA02:** Campos obrigatórios não preenchidos 1. No passo 11 do FB01, o sistema identifica que o campo nome não foi preenchido. 2. O sistema exibe mensagem de erro: "O nome da coleção é obrigatório." 3. O sistema retorna ao passo 4 do FB01. **FA03:** Cancelamento da operação 1. Em qualquer momento entre os passos 4 e 9 do FB01, o usuário pode selecionar "Cancelar". 2. O sistema descarta as informações inseridas. 3. O sistema retorna à categoria pai. 4. O caso de uso é encerrado.
Requisitos Especiais	RNF-01 - A interface deve ser intuitiva. RNF-02 - A coleção deve ser criada em até 3 segundos.
Condições Prévias	O usuário deve estar autenticado. A categoria pai deve existir e pertencer ao usuário.

Continua na próxima página

Quadro 49 – Continuação da página anterior

Item	**Descrição**
Pós-Condições	Uma nova coleção é criada vinculada à categoria pai, com sua privacidade individual configurada. A coleção está pronta para receber itens. Caso um layout tenha sido associado, os itens dessa coleção passam a adotá-lo; caso contrário, a ficha de cada item nasce vazia e o usuário monta o layout de cada um no editor (RF-17). A contagem de coleções da categoria é atualizada. Se a privacidade configurada for 'pública' ou 'somente amigos', a coleção poderá ser seguida por outros usuários, conforme as regras do RF-21: a pública por qualquer usuário, e a somente amigos apenas pelos amigos confirmados.
Pontos de Extensão	E01 - Associar Layout à Coleção (estende o passo 7 do FB01, conforme RF-15). E02 - Configurar Privacidade da Coleção (estende o passo 8 do FB01, conforme RF-08; padrão pública, alterável para privada ou somente amigos).

Fonte: Elaborada pelo autor

5.3.2.9 Gerenciar Privacidade da Coleção

Quadro 50 – Especificação do Caso de Uso: Gerenciar Privacidade da Coleção

Item	**Descrição**
Nome do Caso de Uso	Gerenciar Privacidade da Coleção

Continua na próxima página

Quadro 50 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário defina quem pode visualizar e acompanhar cada uma de suas coleções, escolhendo entre os níveis pública, somente amigos e privada. A privacidade é configurada no nível da coleção; a categoria não possui nível próprio (RF-08 e RF-06).
Fluxo Básico	**FB01:** Alteração do nível de privacidade 1. O usuário acessa uma coleção de sua propriedade. 2. O usuário seleciona a opção de edição da coleção. 3. O sistema exibe o nível de privacidade vigente entre as três opções disponíveis. 4. O usuário seleciona o novo nível desejado. 5. O usuário confirma a alteração. 6. O sistema atualiza a privacidade da coleção. 7. O sistema avalia os acompanhamentos existentes e encerra os que deixaram de ser permitidos [E01]. 8. O sistema exibe mensagem de sucesso e atualiza o indicador de privacidade exibido no *card* da coleção.

Continua na próxima página

Quadro 50 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Restrição para privada 1. No passo 7 do FB01, o novo nível escolhido é “privada”. 2. O sistema encerra o acompanhamento de todos os seguidores da coleção. 3. O sistema remove os eventos dessa coleção dos *feeds* em que apareciam e oculta as notificações relacionadas (RF-22 e RF-24). 4. O fluxo continua no passo 8 do FB01. **FA02:** Restrição de pública para somente amigos 1. No passo 7 do FB01, a coleção passa de “pública” para “somente amigos”. 2. O sistema mantém o acompanhamento dos seguidores com amizade confirmada. 3. O sistema encerra o acompanhamento dos demais seguidores (RF-21). 4. O fluxo continua no passo 8 do FB01. **FA03:** Ampliação da visibilidade 1. No passo 7 do FB01, a coleção passa a um nível mais permissivo, como de “privada” para “pública”. 2. O sistema não restaura acompanhamentos encerrados anteriormente; a coleção volta a poder ser seguida, mas cada usuário precisa segui-la novamente. 3. O fluxo continua no passo 8 do FB01.

Continua na próxima página

Quadro 50 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-03 - As regras de visibilidade devem ser aplicadas no servidor, de modo que uma coleção restrita não seja acessível nem por link direto. RNF-02 - A alteração deve ser concluída em até 3 segundos.
Condições Prévias	O usuário deve estar autenticado e ser o proprietário da coleção.
Pós-Condições	A coleção passa a ter o novo nível de privacidade. Os acompanhamentos incompatíveis com o novo nível são encerrados, e os eventos correspondentes deixam de ser exibidos nos *feeds* dos usuários afetados.
Pontos de Extensão	E01 - Encerrar Acompanhamentos (estende o passo 7 do FB01, conforme RF-21 – executado apenas quando a mudança restringe a visibilidade).

Fonte: Elaborada pelo autor

5.3.2.10 Criar Item

Quadro 51 – Especificação do Caso de Uso: Criar Item

Item	**Descrição**
Nome do Caso de Uso	Criar Item

Continua na próxima página

Quadro 51 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário cadastre um novo item dentro de uma coleção específica e, em seguida, preencha a sua ficha. Ao abrir a ficha, o item recebe uma cópia própria do layout associado à coleção; caso a coleção não tenha layout, a ficha nasce vazia e o próprio usuário a monta (RF-09, RF-15 e RF-17).
Fluxo Básico	**FB01:** Criação de item 1. O usuário acessa uma coleção específica (dentro de uma categoria). 2. O usuário seleciona a opção "Adicionar Item". 3. O sistema exibe o formulário do item. 4. O usuário insere o nome do item e, se quiser, o subtítulo, a cor de fundo do cartão e a imagem de capa [E02]. 5. O usuário confirma a criação do item. 6. O sistema valida os dados inseridos. 7. O sistema cria o item e o vincula à coleção. 8. O sistema abre o editor da ficha do item e cria a ficha própria, copiada do layout da coleção, com seus Elementos, conteúdo e estilos. 9. O usuário altera na ficha os Elementos que desejar, escrevendo sobre eles diretamente, como nos Elementos de Texto, de Data e de Classificação [E01]. 10. O sistema grava cada alteração automaticamente. 11. O usuário sai do editor e o sistema exibe a ficha do item em modo leitura (RF-11).

Continua na próxima página

Quadro 51 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Nenhum layout aplicável 1. No passo 8 do FB01, o sistema identifica que a coleção não possui layout associado. 2. O sistema cria a ficha própria vazia e o usuário monta o layout do item pelo painel de Elementos (RF-17). 3. O fluxo continua no passo 9 do FB01. **FA02:** Nome do item já existe na coleção 1. No passo 6 do FB01, o sistema identifica que já existe um item com o mesmo nome na coleção. 2. O sistema recusa o cadastro e exibe a mensagem: "Já existe um item com este nome nesta coleção." 3. O sistema retorna ao passo 4 do FB01. **FA03:** Campos obrigatórios não preenchidos 1. No passo 6 do FB01, o sistema identifica que o campo nome não foi preenchido. 2. O sistema exibe mensagem de erro indicando os campos que precisam ser preenchidos. 3. O sistema retorna ao passo 4 do FB01. **FA04:** Cancelamento da operação 1. Em qualquer momento entre os passos 3 e 5 do FB01, o usuário pode selecionar "Cancelar". 2. O sistema descarta as informações inseridas. 3. O sistema retorna à visualização da coleção. 4. O caso de uso é encerrado.

Continua na próxima página

Quadro 51 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-01 - A interface deve ser intuitiva. RNF-02 - O sistema deve criar o item em até 3 segundos. RNF-07 - As alterações na ficha devem ser refletidas no editor em até 200 milissegundos.
Condições Prévias	O usuário deve estar autenticado. A coleção de destino deve existir dentro de uma categoria válida.
Pós-Condições	Um novo item é criado e vinculado à coleção, com a sua ficha própria, copiada do layout da coleção ou vazia (RF-17). A ficha individual do item passa a ser exibida em modo leitura (RF-11). O item aparece na visualização da coleção. A contagem de itens da coleção e da categoria pai é atualizada.
Pontos de Extensão	E01 - Modificar Layout do Item (estende o passo 9 do FB01, conforme RF-17). E02 - Enquadrar a Imagem de Capa (estende o passo 4 do FB01).

Fonte: Elaborada pelo autor

5.3.2.11 Exibir Ficha de Item em Modo Leitura

Quadro 52 – Especificação do Caso de Uso: Exibir Ficha de Item em Modo Leitura

Item	**Descrição**
Nome do Caso de Uso	Exibir Ficha de Item em Modo Leitura

Continua na próxima página

Quadro 52 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso descreve a exibição da ficha individual de um item, com os Elementos do seu layout efetivo, tanto para o dono do acervo quanto para visitantes. A ficha é a tela de leitura do item: para o dono, é dela que se acessa a edição do layout do próprio item; para o visitante, ela é estritamente somente leitura (RF-11 e RF-17).
Fluxo Básico	**FB01:** Exibição da ficha para o dono 1. O usuário seleciona um item na listagem da coleção. 2. O sistema determina o layout efetivo do item, nesta ordem: o layout próprio do item, quando ele foi modificado individualmente (RF-17); na falta dele, o layout associado à sua coleção (RF-15); e, na falta dos dois, nenhum layout, restando à ficha apenas o nome e a data de criação do item. 3. O sistema recupera os Elementos desse layout, com seu conteúdo, posição, tamanho, camada e propriedades visuais. 4. O sistema exibe a ficha, desenhando cada Elemento na posição e com a aparência definidas, e respeitando a aparência global configurada pelo dono (RF-18). 5. O sistema marca os Elementos sem conteúdo, identificando cada um pelo seu nome. 6. O sistema disponibiliza ao dono o acesso à edição do layout do item [E01]. 7. O usuário retorna à listagem da coleção.

Continua na próxima página

Quadro 52 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Item sem layout efetivo 1. No passo 2 do FB01, o sistema identifica que o item não possui layout próprio e que sua coleção também não possui layout associado. 2. O sistema exibe a ficha sem Elementos, apresentando apenas o nome e a data de criação do item. 3. O fluxo segue no passo 6 do FB01. **FA02:** Acesso por visitante 1. No passo 1 do FB01, quem acessa a ficha não é o dono do item. 2. O sistema verifica a privacidade da coleção a que o item pertence e o vínculo entre os dois usuários (RF-08). 3. Sendo o acesso permitido, o sistema exibe os mesmos Elementos do FB01, inclusive os vazios, e omite o passo 6: nenhuma opção de edição é apresentada. **FA03:** Acesso não permitido 1. No passo 2 do FA02, o sistema identifica que a privacidade da coleção não permite o acesso àquele visitante. 2. O sistema informa que o conteúdo não está disponível e não exibe a ficha. 3. O caso de uso é encerrado.

Continua na próxima página

Quadro 52 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-02 - A ficha deve ser carregada em até 3 segundos. RNF-03 - A verificação de privacidade ocorre no servidor, e não apenas na interface. RNF-04 - A ficha deve permanecer legível em diferentes resoluções de tela, preservando as posições relativas entre os Elementos.
Condições Prévias	O item deve existir. O acesso à coleção a que ele pertence deve ser permitido a quem visualiza, seja por ser o dono, seja pelo nível de privacidade configurado (RF-08).
Pós-Condições	A ficha do item é exibida em modo leitura, com os Elementos do seu layout efetivo. Nenhum dado do item é alterado. Para o dono, o acesso à edição do layout do item fica disponível; para o visitante, nenhuma ação de edição é oferecida. As ações de alterar os dados do item e de excluí-lo permanecem no menu de ações da listagem da coleção, e não na ficha.
Pontos de Extensão	E01 - Modificar Layout do Item (estende o passo 6 do FB01, conforme RF-17; disponível apenas para o dono do item).

Fonte: Elaborada pelo autor

5.3.2.12 Visualizar Itens da Coleção

Quadro 53 – Especificação do Caso de Uso: Visualizar Itens da Coleção

Item	**Descrição**
Nome do Caso de Uso	Visualizar Itens da Coleção

Continua na próxima página

Quadro 53 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário visualize todos os itens de uma coleção, aplicando um critério de ordenação e uma busca pelo nome e pelo subtítulo (RF-12).
Fluxo Básico	**FB01:** Visualização dos itens 1. O usuário acessa uma coleção a partir da categoria correspondente. 2. O sistema recupera os itens vinculados à coleção, na ordenação previamente escolhida pelo usuário. 3. O sistema exibe os itens na grade. 4. O usuário pode alterar o critério de ordenação [E01]. 5. O usuário pode escrever um termo de busca [E02]. 6. O sistema atualiza a listagem conforme os critérios aplicados. 7. O usuário seleciona um item para abrir sua ficha [include: Exibir Ficha de Item em Modo Leitura].

Continua na próxima página

Quadro 53 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Coleção sem itens 1. No passo 2 do FB01, o sistema identifica que a coleção não possui itens. 2. O sistema exibe mensagem informativa e um atalho para o cadastro do primeiro item. 3. O caso de uso é encerrado. **FA02:** Nenhum item corresponde à busca 1. No passo 6 do FB01, nenhum item tem o termo no nome nem no subtítulo. 2. O sistema exibe mensagem informando que não há resultados. 3. O sistema retorna ao passo 5 do FB01. **FA03:** Saída da coleção 1. O usuário navega para fora da coleção. 2. O sistema descarta a busca, que vale apenas durante a permanência na coleção. 3. O sistema preserva o critério de ordenação, que vale para todas as coleções até ser alterado.
Requisitos Especiais	RNF-02 - A listagem deve ser carregada em até 3 segundos, com carregamento progressivo acima de 20 itens. RNF-04 - A grade deve permanecer legível em diferentes resoluções de tela.
Condições Prévias	O usuário deve estar autenticado. A coleção deve existir e ser visível para ele, seja por ser de sua propriedade, seja pelo nível de privacidade configurado (RF-08).

Continua na próxima página

Quadro 53 – Continuação da página anterior

Item	**Descrição**
Pós-Condições	O usuário visualiza os itens da coleção conforme os critérios aplicados. O critério de ordenação escolhido é persistido para as demais coleções; a busca é descartada ao sair da coleção.
Pontos de Extensão	E01 - Ordenar Itens (estende o passo 4 do FB01, conforme RF-12). E02 - Buscar Itens (estende o passo 5 do FB01, conforme RF-12; o termo é comparado com o nome e o subtítulo).

Fonte: Elaborada pelo autor

5.3.2.13 Visualizar Dashboard de Categorias

Quadro 54 – Especificação do Caso de Uso: Visualizar Dashboard de Categorias

Item	**Descrição**
Nome do Caso de Uso	Visualizar Dashboard de Categorias
Descrição Resumida	Este caso de uso permite que o usuário visualize todas as suas categorias em uma tela dedicada, com informações resumidas de cada uma (nome, ícone, número de coleções e número total de itens) e acesso rápido às coleções que pertencem a cada categoria. O dashboard é o ponto central de navegação do sistema e permite alternar entre modos de visualização, aplicar filtros e ordenações (RF-13).

Continua na próxima página

Quadro 54 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Visualização do dashboard 1. O usuário acessa o menu principal do sistema. 2. O usuário seleciona a opção “Minhas Categorias” ou equivalente. 3. O sistema recupera todas as categorias do usuário no banco de dados. 4. O sistema exibe o dashboard com cards de todas as categorias, aplicando as preferências de visualização previamente salvas (modo, ordenação e filtros). 5. Para cada categoria, o sistema exibe: nome, ícone, contagem de coleções e contagem total de itens (somando todas as coleções). 6. O usuário pode alternar entre modo grade e modo lista [E01]. 7. O usuário pode aplicar filtros e ordenações (por nome, data de criação ou tipo/tema da categoria) [E02]. 8. O usuário clica em uma categoria para acessar suas coleções.

Continua na próxima página

Quadro 54 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Usuário sem categorias criadas 1. No passo 3 do FB01, o sistema identifica que o usuário não possui categorias criadas. 2. O sistema exibe mensagem informativa: "Você ainda não criou nenhuma categoria. Crie sua primeira categoria para começar!" 3. O sistema exibe botão destacado "Criar Nova Categoria". 4. Se o usuário clicar no botão, o sistema aciona o caso de uso "Criar Categoria". **FA02:** Erro ao carregar categorias 1. No passo 3 do FB01, o sistema encontra erro ao recuperar as categorias do banco de dados. 2. O sistema exibe mensagem de erro: "Não foi possível carregar suas categorias. Por favor, tente novamente." 3. O sistema oferece opção para recarregar a página. **FA03:** Categoria vazia ao acessar 1. No passo 8 do FB01, o usuário clica em uma categoria que não possui coleções criadas. 2. O sistema exibe a página da categoria com mensagem: "Esta categoria ainda não tem coleções. Crie sua primeira coleção para começar a adicionar itens." 3. O sistema exibe atalho destacado "Criar Primeira Coleção". 4. Se o usuário clicar no atalho, o sistema aciona o caso de uso "Criar Coleção".

Continua na próxima página

Quadro 54 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-02 - O sistema deve carregar as categorias em até 3 segundos. RNF-04 - O dashboard deve ser responsivo em dispositivos móveis. RNF-09 - A interface deve seguir padrões de acessibilidade.
Condições Prévias	O usuário deve estar autenticado no sistema.
Pós-Condições	O usuário visualiza todas as suas categorias. O usuário pode navegar para qualquer categoria e, dentro dela, para qualquer coleção ou item. O estado de visualização (grade/lista, filtros, ordenação) é persistido para futuras sessões.
Pontos de Extensão	E01 - Alternar modo de visualização (grade/lista, conforme RF-13). E02 - Filtrar e Ordenar Categorias (permite organizar por diferentes critérios, conforme RF-13).

Fonte: Elaborada pelo autor

5.3.2.14 Criar Layout

Quadro 55 – Especificação do Caso de Uso: Criar Layout

Item	**Descrição**
Nome do Caso de Uso	Criar Layout

Continua na próxima página

Quadro 55 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário crie um layout personalizado no editor visual, acrescentando à área de edição os Elementos que comporão a ficha dos itens e definindo a posição, o tamanho e a aparência de cada um. O layout pode ser criado do zero ou por duplicação de um layout já existente na biblioteca e, posteriormente, associado a uma ou mais coleções, conforme RF-14 e RF-15.

Continua na próxima página

Quadro 55 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Criação de layout 1. O usuário acessa a área de gerenciamento de layouts. 2. O usuário seleciona a opção “Criar Layout”. 3. O sistema exibe o editor com a área de edição vazia e o painel de Elementos ao lado. 4. O usuário insere um nome para o layout. 5. O usuário insere uma descrição para o layout (opcional). 6. O usuário acrescenta Elementos à área de edição pelo painel, escolhendo entre os seis tipos disponíveis: texto, imagem, forma, ícone, classificação e data [E01]. 7. O usuário posiciona e redimensiona cada Elemento arrastando-o pela área de edição e ajustando sua alça, e define a camada em que ele se sobrepõe aos demais. 8. O usuário escreve o conteúdo de um Elemento diretamente sobre ele, na própria área de edição. 9. O usuário ajusta as propriedades visuais do Elemento selecionado: no texto, fonte, tamanho, peso, itálico, sublinhado, riscado, alinhamento horizontal e vertical, nível de título, entrelinha, espaçamento entre letras e apresentação em lista; na forma e na imagem, forma do elemento, arredondamento e borda (cor, espessura e traço); na forma, ainda, curva e ponta da linha; na imagem, ajuste; na classificação, nota máxima e ícone; na data, formato e estética; e, em qualquer Elemento, cor, cor de fundo, opacidade e rotação. 10. O sistema grava cada alteração concluída automaticamente, sem exigir confirmação, e avisa o usuário caso ele tente sair da página com alteração ainda por gravar. 11. O usuário retorna à lista de layouts, na qual o novo layout já aparece disponível para associação a coleções (RF-15).

Continua na próxima página

Quadro 55 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Nome do layout já existe 1. No passo 4 do FB01, o sistema identifica que já existe um layout com o mesmo nome na biblioteca do usuário. 2. O sistema exibe mensagem de erro: "Já existe um layout com este nome. Por favor, escolha outro nome." 3. O sistema retorna ao passo 4 do FB01. **FA02:** Layout sem Elementos 1. No passo 11 do FB01, o usuário volta à lista sem ter acrescentado nenhum Elemento. 2. O sistema mantém o layout gravado, vazio, na biblioteca (RF-14), de onde ele pode ser reaberto e preenchido ou associado a uma coleção. 3. O caso de uso é encerrado. **FA03:** Duplicação de layout existente 1. No passo 2 do FB01, o usuário pode optar por "Duplicar Layout Existente" [E02]. 2. O sistema exibe a biblioteca de layouts do usuário, que reúne os layouts criados por ele, os layouts padrão oferecidos pelo sistema e os layouts importados de outros usuários (RF-16). 3. O usuário seleciona um layout para duplicar. 4. O sistema carrega a estrutura do layout selecionado no editor, pré-preenchendo o nome com o original acrescido de "(cópia)". 5. O fluxo continua no passo 4 do FB01, com o usuário podendo renomear e ajustar a cópia livremente. 6. Alterações realizadas no layout duplicado não refletem no layout de origem. **FA04:** Desfazer uma alteração 1. Em qualquer momento entre os passos 6 e 10 do FB01, o usuário pode desfazer a última alteração, e também refazê-la. 2. O sistema restaura o estado anterior da área de edição e o grava, percorrendo o histórico de alterações da sessão de edição. 3. O fluxo retorna ao passo em que o usuário estava. 4. Caso o usuário feche a página havendo alterações ainda

Quadro 55 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-01 - O editor deve ser intuitivo e facilitar a personalização. RNF-07 - As alterações devem ser gravadas e refletidas na prévia sem recarregar a página e sem travamentos. RNF-04 - Compatibilidade com navegadores modernos.
Condições Prévias	O usuário deve estar autenticado. Para duplicação (FA03), deve existir ao menos um layout na biblioteca do usuário.
Pós-Condições	Um novo layout é criado e armazenado no banco de dados, com os Elementos acrescentados pelo usuário, suas posições, tamanhos, camadas, conteúdos e propriedades visuais. O layout está disponível para associação a uma ou mais coleções, conforme RF-15, e aparece na biblioteca de layouts do usuário. Se criado por duplicação, o layout de origem permanece inalterado.
Pontos de Extensão	E01 - Editar Layout (estende o passo 6 do FB01, permitindo acrescentar, alterar e remover Elementos a qualquer momento, conforme RF-14). E02 - Duplicar Layout (estende o passo 2 do FB01, permitindo criar um layout a partir de outro já existente na biblioteca, conforme RF-14).

Fonte: Elaborada pelo autor

5.3.2.15 Compartilhar Layout

Quadro 56 – Especificação do Caso de Uso: Compartilhar Layout

Item	**Descrição**
Nome do Caso de Uso	Compartilhar Layout
Descrição Resumida	Este caso de uso permite que um usuário compartilhe um layout de sua biblioteca com outra pessoa, por meio do identificador do layout, e que o destinatário o importe, recebendo uma cópia independente em sua própria biblioteca (RF-16).

Continua na próxima página

Quadro 56 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Compartilhamento e importação 1. O usuário proprietário acessa sua biblioteca de layouts. 2. O usuário seleciona o layout desejado e escolhe a opção de compartilhar. 3. O sistema apresenta o identificador do layout, com a opção de copiá-lo. 4. O usuário envia o identificador ao destinatário pelo meio que preferir, externo à plataforma. 5. O destinatário, autenticado, escolhe importar na sua biblioteca de layouts e cola o identificador recebido. 6. O sistema verifica se o layout existe e se não pertence ao próprio destinatário. 7. O sistema exibe uma prévia da estrutura do layout e solicita confirmação da importação. 8. O destinatário confirma a importação. 9. O sistema cria, na biblioteca do destinatário, uma cópia integral da estrutura do layout, incluindo Elementos, conteúdo, estilos, posições e tamanhos. 10. O sistema registra a cópia com a finalidade de *template* importado e guarda a referência ao layout de origem. 11. O sistema exibe mensagem de sucesso e apresenta o layout importado na biblioteca do destinatário.

Continua na próxima página

Quadro 56 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Layout de origem inexistente 1. No passo 6 do FB01, o sistema identifica que o layout foi excluído pelo proprietário. 2. O sistema exibe mensagem informando que o layout não está mais disponível. 3. O caso de uso é encerrado. **FA02:** Importação do próprio layout 1. No passo 6 do FB01, o sistema identifica que o layout já pertence ao usuário que informou o identificador. 2. O sistema informa que, para obter outra cópia de um layout próprio, deve-se utilizar a duplicação (RF-14). 3. O caso de uso é encerrado. **FA03:** Nome de layout já existente na biblioteca do destinatário 1. No passo 9 do FB01, o sistema identifica que o destinatário já possui um layout com o mesmo nome. 2. O sistema dá à cópia o nome original acrescido de "(cópia)", ou de "(cópia 2)" e assim por diante, como na duplicação (RF-14). 3. O fluxo continua no passo 10 do FB01; o destinatário pode renomear a cópia depois. **FA04:** Alteração posterior no layout de origem 1. Após a importação, o proprietário altera ou exclui o layout de origem. 2. As cópias já importadas permanecem inalteradas, pois são independentes. 3. Na exclusão, o sistema apenas zera a referência à origem, preservando a cópia.

Continua na próxima página

Quadro 56 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-03 - A importação deve exigir usuário autenticado, e o compartilhamento não deve expor dados do proprietário além da estrutura do layout. RNF-02 - A cópia deve ser concluída em até 3 segundos.
Condições Prévias	Ambos os usuários devem estar cadastrados. O layout de origem deve existir e pertencer a quem compartilha. Não é exigido vínculo de amizade entre os dois.
Pós-Condições	Uma cópia independente do layout passa a existir na biblioteca do destinatário, utilizável e editável como se tivesse sido criada por ele. O layout de origem permanece inalterado, e nenhum item ou personalização individual do proprietário é compartilhado.
Pontos de Extensão	E01 - Visualizar Prévia do Layout (estende o passo 7 do FB01 – exibe a estrutura antes de confirmar a importação).

Fonte: Elaborada pelo autor

5.3.2.16 Adicionar Amigo

Quadro 57 – Especificação do Caso de Uso: Adicionar Amigo

Item	**Descrição**
Nome do Caso de Uso	Adicionar Amigo

Continua na próxima página

Quadro 57 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário envie solicitações de amizade para outros usuários da plataforma. A busca pode ser realizada pelo código de usuário ou pelo nome, e a solicitação fica pendente até que o destinatário a aceite ou recuse (RF-20 e RF-24).

Continua na próxima página

Quadro 57 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Envio de solicitação de amizade 1. O usuário acessa a área de busca de usuários ou área social. 2. O usuário insere o código do usuário ou o nome da pessoa que deseja adicionar. 3. O sistema busca usuários correspondentes no banco de dados, conforme o critério informado. 4. O sistema exibe uma lista de resultados com nome, foto e código de usuário. 5. O usuário identifica a pessoa desejada e clica em "Adicionar Amigo". 6. O sistema verifica se já existe uma relação de amizade ou solicitação pendente. 7. O sistema cria uma solicitação de amizade pendente no banco de dados. 8. O sistema gera notificação in-app ao destinatário informando sobre a solicitação [include: Gerenciar Notificações do Sistema]. 9. O sistema exibe mensagem de sucesso: "Solicitação de amizade enviada." 10. O sistema atualiza o botão para "Solicitação Pendente" no perfil do usuário destinatário.

Continua na próxima página

Quadro 57 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Usuário não encontrado 1. No passo 3 do FB01, o sistema não encontra nenhum usuário correspondente ao critério informado. 2. O sistema exibe mensagem: “Nenhum usuário encontrado.” 3. O sistema retorna ao passo 2 do FB01. **FA02:** Já são amigos 1. No passo 6 do FB01, o sistema identifica que os usuários já são amigos. 2. O sistema exibe mensagem: “Vocês já são amigos.” 3. O sistema oferece opção de visualizar o perfil do amigo. 4. O caso de uso é encerrado. **FA03:** Solicitação já existe 1. No passo 6 do FB01, o sistema identifica que já existe uma solicitação pendente. 2. O sistema exibe mensagem: “Você já enviou uma solicitação de amizade para este usuário.” 3. O sistema oferece opção de cancelar a solicitação pendente. 4. O caso de uso é encerrado. **FA04:** Solicitação recebida do mesmo usuário 1. No passo 6 do FB01, o sistema identifica que existe uma solicitação pendente enviada pelo usuário que está sendo adicionado. 2. O sistema exibe mensagem: “Este usuário já enviou uma solicitação de amizade para você.” 3. O sistema oferece opção de aceitar a solicitação existente. 4. Se o usuário aceitar, o sistema aciona o caso de uso “Gerenciar Solicitações de Amizade”.

Continua na próxima página

Quadro 57 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-02 - A busca e envio de solicitação devem ocorrer em até 3 segundos. RNF-03 - A privacidade dos usuários deve ser respeitada.
Condições Prévias	O usuário deve estar autenticado no sistema. O usuário destinatário deve existir e ter um perfil ativo.
Pós-Condições	Uma solicitação de amizade é criada com status "pendente". O destinatário recebe notificação da solicitação (RF-24). A solicitação aparece na lista de pendentes de ambos os usuários. O botão no perfil do destinatário muda para "Solicitação Pendente".
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

5.3.2.17 Gerenciar Solicitações de Amizade

Quadro 58 – Especificação do Caso de Uso: Gerenciar Solicitações de Amizade

Item	**Descrição**
Nome do Caso de Uso	Gerenciar Solicitações de Amizade

Continua na próxima página

Quadro 58 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário visualize, aceite ou recuse solicitações de amizade recebidas de outros usuários da plataforma. Ao aceitar, o vínculo de amizade é criado mutuamente e o solicitante é notificado. Ao recusar, a solicitação é marcada como recusada sem notificação ao solicitante (RF-20 e RF-24).
Fluxo Básico	**FB01:** Aceitação de solicitação de amizade 1. O usuário acessa a área de notificações ou de solicitações pendentes. 2. O sistema exibe a lista de solicitações de amizade com status “pendente”, contendo nome, foto e data de envio de cada solicitante. 3. O usuário seleciona a solicitação desejada e clica em “Aceitar”. 4. O sistema atualiza o status da solicitação para “aceita” no banco de dados. 5. O sistema cria o vínculo de amizade mútuo entre os dois usuários. 6. O sistema envia notificação ao solicitante informando que a solicitação foi aceita [include: Gerenciar Notificações do Sistema]. 7. O sistema exibe mensagem de sucesso: “Vocês agora são amigos!” 8. O sistema atualiza a lista de solicitações, removendo a solicitação aceita.

Continua na próxima página

Quadro 58 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Recusa de solicitação 1. No passo 3 do FB01, o usuário clica em "Recusar" na solicitação desejada. 2. O sistema atualiza o status da solicitação para "recusada" no banco de dados. 3. O sistema não gera notificação ao solicitante; a recusa é silenciosa. 4. O sistema remove a solicitação da lista de pendentes do usuário. 5. O caso de uso é encerrado. **FA02:** Nenhuma solicitação pendente 1. No passo 2 do FB01, o sistema identifica que não há solicitações pendentes. 2. O sistema exibe mensagem: "Você não possui solicitações de amizade pendentes." 3. O caso de uso é encerrado. **FA03:** Solicitação cancelada pelo remetente 1. No passo 3 do FB01, o usuário tenta aceitar uma solicitação que foi cancelada pelo remetente antes da resposta. 2. O sistema exibe mensagem: "Esta solicitação não está mais disponível." 3. O sistema remove a solicitação da lista de pendentes. 4. O caso de uso é encerrado.

Continua na próxima página

Quadro 58 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-02 - A operação deve ser concluída em até 3 segundos. A notificação de aceite é registrada na mesma operação (RF-24); recusas não geram notificação.
Condições Prévias	O usuário deve estar autenticado.
Pós-Condições	Se aceita: vínculo de amizade mútuo criado; ambos os usuários aparecem na lista de amigos um do outro e o solicitante é notificado. Se recusada: status da solicitação atualizado para "recusada"; nenhum vínculo ou notificação é criado.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

Fonte: Elaborada pelo autor

5.3.2.18 Visualizar Perfil de Usuário

Quadro 59 – Especificação do Caso de Uso: Visualizar Perfil de Usuário

Item	**Descrição**
Nome do Caso de Uso	Visualizar Perfil de Usuário

Continua na próxima página

Quadro 59 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário acesse o perfil público de outro usuário (amigo ou não), visualizando suas informações básicas e as coleções disponíveis conforme o nível de privacidade configurado em cada uma delas. As coleções visíveis são agrupadas pelas respectivas categorias, e a navegação nas coleções e itens do usuário visitado é restrita ao modo somente leitura (RF-23 e RF-08).

Continua na próxima página

Quadro 59 – Continuação da página anterior

Item	**Descrição**
Fluxo Básico	**FB01:** Visualização do perfil 1. O usuário acessa a lista de amigos ou realiza busca pelo código do usuário desejado. 2. O usuário seleciona o perfil desejado. 3. O sistema recupera as informações públicas do perfil (nome, foto, biografia). 4. O sistema recupera as coleções do usuário visitado com privacidade "pública" ou "somente amigos" (esta última apenas quando há vínculo de amizade confirmada), agrupando-as pelas respectivas categorias. 5. O sistema exibe o perfil com suas informações e as coleções acessíveis, agrupadas por categoria. 6. Coleções com privacidade "privada" não são listadas nem mencionadas. Categorias que, para esse visitante, não tenham nenhuma coleção visível também são ocultadas integralmente. 7. O usuário navega pelas coleções visíveis e pelos itens contidos nelas, em modo somente leitura. 8. O usuário pode seguir uma coleção pública independentemente de amizade; para coleções "somente amigos", o acompanhamento exige amizade confirmada [E01].

Continua na próxima página

Quadro 59 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Perfil sem coleções acessíveis 1. No passo 4 do FB01, o sistema identifica que todas as coleções do usuário visitado são privadas ou que ele não possui coleções acessíveis ao visitante. 2. O sistema exibe o perfil básico do usuário visitado com a mensagem: "Este usuário não possui coleções públicas no momento." 3. O caso de uso continua sem exibir coleções ou categorias. **FA02:** Tentativa de acesso a coleção privada por link direto 1. O usuário tenta acessar uma coleção privada de outro usuário por meio de link direto. 2. O sistema verifica a privacidade da coleção. 3. O sistema exibe mensagem: "Esta coleção não está disponível." 4. O sistema não exibe nenhum dado da coleção, nem mesmo o nome. **FA03:** Usuário acessado não é amigo confirmado 1. No passo 4 do FB01, o sistema identifica que não existe vínculo de amizade confirmado entre os usuários. 2. O sistema exibe apenas as coleções com privacidade "pública" do perfil visitado, agrupadas pelas respectivas categorias. 3. Coleções com privacidade "somente amigos" não são exibidas. 4. Categorias sem nenhuma coleção pública visível são ocultadas integralmente.

Continua na próxima página

Quadro 59 – Continuação da página anterior

Item	**Descrição**
Requisitos Especiais	RNF-03 - A privacidade das coleções deve ser respeitada rigorosamente em toda a navegação. RNF-02 - O perfil deve ser carregado em até 3 segundos. RNF-04 - A visualização deve ser responsiva em dispositivos móveis.
Condições Prévias	O usuário deve estar autenticado. O usuário visitado deve existir na plataforma.
Pós-Condições	O usuário visualiza o perfil e as coleções acessíveis do usuário visitado, agrupadas por categoria. Nenhuma alteração é feita nos dados do usuário visitado. O usuário pode iniciar o fluxo de seguir uma coleção a partir desta tela.
Pontos de Extensão	E01 - Seguir Coleção (estende o passo 8 do FB01, conforme RF-21 – permite ao usuário acompanhar uma coleção pública ou somente amigos a partir do perfil visitado).

Fonte: Elaborada pelo autor

5.3.2.19 Visualizar Feed de Atualizações

Quadro 60 – Especificação do Caso de Uso: Visualizar Feed de Atualizações

Item	**Descrição**
Nome do Caso de Uso	Visualizar Feed de Atualizações

Continua na próxima página

Quadro 60 – Continuação da página anterior

Item	**Descrição**
Descrição Resumida	Este caso de uso permite que o usuário acompanhe as atualizações das coleções que segue, visualizando eventos de adição, edição e remoção de itens realizados por seus proprietários, ordenados cronologicamente do mais recente ao mais antigo (RF-22).
Fluxo Básico	**FB01:** Exibição do feed 1. O usuário abre as novidades das coleções que segue, junto ao sino de notificações do cabeçalho. 2. O sistema identifica todas as coleções que o usuário segue ativamente (conforme RF-21). 3. O sistema recupera os eventos de atualização associados a essas coleções (adição/edição/remoção de itens), ordenados do mais recente ao mais antigo. 4. O sistema verifica a privacidade atual de cada coleção antes de exibir seus eventos (RF-08). 5. O sistema exibe o feed com os eventos, contendo: nome do autor da ação, tipo de evento, nome do item afetado, nome da coleção, nome da categoria pai e data/hora relativa do evento. 6. O feed é atualizado automaticamente, sem necessidade de recarregar a página, por consulta periódica à API (RF-22). 7. O usuário pode rolar o feed para carregar eventos mais antigos progressivamente (paginação).

Continua na próxima página

Quadro 60 – Continuação da página anterior

Item	**Descrição**
Fluxos Alternativos	**FA01:** Usuário não segue nenhuma coleção 1. No passo 2 do FB01, o sistema identifica que o usuário não segue nenhuma coleção. 2. O sistema exibe mensagem: "Seu feed está vazio. Siga coleções para acompanhar suas atualizações." 3. O sistema exibe atalho para a tela Explorar, que reúne coleções de outras pessoas pelas tags (RF-07), de onde se chega aos perfis (RF-23). **FA02:** Coleção seguida alterada para privada 1. No passo 4 do FB01, o sistema identifica que uma coleção anteriormente seguida foi alterada para privada. 2. O sistema não exibe os eventos dessa coleção, incluindo eventos anteriores à mudança de privacidade. 3. O sistema encerra automaticamente o acompanhamento dessa coleção. 4. Os demais eventos do feed continuam sendo exibidos normalmente. **FA03:** Erro ao carregar o feed 1. No passo 3 do FB01, o sistema encontra erro ao recuperar os eventos. 2. O sistema exibe mensagem: "Não foi possível carregar o feed. Tente novamente." 3. O sistema oferece botão para recarregar.
Requisitos Especiais	RNF-02 - O carregamento inicial do feed deve ocorrer em até 3 segundos. RNF-03 - Eventos de coleções privadas não devem ser exibidos em nenhuma circunstância.

Continua na próxima página

Quadro 60 – Continuação da página anterior

Item	**Descrição**
Condições Prévias	O usuário deve estar autenticado.
Pós-Condições	O usuário visualiza os eventos recentes das coleções que segue. O feed é atualizado periodicamente durante a navegação (RF-22). Coleções tornadas privadas são removidas do feed imediatamente, com seu acompanhamento encerrado.
Pontos de Extensão	Nenhum ponto de extensão identificado para este caso de uso.

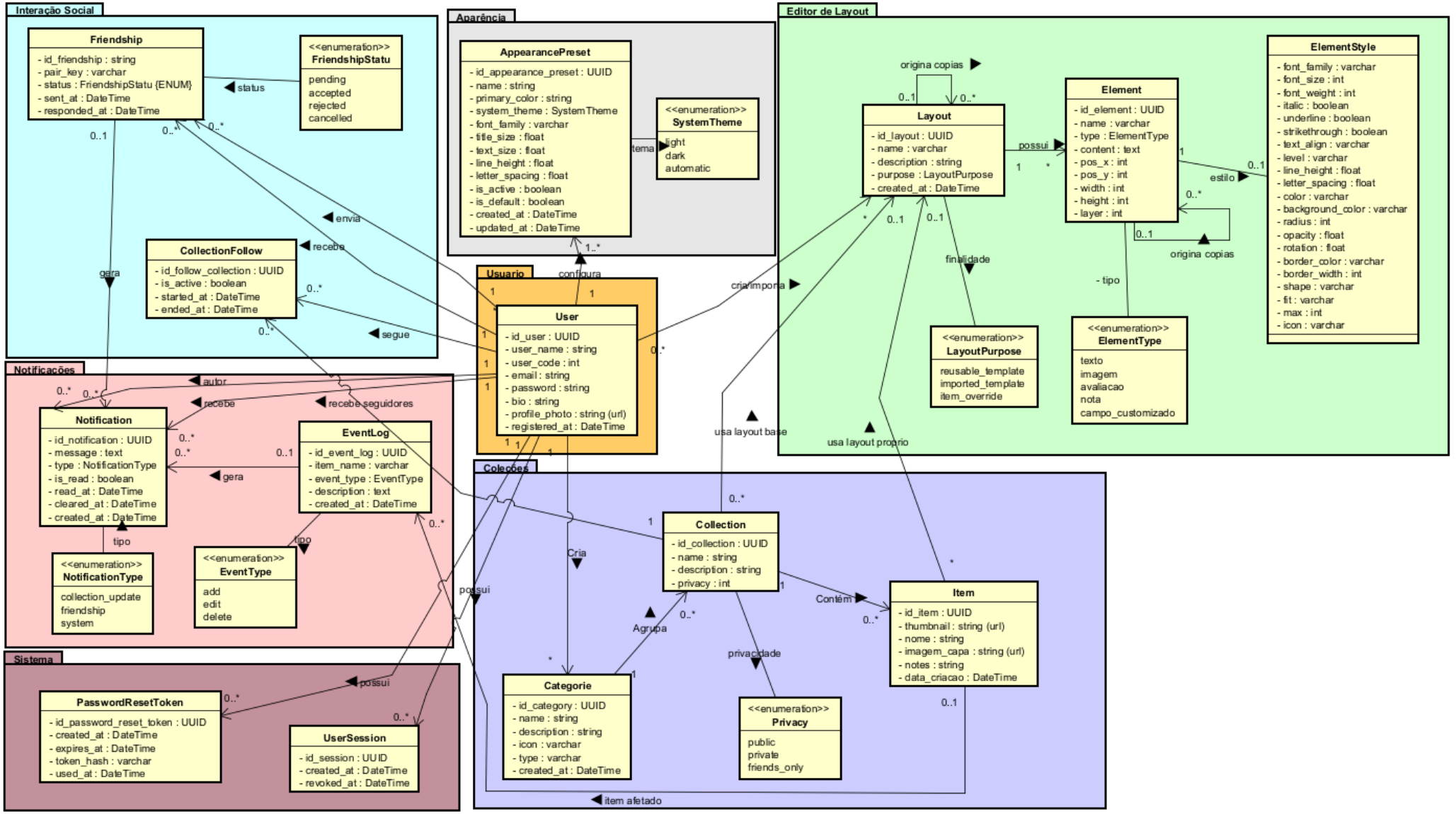
Fonte: Elaborada pelo autor

5.4 Modelo de Domínio

De acordo com (LARMAN, 2007), o modelo de domínio é uma representação visual das classes conceituais ou objetos do mundo real em um domínio de interesse, ilustrando conceitos significativos, atributos e associações entre eles.

A Figura 8 apresenta o modelo de domínio desenvolvido para o escopo deste trabalho, evidenciando a organização nos sete pacotes descritos na Subseção 5.1.1, bem como as principais classes conceituais do sistema e as associações que estabelecem entre si.

Figura 8 – Modelo de Domínio



Fonte: Elaborada pelo autor

6 PROJETO DE SOFTWARE

Este capítulo apresenta a arquitetura técnica e as especificações de projeto que fundamentam o sistema de gerenciamento de coleções. Nele, detalham-se as interfaces, a modelagem de dados e as regras de persistência utilizadas no desenvolvimento da aplicação.

6.1 Projeto de Interface

As interfaces do sistema foram projetadas na ferramenta **Figma** e adotam um partido visual deliberadamente simples, construído sobre poucos componentes reutilizáveis, uma tipografia única e uma paleta reduzida, ancorada em um tom de roxo. A escolha é coerente com a proposta do produto: como o conteúdo das fichas é inteiramente definido pelo usuário no editor de *layouts*, a interface que o cerca deve permanecer discreta, para não competir visualmente com aquilo que o próprio usuário compôs. Convém registrar que os protótipos aqui descritos foram elaborados na fase de concepção, antes da implementação, e representam a intenção de projeto: definem a organização e o comportamento pretendidos para as telas, e não seu resultado final, que pode incorporar ajustes decorrentes do desenvolvimento.

Os esquemas de tela elaborados nessa fase são apresentados nas Figuras 9 a 18. Eles registram a organização pretendida para cada tela e serviram de base para o desenvolvimento, sem a pretensão de corresponder ponto a ponto à interface final.

O sistema divide-se em duas regiões visualmente distintas. Antes da autenticação, na página inicial, no cadastro e nas telas de *login* e de recuperação de senha, o conteúdo aparece centralizado e isolado, sem elementos de navegação, já que ainda não há acervo a percorrer. A Figura 9 mostra esse arranjo na tela de entrada, reduzida à identidade da plataforma e às duas ações possíveis. Após a autenticação, as telas passam a compartilhar uma mesma moldura: uma barra lateral fixa, que dá acesso às categorias do usuário e à área social; uma barra superior, que reúne a

identificação da tela atual, o acesso às notificações e o menu do usuário; e a área central, onde o conteúdo é efetivamente apresentado.

Figura 9 – Esquema da tela inicial

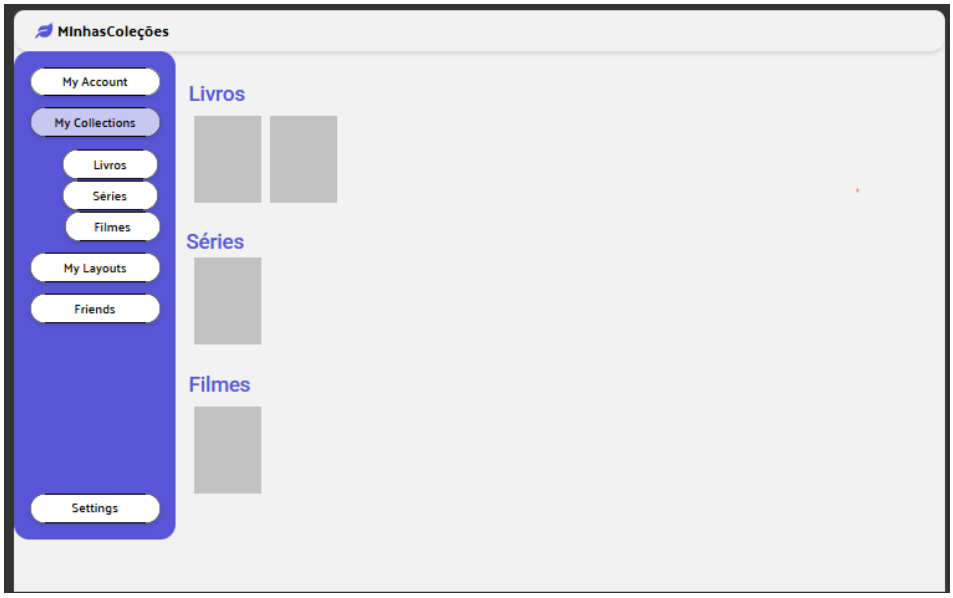


Fonte: Elaborada pelo autor

A navegação pela hierarquia de três níveis é o eixo da área central. Categorias, coleções e itens são exibidos em uma grade de *cards*, e uma trilha de navegação no topo indica a posição do usuário, por exemplo "Categorias > Livros > Lidos", permitindo retornar a qualquer nível anterior com um clique. Os *cards* adaptam-se ao que representam: os de coleção exibem o indicador de privacidade e a contagem de itens, enquanto os de item privilegiam a imagem de capa. Acima da grade situam-se os controles de modo de exibição, ordenação e filtro previstos nos requisitos, e um botão flutuante, no canto inferior, cria um novo registro no contexto em que o usuário se encontra.

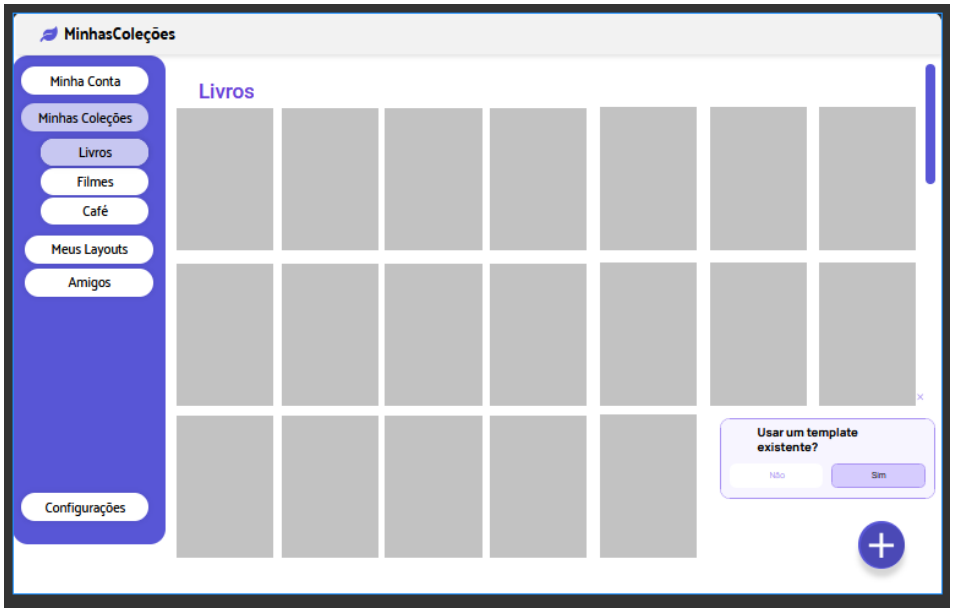
A Figura 10 apresenta o esquema da tela que reúne as coleções do usuário, agrupadas sob as categorias a que pertencem, com a barra lateral dando acesso direto a cada uma delas. A Figura 11, por sua vez, mostra a listagem de itens de uma coleção, com a grade de *cards* ocupando a área central e o botão flutuante de criação no canto inferior direito.

Figura 10 – Esquema da tela de coleções do usuário



Fonte: Elaborada pelo autor

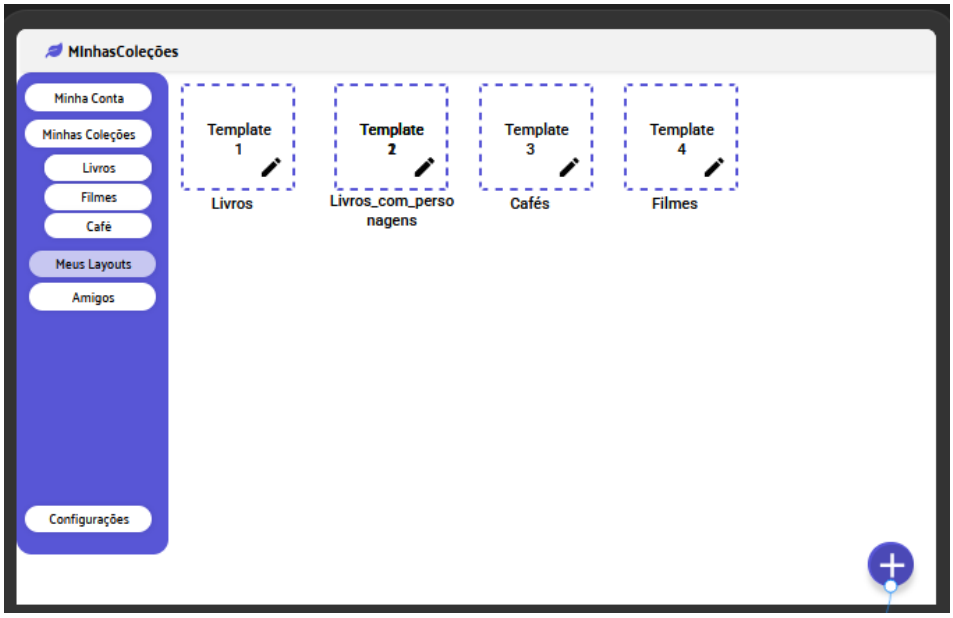
Figura 11 – Esquema da listagem de itens de uma coleção



Fonte: Elaborada pelo autor

A biblioteca de layouts, apresentada na Figura 12, reúne em uma única tela os layouts disponíveis ao usuário, cada um identificado pelo nome que ele lhe deu e acompanhado do atalho para edição.

Figura 12 – Esquema da biblioteca de layouts



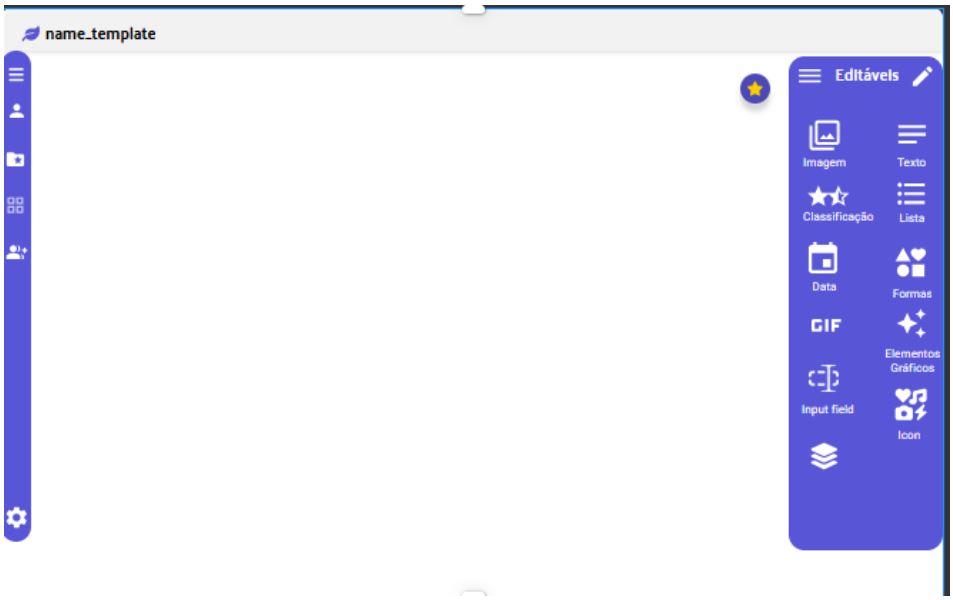
Fonte: Elaborada pelo autor

O editor de *layouts* recebeu tratamento próprio, por ser a tela em que a proposta da plataforma se materializa. Divide-se entre uma área de edição, que representa a ficha em construção, e um painel lateral com os tipos de Elemento disponíveis. O usuário acrescenta a peça à área de edição pelo painel e, uma vez ali, posiciona e redimensiona o Elemento livremente, escrevendo seu conteúdo sobre ele próprio. O editor não possui botão de gravação: cada alteração concluída é gravada automaticamente, cabendo ao desfazer e ao refazer o papel de reverter uma alteração indesejada, e ao aviso de saída o de alertar quando ainda há alteração por gravar. Também não há alternância entre modos de edição e de prévia: a própria área de edição desenha a ficha como ela aparecerá em modo leitura, de modo que cada ajuste é conferido no lugar em que é feito.

As Figuras 13 e 14 mostram as duas situações do editor. Na primeira, a área de edição está vazia, como no início da montagem de um layout; na segunda,

ela exibe uma ficha de livro já composta, com Elementos de imagem, texto, data e classificação dispostos livremente. Cabe observar que o painel lateral desses esquemas reflete o conjunto de peças cogitado na concepção, anterior à definição dos seis tipos consolidados no Capítulo 5.

Figura 13 – Esquema do editor de layout com a área de edição vazia



Fonte: Elaborada pelo autor

Figura 14 – Esquema do editor de layout com uma ficha composta



Fonte: Elaborada pelo autor

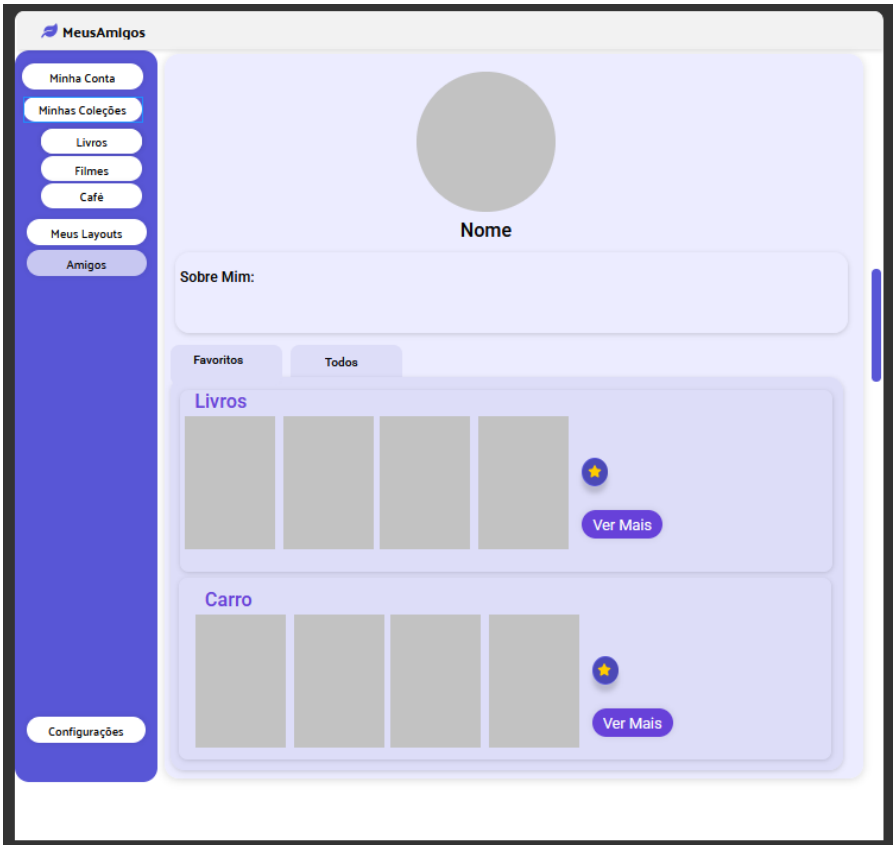
As telas de interação social seguem a mesma moldura. A área de amigos, na Figura 15, separa em abas os vínculos já estabelecidos e as solicitações pendentes, e oferece a busca por código de usuário ou por nome prevista no RF-20. No perfil de outro usuário, na Figura 16, exibem-se os dados públicos e as coleções visíveis conforme o vínculo existente, cada uma acompanhada da ação de seguir ou deixar de seguir.

Figura 15 – Esquema da tela de amigos



Fonte: Elaborada pelo autor

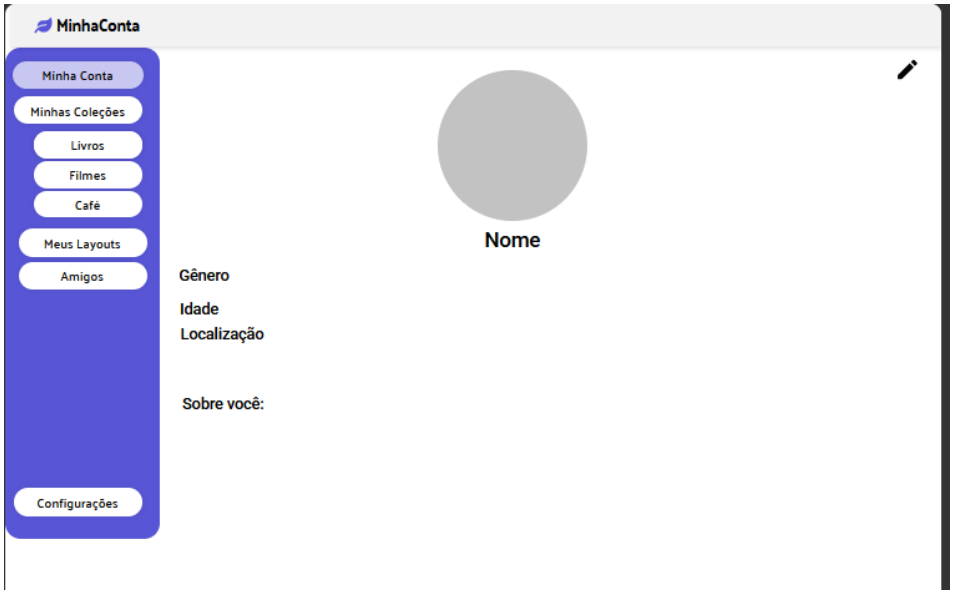
Figura 16 – Esquema do perfil de outro usuário



Fonte: Elaborada pelo autor

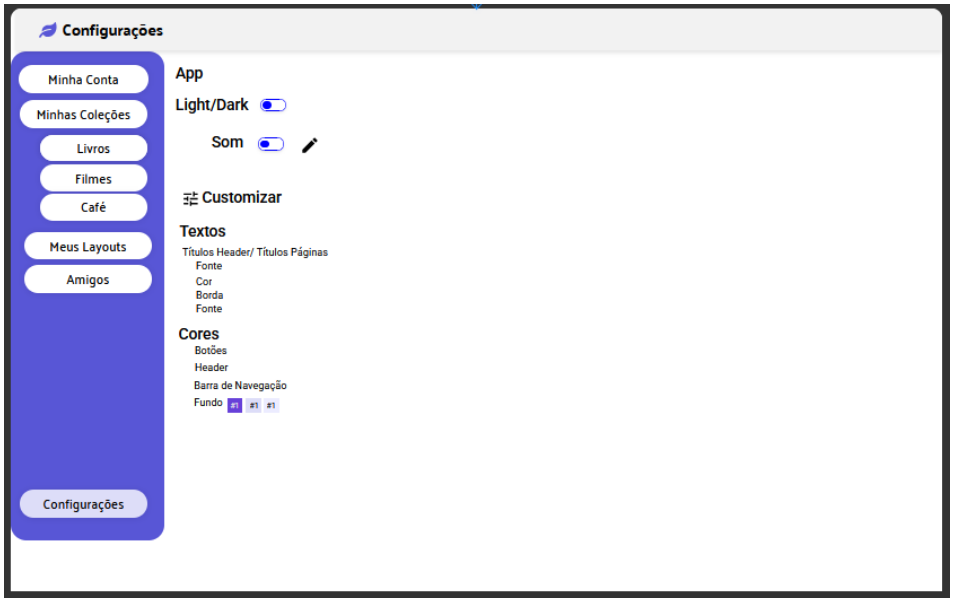
Completam o conjunto as telas de conta e de aparência. O perfil do próprio usuário, na Figura 17, reúne a foto, o nome e os demais dados de identificação, com o atalho para edição no canto superior. A tela de configurações, na Figura 18, concentra a personalização visual global prevista nos RF-18 e RF-19: a alternância entre os temas claro e escuro e o ajuste de tipografia e de cores da interface. Na especificação consolidada no Capítulo 5, essa personalização passou a se organizar como uma biblioteca de paletas, cada uma com cores e tipografia próprias, e a escolha do estilo claro, escuro ou automático foi deslocada para o menu da conta; o esquema registra a concepção inicial.

Figura 17 – Esquema do perfil do próprio usuário



Fonte: Elaborada pelo autor

Figura 18 – Esquema da tela de configurações de aparência



Fonte: Elaborada pelo autor

6.2 Projeto de Dados

A persistência de informações utiliza o Sistema Gerenciador de Banco de Dados (SGBD) **PostgreSQL 16**, executado em ambiente isolado através de **Docker**. A manipulação e a administração dos dados são realizadas por meio do **DBeaver**, cliente gráfico de administração de bancos de dados relacionais.

O banco de dados foi estruturado para suportar tipos de dados modernos e performáticos, incluindo:

- **UUID (Universally Unique Identifier):** Utilizado para todas as chaves primárias e estrangeiras;
- **JSONB:** Para o armazenamento eficiente de configurações dinâmicas de interface;
- **Varchar(n):** Para cadeias de caracteres de tamanho limitado, com restrição imposta no nível do banco;
- **Text:** Para cadeias de caracteres extensas, sem limite explícito de tamanho;

- **Timestamp:** Para o registro cronológico de eventos e criações;
- **Float:** Para valores numéricos fracionários, utilizados em ajustes tipográficos;
- **Boolean:** Para o registro de estados binários (verdadeiro/falso);
- **Enum:** Para campos com domínio restrito a um conjunto pré-definido de valores.

6.2.1 Mapeamento Objeto-Relacional

O mapeamento entre o modelo de domínio (TypeScript) e o modelo relacional (PostgreSQL) é gerenciado pelo **Prisma ORM**. Para garantir a integridade dos dados e a fidelidade aos requisitos, adotaram-se as seguintes regras de mapeamento:

- **Idioma dos identificadores:** Os nomes conceituais empregados no texto desta monografia estão em português (Elemento, EstiloDeElemento, TipoElemento, Categoria, Coleção), ao passo que os identificadores do código e do banco permanecem em inglês (`Element`, `ElementStyle`, `ElementType`, `categories`, `collections`). A correspondência entre uns e outros é direta, e adotou-se essa separação para que o texto permaneça legível em português sem divergir dos nomes efetivamente encontrados no código-fonte;
- **Nomenclatura:** As entidades em *camelCase* no código são mapeadas para tabelas em *snake_case* no banco de dados através da diretiva `@@map`;
- **Tipos Físicos:** Os atributos do tipo `String` no Prisma são explicitamente mapeados para os tipos físicos `VARCHAR(n)` ou `TEXT` no PostgreSQL através das diretivas `@db.VarChar(n)` e `@db.Text`, conforme o tamanho esperado do conteúdo. Esses tamanhos são tetos físicos, por vezes mais largos que os limites dos requisitos; os limites de cada requisito, como os 100 caracteres do nome de uma categoria (RF-06), são aplicados pela validação dos dados na API (Seção 7.3);
- **Chaves Primárias:** Todos os modelos possuem um identificador único do tipo UUID gerado automaticamente;

- **Relações 1:N:** Implementadas via chaves estrangeiras (*foreign keys*), obrigatórias quando o registro não existe sem o pai, como o item sem a coleção, e opcionais quando a ligação pode faltar, como o *layout* de uma coleção; a integridade referencial é garantida por regras de *cascade* ou de *set null*, conforme a semântica da relação, e o *set null* só se aplica às opcionais;
- **Relações N:N:** Implementadas via tabela associativa, como `collection_follows`, que vincula usuários às coleções que acompanham e é protegida por chave única composta, impedindo a duplicação do par, e `_CollectionToTag`, que liga as coleções às suas tags;
- **Relações Reflexivas:** Aparecem em dois contextos. Nas interações sociais, a amizade é mapeada como relação nomeada, para distinguir os papéis de remetente e destinatário. No editor de *layouts*, tanto `layouts` quanto `elements` referenciam a própria tabela para registrar de qual registro a cópia se originou, com regra *set null*, de modo que a cópia nunca depende do original para existir;
- **Índices:** Foram criados índices secundários sobre as chaves estrangeiras de maior frequência de consulta, visando a otimização das operações de leitura.

Cabe registrar que nem toda regra de unicidade prevista nos requisitos está expressa como restrição do banco. As unicidades de nome de categoria e de nome de *layout*, ambas por usuário, são garantidas por chave única. Já a proibição de dois itens homônimos na mesma coleção e de duas coleções homônimas na mesma categoria é verificada na camada de lógica de negócio, antes da escrita. A diferença decorre da ordem em que os requisitos foram implementados e não altera o comportamento observado pelo usuário, mas convém explicitá-la, pois a leitura isolada do esquema não a revelaria.

6.2.2 Estrutura das Tabelas no Banco de Dados

Nesta seção, descreve-se o padrão adotado para a nomenclatura dos objetos do banco de dados, incluindo chaves primárias, chaves estrangeiras e índices únicos.

A padronização visa garantir a manutenibilidade do sistema e a integridade referencial. Foi convencionado que os nomes dos objetos devem obedecer aos padrões definidos na Tabela 1.

Tabela 1 – Convenção para Nome dos Objetos no Banco de Dados

Objeto	**Padrão Adotado (com exemplo)**
Chave Primária	nome_da_tabela_pkey (ex.: `users_pkey`)
Chave Estrangeira	nome_da_tabela_nome_do_campo_fkey (ex.: `elements_layout_id_fkey`)
Chave Única (Unique)	nome_da_tabela_nome_do_campo_key (ex.: `users_email_key`)
Índices	nome_da_tabela_nome_do_campo_idx (ex.: `elements_layout_id_idx`)

Fonte: Elaborada pelo autor

Para melhor compreensão da arquitetura de dados, as 15 tabelas declaradas no esquema estão consolidadas na Tabela 2, ao lado da tabela associativa `_CollectionToTag`, gerada pelo próprio Prisma e descrita junto à tabela `tags`. A Tabela 2 serve como índice para o detalhamento físico apresentado na subseção seguinte. A ordem de apresentação acompanha os pacotes do modelo de domínio definidos no Capítulo 5.

Tabela 2 – Tabelas Identificadas neste Trabalho

Tabela do Banco de Dados	**Referência no Documento**
users	Tabela 3
user_sessions	Tabela 5
email_verification_codes	Tabela 4
password_reset_tokens	Tabela 6
categories	Tabela 7
layouts	Tabela 8
collections	Tabela 9
tags e _CollectionToTag	Tabela 10
elements	Tabela 11
items	Tabela 12
notifications	Tabela 13
event_logs	Tabela 14
friendships	Tabela 15
collection_follows	Tabela 16
appearance_presets	Tabela 17

Fonte: Elaborada pelo autor

6.2.2.1 Detalhamento da Estrutura das Tabelas

Nesta subseção, apresenta-se o detalhamento físico das 15 tabelas declaradas no esquema do banco de dados do sistema, especificando tipos de dados, chaves e restrições. Para a leitura das tabelas de detalhamento, foram adotadas as seguintes convenções de cabeçalho:

- **Campo:** Nome técnico do atributo mapeado na tabela do banco de dados;
- **Tipo:** Tipo de dado nativo do PostgreSQL. Para cadeias de caracteres, indica-se *Varch.(n)* para campos com limite, onde n é o tamanho máximo, ou *Text* para campos sem restrição;
- **Obr.?:** Indica se o campo é de preenchimento obrigatório (X) ou opcional (vazio);
- **PK?:** Indica se o campo compõe a Chave Primária (*Primary Key*) da tabela;

- **Tab.:** Em caso de Chave Estrangeira (*Foreign Key*), indica a tabela de referência;
- **Cmp.:** Indica o campo específico de referência na tabela estrangeira;
- **Grp.:** Identifica o grupo de Chave Única (*Unique Key*) ao qual o campo pertence (ex: UK_01);
- **Ord.:** Define a sequência do campo dentro de uma composição de chave única ou índice.

Tabela 3 – Usuário (users)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_user	UUID	X	X				
user_name	Varch.(100)	X					
user_code	Varch.(110)	X				UK_01	1
email	Varch.(150)	X				UK_02	1
password	Varch.(255)	X					
bio	Varch.(500)						
profile_photo	Varch.(500)						
profile_photo_original	Varch.(500)						
profile_photo_crop_x, _y, _width, _height	Int						
profile_photo_color	Varch.(7)						
cover_image	Varch.(500)						
cover_original	Varch.(500)						
cover_crop_x, _y, _width, _height	Int						
cover_color	Varch.(7)						
registered_at	Time.	X					
email_verified_at	Time.						
default_layouts_version	Int	X					
onboarding_tour_at	Time.						
onboarding_checklist_hidden_at	Time.						
onboarding_tips_seen	Text[]	X					

Fonte: Elaborada pelo autor

O campo `user_code` merece destaque: trata-se do identificador público do usuário, gerado automaticamente no cadastro (RF-01) e imutável, utilizado nas

buscas e nas solicitações de amizade (RF-20). Sua unicidade é garantida em toda a plataforma, de modo independente da unicidade do endereço de e-mail.

A foto e a capa do perfil seguem o mesmo arranjo das imagens da categoria (RF-05 e RF-06): a coluna principal guarda o recorte exibido, a coluna `_original` guarda o arquivo enviado, e as quatro colunas `_crop_*` registram a área recortada, o que permite refazer o enquadramento sem novo envio. As colunas `_color` guardam a cor usada enquanto a imagem não existe.

O `email_verified_at` registra quando o usuário confirmou o e-mail digitando o código recebido (RF-01). Enquanto é nulo, a conta não recebe sessão no login. Guardar a data, e não apenas um indicador, informa também quando a confirmação ocorreu; as contas que já existiam quando a confirmação foi introduzida receberam, na migração, a data do próprio cadastro.

Já o `default_layouts_version` registra até que versão do conjunto de *layouts* padrão (RF-01) a conta já recebeu. Cada *layout* padrão declara a versão em que passou a existir, e a conta recebe, na abertura da biblioteca, os que forem mais novos que o seu número. Como a versão só aumenta, um *layout* padrão excluído de propósito não reaparece, o que aconteceria caso a verificação se apoiasse na simples presença dos modelos pelo nome. As três colunas `onboarding_*` guardam o estado do guia de primeiros passos (Seção 7.5): quando o tour inicial foi concluído ou pulado, quando a lista de primeiros passos foi escondida e quais dicas de tela já foram vistas.

Tabela 4 – Código de Confirmação de E-mail (email_verification_codes)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_email_verification_code	UUID	X	X				
code_hash	Varch.(64)	X					
attempts	Int	X					
created_at	Time.	X					
expires_at	Time.	X					
used_at	Time.						
user_id	UUID	X		users	id_user		

Fonte: Elaborada pelo autor

Sustenta a confirmação do e-mail no cadastro (RF-01). Como no *token* de

redefinição de senha, guarda-se apenas o *hash* do código de 6 dígitos. O campo `attempts` conta as tentativas erradas, que se esgotam em cinco; `expires_at` fixa a validade de 15 minutos; e `used_at` marca o código consumido ou substituído, pois emitir um código novo aposenta os anteriores, de modo que exista apenas um válido por vez.

Tabela 5 – Sessão de Usuário (user_sessions)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_session	UUID	X	X				
created_at	Time.	X					
revoked_at	Time.						
user_id	UUID	X		users	id_user		

Fonte: Elaborada pelo autor

Esta tabela é o que torna possível revogar um acesso antes de seu prazo de expiração. O identificador da sessão viaja no *token* emitido no login, e a cada requisição autenticada o sistema confere se o registro correspondente ainda tem `revoked_at` nulo. Sem esse controle, um *token* permaneceria válido até expirar, e o *logout* seria apenas uma formalidade do lado do navegador (RF-02).

Tabela 6 – Token de Redefinição de Senha (password_reset_tokens)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_password_reset_token	UUID	X	X				
token_hash	Varch.(64)	X				UK_01	1
created_at	Time.	X					
expires_at	Time.	X					
used_at	Time.						
user_id	UUID	X		users	id_user		

Fonte: Elaborada pelo autor

Sustenta o fluxo de recuperação de senha (RF-04). Armazena-se o *hash* do *token*, e não seu valor original, de modo que o acesso à base não permita forjar um link de redefinição válido. Os campos `expires_at` e `used_at` implementam, respectivamente, o prazo de validade e o uso único exigidos pelo requisito.

Alguns campos desta tabela envolvem decisões que a leitura isolada do esquema não revelaria. O `icon` guarda o emoji da categoria, exibido sobre o banner

Tabela 7 – Categoria (categories)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_category	UUID	X	X				
name	Varch.(150)	X				UK_01	1
type	Varch.(50)						
description	Text						
icon	Varch.(500)						
cover_image	Varch.(500)						
cover_original	Varch.(500)						
cover_crop_x	Int						
cover_crop_y	Int						
cover_crop_width	Int						
cover_crop_height	Int						
background_color	Varch.(7)						
created_at	Time.	X					
user_id	UUID	X		users	id_user	UK_01	2

Fonte: Elaborada pelo autor

do cartão e no modo lista. A imagem do banner tem colunas próprias porque, na categoria, o emoji e a imagem coexistem (RF-06): o `cover_image` guarda o recorte feito pelo usuário, já na proporção do banner, que é o que os cartões carregam, e o `cover_original` guarda o arquivo enviado, intacto, que nenhuma tela exibe. Os quatro campos `cover_crop_*` registram a área recortada, em pixels do original. É esse par, original e área, que permite reabrir o enquadramento onde ele estava e refazê-lo sem novo envio; os campos da área ficam nulos quando a capa é anterior ao recorte, e nesse caso o enquadramento reabre centralizado. As colunas de arquivo têm 500 caracteres porque guardam a URL pública completa do arquivo armazenado.

Já o `background_color` guarda a cor em notação hexadecimal, e não como índice de uma paleta do sistema. A consequência é que a cor pertence ao usuário, e não ao tema: ela permanece a mesma na alternância entre os modos claro e escuro, comportamento adotado também por ferramentas como o Notion e o Trello nas cores de etiqueta. Como o nome da categoria assenta sobre o banner, seja sobre a cor, seja sobre a capa, a legibilidade fica a cargo da interface: um escurecimento suave na metade inferior do banner e uma sombra discreta no texto branco garantem a

leitura sobre qualquer fundo (RNF-05), sem desenhar uma faixa sobre a imagem.

Tabela 8 – Layout Customizado (layouts)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_layout	UUID	X	X				
name	Varch.(150)	X				UK_01	2
description	Text						
purpose	Enum	X					
background	Varch.(24)						
created_at	Time.	X					
source_layout_id	UUID			layouts	id_layout		
user_id	UUID	X		users	id_user	UK_01	1

Fonte: Elaborada pelo autor

Três campos desta tabela sustentam requisitos do Editor de Layout. O `purpose` distingue as três finalidades que um *layout* pode ter: *template* reutilizável, criado pelo próprio usuário; *template* importado, resultante do compartilhamento (RF-16); e cópia individual de item, gerada quando o usuário personaliza a ficha de um item específico (RF-17). Já o `source_layout_id` registra de qual *layout* a cópia se originou, na duplicação (RF-14), na importação (RF-16) e na ficha própria de um item (RF-17). Nas duas primeiras, a regra *set null* faz com que excluir o original apenas zere a referência e preserve a cópia. Com as fichas próprias, a aplicação é mais estrita: recusa a exclusão de um *layout* enquanto ele estiver associado a coleções ou tiver originado fichas de item, e a desassociação prevista no RF-14 é que zera antes essas referências, depois de dar uma ficha própria a cada item que ainda exibia o *layout*.

Por fim, o `background` guarda o fundo da ficha (RF-14): uma cor hexadecimal ou um degradê de duas cores com ângulo e ponto de início, gravado em formato compacto e validado, e nunca como CSS livre, o que abriria espaço para injeção de estilo na página de quem visita a ficha. É coluna do *layout*, e não propriedade de um Elemento, porque não pertence a nenhum deles: é a superfície sobre a qual todos são desenhados. Como a ficha própria de um item também é um registro desta tabela (RF-17), a mesma coluna atende aos dois casos, e o valor nulo corresponde ao fundo branco padrão.

Como `user_id` é obrigatório, um *layout* não existe sem dono, e é isso que define onde vivem os *layouts* padrão oferecidos pelo sistema: eles são criados na biblioteca de cada usuário no momento do cadastro (RF-01), com a finalidade de *template* reutilizável, do mesmo modo que as paletas predefinidas de aparência (RF-18). A partir daí comportam-se como qualquer outro *layout* do usuário, podendo ser associados a coleções, editados ou duplicados.

Apenas os *layouts* de finalidade *template*, próprios ou importados, aparecem na biblioteca e podem ser associados a coleções. As cópias individuais de item ficam de fora dessa listagem, pois pertencem à ficha de um item específico e não fazem sentido como modelo a reutilizar.

Tabela 9 – Coleção (collections)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_collection	UUID	X	X				
name	Varch.(150)	X					
description	Text						
privacy	Enum	X					
icon	Varch.(500)						
icon_original	Varch.(500)						
icon_crop_x	Int						
icon_crop_y	Int						
icon_crop_width	Int						
icon_crop_height	Int						
background_color	Varch.(7)						
created_at	Time.	X					
category_id	UUID	X		categories	id_cat.		
layout_id	UUID			layouts	id_lay.		

Fonte: Elaborada pelo autor

Na coleção, o ícone é emoji **ou** imagem, e escolher uma forma descarta a outra (RF-07). Por isso, ao contrário da categoria, não há coluna própria para a imagem: o recorte quadrado feito pelo usuário fica no próprio `icon`, e o `icon_original` e os campos `icon_crop_*` cumprem o mesmo papel que têm na capa da categoria, permitindo refazer o enquadramento sem novo envio.

A tabela `tags` é o vocabulário compartilhado do RF-07: o `name` é único no sistema inteiro, e não por usuário, de modo que a mesma palavra ocupa uma única

Tabela 10 – Tag de Coleção (tags)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_tag	UUID	X	X				
name	Varch.(30)	X				UK_01	1
created_at	Time.	X					

Fonte: Elaborada pelo autor

linha, qualquer que seja a conta que a escreveu. É isso que permite ao formulário sugerir as tags já existentes. O nome é gravado já na forma canônica (minúsculas, sem espaços nas pontas e sem acentuação), pela mesma rotina que normaliza o tema da categoria; gravar a forma original quebraria a unicidade, pois 'Fantasia' e 'fantasia' seriam duas linhas. O limite de 30 caracteres, menor que os 50 do tema, reflete o fato de a tag ser um rótulo curto, e várias delas dividirem a largura de um cartão.

A ligação entre coleções e tags é muitos-para-muitos e fica na tabela associativa `_CollectionToTag`, criada e mantida pelo próprio Prisma, com as colunas `A` (referência a `collections`) e `B` (referência a `tags`). É o único nome do esquema fora do padrão *snake_case*, justamente por ser gerado pelo ORM; em troca, as tags de uma coleção são lidas diretamente como lista, sem um nível intermediário de objetos. Excluir uma coleção desfaz apenas a ligação: a tag permanece no vocabulário, ao contrário do tema, que é derivado das categorias existentes e desaparece com a última que o utilizava.

Esta é a tabela central do diferencial da plataforma. Nela residem a geometria de cada Elemento (`pos_x`, `pos_y`, `width` e `height`), a camada em que ele se sobrepõe aos demais (`layer`), o conteúdo exibido (`content`) e toda a sua aparência (`style`). Essa concentração é o que permite tratar o Elemento como objeto visual editável, à maneira de um editor gráfico, em vez de separar a estrutura do dado que a preenche. O campo `type` corresponde à enumeração TipoElemento, com os seis valores fixos definidos no Capítulo 5.

A coluna `style` corresponde ao **EstiloDeElemento** do modelo de domínio, apresentado na Seção 5.4. Ali ele aparece como uma classe de valor composta no Elemento, com uma propriedade visual por atributo, o que torna visível o

Tabela 11 – Elemento do Layout (elements)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_element	UUID	X	X				
name	Varch.(100)	X					
type	Enum	X					
content	Text						
pos_x	Int	X					
pos_y	Int	X					
width	Int	X					
height	Int	X					
layer	Int	X					
style	JSONB						
source_element_id	UUID			elements	id_element		
layout_id	UUID	X		layouts	id_layout		

Fonte: Elaborada pelo autor

que o editor oferece; aqui ele é gravado como um único documento `JSONB`. A correspondência é direta, e os nomes das propriedades no documento são os mesmos atributos da classe.

A divisão entre coluna e `JSONB` obedece a um critério: é coluna aquilo que o banco precisa consultar, ordenar ou indexar, e é `JSONB` aquilo que ele apenas guarda e devolve, cabendo ao navegador interpretar. Por esse critério, `content` é coluna, pois é o valor do item, e não sua aparência: é o que muda de um item para outro, o que a mescla de fichas transfere para o *layout* novo (RF-15) e o que a ordenação pela classificação, registrada entre os requisitos adiados (Subseção 5.2.4), precisará ler no banco. A cor de fundo de um Elemento, ao contrário, não é coluna: nenhuma consulta pergunta quais Elementos são azuis. Uma coluna por propriedade traria o inconveniente oposto: o editor cresce em propriedades visuais, e cada nova opção exigiria alterar o esquema do banco. Ser `JSONB` não afrouxa a validação: o objeto é validado atributo a atributo na camada de aplicação, e uma chave desconhecida faz a requisição ser rejeitada como qualquer outra.

A geometria é expressa em inteiros porque a unidade não é o *pixel*, e sim a milésima parte da largura do quadro da ficha. Como a ficha ocupa a largura disponível e escala junto com ela, uma medida absoluta significaria coisas diferentes

em cada tela; com a unidade relativa, mudar a largura disponível altera apenas a escala, e nenhum Elemento se reposiciona.

O campo `layer` não acompanha `pos_x` e `pos_y` como uma terceira coordenada, e por isso não se chama `pos_z`: a ficha é plana, e o que ele determina é a ordem de pintura entre Elementos que ocupam a mesma região. Sem ele, essa ordem viria da ordem de retorno da consulta, que o SGBD não garante, e dois Elementos sobrepostos poderiam trocar de posição entre um carregamento e outro.

Tabela 12 – Item da Coleção (items)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_item	UUID	X	X				
name	Varch.(255)	X					
subtitle	Varch.(150)						
thumbnail	Varch.(500)						
cover_image	Varch.(500)						
cover_original	Varch.(500)						
cover_crop_x, _y, _width, _height	Int						
background_color	Varch.(7)						
notes	Text						
created_at	Time.	X					
collection_id	UUID	X		collections	id_coll.		
layout_id	UUID			layouts	id_layout		

Fonte: Elaborada pelo autor

O item é a única das três entidades que recebe cor de fundo sem receber `icon`: já dispõe de `thumbnail` e `cover_image` para imagem, e um terceiro campo de imagem seria repetição. A cor atende justamente ao item sem miniatura, que de outro modo apareceria como um retângulo vazio na listagem da coleção. A capa do cartão segue o arranjo de recorte da capa da categoria, com o original e a área guardados, e o `subtitle` é a linha exibida sob o nome no cartão, que a busca da listagem também consulta (RF-12).

A tabela distingue dois papéis de usuário: `user_id` é o destinatário da notificação e `author_id` é quem originou o evento, campo que permite compor mensagens como "João Silva enviou uma solicitação de amizade". As chaves `friendship_id` e `event_log_id` vinculam a notificação ao fato que a gerou, conforme o tipo. Quanto

Tabela 13 – Notificações (notifications)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_notification	UUID	X	X				
message	Text	X					
type	Enum	X					
is_read	Bool	X					
read_at	Time.						
cleared_at	Time.						
created_at	Time.	X					
user_id	UUID	X		users	id_user		
author_id	UUID			users	id_user		
friendship_id	UUID			friendships	id_friendship		
event_log_id	UUID			event_logs	id_event_log		

Fonte: Elaborada pelo autor

ao estado de leitura, `is_read` indica a situação atual, `read_at` registra o instante em que a leitura ocorreu e `cleared_at` marca as notificações que o usuário optou por limpar da listagem, ocultando-as de sua área sem excluir o registro, o que preserva o histórico (RF-24).

Tabela 14 – Log de Eventos (event_logs)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_event_log	UUID	X	X				
event_type	Enum	X					
item_name	Varch.(255)	X					
description	Text						
created_at	Time.	X					
user_id	UUID	X		users	id_user		
collection_id	UUID	X		collections	id_coll.		
item_id	UUID			items	id_item		

Fonte: Elaborada pelo autor

Esta tabela alimenta o *feed* de atualizações (RF-22). O vínculo com a coleção é obrigatório, pois é por ele que se determina quais usuários devem receber o evento; já o vínculo com o item é opcional e adota a regra *set null*. A razão está no campo `item_name`: como eventos de remoção precisam permanecer legíveis depois que o item deixa de existir, guarda-se o nome do item no próprio registro do evento.

Assim, excluir um item zera a referência sem apagar o histórico, e o *feed* continua exibindo a informação de forma compreensível. Os campos estruturados são a fonte primária dos dados apresentados; `description` guarda apenas texto auxiliar.

Tabela 15 – Relacionamentos (friendships)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_friendship	UUID	X	X				
pair_key	Varch.(73)	X				UK_01	1
status	Enum	X					
sent_at	Time.	X					
responded_at	Time.						
sender_id	UUID	X		users	id_user		
receiver_id	UUID	X		users	id_user		

Fonte: Elaborada pelo autor

A unicidade desta tabela recai sobre `pair_key` e não sobre o par `sender_id`–`receiver_id`, e a diferença é deliberada. Uma chave única composta pelas duas colunas impediria a repetição da mesma solicitação, mas não a solicitação inversa: se A convida B e, antes da resposta, B convida A, os dois registros seriam aceitos, e passariam a existir dois vínculos entre as mesmas duas pessoas. O `pair_key` resolve isso ao combinar os dois identificadores em uma ordem fixa, independentemente de quem enviou, de modo que qualquer par de usuários corresponda a um único valor possível.

Tabela 16 – Seguidores de Coleção (collection_follows)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_follow_collection	UUID	X	X				
is_active	Bool	X					
started_at	Time.	X					
ended_at	Time.						
user_id	UUID	X		users	id_user	UK_01	1
collection_id	UUID	X		collections	id_coll.	UK_01	2

Fonte: Elaborada pelo autor

Cada paleta pertence a um usuário, e seu nome não se repete entre as paletas da mesma conta, o que a chave única composta por `user_id` e `name` garante; usuários diferentes podem ter paletas de mesmo nome. O campo `subcolors`

Tabela 17 – Preferências de Aparência (appearance_presets)

Campo	**Tipo**	**Obr.?**	**PK?**	**Tab.**	**Cmp.**	**Grp.**	**Ord.**
id_appearance_preset	UUID	X	X				
name	Varch.(50)	X				UK_01	2
primary_color	Varch.(9)	X					
subcolors	JSONB	X					
system_theme	Enum	X					
font_family	Varch.(100)	X					
title_size	Float	X					
text_size	Float	X					
line_height	Float	X					
letter_spacing	Float	X					
font_weight	Int	X					
font_italic	Bool	X					
is_active	Bool	X					
is_default	Bool	X					
created_at	Time.	X					
updated_at	Time.	X					
user_id	UUID	X		users	id_user	UK_01	1

Fonte: Elaborada pelo autor

emprega o tipo JSONB para guardar as doze subcores em ordem fixa, na qual cada posição corresponde a um papel na interface: fundo da página, cartões, títulos, texto nos cartões, barra lateral, cabeçalho e modais, seguidos do texto no fundo da página, na barra lateral, no cabeçalho, nos modais e sobre o botão de destaque. As cinco últimas foram acrescentadas ao fim da lista, para que as sete primeiras não mudassem de posição. A quantidade exata é garantida pela validação na camada de aplicação, pois uma paleta é sempre completa (RF-18). O `system_theme` guarda o estilo em vigor enquanto a paleta está em uso: nasce do lado para o qual ela foi desenhada e é atualizado quando o usuário altera o estilo no menu da conta. A tipografia (`font_family`, `title_size`, `text_size`, `line_height`, `letter_spacing`, `font_weight` e `font_italic`) também é gravada em cada paleta (RF-19). Os dois indicadores booleanos têm papéis distintos: `is_active` aponta a paleta em uso, e um índice único parcial garante uma única por usuário; `is_default` indica apenas que a paleta foi oferecida pelo sistema no cadastro (RF-01), sem protegê-la de edição ou exclusão. No cadastro, fica ativa a paleta ‘Claro’ ou a

‘Escuro’, conforme o tema do sistema operacional naquele momento, e a conta nunca fica sem paleta porque a paleta em uso não pode ser excluída.

7 IMPLEMENTAÇÃO

Este capítulo descreve como o sistema especificado nos Capítulos 5 e 6 foi construído. São apresentados o ambiente de desenvolvimento, o servidor de dados, o servidor de aplicação, a organização do código do *backend* e da aplicação cliente, a forma como a implementação foi verificada ao longo do desenvolvimento e as limitações conhecidas da versão atual. As ferramentas empregadas já foram caracterizadas na Seção 3.3; aqui o foco recai sobre as versões adotadas, a configuração do ambiente e os mecanismos implementados.

7.1 Ambiente de Desenvolvimento

O sistema é composto por dois projetos independentes, versionados em repositórios Git distintos: o *backend*, uma API REST escrita em TypeScript sobre o NestJS, e o *frontend*, uma aplicação de página única escrita em TypeScript com React. Ambos são executados sobre o Node.js e têm suas dependências gerenciadas pelo pnpm, cuja versão fica registrada no próprio arquivo `package.json` de cada projeto, de modo que qualquer máquina que instale as dependências utilize a mesma versão do gerenciador. O Quadro 61 reúne as principais tecnologias e as versões empregadas.

Durante o desenvolvimento, o sistema é executado localmente por três processos: o contêiner do PostgreSQL, na porta 5432; a API, na porta 3000; e o servidor de desenvolvimento do Vite, na porta 3001. A porta do *frontend* é fixada na configuração do Vite por uma razão funcional, e não de conveniência: é a origem que a API autoriza a realizar requisições pelo navegador, e servir a interface em outra porta faria o navegador bloquear todas as chamadas por violação da política de CORS.

Quadro 61 – Tecnologias e versões utilizadas na implementação

Camada	**Tecnologia e versão**
Ambiente de execução	Node.js 22.15 e gerenciador de pacotes pnpm 11.20
Linguagem	TypeScript 5.7 no *backend* e 6.0 no *frontend*
Servidor de aplicação	NestJS 11, com Passport e JWT para autenticação, class-validator para validação, `@nestjs/throttler` para limitação de requisições e Nodemailer para envio de e-mail
Persistência	PostgreSQL 16, executado em contêiner Docker, e Prisma ORM 7.8 com o adaptador para o *driver* `pg`
Armazenamento de arquivos	Disco local ou Cloudflare R2, este acessado pelo SDK S3 da AWS
Interface	React 19.2, Vite 8, React Router 8, TanStack Query 5 e Axios 1.19
Testes	Jest 30 com ts-jest
Qualidade de código	ESLint e Prettier no *backend*; Oxlint no *frontend*

Fonte: Elaborada pelo autor

A configuração do *backend* é feita por variáveis de ambiente, lidas de um arquivo `.env`. O repositório inclui o modelo `.env.example`, que relaciona as variáveis sem seus valores: a cadeia de conexão com o banco (`DATABASE_URL`), o segredo de assinatura dos *tokens* (`JWT_SECRET`), o endereço do *frontend* (`FRONTEND_URL`), os parâmetros do servidor de e-mail (`SMTP_*`) e as credenciais do armazenamento em nuvem (`R2_*`). Manter esses valores fora do código evita que credenciais sejam versionadas e permite que o mesmo código seja executado em ambientes diferentes apenas com a troca do arquivo de configuração.

7.2 Servidor de Dados

Em desenvolvimento, o PostgreSQL é provisionado pelo Docker Compose, a partir do arquivo `docker-compose.dev.yml` mantido no repositório do *backend*; o `docker-compose.yml` do mesmo repositório é o da implantação, descrito no Capítulo 9. O arquivo de desenvolvimento declara um único serviço, baseado na imagem oficial `postgres:16`, que expõe a porta 5432 e grava os dados em um volume nomeado. O volume desacopla os dados do ciclo de vida do contêiner:

recriar ou atualizar o contêiner não apaga o banco. O serviço conta ainda com uma verificação de saúde que executa o utilitário `pg_isready` periodicamente, o que permite saber quando o banco está de fato pronto para aceitar conexões, e não apenas quando o contêiner foi iniciado.

O esquema do banco é definido com o Prisma e fica dividido em oito arquivos no diretório `prisma/schema`: um arquivo principal, com a configuração, e sete arquivos por assunto (usuário, coleções, *tags*, *layout*, aparência, interação social e notificações). A divisão aproxima-se dos pacotes do modelo de domínio apresentados no Capítulo 5, com duas diferenças: as tabelas do pacote Sistema, de sessões, de redefinição de senha e de confirmação de e-mail, ficam no arquivo de usuário, ao qual se ligam, e as *tags* ganharam arquivo próprio por formarem um vocabulário compartilhado entre todas as contas. Cada alteração no esquema gera uma migração versionada, contendo o SQL correspondente; no momento da redação deste trabalho, o histórico reúne 54 migrações, que reconstroem o banco do zero de forma determinística. Quando uma mudança de modelo afetava dados já existentes, a migração foi escrita para convertê-los em vez de descartá-los. Foi o caso da redução dos tipos de Elemento: como o PostgreSQL não permite remover valores de um tipo enumerado, a migração cria o novo tipo, converte a coluna e só então descarta o tipo antigo, preservando o conteúdo de todos os Elementos.

O cliente do Prisma, que oferece acesso tipado às tabelas, é gerado automaticamente a cada instalação de dependências, por meio do *script* `postinstall`. Na aplicação, ele é encapsulado em um serviço do NestJS (`PrismaService`), injetado nos módulos que acessam o banco. As operações que precisam gravar vários registros de forma indivisível são executadas em transações. É o caso da criação da ficha própria de um item (RF-17), que copia o *layout* de origem e todos os seus Elementos: ou a cópia é criada por inteiro, ou nada é gravado. O mapeamento entre o modelo de objetos e as tabelas está detalhado no Capítulo 6.

7.3 Servidor de Aplicação

A API é uma aplicação NestJS que atende na porta 3000, sem prefixo de rota. No momento da redação, ela é composta por 19 controladores, que expõem

100 rotas: 36 de consulta (GET), 31 de criação ou de ação (POST), 16 de alteração parcial (PATCH), uma de substituição (PUT) e 16 de exclusão (DELETE). Toda requisição percorre uma sequência fixa de etapas antes de alcançar a regra de negócio, configurada nos arquivos `main.ts` e `app.module.ts`:

1. **CORS:** somente a origem informada em `FRONTEND_URL` é autorizada a chamar a API pelo navegador.
2. **Limitação de requisições:** um *guard* global limita cada endereço de origem a 100 requisições por minuto. As rotas sensíveis sobrescrevem esse limite: o *login* aceita cinco tentativas por minuto, e a solicitação de recuperação de senha, três a cada dez minutos, já que cada chamada dispara um e-mail. O endereço de origem é o do cliente mesmo quando a API está atrás de um *proxy* reverso, pois a aplicação confia em exatamente um intermediário para ler o cabeçalho `X-Forwarded-For` (Capítulo 9).
3. **Autenticação:** um segundo *guard* global exige um *token* JWT válido em todas as rotas. Uma rota só fica aberta quando é marcada explicitamente como pública, o que faz com que o esquecimento dessa marcação resulte em uma rota fechada, e não em uma rota exposta sem que ninguém perceba.
4. **Validação da sessão:** além de verificar a assinatura e a validade do *token*, a estratégia de autenticação consulta a tabela de sessões e confirma que a sessão indicada no *token* não foi revogada. Com isso, o *logout* tem efeito imediato: um *token* copiado antes da saída deixa de funcionar no mesmo instante, ao custo de uma consulta ao banco por requisição autenticada.
5. **Validação dos dados:** o corpo, os parâmetros e a consulta de cada requisição são validados contra classes de transferência de dados (DTOs). A validação é estrita: além de rejeitar valores inválidos, ela rejeita qualquer campo não previsto, o que torna visível uma divergência de contrato entre *frontend* e *backend* no momento em que ela surge.
6. **Regra de negócio:** validada a requisição, o controlador repassa a operação ao serviço do módulo, que aplica as regras do domínio e acessa o banco pelo Prisma.

A ordem entre as etapas 2 e 3 é deliberada. Como os *guards* executam na ordem em que são registrados, a limitação de requisições atua antes da autenticação, de modo que um volume excessivo de chamadas é barrado antes de consumir a consulta ao banco que valida a sessão, que é justamente o recurso que se deseja proteger. Os erros produzidos em qualquer etapa são convertidos em respostas padronizadas por um filtro de exceções global, com mensagens em português que a interface exibe diretamente ao usuário.

7.3.1 Armazenamento de Arquivos

As imagens enviadas pelos usuários não são gravadas no banco de dados: o banco guarda apenas o caminho público do arquivo. O armazenamento em si é definido por uma interface com três operações, gravar, ler e remover, e por duas implementações dessa interface, uma que grava no disco da máquina e outra que envia o arquivo ao Cloudflare R2, serviço de armazenamento de objetos compatível com a API do Amazon S3. A escolha entre elas é feita em um único ponto, no módulo de armazenamento, a partir das credenciais presentes na configuração: havendo as variáveis do R2, os arquivos seguem para a nuvem; faltando qualquer uma delas, o sistema grava em disco e registra um aviso na inicialização. Como os demais módulos dependem apenas da interface, a troca de destino não exige alteração em nenhum controlador ou serviço. A leitura atende a uma única rota autenticada, que devolve à própria interface os arquivos já guardados, para que ela possa redesenhá-los no navegador, o que o armazenamento em nuvem não permite a uma página de outra origem; a rota recusa qualquer endereço que não pertença ao armazenamento, para não servir de intermediário a outros sites.

O formato das imagens é verificado a partir do conteúdo do arquivo, e não da declaração enviada pelo cliente. O cabeçalho `Content-Type` de um envio é informado pelo próprio navegador ou por quem monta a requisição, e pode ser falsificado trivialmente. Por isso, a aplicação lê a assinatura presente nos primeiros *bytes* do arquivo, que identifica os formatos JPEG e PNG exigidos pelos requisitos, e é essa assinatura que decide se o arquivo é aceito e qual extensão ele recebe. O nome do arquivo gravado é sempre gerado pelo servidor, o que impede que um nome malicioso enviado pelo cliente direcione a gravação para fora do diretório

previsto.

7.4 Organização do Código do *Backend*

O código do *backend* segue a organização modular do NestJS, conforme o Quadro 62. Cada funcionalidade do domínio corresponde a um módulo, e os módulos de domínio adotam a mesma estrutura interna: um diretório `controller`, que recebe as requisições HTTP; um diretório `service`, que concentra as regras de negócio; e um diretório `dto`, que define e valida os formatos de entrada. Os módulos de infraestrutura fogem desse molde: `mail` e `infra` não têm controlador, por não receberem requisições, e `storage` reúne seus poucos arquivos na raiz do módulo.

Quadro 62 – Estrutura de diretórios do *backend*

```
src/
  main.ts
  app.module.ts
  common/
    dto/
    filters/
    validators/
  modules/
    auth/
    user/
    category/
    collection/
    item/
    layout/
    appearance-preset/
    social/
    tag/
    notification/
    mail/
    storage/
    infra/database/prisma/
prisma/
  schema/
    migrations/
```

Fonte: Elaborada pelo autor

Os módulos espelham os pacotes do modelo de domínio descritos na Subseção 5.1.1: o pacote Usuário corresponde ao módulo `user`; o pacote Sistema, ao

módulo `auth`, que trata de *login*, *logout*, sessões e redefinição de senha, com exceção da confirmação do e-mail, que fica no módulo `user`, junto do cadastro que a exige; o pacote Coleções, aos módulos `category`, `collection` e `item`; o pacote Editor de Layout, ao módulo `layout`; o pacote Aparência, ao módulo `appearance-preset`; o pacote Interação Social, ao módulo `social`; e o pacote Notificações, ao módulo `notification`. O módulo `tag`, que consulta o vocabulário de *tags* para as sugestões do formulário de coleção (RF-07), pertence ao pacote Coleções. Os módulos `mail`, `storage` e `infra` não têm correspondente no domínio, pois oferecem infraestrutura, respectivamente o envio de e-mail, o armazenamento de arquivos e o acesso ao banco, a todos os demais. Os arquivos de teste ficam ao lado do código que verificam, identificados pelo sufixo `.spec.ts`.

Três decisões de implementação merecem registro por não serem visíveis no esquema do banco. A primeira diz respeito à ficha própria de um item (RF-17). Na primeira abertura do editor da ficha, que a interface faz logo após o cadastro do item, o serviço responsável cria um novo *layout*, de finalidade exclusiva daquele item, e copia para ele os Elementos do *layout* da coleção, guardando em cada cópia a referência ao original. Quando a coleção não possui *layout*, a cópia é criada vazia, e o usuário monta a ficha a partir do zero. A partir desse momento, qualquer alteração afeta apenas a cópia. A criação é idempotente: abrir o editor de novo devolve a ficha que já existe.

A segunda decisão diz respeito ao que acontece com as fichas quando o *layout* da coleção muda (RF-14 e RF-15). As fichas próprias não acompanham o *layout* novo por conta própria; o que o sistema oferece é a mescla, pedida pelo usuário para os itens que ele escolher. A regra é a mesma na interface, que mostra a prévia, e na API, que grava: cada Elemento do *layout* novo recebe no máximo um Elemento da ficha antiga, primeiro pela mesma origem, depois pelo mesmo tipo e nome e, por fim, pelo mesmo tipo na ordem de leitura; do antigo vem apenas o valor, e o que tinha valor sem encontrar lugar permanece na ficha. Para excluir um *layout* em uso, a API oferece ainda a desassociação: em uma única transação, dá uma ficha própria a cada item que ainda exibia o *layout*, solta-o das coleções e zera a referência das fichas que dele derivaram, de modo que a exclusão passa a ser permitida sem que nenhuma ficha mude.

A terceira decisão diz respeito ao conteúdo que toda conta recebe no cadastro: os *layouts* padrão (RF-14) e as paletas predefinidas (RF-18). Esse conteúdo não fica armazenado em registros de um usuário especial do sistema, mas definido no próprio código, nos arquivos `layouts-padrao.ts` e `paletas-padrao.ts`, e copiado para cada nova conta na mesma operação que cria o usuário. A razão é a portabilidade: guardados apenas no banco de desenvolvimento, esses modelos não existiriam em nenhuma outra instalação, e a funcionalidade pareceria defeituosa sem estar. Definidos em código, eles acompanham o repositório, e suas alterações ficam registradas no histórico de versões. As cópias, e não referências a um original compartilhado, são o que permite ao usuário editar e excluir esses modelos como os seus próprios, sem afetar os de outras pessoas. Cada *layout* padrão declara a versão do conjunto em que passou a existir, e cada conta guarda até que versão já recebeu (Capítulo 6). Assim, um padrão acrescentado depois chega também às contas existentes, na abertura da biblioteca. A entrega é controlada por uma atualização condicional no banco, que sobe a versão da conta na mesma transação que cria os *layouts* e apenas se ela ainda for a que foi lida. Desse modo, duas requisições simultâneas não produzem cópias em duplicidade: o próprio banco decide qual delas realiza a entrega.

7.5 Aplicação Cliente

O *frontend* é uma aplicação de página única, empacotada pelo Vite, cuja estrutura de diretórios é apresentada no Quadro 63. O diretório `pages` reúne as 20 telas da aplicação, uma por rota. Sete atendem ao acesso e ao acervo do usuário: cadastro, confirmação do e-mail, *login*, solicitação e redefinição de senha, categorias e coleções. Cinco tratam dos itens e dos *layouts*: listagem de itens, ficha do item, editor da ficha do item, biblioteca e editor de *layouts*. Duas tratam da aparência: biblioteca e editor de paletas. E seis atendem à interação social: amigos, coleções seguidas, Explorar, perfil de um usuário, que serve também ao próprio perfil, e a categoria e a coleção de outro usuário vistas em modo somente leitura. As notificações e as novidades do *feed* não têm tela própria: abrem a partir do sino do cabeçalho, comum a todas as telas autenticadas. A edição do perfil é feita em uma janela sobre o próprio perfil. As partes reutilizáveis dessas telas ficam em

`components`; a lógica de estado compartilhada entre telas, em `hooks`; as funções puras, como os cálculos de cor e de geometria do editor, em `lib`; as chamadas à API, em `api`; e os estilos, em `styles`, organizados em uma camada de variáveis visuais, uma de estilos base e uma de componentes.

Dois diretórios ficam fora dessa divisão. O `pagina-inicial` contém a página pública de apresentação da plataforma, a única acessível sem conta, que explica o funcionamento com um diagrama da relação entre o *layout*, a coleção e as fichas. Ela é escrita em React, como o restante, mas é transformada em HTML pronto durante a construção, de modo que o navegador a recebe sem depender de JavaScript; todos os demais endereços são atendidos pela aplicação de página única. O `onboarding` contém o guia de primeiros passos da área autenticada: um tour apresentado no primeiro acesso, que pode ser pulado e revisto, uma lista de primeiros passos cujo progresso é calculado a partir do que já existe na conta e dicas exibidas na primeira visita a cada tela. O que já foi visto fica registrado nos campos de *onboarding* da tabela de usuários (Capítulo 6), para que o guia não se repita em outro dispositivo.

Quadro 63 – Estrutura de diretórios do *frontend*

```
src/
  api/
  assets/
  components/
  dados/
  hooks/
  lib/
  onboarding/
  pages/
  pagina-inicial/
  styles/
  types/
```

Fonte: Elaborada pelo autor

7.5.1 Comunicação com a API

Toda a comunicação com o *backend* passa por uma única instância do Axios, configurada com dois interceptadores. O primeiro atua antes de cada requisição e acrescenta o *token* de acesso ao cabeçalho `Authorization`, formato exigido pela

estratégia de autenticação da API. O segundo atua sobre as respostas com erro: converte cada falha em uma mensagem exibível, dando preferência à mensagem em português enviada pelo servidor, e, quando a API responde que a sessão expirou ou foi revogada, remove o *token* armazenado e conduz o usuário de volta à tela de *login*. Centralizar esse tratamento evita que cada tela precise repetir a montagem do cabeçalho e a interpretação dos erros.

As leituras de dados são gerenciadas pelo TanStack Query, que guarda em memória as respostas já obtidas, identificadas por uma chave que inclui os parâmetros da consulta. Como a API recebe a ordenação e os filtros como parâmetros, cada combinação corresponde a uma requisição distinta; com a memória por chave, voltar a uma combinação já consultada exibe o resultado imediatamente. As operações de gravação, por sua vez, sinalizam quais consultas ficaram desatualizadas, e as listagens se atualizam sem que a tela precise solicitar os dados novamente. A movimentação de Itens entre Coleções vai além e é aplicada de forma otimista, como exige o RNF-06: o Item sai da lista da Coleção de origem antes da resposta da API, a lista é restaurada se o servidor recusar a operação, e, ao final, as listagens são relidas do servidor. No *logout*, essa memória é esvaziada junto com o *token*, para que nenhum dado da sessão encerrada fique disponível a quem utilizar o mesmo navegador em seguida.

7.5.2 Editor de Layouts

O editor de *layouts* é a parte mais extensa da aplicação cliente, e três decisões de implementação sustentam seu funcionamento. A primeira é a gravação automática. O editor não possui botão de salvar: cada alteração concluída, como soltar um Elemento após arrastá-lo ou terminar de editar um texto, é enviada à API após uma pausa de 400 milissegundos, intervalo que agrupa movimentos feitos em sequência em uma única requisição. A gravação automática só é segura porque vem acompanhada do desfazer e do refazer, acionados pelos atalhos `Ctrl+Z` e `Ctrl+Shift+Z` e apoiados em um histórico de 50 passos; sem eles, um movimento equivocado se tornaria permanente no instante em que fosse feito. Enquanto houver alteração ainda não gravada, o navegador avisa o usuário caso ele tente fechar a página.

A segunda decisão é a separação entre a identidade de um Elemento na tela e sua identidade no banco. Durante a edição, cada Elemento recebe um identificador gerado no próprio navegador, que nunca muda, e o identificador do banco é mantido à parte, em um mapa consultado apenas pela sincronização. Essa separação é o que permite desfazer uma exclusão: o Elemento restaurado precisa ser criado novamente no banco, com um novo identificador, mas permanece o mesmo na tela. A sincronização compara o estado da tela com uma cópia do último estado confirmado pelo servidor e envia somente as diferenças: cria os Elementos novos, altera apenas os campos modificados dos existentes e remove os que deixaram de existir. Quando uma requisição falha, a cópia de referência não é atualizada, de modo que a tentativa seguinte reenvia automaticamente o que ficou pendente.

A terceira decisão diz respeito ao desenho dos Elementos do tipo forma, que são construídos como gráficos vetoriais (SVG) em vez de caixas estilizadas por CSS. O SVG permite aplicar preenchimento, cor, espessura e tracejado da borda a qualquer uma das doze silhuetas disponíveis, o que o recorte de caixas por CSS não oferece. A área de desenho de cada forma adota as mesmas dimensões do Elemento, de modo que a espessura do traço é expressa na mesma unidade relativa das posições e dos tamanhos, descrita no Capítulo 6, e acompanha a escala da ficha quando a largura disponível muda.

7.5.3 Aparência e Tipografia

A aparência da interface é aplicada por meio de variáveis CSS. Uma função pura converte a paleta em uso no conjunto de variáveis que colore a interface, e a mesma função pinta a área de prévia do editor de paletas e as miniaturas da biblioteca, o que garante que a prévia nunca diverge do resultado aplicado. Quando o estilo global em vigor é o oposto ao da paleta em uso, a claridade de todas as cores é invertida no espaço de cor OKLCH, que, ao contrário do HSL, representa a claridade de acordo com a percepção humana, preservando o matiz e a saturação de cada cor.

A preferência de estilo é persistida no banco, por fazer parte da paleta, e replicada no armazenamento local do navegador. A cópia local atende aos momentos em que a API não pode responder: as telas anteriores ao *login*, que ainda não

sabem de qual conta se trata, e o primeiro quadro de cada carregamento da página. Ela é lida por um pequeno *script* no cabeçalho do documento, executado antes do carregamento dos estilos, o que evita que a página seja exibida por um instante com o estilo errado. Havendo divergência entre as duas fontes, prevalece o valor do banco.

As fontes tipográficas oferecidas além das do sistema operacional são servidas pelo próprio site, e não carregadas de serviços externos como o Google Fonts. Carregar uma fonte de um serviço de terceiro faz o navegador de cada visitante enviar a esse serviço seu endereço IP, sem base legal que autorize esse tratamento nos termos da Lei Geral de Proteção de Dados Pessoais. Servindo os arquivos do mesmo domínio, não há terceiro envolvido. O navegador continua baixando cada família apenas quando algum texto exibido a utiliza.

7.6 Verificação Durante a Implementação

A implementação foi acompanhada por testes automatizados no *backend*, escritos com o Jest. Na execução de 8 de outubro de 2026, relatada no Capítulo 8, o projeto contava com 58 conjuntos de testes, que somavam 520 casos, todos aprovados. Os testes verificam serviços, controladores e DTOs dos módulos, substituindo o acesso ao banco e ao armazenamento por objetos simulados, de modo que cada regra de negócio seja exercitada de forma isolada e sem depender de um banco em execução. No *frontend*, a etapa de construção executa a verificação de tipos do TypeScript antes de gerar o pacote de produção, o que impede que uma divergência entre os tipos da interface e os formatos da API passe despercebida. Complementarmente, cada funcionalidade concluída foi verificada manualmente na interface, com apoio do DBeaver para inspecionar o estado do banco. Os procedimentos e os resultados dos testes são detalhados no capítulo dedicado aos testes.

7.7 Limitações da Implementação

A versão atual do sistema possui limitações conhecidas, registradas a seguir:

- **Limitação de requisições em memória:** os contadores de requisições são mantidos na memória do processo da API. Reiniciar o servidor os zera, e várias instâncias da aplicação não compartilhariam esses contadores. Em uma implantação com múltiplas instâncias, seria necessário um armazenamento compartilhado.
- **Ausência de comunicação em tempo real:** a API é exclusivamente REST, sem canal de comunicação contínua com o navegador. O *feed* de atualizações e as notificações são obtidos quando a interface os solicita, a cada minuto e sempre que a janela volta a ter foco, e não enviados pelo servidor no momento em que os eventos ocorrem (Subseção 5.2.4).
- **Ausência de documentação automática da API:** não há documentação no padrão OpenAPI; a referência das rotas é mantida manualmente no repositório do *backend*.
- **Armazenamento em disco sem credenciais de nuvem:** na ausência das credenciais do R2, os arquivos enviados ficam no disco da máquina que executa a API e não estão disponíveis em outra máquina, perdendo-se em ambientes de hospedagem cujo sistema de arquivos é recriado a cada publicação.

8 TESTE

Este capítulo apresenta a verificação do sistema, nas três frentes da estratégia descrita no Capítulo 3. A verificação automatizada concentra-se no *backend*, onde residem as regras de acesso, privacidade, sessão e consistência dos dados, e foi conduzida por meio de testes de unidade e de testes de ponta a ponta da API, estes executados sobre um banco de dados real. A verificação manual cobre a interface, que os testes automatizados não alcançam. As medições de desempenho verificam os limites de tempo fixados pelos requisitos não funcionais. São descritos o ambiente em que os testes automatizados são executados, uma visão geral do conjunto de testes por módulo, a especificação dos principais casos de teste, os testes de ponta a ponta, os resultados dos testes automatizados, a verificação manual e o método das medições, cujos resultados, por dependerem do juízo sobre cada requisito, são apresentados no Capítulo 10.

8.1 Ambiente e Execução dos Testes

Os testes foram escritos com o *framework* Jest, na versão 30, com o *ts-jest* responsável por interpretar o código TypeScript. Cada arquivo de teste fica ao lado do código que verifica e é identificado pelo sufixo `.spec.ts`, e o conjunto completo é executado pelo comando `pnpm test`, na raiz do projeto do *backend*.

Os testes são de unidade: cada um exercita uma parte do sistema de forma isolada. Para isso, as dependências externas, como o serviço de acesso ao banco de dados e o serviço de armazenamento de arquivos, são substituídas por objetos simulados, cujas respostas são definidas pelo próprio teste. Essa abordagem permite reproduzir situações difíceis de obter manualmente, como duas requisições concorrentes sobre o mesmo registro, e dispensa um banco de dados em execução. Em contrapartida, o teste verifica as chamadas que a regra de negócio faz ao banco, e não o resultado dessas chamadas em um banco real. Esse resultado é verificado pelos testes de ponta a ponta, descritos na Seção 8.4.

Os testes se dividem em três alvos. Os testes de serviços verificam as regras de negócio propriamente ditas. Os testes de controladores verificam que cada operação é executada em nome do usuário da sessão autenticada, e não de um identificador enviado na requisição. Os testes de DTOs verificam a validação dos dados de entrada, como formatos, limites e campos obrigatórios.

8.2 Visão Geral dos Testes

O Quadro 64 apresenta a distribuição dos testes de unidade pelos módulos do *backend*, com os requisitos funcionais a que cada módulo se relaciona. Os módulos `storage`, `mail` e `common` não correspondem a um requisito específico, pois reúnem funcionalidades transversais: o armazenamento de arquivos, o envio de e-mail, a validação dos Elementos e das imagens enviadas, a regra de mescla de fichas (RF-15) e o tratamento global de erros. O módulo `tag`, que apenas lê o vocabulário de *tags* para as sugestões do formulário de coleção, não tem testes de unidade e por isso não aparece no quadro; a gravação das *tags*, que liga as existentes e cria as novas, é verificada nos testes do módulo `collection`.

O módulo `layout` concentra o maior número de testes porque a maior parte deles é parametrizada: o mesmo teste é repetido para cada *layout* padrão e para cada forma desenhada que o sistema fornece, submetendo-os às mesmas regras de validação que a API aplica aos Elementos recebidos, de modo que um conteúdo padrão inválido seja detectado no teste, e não na biblioteca do usuário.

8.3 Casos de Teste

Os casos de teste a seguir foram especificados segundo a estrutura proposta por Marques (2018), composta por título, objetivo, pré-condição, passos e resultados esperados. Como os casos são executados automaticamente pelo Jest, os passos descrevem as ações realizadas pelo teste automatizado: preparar o estado simulado, acionar a operação e verificar o retorno. Os resultados esperados descrevem o comportamento do sistema com verbos no presente, conforme recomenda o autor. Ao final de cada caso, são indicados os testes automatizados que o implementam,

Quadro 64 – Distribuição dos testes de unidade por módulo

Módulo	**Requisitos relacionados**	**Arquivos**	**Testes**
`layout`	RF-01 (*layouts* padrão), RF-14 e RF-16	5	140
`social`	RF-20 a RF-23	20	103
`user`	RF-01, RF-03 e RF-05	5	70
`common`	Transversal (validação de Elementos e de imagens, mescla de fichas e tratamento de erros)	5	44
`item`	RF-09, RF-10, RF-12, RF-17 e RF-22	4	38
`category`	RF-06 e RF-13	4	24
`appearance-preset`	RF-18 e RF-19	3	23
`notification`	RF-24	3	23
`auth`	RF-01, RF-02 e RF-04	5	22
`storage`	Transversal (armazenamento de arquivos)	2	16
`collection`	RF-07, RF-08 e RF-15	1	11
`mail`	Transversal (envio de e-mail)	1	6
Total		**58**	**520**

Fonte: Elaborada pelo autor

identificados pelo nome registrado no código. Foram selecionadas as regras de maior impacto sobre a segurança e a consistência dos dados, apresentadas nos Quadros 65 a 72.

Quadro 65 – Caso de teste CT01: validar a revogação da sessão no *logout*

Objetivo: Verificar se o *logout* revoga a sessão atual e se um *token* associado a uma sessão revogada deixa de ser aceito (RF-02).
Pré-condição: Usuário cadastrado, com uma sessão ativa registrada no banco e um *token* de acesso que referencia essa sessão.
Passos: 1. Acionar o *logout* informando o usuário e a sessão atual. 2. Verificar a atualização enviada à tabela de sessões. 3. Validar um *token* cuja sessão está ativa. 4. Validar um *token* cuja sessão não existe ou foi revogada.
Resultados esperados: 1. O sistema registra a data de revogação somente na sessão atual do usuário, e apenas se ela ainda não estiver revogada. 2. O sistema aceita o *token* da sessão ativa e devolve os dados do usuário autenticado. 3. O sistema recusa o *token* da sessão inexistente ou revogada, respondendo como acesso não autorizado.
Testes automatizados: `auth.service.spec.ts`: "revokes only the current session on logout"; `jwt.strategy.spec.ts`: "accepts an active session" e "rejects a missing or revoked session".

Fonte: Elaborada pelo autor

Quadro 66 – Caso de teste CT02: validar a redefinição de senha por *token* temporário

Objetivo: Verificar se a recuperação de senha gera um *token* temporário armazenado de forma protegida, não revela se um e-mail está cadastrado e invalida o *token* e as sessões após o uso (RF-04).
Pré-condição: Usuário cadastrado com e-mail conhecido.
Passos: 1. Solicitar a recuperação de senha para o e-mail cadastrado. 2. Solicitar a recuperação de senha para um e-mail não cadastrado. 3. Redefinir a senha com um *token* válido e uma nova senha confirmada. 4. Redefinir a senha com um *token* inválido ou expirado.
Resultados esperados: 1. O sistema envia por e-mail um *token* de 64 caracteres, grava no banco apenas o seu *hash* SHA-256, com data de expiração, e responde com uma mensagem neutra, iniciada por "Se o e-mail estiver cadastrado". 2. O sistema responde com a mesma mensagem neutra, sem criar *token* e sem enviar e-mail. 3. O sistema grava a nova senha criptografada com bcrypt, marca o *token* como utilizado somente se ele ainda não tiver sido usado nem expirado, e revoga todas as sessões ativas do usuário. 4. O sistema recusa a requisição como inválida e não altera a senha.
Testes automatizados: `auth.service.spec.ts`: "creates a hashed, temporary password reset token", "uses the same response without creating a token for an unknown email", "resets the password, consumes the token and revokes all sessions" e "rejects an invalid or expired reset token".

Fonte: Elaborada pelo autor

Quadro 67 – Caso de teste CT03: validar o encerramento de acompanhamentos ao restringir a privacidade

Objetivo: Verificar se, ao tornar uma coleção mais restrita, o sistema encerra os acompanhamentos dos usuários que perderam acesso a ela, preservando os demais (RF-08 e RF-21).
Pré-condição: Coleção pertencente ao usuário autenticado, com acompanhamentos ativos de outros usuários.
Passos: 1. Alterar a privacidade da coleção para privada. 2. Alterar a privacidade da coleção para somente amigos. 3. Alterar a privacidade de uma coleção somente para amigos para pública. 4. Verificar, em cada caso, as atualizações enviadas às tabelas de coleções e de acompanhamentos.
Resultados esperados: 1. Ao tornar a coleção privada, o sistema altera a privacidade e encerra todos os acompanhamentos ativos na mesma transação, registrando a data de encerramento. 2. Ao tornar a coleção somente para amigos, o sistema encerra apenas os acompanhamentos de usuários sem amizade aceita com o dono, mantendo os dos amigos. 3. Ao tornar a coleção menos restrita, o sistema altera a privacidade sem modificar nenhum acompanhamento.
Testes automatizados: `collection.service.spec.ts`: “ends every active follow when a collection becomes private”, “keeps friend follows and ends non-friend follows when becoming friends-only” e “does not touch follows when visibility becomes less restrictive”.

Fonte: Elaborada pelo autor

Quadro 68 – Caso de teste CT04: validar o acesso a coleções conforme privacidade e amizade

Objetivo: Verificar se cada usuário visualiza e acompanha somente as coleções permitidas pelo seu vínculo com o dono e pelo nível de privacidade de cada coleção (RF-08, RF-21 e RF-23).
Pré-condição: Dois usuários cadastrados: o dono das coleções e um visitante autenticado, com ou sem amizade aceita entre eles.
Passos: 1. Determinar o nível de acesso quando o visitante é o próprio dono. 2. Determinar o nível de acesso quando há amizade aceita entre visitante e dono. 3. Determinar o nível de acesso quando não há amizade, ou quando a solicitação está pendente, recusada ou cancelada. 4. Avaliar, para cada nível de acesso, a visualização de coleções públicas, somente para amigos e privadas. 5. Solicitar o acompanhamento de uma coleção pública, de uma coleção somente para amigos e de uma coleção do próprio usuário. 6. Consultar uma coleção inexistente e uma coleção que o visitante não pode ver.
Resultados esperados: 1. O sistema concede ao dono acesso às três privacidades, sem consultar amizades. 2. O sistema concede ao amigo acesso às coleções públicas e somente para amigos. 3. O sistema concede aos demais usuários acesso apenas às coleções públicas, inclusive quando a solicitação de amizade não foi aceita. 4. O sistema permite que não amigos acompanhem coleções públicas e que amigos acompanhem coleções somente para amigos, e recusa o acompanhamento de uma coleção do próprio usuário. 5. O sistema responde que a coleção não foi encontrada tanto quando ela não existe quanto quando o visitante não tem permissão para vê-la, sem revelar a existência de coleções invisíveis.
Testes automatizados: `social-access-policy.service.spec.ts`: todos os testes do arquivo, incluindo as combinações parametrizadas de nível de acesso e privacidade; `social-content.service.spec.ts`: “returns not found for a missing or invisible collection”.

Fonte: Elaborada pelo autor

Quadro 69 – Caso de teste CT05: validar a criação da ficha própria de um item

Objetivo: Verificar se a personalização de um item cria uma cópia própria do *layout*, com ou sem *layout* na coleção, e se as alterações afetam somente essa cópia (RF-17).
Pré-condição: Item pertencente ao usuário autenticado, em uma coleção com *layout* associado ou sem *layout*.
Passos: 1. Iniciar a personalização de um item cuja coleção possui *layout* com Elementos. 2. Iniciar a personalização de um item cuja coleção não possui *layout*. 3. Adicionar um Elemento a um item que já possui ficha própria. 4. Tentar alterar um Elemento de um item que ainda não possui ficha própria.
Resultados esperados: 1. O sistema cria um *layout* de uso exclusivo do item, com referência ao *layout* da coleção, copia cada Elemento com seu conteúdo, posição, tamanho, camada e estilo, registrando a referência ao Elemento original, e associa o novo *layout* ao item. 2. O sistema cria o *layout* do item vazio, sem referência de origem e sem copiar Elementos, e o associa ao item. 3. O sistema grava o novo Elemento somente no *layout* próprio do item. 4. O sistema recusa a alteração por conflito de estado e não modifica nenhum Elemento.
Testes automatizados: `item-layout-override.service.spec.ts`: “creates an editable item layout copy from the collection layout”, “creates an empty override when the collection has no layout”, “adds a visual element only to the item override layout” e “rejects editing item visual elements before an override exists”.

Fonte: Elaborada pelo autor

Quadro 70 – Caso de teste CT06: validar a paleta em uso

Objetivo: Verificar se o cadastro deixa em uso a paleta correspondente ao tema do sistema operacional, se a troca de paleta mantém uma única paleta em uso e se a paleta em uso não pode ser excluída (RF-01 e RF-18).
Pré-condição: Para o cadastro, dados válidos de um novo usuário; para as demais operações, usuário autenticado com paletas na biblioteca.
Passos: 1. Cadastrar um usuário informando que o sistema operacional está no tema escuro. 2. Selecionar como paleta em uso uma paleta do usuário que não está ativa. 3. Tentar excluir a paleta em uso. 4. Excluir uma paleta predefinida que não está em uso.
Resultados esperados: 1. O sistema cria as paletas predefinidas com exatamente uma em uso, a paleta "Escuro", de tema escuro. 2. O sistema desativa a paleta anterior e ativa a selecionada em uma única transação. 3. O sistema recusa a exclusão por conflito e não remove a paleta. 4. O sistema remove a paleta predefinida, que pertence ao usuário como as demais.
Testes automatizados: `user.service.spec.ts`: "starts the account on the dark palette when the system is dark"; `appearance-preset.service.spec.ts`: "activates an owned preset and deactivates the previous one atomically", "rejects deleting the active preset" e "deletes a system palette that is not in use".

Fonte: Elaborada pelo autor

Quadro 71 – Caso de teste CT07: validar a resposta a uma solicitação de amizade

Objetivo: Verificar se a resposta a uma solicitação de amizade altera o estado apenas enquanto a solicitação está pendente, notifica o remetente na mesma transação e não produz efeitos duplicados em respostas concorrentes (RF-20 e RF-24).
Pré-condição: Solicitação de amizade enviada por um usuário ao usuário autenticado.
Passos: 1. Aceitar uma solicitação pendente. 2. Recusar uma solicitação que já foi cancelada pelo remetente. 3. Aceitar uma solicitação pendente que outra requisição simultânea já respondeu.
Resultados esperados: 1. O sistema altera a solicitação para aceita somente se ela ainda estiver pendente e destinada ao usuário autenticado, registra a data da resposta e cria, na mesma transação, a notificação “João aceitou sua solicitação de amizade.” para o remetente. 2. O sistema recusa a operação por conflito, sem alterar a solicitação e sem criar notificação. 3. O sistema identifica que nenhum registro foi alterado, recusa a operação por conflito e não cria notificação.
Testes automatizados: `friendship.service.spec.ts`: “accepts a pending request and notifies its sender transactionally”, “rejects a request that is no longer pending” e “rejects a concurrent second response without creating a notification”.

Fonte: Elaborada pelo autor

Quadro 72 – Caso de teste CT08: validar o formato real da imagem enviada

Objetivo: Verificar se o sistema identifica o formato de uma imagem pelo seu conteúdo, e não pelo nome ou pelo tipo declarado, e se recusa arquivos que apenas aparentam ser imagens (RF-09).
Pré-condição: Item pertencente ao usuário autenticado, sem imagem de capa.
Passos: 1. Verificar o formato de arquivos que começam com as assinaturas de JPEG e de PNG. 2. Verificar o formato de um arquivo de texto renomeado como imagem. 3. Enviar como imagem de capa do item um arquivo cujo conteúdo é texto.
Resultados esperados: 1. O sistema reconhece os formatos JPEG e PNG pelas assinaturas no início do arquivo. 2. O sistema não reconhece o arquivo de texto como imagem. 3. O sistema recusa o envio como requisição inválida e não grava nenhum arquivo no armazenamento.
Testes automatizados: `imagem-real.validator.spec.ts`: “recognizes JPEG and PNG by their signature” e “rejects content that only claims to be an image”; `item.service.spec.ts`: “rejects a file whose bytes are not JPEG or PNG”.

Fonte: Elaborada pelo autor

8.4 Testes de Ponta a Ponta da API

Os testes de ponta a ponta exercitam a API como um cliente a usa: cada teste envia requisições HTTP à aplicação completa, com os mesmos módulos, a mesma validação dos dados de entrada e o mesmo tratamento de erros da versão publicada, e confere a resposta e, quando necessário, o que ficou gravado no banco. Assim, um mesmo teste verifica em conjunto a autenticação, a validação, a regra de negócio e o comportamento do banco de dados, inclusive as restrições de unicidade, que os testes de unidade não alcançam.

Os testes são executados pelo Jest, com a biblioteca *supertest* enviando as requisições, sobre um PostgreSQL 16 em um contêiner Docker criado apenas para eles, separado do banco de desenvolvimento e do banco da plataforma publicada. Antes da execução, as migrações do projeto são aplicadas a esse banco, de modo que as tabelas, os índices e as restrições são os mesmos da produção, e cada arquivo de teste começa com as tabelas vazias. A única peça substituída é o envio de e-mail: a mensagem com o código de confirmação do cadastro é capturada pelo teste, em vez de enviada. Uma verificação interrompe a execução se o endereço do banco não for o da própria máquina, o que impede que os testes, que esvaziam as tabelas, sejam executados por engano contra dados reais. O conjunto é executado pelo comando `pnpm test:e2e`, na raiz do projeto do *backend*.

Os fluxos verificados estão distribuídos em quatro arquivos, apresentados no Quadro 73. O limite de tentativas de *login* tem arquivo próprio porque a API conta as tentativas por aplicação em execução, e as tentativas dos demais testes interfeririam na contagem.

Quadro 73 – Testes de ponta a ponta da API

Arquivo	**Requisitos**	**Testes**	**O que verifica**
`autenticacao`	RF-01, RF-02 e RNF-03	9	Cadastro com dados inválidos recusado; senha gravada apenas como *hash*; acesso negado antes da confirmação do e-mail; código de confirmação errado recusado; mesma mensagem para senha errada e e-mail inexistente; rota protegida recusada sem sessão; sessão encerrada no *logout*
`limite-de-login`	RNF-03	1	Sexta tentativa de *login* no mesmo minuto recusada, mesmo com a senha correta
`acervo`	RF-06, RF-07, RF-09, RF-10 e RF-17	9	Categoria com nome repetido recusada pela restrição de unicidade do banco; Item com nome repetido na Coleção recusado; movimentação de Item com registro do evento e recusa quando o destino já tem Item de mesmo nome; Item invisível e inalterável para outro usuário; exclusão do Item com a sua ficha própria; exclusão da Categoria com as suas Coleções e Itens
`privacidade-e-social`	RF-08 e RF-20 a RF-24	7	Coleção visível conforme a privacidade e a amizade, com 404 para a invisível; acompanhamento negado a Coleção invisível; notificação do aceite da amizade; Item novo no *feed* de quem acompanha; acompanhamento encerrado quando a Coleção se torna privada; uma única amizade para dois pedidos enviados ao mesmo tempo
Total		**26**	

Fonte: Elaborada pelo autor

8.5 Resultados e Limitações dos Testes Automatizados

Na execução realizada em 8 de outubro de 2026, os 58 conjuntos de testes de unidade, com 520 testes no total, foram aprovados, em aproximadamente 7 segundos. Todos os casos de teste especificados na Seção 8.3 obtiveram, portanto, os resultados esperados. Na mesma data, os quatro arquivos de testes de ponta a ponta, com 26 testes, foram aprovados em aproximadamente 9 segundos, em três execuções consecutivas.

A estratégia adotada possui limitações que devem ser consideradas na interpretação desses resultados:

- **Alcance dos testes de ponta a ponta:** eles cobrem os fluxos de autenticação, de organização do acervo e de privacidade e interação social, mas não todas as rotas da API; as rotas do editor de *layouts*, das paletas e do envio de imagens são verificadas apenas pelos testes de unidade.
- **Escopo da verificação automatizada:** os testes automatizados concentram-se nas regras de negócio do *backend* e não abrangem os aspectos visuais e de interação da aplicação cliente, que ficam a cargo da verificação manual e das medições no navegador, descritas nas seções seguintes.

8.6 Verificação Manual

Os aspectos que os testes de unidade não alcançam, como a aparência das telas e a interação com o editor de *layouts*, foram verificados manualmente, na segunda frente da estratégia descrita no Capítulo 3. A cada funcionalidade concluída, ela foi exercitada pela interface, e o estado do banco de dados após cada operação foi conferido com o DBeaver, o que permitiu confirmar, por exemplo, que um Item ficava vinculado à Coleção correta e usava os Elementos do *layout* aplicável. Depois da publicação descrita no Capítulo 9, a plataforma passou a ser usada pela autora e por duas pessoas que não participaram do desenvolvimento, uso relatado na Seção 10.4. Somou-se a isso a inspeção do código da aplicação cliente e da API, para confirmar como cada regra foi implementada quando o comportamento observado na tela não bastava para isso.

Por ser exploratória, essa verificação não tem casos especificados como os da Seção 8.3. No Capítulo 10, ela aparece como *uso* e *inspeção* entre os instrumentos de validação dos requisitos funcionais, ao lado dos testes automatizados.

8.7 Medições de Desempenho

Três requisitos não funcionais fixam limites de tempo: o RNF-02, para o carregamento das telas e as respostas da API; o RNF-06, para a movimentação de Itens entre Coleções; e o RNF-07, para a resposta do editor de *layouts*. Esses limites não se verificam por testes de unidade, e foram medidos com *scripts* mantidos no repositório da API, na pasta `scripts/medicoes`. Os *scripts* gravam cada amostra individualmente, além da descrição do ambiente e dos resumos estatísticos, de modo que todo número citado pode ser conferido e a medição, repetida. Esta seção descreve como as medições foram feitas; os resultados e o juízo sobre cada requisito estão na Seção 10.3.

8.7.1 Ambiente e Acervo

As medições foram feitas em 8 de outubro de 2026, no computador descrito no Quadro 1. A API foi executada em modo de produção, conectada ao banco de dados do Supabase pela internet, de modo que cada tempo medido inclui a ida e a volta até o banco. A interface foi compilada para produção e servida pela mesma máquina que a API. Os *scripts* criam, pela própria API, uma conta de teste e um acervo sintético com uma Categoria, duas Coleções, cinquenta Itens, cinco deles com ficha própria, e um *layout* de quinze Elementos, quantidade citada no RNF-07. A confirmação do e-mail da conta de teste passa pela mesma rota usada pela interface, com o código lido do registro da API, executada em um modo que não envia e-mail. Ao final, os *scripts* excluem a conta e confirmam a exclusão.

As medições não foram repetidas no ambiente de produção descrito no Capítulo 9, de modo que os tempos caracterizam a API, o banco de dados e a interface, mas não a distância entre a máquina virtual e o Supabase nem a rede de quem usa a plataforma publicada.

8.7.2 Tempo de Resposta da API

O tempo de resposta foi medido em oito rotas: as leituras do painel de Categorias, das Coleções de uma Categoria, da lista de cinquenta Itens e da ficha própria de quinze Elementos; o cadastro e a exclusão de um Item; a movimentação

de um Item entre Coleções; e a gravação de um Elemento no editor. Cada rota recebeu cinco requisições de aquecimento, descartadas, e cinquenta medidas. O tempo de cada requisição foi medido do instante anterior ao envio até a leitura completa do corpo da resposta, uma requisição de cada vez, com 700 milissegundos entre elas. O intervalo mantém o ritmo abaixo do limite de 100 requisições por minuto que a API impõe a cada endereço de origem; pela mesma razão, não foi feito ensaio de carga, que, a partir de uma única máquina, mediria o limitador, e não a API.

8.7.3 Medições no Navegador

As telas, a movimentação de Itens e o editor foram medidos no Google Chrome 154. Três *scripts* da mesma pasta foram executados na própria página, sem alteração, como se colados no console do navegador.

O primeiro registra, a cada carregamento de uma tela, o instante da primeira pintura de conteúdo (FCP), o do maior elemento de conteúdo pintado (LCP) e o início e a duração de cada chamada à API, todos contados a partir do início da navegação. Foram feitos dez recarregamentos da lista de cinquenta Itens e dez da ficha de um Item com ficha própria, com a rede limitada a 10 Mbps pelas ferramentas de desenvolvimento do navegador, como prevê o RNF-02, em duas condições: com o cache do navegador desligado, que equivale à primeira visita, em que toda a interface é baixada, e com o cache ligado, que equivale ao retorno de quem já visitou a plataforma.

O segundo mede, para cada movimentação de Item feita pela tela, o tempo entre o clique no botão que confirma a movimentação e o primeiro quadro pintado sem o cartão do Item, e registra quando chegou a resposta do servidor. Foram feitas dez movimentações, também a 10 Mbps.

O terceiro usa a interface de temporização de eventos do navegador (*Event Timing API*), base da métrica INP, que informa, para cada interação, o tempo entre o gesto e a pintura do quadro seguinte. Os eventos de um mesmo gesto, como os de um arraste, são agrupados, e cada gesto conta pela sua maior duração. No editor do *layout* de quinze Elementos foram feitos 31 gestos com o *mouse*: nove seleções, seis

mudanças de propriedade de texto, seis arrastes, quatro redimensionamentos pelas alças, quatro inclusões de Elementos pelo painel, uma operação de desfazer e outra de refazer. A interface só informa interações que duram ao menos 16 milissegundos.

As telas e a movimentação foram medidas com uma janela de 1584 por 805 pixels de área útil; o editor, cuja resposta não depende da rede, foi medido em outra sessão, sem limitação de rede e com uma janela de 1024 por 577 pixels. Em nenhuma das medições houve limitação do processador.

8.7.4 Cálculo dos Percentis

Os percentis foram calculados pelo método do posto mais próximo, sem interpolação, de modo que todo valor relatado é um tempo que de fato ocorreu; com cinquenta amostras, o percentil 99 coincide com o máximo. O juízo sobre a API usa o percentil 95, e o juízo sobre as telas e o editor, o percentil 75, convenção das métricas LCP e INP; a escolha é justificada com cada requisito na Seção 10.3.

9 IMPLANTAÇÃO

A Subseção 3.4.3 e os Capítulos 6 e 7 trataram o sistema como artefato lógico: a divisão em camadas, a aplicação cliente em React, a API em NestJS e a persistência em PostgreSQL por meio do Prisma. Este capítulo trata de onde cada uma dessas partes é executada. A distinção é necessária porque o desenho em camadas não determina, por si só, a distribuição física: a mesma arquitetura poderia ser publicada em uma única máquina ou repartida entre vários provedores, e a escolha entre essas alternativas responde a critérios de custo, de carga esperada e de esforço de operação, e não a critérios de projeto.

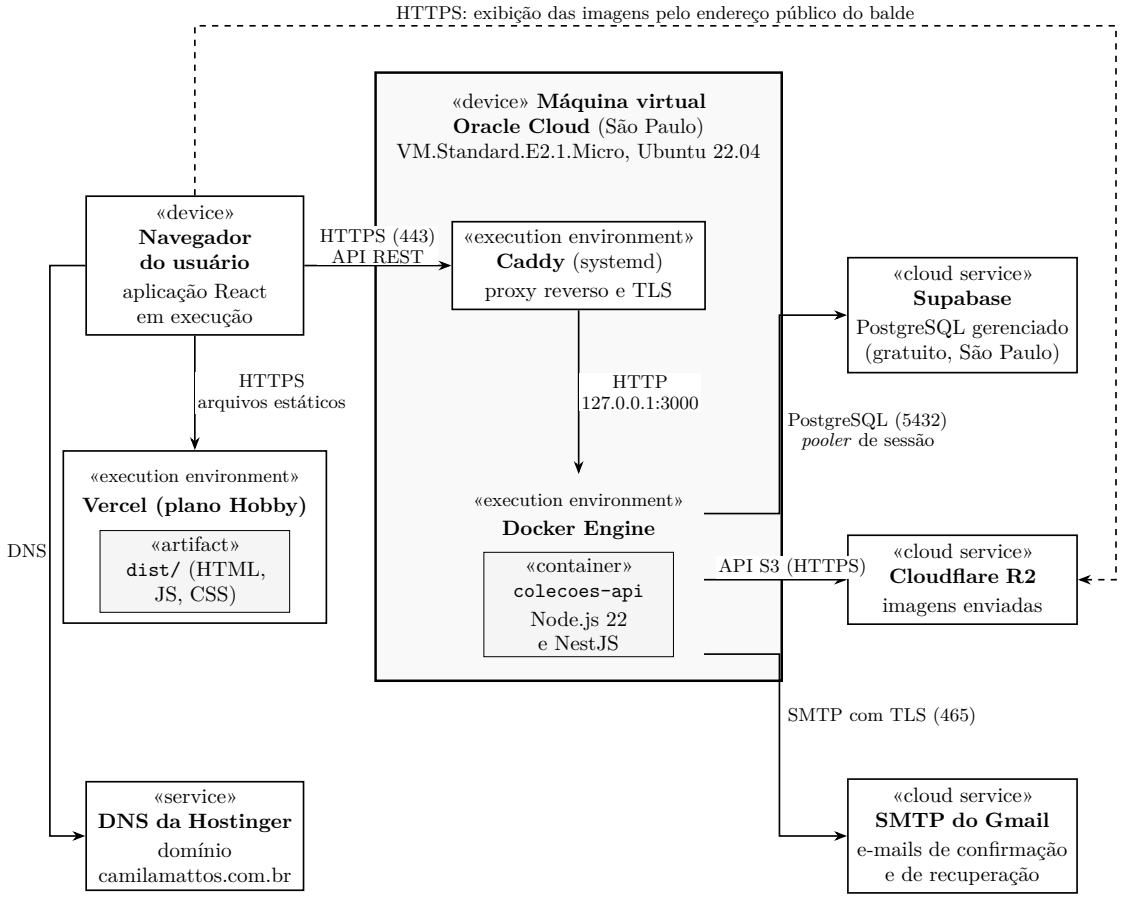
O ambiente aqui descrito é o de produção, acessível pela internet, e foi montado com planos gratuitos dos provedores envolvidos, com uma única despesa: o registro do domínio próprio `camilamattos.com.br`. Essa restrição de custo é o fio que explica a maior parte das decisões registradas a seguir. A Seção 9.1 apresenta os ambientes de execução e a distribuição dos componentes entre eles. A Seção 9.2 descreve como cada componente é configurado e qual superfície da máquina permanece exposta. A Seção 9.3 percorre o caminho que uma nova versão do sistema atravessa até entrar em operação.

9.1 Infraestrutura

A alocação dos componentes segue um critério simples: cada parte do sistema é publicada no tipo de ambiente que corresponde à sua natureza de execução. A aplicação cliente, que se reduz a arquivos estáticos, vai para uma plataforma de hospedagem de arquivos estáticos; a API, que é um processo de longa duração, vai para uma máquina virtual; e os dados, que precisam sobreviver à máquina, ficam em serviços gerenciados. Desse critério decorre a distribuição apresentada na Figura 19.

Aplicação cliente. A aplicação cliente é publicada na Vercel, no plano gratuito (Hobby), sob o endereço <https://tcc.camilamattos.com.br>, subdomínio

Figura 19 – Diagrama de implantação do sistema



Fonte: Elaborada pelo autor

do domínio próprio ligado à Vercel por um registro DNS do tipo CNAME. A separação já era imposta pela arquitetura: a aplicação executa no navegador do usuário, e o resultado da sua construção (`tsc -b` seguido de `vite build`) é um conjunto de arquivos estáticos, sem estado de servidor. Qualquer hospedagem de arquivos estáticos poderia servi-los; a Vercel foi escolhida porque constrói e publica o projeto diretamente a partir do repositório no GitHub, a cada alteração no ramo principal (Vercel, a), o que dispensa qualquer etapa manual para essa parte do sistema. Servir esses arquivos a partir da mesma máquina que executa a API foi descartado, por consumir a memória escassa dessa máquina em uma função que não exige nenhuma de suas características. Como a navegação entre telas é feita no próprio navegador, um endereço interno aberto diretamente, como o de

uma Coleção, precisa ser respondido com o arquivo da aplicação, `app.html`, e não com o `index.html`, que é a página inicial pública (Seção 7.5); em aplicações Vite publicadas na Vercel, isso é feito por uma regra de reescrita declarada no arquivo `vercel.json` do repositório (Vercel, c), que preserva a raiz do site para a página inicial.

API. A API é executada em uma máquina virtual da Oracle Cloud Infrastructure, criada na região de São Paulo com a forma `VM.Standard.E2.1.Micro`, que integra o nível gratuito permanente (*Always Free*) do provedor. A documentação da Oracle descreve essa forma com um oitavo de OCPU, com possibilidade de usar recursos adicionais de processamento, e 1 GB de memória (Oracle,); o sistema operacional é o Ubuntu 22.04. Diferentemente de arquiteturas baseadas em serviços, em que a decisão de implantação envolve distribuir vários processos entre máquinas, a API deste trabalho é um monólito: um único processo, empacotado em um único contêiner. A questão de distribuição, portanto, não se coloca, e o dimensionamento reduz-se a saber se a máquina comporta esse processo. Com 1 GB, a memória é o recurso que se esgota primeiro, antes do processamento e do disco. O efeito foi observado já na primeira publicação: como a imagem do contêiner é construída na própria máquina, a instalação das dependências e a compilação só foram concluídas depois da criação de uma área de troca (*swap*) em disco.

Banco de dados. O PostgreSQL de produção é fornecido em regime gerenciado pelo Supabase, no plano gratuito, em projeto criado na região de São Paulo, a mesma da máquina virtual, o que mantém curto o percurso de ida e volta entre a API e o banco. Durante o desenvolvimento, o banco local em contêiner Docker, descrito nos Capítulos 3 e 7, continua disponível como alternativa. A escolha de um serviço gerenciado separa duas responsabilidades: manter o processo do banco em execução e atualizado, que passa a ser do provedor, e guardar a única cópia dos dados fora da máquina virtual, que de outro modo dividiria com a API o mesmo gigabyte de memória. A API conecta-se pelo *pooler* de sessão do Supabase, na porta 5432. A escolha não é de desempenho, mas de rede: no plano gratuito, a conexão direta ao banco é feita apenas por IPv6, e o próprio provedor indica o modo sessão do *pooler* como alternativa para clientes em redes apenas IPv4 (Supabase, b), situação da máquina virtual, que dispõe de um endereço público

IPv4. A mesma cadeia de conexão atende às migrações e à API.

O plano gratuito impõe três limites que convém registrar. O banco é limitado a 500 MB por projeto (Supabase, a). O provedor não faz cópia de segurança automática de projetos gratuitos e recomenda que eles exportem os dados regularmente com o comando `db dump` da sua ferramenta de linha de comando, mantendo as cópias fora da plataforma (Supabase, c). E projetos gratuitos com pouca atividade ao longo de sete dias são pausados, podendo ser restaurados pelo painel (Supabase, d). O primeiro limite está longe de ser atingido por uma operação de validação; os dois últimos, ao contrário, afetam diretamente a confiabilidade do ambiente: o que a delegação ao serviço gerenciado removeu foi a operação do banco, e não a garantia de recuperação dos dados, que depende de uma exportação feita pelo próprio responsável pela plataforma.

Arquivos enviados. As imagens enviadas pelos usuários (capas, ícones, fotos de perfil e imagens dos Elementos) são gravadas no Cloudflare R2, acessado pela API S3 por meio do SDK da AWS, conforme o Capítulo 7. A consequência prática é que a máquina virtual não guarda estado: o banco está no Supabase e os arquivos estão no R2, de modo que perder a máquina exige apenas reinstalar a API, sem perda de Coleções, Itens ou imagens. Essa propriedade depende das credenciais do R2 estarem configuradas; sem elas, a API gravaria os arquivos no disco do contêiner, limitação já registrada no Capítulo 7, e eles se perderiam a cada nova construção da imagem.

Envio de e-mail. Os e-mails com o código de confirmação do cadastro e com o link de recuperação de senha são enviados pelo servidor SMTP do Gmail, com conexão cifrada na porta 465 e autenticação por senha de aplicativo da conta Google. Não há serviço de e-mail próprio: o envio depende apenas das variáveis de configuração descritas na Seção 9.2.

Nome de domínio. O domínio `camilamattos.com.br` foi registrado na Hostinger, que também mantém os seus registros DNS. Dois subdomínios atendem à plataforma: `tcc.camilamattos.com.br`, que aponta para a Vercel e serve a aplicação cliente, e `api.camilamattos.com.br`, cujo registro do tipo A aponta para o endereço público da máquina virtual. A necessidade de um nome para a API

decorre de uma cadeia de exigências. A aplicação cliente é servida pela Vercel em HTTPS, e requisições `fetch()` ou `XMLHttpRequest` feitas por uma página segura a um endereço HTTP são conteúdo misto bloqueável, recusado pelo navegador (MDN Web Docs,); a API precisa, portanto, responder em HTTPS. Para isso, o Caddy obtém e renova automaticamente certificados TLS para nomes de domínio públicos (Caddy Web Server, a), o que pressupõe um nome que resolva para a máquina, e o subdomínio da API atende a essa exigência. Reunir a aplicação e a API sob o mesmo domínio também dá à plataforma um endereço estável, que não depende do provedor de hospedagem. A contrapartida é a dependência do serviço de DNS do registrador: uma falha na resolução do nome tornaria a plataforma inacessível, ainda que a máquina estivesse em funcionamento, e o registro precisa ser renovado para que o domínio continue ativo.

O Quadro 74 sintetiza os alvos de implantação e o motivo da alocação de cada um.

Quadro 74 – Alvos de implantação do sistema

Componente	**Onde executa**	**Plano**	**Motivo da alocação**
Aplicação cliente (React)	Vercel	Hobby (gratuito)	Arquivos estáticos sem estado de servidor; construção e publicação automáticas a partir do repositório
API (NestJS)	Máquina virtual Oracle Cloud, em contêiner Docker, atrás do Caddy	*Always Free*	Processo de longa duração; um único contêiner, por ser um monólito
Banco de dados (PostgreSQL)	Supabase, região de São Paulo	Gratuito	Operação do banco delegada ao provedor; dados fora da máquina virtual
Arquivos enviados	Cloudflare R2	Gratuito	Mantém a máquina virtual sem estado
E-mail	SMTP do Gmail	Conta Google	Envio dos e-mails de confirmação de cadastro e de recuperação de senha sem servidor próprio
Nome de domínio	Hostinger	Pago (registro do domínio)	Endereços públicos da aplicação e da API; nome necessário para o certificado TLS emitido pelo Caddy

Fonte: Elaborada pelo autor

9.2 Configuração dos Serviços

A configuração segue um princípio único: o código é o mesmo em todos os ambientes, e tudo o que varia entre eles é fornecido de fora, no momento da execução ou da construção. Na máquina virtual, os valores ficam em um arquivo `.env` ao lado do repositório; na Vercel, nas variáveis de ambiente do projeto. O Quadro 75 relaciona essas variáveis, sem seus valores.

Quadro 75 – Variáveis de ambiente de produção

Variável	**Onde**	**Função**
`DATABASE_URL`	Máquina virtual	Cadeia de conexão do *pooler* de sessão do Supabase, usada pela API e pelas migrações
`JWT_SECRET`	Máquina virtual	Segredo de assinatura dos *tokens* de sessão
`FRONTEND_URL`	Máquina virtual	Endereço publicado da aplicação cliente; única origem autorizada pelo CORS
`PORT`	Máquina virtual	Porta em que a API escuta dentro do contêiner
`SMTP_*`	Máquina virtual	Servidor, porta, cifra, usuário, senha de aplicativo e remetente do Gmail
`R2_*`	Máquina virtual	Conta, chaves de acesso, balde e endereço público do Cloudflare R2
`API_URL`	Vercel	Endereço HTTPS da API, embutido nos arquivos estáticos durante a construção

Fonte: Elaborada pelo autor

A variável `API_URL` tem natureza diferente das demais. Como a aplicação cliente não tem servidor próprio, não há momento de execução em que ela pudesse ler a configuração: o endereço da API é embutido nos arquivos durante a construção, e alterá-lo exige uma nova publicação. O nome sem o prefixo `VITE_`, que o Vite exige por padrão, é aceito por uma configuração explícita do prefixo no `vite.config.ts`, restrita a esse nome exato para que nenhuma outra variável do ambiente seja exposta nos arquivos públicos por acidente.

9.2.1 Contêiner da API

A API é construída e executada pelo Docker, a partir de um `Dockerfile` e de um `docker-compose.yml` versionados no repositório do *backend*. O `Dockerfile`,

reproduzido no Quadro 76, é dividido em três estágios. O estágio `base` parte da imagem `node:22-slim` e acrescenta o OpenSSL, exigido pelo motor do Prisma e ausente da imagem reduzida. O estágio `build` instala as dependências, o que gera o cliente do Prisma, compila o código e aplica as migrações pendentes ao banco. O estágio `runtime`, que é o único que compõe a imagem final, copia apenas o `package.json`, as dependências e o código compilado, sem o código-fonte.

Quadro 76 – `Dockerfile` da API

```
1  FROM node:22-slim AS base
2  RUN apt-get update && apt-get install -y --no-install-recommends
       openssl \
3      && rm -rf /var/lib/apt/lists/*
4  RUN corepack enable
5  WORKDIR /app
6
7  FROM base AS build
8  COPY package.json pnpm-lock.yaml pnpm-workspace.yaml prisma.config
       .ts ./
9  COPY prisma ./prisma
10 RUN pnpm install --frozen-lockfile
11 COPY . .
12 RUN pnpm build
13 RUN --mount=type=secret,id=backend_env,target=/app/.env \
14     pnpm exec prisma migrate deploy
15
16 FROM base AS runtime
17 ENV NODE_ENV=production
18 COPY --from=build /app/package.json ./
19 COPY --from=build /app/node_modules ./node_modules
20 COPY --from=build /app/dist ./dist
21 EXPOSE 3000
22 CMD ["node", "dist/src/main.js"]
```

Fonte: Elaborada pelo autor

Dois pontos desse arquivo merecem comentário. O primeiro é a forma como a migração recebe a cadeia de conexão. Argumentos de construção e variáveis de ambiente são inadequados para passar segredos ao processo de construção, porque persistem na imagem final; a montagem de segredo (`RUN --mount=type=secret`) torna o arquivo disponível apenas durante a instrução que o utiliza (Docker, a). O `.env` é declarado no `docker-compose.yml` como segredo de construção e montado somente no passo da migração, de modo que nenhuma camada da imagem contém as

credenciais. Em execução, o mesmo arquivo é entregue ao contêiner como conjunto de variáveis de ambiente (`env_file`).

O segundo é o momento da migração. Executada durante a construção, ela altera o esquema do banco antes que o contêiner novo substitua o antigo, e o contêiner antigo continua atendendo requisições sobre o esquema já alterado até ser substituído. A vinculação tem uma consequência desejável e uma que precisa ser observada. A desejável é que uma versão cujo esquema não pôde ser aplicado não chega a entrar em execução, porque a falha da migração interrompe a construção. A que precisa ser observada é que, durante a janela de construção, o código antigo opera sobre o esquema novo: em migrações que apenas acrescentam tabelas ou colunas, o efeito é nulo; em migrações que removem ou renomeiam estruturas, abre-se uma janela de erro. Manter cada migração compatível com a versão anterior do código passa, assim, a ser condição do procedimento tal como ele está construído.

9.2.2 Superfície Exposta

O Quadro 77 reproduz a declaração do contêiner e a configuração do Caddy. A porta da API é publicada apenas na interface de retorno da máquina (`127.0.0.1`), o que a torna inalcançável de fora; o único caminho até ela passa pelo Caddy, instalado diretamente no sistema operacional e executado como serviço do `systemd`. A restrição no próprio mapeamento de portas não é redundante em relação ao *firewall*: quando o Docker publica a porta de um contêiner, o tráfego é desviado antes de passar pelas regras do UFW, que ficam, na prática, sem efeito para essa porta (Docker, b). Publicada em todas as interfaces, a API ficaria acessível sem cifra, contornando o Caddy, ainda que o UFW não a liberasse.

Quadro 77 – Declaração do contêiner da API e configuração do Caddy

```
# docker-compose.yml (trecho)
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
      secrets:
        - backend_env
    container_name: colecoes-api
    restart: unless-stopped
    env_file:
      - .env
    ports:
      - "127.0.0.1:3000:3000"

# /etc/caddy/Caddyfile
api.camilamattos.com.br {
    reverse_proxy localhost:3000
}
```

Fonte: Elaborada pelo autor

O bloco do Caddyfile é suficiente para o arranjo inteiro: por se tratar de um nome de domínio público, o Caddy obtém e renova o certificado TLS e redireciona o tráfego HTTP para HTTPS sem configuração adicional (Caddy Web Server, a). A superfície de rede da máquina reduz-se, assim, a três portas, liberadas tanto nas regras de entrada da rede virtual da Oracle quanto no UFW: a 22, para administração por SSH com autenticação por chave; a 443, atendida pelo Caddy; e a 80, que não serve conteúdo algum e responde apenas pelo redirecionamento para HTTPS e pela validação dos certificados.

Ao repassar uma requisição, o Caddy acrescenta o cabeçalho `X-Forwarded-For` com o endereço do cliente (Caddy Web Server, b). A informação é necessária para a limitação de requisições descrita no Capítulo 7, que identifica o cliente pelo endereço IP: sem ela, todas as requisições chegariam à API vindas do próprio Caddy, e o limite de cinco tentativas de *login* por minuto passaria a valer para a plataforma inteira, e não para cada cliente. A API é, por isso, configurada para confiar em exatamente um intermediário, o Caddy. Confiar em qualquer valor do cabeçalho, em vez de em um único salto, permitiria a um

cliente forjar o próprio endereço e escapar do limite. Pela mesma razão de restringir o que é aceito, o CORS da API autoriza uma única origem, a da aplicação cliente publicada, lida da variável `FRONTEND_URL`.

9.2.3 Continuidade de Execução

A recuperação após falhas é delegada a dois supervisores já presentes na máquina. O contêiner é declarado com a política `restart: unless-stopped`, de modo que o Docker o reinicia se o processo terminar inesperadamente, e o Docker e o Caddy são serviços do `systemd` habilitados na inicialização, o que faz a plataforma voltar sozinha depois de uma reinicialização da máquina virtual. Não há, contudo, rota dedicada à verificação de saúde: a forma mais simples de saber se a API está operando é uma requisição sem *token* a uma rota protegida, como `GET /me`, cuja resposta 401 indica que o processo, o roteamento e a verificação de autenticação estão funcionando, embora não comprove o acesso ao banco.

Os segredos e parâmetros da API são mantidos no arquivo `.env`, fora do controle de versão e redigido diretamente na máquina virtual, por acesso remoto. Não há cofre de segredos nem cópia desse arquivo em outro lugar. A consequência é que a máquina guarda a única cópia existente da configuração de produção: perdê-la não implicaria perda de dados, que estão no Supabase e no R2, mas exigiria reconstituir as credenciais de cada serviço. É uma limitação registrada, de correção simples, e não uma decisão de projeto.

9.3 Procedimento de Implantação

A implantação envolve duas atividades distintas: o provisionamento inicial do ambiente, feito uma única vez, e a publicação de novas versões, repetida a cada alteração do sistema.

9.3.1 Provisionamento Inicial

O ambiente foi preparado na seguinte ordem:

1. **Serviços de dados:** criação do projeto no Supabase, na região de São Paulo, e do balde no Cloudflare R2, com acesso público de leitura para a exibição das imagens; geração da senha de aplicativo da conta Google usada no envio de e-mails.

2. **Máquina virtual:** criação da instância na Oracle Cloud, com acesso por SSH autenticado por chave; liberação das portas 80 e 443 nas regras de entrada da rede virtual; atualização dos pacotes do sistema e ativação do UFW, permitindo apenas as portas 22, 80 e 443.

3. **Área de troca:** criação de um arquivo de *swap*, necessário para que a construção da imagem caiba na memória da máquina.

4. **Docker e Caddy:** instalação do Docker pelo *script* oficial do projeto e do Caddy pelo seu repositório de pacotes, habilitado como serviço do `systemd`.

5. **Nomes de domínio:** no painel de DNS da Hostinger, criação do registro do tipo A de `api.camilamattos.com.br`, apontando para o endereço público da máquina, e do registro do tipo CNAME de `tcc.camilamattos.com.br`, indicado pela Vercel ao adicionar o domínio ao projeto.

6. **Código:** geração de uma chave SSH na máquina, cadastrada como chave de implantação (*deploy key*) no repositório do *backend* no GitHub, e clonagem do repositório no diretório `/opt/apps`.

7. **Configuração:** redação do `.env` a partir do modelo `.env.example` e do `Caddyfile` com o nome da API.

8. **Primeira execução:** construção e início do contêiner com `docker compose up -d --build`, que aplica todas as migrações ao banco vazio, e recarga do Caddy, que obtém o certificado.

9. **Aplicação cliente:** importação do repositório do *frontend* na Vercel, definição da variável `API_URL` com o endereço `https://api.camilamattos.com.br`, antes da primeira construção, e associação do domínio `tcc.camilamattos.com.br` ao projeto. Na máquina virtual, a variável `FRONTEND_URL` recebe

`https://tcc.camilamattos.com.br`, a única origem autorizada pelo CORS.

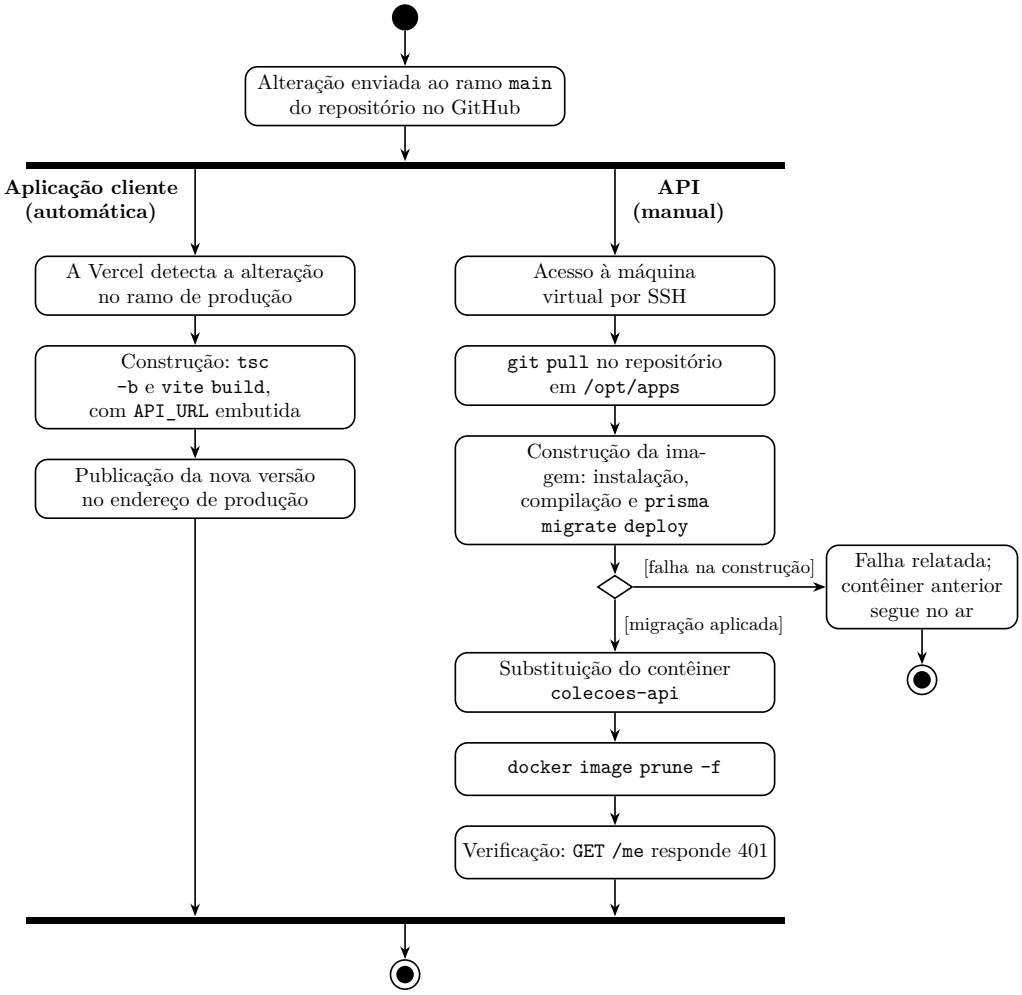
9.3.2 Publicação de Novas Versões

A publicação de uma nova versão segue caminhos diferentes para as duas partes do sistema, como mostra o diagrama de atividades da Figura 20. A aplicação cliente é publicada automaticamente: a cada alteração no ramo principal do repositório, a Vercel constrói o projeto e cria uma nova publicação de produção (Vercel, a). A API é publicada manualmente: o responsável acessa a máquina por SSH e executa o *script* de implantação `scripts/deploy.sh`, versionado no repositório do *backend*, que obtém a versão mais recente do repositório (`git pull`), reconstrói a imagem e substitui o contêiner (`docker compose up -d --build`) e, por fim, remove as imagens que deixaram de ser usadas (`docker image prune -f`).

A diferença de tratamento entre as duas partes decorre do que cada uma exige. A publicação da aplicação cliente não toca em nenhum recurso da máquina virtual nem em dados, e a plataforma que a recebe já oferece a construção automática; automatizá-la não teve custo. A publicação da API, ao contrário, reconstrói a imagem na mesma máquina que atende à produção e aplica migrações ao banco, e foi mantida manual para que cada publicação seja acompanhada por quem a disparou. A automação por uma esteira de integração contínua, que se conectaria à máquina e executaria o mesmo *script*, é um acréscimo possível que não altera nenhum dos passos descritos.

Duas consequências da publicação da API precisam ser registradas. A primeira é a degradação durante a construção: instalar dependências e compilar o código em uma máquina de 1 GB, com parte da memória em disco, disputa recursos com o contêiner em execução, e as respostas da API tornam-se mais lentas enquanto a construção ocorre. A segunda é a indisponibilidade breve no momento da substituição do contêiner, durante a qual o Caddy não encontra o processo na porta local e a aplicação cliente recebe erro nas requisições. Para a escala de uso do trabalho, ambas são aceitáveis; deixariam de ser em uma operação com compromisso formal de disponibilidade.

Figura 20 – Diagrama de atividades da publicação de uma nova versão



Fonte: Elaborada pelo autor

9.3.3 Reversão

A reversão também segue caminhos distintos. Na aplicação cliente, a Vercel permite devolver a produção a uma publicação anterior sem nova construção, recurso que, no plano Hobby, alcança a publicação imediatamente anterior (Vercel, b). Na API, a reversão percorre o mesmo caminho da publicação: o código é revertido no repositório e o *script* de implantação é executado novamente. Como o último passo da publicação remove as imagens que deixaram de ser usadas, não há artefato preservado da versão substituída, e o tempo de reversão é igual ao de uma

publicação completa, e não menor. Uma migração já aplicada, por sua vez, não é desfeita pela reversão do código; por isso, a compatibilidade entre cada migração e a versão anterior do código, discutida na Seção 9.2, é também o que torna a reversão segura.

As decisões descritas neste capítulo (a máquina gratuita de 1 GB, a construção da imagem no próprio ambiente de produção, a publicação manual da API, o banco sem cópia de segurança automática e a configuração mantida apenas na máquina) têm a mesma natureza: trocam garantias operacionais por um custo restrito ao registro do domínio, em um contexto em que o objetivo da operação é validar o sistema, e não sustentar uma exploração comercial. Nenhuma delas decorre da arquitetura descrita nos Capítulos 6 e 7, e todas podem ser revistas por acréscimo, sem alteração do código da aplicação.

10 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados da validação dos requisitos definidos no Capítulo 5 e discute como a plataforma responde ao problema apresentado no Capítulo 1. A Seção 10.1 descreve a plataforma em operação. A Seção 10.2 apresenta a validação dos requisitos funcionais, e a Seção 10.3, a dos requisitos não funcionais. A Seção 10.4 relata o uso da plataforma publicada, e a Seção 10.5 discute os resultados.

10.1 A Plataforma em Operação

A plataforma está em operação pública desde 3 de outubro de 2026, no ambiente descrito no Capítulo 9. O envio de e-mail está ativo em produção, e as mensagens de confirmação de cadastro e de recuperação de senha chegam ao destinatário.

A operação tem caráter de validação acadêmica. As Categorias, Coleções, Itens e *layouts* da base foram criados, em sua maior parte, pela autora durante o desenvolvimento. A plataforma também foi usada por duas pessoas que não participaram do desenvolvimento, conforme a Seção 10.4.

10.2 Validação dos Requisitos Funcionais

O Capítulo 8 registrou os casos de teste e a execução dos testes automatizados, de unidade e de ponta a ponta, cuja distribuição por requisito consta dos Quadros 64 e 73. Esta seção apresenta o resultado obtido em cada verificação e emite o juízo sobre o atendimento de cada requisito funcional. Além dos testes, os requisitos foram verificados pela inspeção do código da aplicação cliente e da API e pelo uso da plataforma publicada. As subseções seguem os grupos de requisitos do Capítulo 5.

10.2.1 Acesso ao Sistema

No cadastro (RF-01), o CT06 mostrou que a conta nova recebe as paletas predefinidas com exatamente uma em uso, correspondente ao tema do sistema operacional informado. Os testes do serviço de confirmação de e-mail, no módulo `user`, mostraram que apenas o *hash* do código de 6 dígitos é guardado, que um código novo aposenta os anteriores e que os pedidos respeitam o intervalo de um minuto e o limite por hora; os testes de ponta a ponta mostraram o acesso negado antes da confirmação e o código errado recusado (Quadro 73). No uso em produção, o código de confirmação chegou ao e-mail cadastrado e liberou o acesso à conta.

No *login* e no *logout* (RF-02), o CT01 mostrou que o *logout* encerra somente a sessão atual, que o *token* de uma sessão ativa é aceito e que o *token* de uma sessão encerrada é recusado como acesso não autorizado.

Na alteração de senha (RF-03), os testes do módulo `user` mostraram que a API exige a senha atual, recusa uma nova senha igual à atual e encerra as demais sessões do usuário.

Na recuperação de senha (RF-04), o CT02 mostrou que a plataforma responde com a mesma mensagem para e-mails cadastrados e não cadastrados, guarda o *token* apenas como *hash*, com data de expiração, aceita o *token* uma única vez e encerra todas as sessões após a redefinição. No uso em produção, o e-mail com o *link* de redefinição chegou ao destinatário.

Na edição do perfil (RF-05), os testes do módulo `user` mostraram que a edição altera apenas os campos do perfil, sem tocar no e-mail nem no código de usuário, que a foto e a capa são gravadas com o recorte e o original juntos e que o reenquadramento reaproveita o original guardado; o uso mostrou o nome, a descrição, a foto e a capa alterados e exibidos no perfil.

Com esses resultados, o usuário acessa a conta somente depois de confirmar o e-mail, e uma sessão encerrada ou uma senha redefinida deixa de dar acesso. Os cinco requisitos do grupo são considerados atendidos.

10.2.2 Coleção

Nas Categorias, Coleções e Itens (RF-06, RF-07 e RF-09), os testes dos módulos `category`, `collection` e `item` foram aprovados, e o uso mostrou a criação, a edição e a exclusão em cada nível. O CT08 mostrou que a plataforma reconhece JPEG e PNG pela assinatura no início do arquivo e recusa um arquivo de texto renomeado como imagem, sem gravar nada no armazenamento. Na tela principal de Categorias (RF-13), o uso mostrou a quantidade de Coleções e de Itens de cada Categoria, a alternância entre grade e lista e as quatro ordenações.

Na privacidade da Coleção (RF-08), o CT04 mostrou que o dono vê as Coleções das três privacidades, o amigo vê as públicas e as somente para amigos e os demais usuários veem apenas as públicas. Para uma Coleção que o visitante não pode ver, a resposta é a mesma de uma Coleção inexistente. O CT03 mostrou que, quando a Coleção se torna privada, todos os acompanhamentos são encerrados na mesma transação, e que, quando ela se torna somente para amigos, são encerrados apenas os acompanhamentos de quem não é amigo do dono.

Na movimentação de Itens (RF-10), os testes do módulo `item` mostraram que o Item com ficha própria a mantém ao mudar de Coleção, e o uso mostrou a movimentação concluída em três passos, abrir, escolher o destino e confirmar, com o botão de desfazer disponível durante trinta segundos.

Na ficha em modo leitura (RF-11), o uso mostrou a ficha aberta sobre a listagem, em modo somente leitura para o visitante. Na listagem de Itens (RF-12), os testes do módulo `item` mostraram as quatro ordenações por nome e por data, com a mais recente primeiro quando nenhuma é pedida, e o uso mostrou a busca pelo nome e pelo subtítulo, feita na própria interface. A ordenação pela classificação e os filtros pelos Elementos não fazem parte do requisito nesta versão (Subseção 5.2.4).

Com esses resultados, cada Coleção é vista apenas por quem a privacidade permite, e a plataforma aceita somente imagens reais. Os oito requisitos do grupo são considerados atendidos.

10.2.3 Editor de *Layout*

No editor (RF-14), os testes do módulo `layout` foram aprovados, e a inspeção mostrou os seis tipos de Elemento e o mesmo componente desenhando a ficha no editor e no modo leitura. Os testes mostraram também que, na desassociação que antecede a exclusão de um *layout* em uso (RF-14, restrição 13), cada Item que ainda o exibia recebe uma cópia própria dele antes que o *layout* seja solto das Coleções.

Na associação a Coleções (RF-15), os testes da mescla de fichas no módulo `common` mostraram que cada Elemento da ficha própria é associado ao novo *layout* pela mesma origem, depois pelo mesmo tipo e nome e, por fim, pelo mesmo tipo na ordem de leitura. O uso mostrou a troca de *layout* precedida da prévia das fichas próprias e da escolha dos Itens a mesclar; os não escolhidos mantiveram sua ficha.

No compartilhamento (RF-16), os testes do módulo `layout` mostraram que o *layout* importado pelo identificador se torna uma cópia independente e que a importação de um *layout* da própria biblioteca é recusada.

Na personalização de um Item (RF-17), o CT05 mostrou que a primeira abertura do editor da ficha cria um *layout* exclusivo do Item, copiado do *layout* da Coleção ou vazio quando a Coleção está sem *layout*. As alterações seguintes ficam somente nessa cópia, e uma edição enviada antes da cópia é recusada por conflito.

Com esses resultados, personalizar um Item altera somente a ficha dele, e um *layout* importado é independente do original. Os quatro requisitos do grupo são considerados atendidos.

10.2.4 Aparência

Nas paletas (RF-18), o CT06 mostrou que a troca de paleta desativa a anterior e ativa a nova em uma única transação e que a exclusão da paleta em uso é recusada. Os testes do módulo `appearance-preset` mostraram cada paleta com a cor primária e as doze subcores preenchidas. O uso mostrou a paleta aplicada a toda a interface no momento da escolha e mantida entre sessões e dispositivos, e o aviso do editor de paletas quando o contraste entre texto e fundo fica abaixo de

4,5:1.

Na tipografia (RF-19), os testes do módulo `appearance-preset` mostraram o tamanho, o espaçamento entre linhas e o peso da fonte aceitos somente dentro das faixas do requisito.

Com esses resultados, a conta tem sempre exatamente uma paleta em uso. Os dois requisitos do grupo são considerados atendidos.

10.2.5 Interação Social

Na amizade (RF-20), o CT07 mostrou que o aceite altera a solicitação somente enquanto ela está pendente e cria, na mesma transação, a notificação para quem fez o pedido. Quando duas respostas chegam ao mesmo tempo, apenas a primeira é registrada, e a segunda é recusada por conflito.

No acompanhamento de Coleções (RF-21), o CT04 mostrou que qualquer usuário pode seguir Coleções públicas, que o amigo pode seguir também as somente para amigos e que seguir uma Coleção própria é recusado.

No *feed* de atualizações (RF-22), os testes do módulo `social` mostraram os eventos das Coleções seguidas, entregues em páginas, com as Coleções somente para amigos incluídas apenas enquanto a amizade estiver aceita e as Coleções privadas excluídas. A inspeção da interface mostrou a atualização automática das novidades por consulta à API a cada minuto e ao voltar à janela, sem recarregar a página.

Na visita a outros usuários (RF-23), o CT04 mostrou a mesma resposta para a Coleção invisível e para a inexistente, e o uso mostrou o perfil, a Categoria e a Coleção visitados em modo somente leitura.

Com esses resultados, cada usuário vê e segue apenas o que a privacidade permite, e cada amizade gera um único vínculo. Os quatro requisitos do grupo são considerados atendidos.

10.2.6 Notificação

Nas notificações (RF-24), os testes do módulo `notification` mostraram a contagem das não lidas, a marcação de uma ou de todas as notificações como

lidas e a limpeza da área, sempre restritas ao usuário autenticado. O uso mostrou o sino do cabeçalho com a quantidade de notificações não lidas, atualizada a cada minuto, e a abertura de uma notificação levando ao que a originou.

Com esses resultados, cada usuário vê e altera apenas as próprias notificações. O requisito é considerado atendido.

10.2.7 Síntese

O Quadro 78 reúne o instrumento, o resultado e a situação de cada requisito funcional.

Os vinte e quatro requisitos funcionais são, portanto, considerados atendidos, tal como especificados no Capítulo 5. O juízo não alcança os comportamentos retirados da especificação desta versão e registrados entre os requisitos adiados (Subseção 5.2.4): a alteração do e-mail cadastrado, a entrega imediata de eventos pelo servidor e a ordenação dos Itens pela classificação, que permanecem como trabalho futuro.

10.3 Validação dos Requisitos Não Funcionais

Os requisitos não funcionais do Capítulo 5 não são o eixo deste trabalho, cuja contribuição está nas funcionalidades verificadas na seção anterior, mas cada um deles merece um juízo explícito sobre o seu atendimento. A situação de cada requisito assume um de quatro valores: **atendido**; **atendido com ressalva**, quando a métrica foi cumprida, mas alguma restrição do requisito comprovadamente não foi; **não verificado**, quando faltou o instrumento ou a medição; e **não atendido**, quando o resultado contraria a métrica. A situação é determinada pela métrica, e uma restrição não cumprida impede que o requisito seja considerado plenamente atendido, ainda que a métrica o seja. Uma restrição que não pôde ser verificada não é contada como descumprida, mas é apontada no juízo do requisito. A posição secundária dos requisitos não funcionais refere-se à contribuição do trabalho, e não à sua importância: vários deles têm prioridade essencial no Capítulo 5, e por isso os que ficaram não verificados ou não atendidos são registrados como lacunas, e

não descartados.

10.3.1 Resultados das Medições

As medições de desempenho seguiram o método descrito na Seção 8.7, em 8 de outubro de 2026. Nenhuma das 441 requisições da medição da API terminou em erro: as 55 de cada uma das oito rotas, contadas as cinco de aquecimento, e uma requisição de ajuste ao final do *script*, fora das amostras. O Quadro 79 apresenta os tempos de resposta da API.

Quadro 78 – Síntese da validação dos requisitos funcionais

Req.	**Instrumento**	**Resultado**	**Situação**
RF-01	CT06; módulo `user`; ponta a ponta; uso	Paletas iniciais com uma em uso; acesso só depois do e-mail confirmado por código	Atendido
RF-02	CT01; módulo `auth`	*Logout* encerra a sessão atual; sessão encerrada recusada	Atendido
RF-03	Módulo `user`; inspeção	Senha atual exigida; demais sessões encerradas	Atendido
RF-04	CT02; módulo `auth`; uso	*Token* de uso único; *link* entregue por e-mail	Atendido
RF-05	Módulo `user`; uso	Só campos do perfil alterados; foto e capa enquadradas, com original guardado	Atendido
RF-06	Módulo `category`; uso	Categorias criadas, editadas e excluídas	Atendido
RF-07	Módulo `collection`; uso	Coleções criadas, editadas e excluídas	Atendido
RF-08	CT03; CT04	Acesso e acompanhamento conforme a privacidade	Atendido
RF-09	CT08; módulo `item`; uso	Imagem reconhecida pelo conteúdo do arquivo	Atendido
RF-10	Módulo `item`; uso	Ficha própria mantida; desfazer por 30 s	Atendido
RF-11	Inspeção; uso	Ficha sobre a listagem, somente leitura ao visitante	Atendido
RF-12	Módulo `item`; uso	Quatro ordenações; busca por nome e subtítulo	Atendido
RF-13	Módulo `category`; uso	Contagens, grade e lista, ordenações	Atendido
RF-14	Módulo `layout`; inspeção	Seis tipos de Elemento; prévia igual à ficha	Atendido
RF-15	Módulos `collection` e `common`; uso	Troca com prévia e mescla das fichas escolhidas	Atendido
RF-16	Módulo `layout`	Cópia independente; importação da própria biblioteca recusada	Atendido
RF-17	CT05	Ficha própria criada na primeira abertura do editor	Atendido
RF-18	CT06; módulo `appearance-preset`; uso	Troca em uma transação; paleta em uso protegida	Atendido
RF-19	Módulo `appearance-preset`	Tipografia nas faixas do requisito	Atendido
RF-20	CT07	Um único vínculo; solicitante notificado	Atendido
RF-21	CT03; CT04; módulo `social`	Acompanhamento conforme a privacidade	Atendido
RF-22	Módulo `social`; inspeção	Eventos das Coleções seguidas, em páginas, com atualização periódica	Atendido
RF-23	CT04; módulo `social`; uso	Perfis e Coleções em modo somente leitura	Atendido
RF-24	Módulo `notification`; uso	Notificações restritas ao usuário; sino com contador	Atendido

Fonte: Elaborado pelo autor

Quadro 79 – Tempo de resposta da API, em milissegundos (n = 50 por rota)

Rota	**RNF**	**Mínimo**	**p50**	**p95**	**Máximo**
Painel de Categorias (`GET /category`)	RNF-02	83,6	89,4	95,1	99,1
Coleções da Categoria (`GET /collection`)	RNF-02	103,3	110,7	115,1	118,7
Lista de 50 Itens (`GET /item`)	RNF-02	146,0	153,1	160,2	177,1
Ficha própria com 15 Elementos (`GET /item/:id/layout`)	RNF-02	109,1	117,1	122,4	123,5
Cadastro de Item (`POST /item`)	RNF-02	338,0	356,2	371,9	373,4
Exclusão de Item (`DELETE /item/:id`)	RNF-02	163,0	172,8	183,0	189,2
Mover Item entre Coleções (`PATCH /item/:id`)	RNF-06	341,7	359,9	382,2	384,1
Gravação de Elemento no editor (`PATCH /layout/:id/elements/:id`)	RNF-07	121,8	132,1	139,2	201,9

Fonte: Elaborada pelo autor

Como a medição não foi repetida no ambiente de produção, esses tempos caracterizam a API e o banco de dados, e não a rede de quem usa a plataforma publicada. O Quadro 80 apresenta o carregamento das telas no navegador, a 10 Mbps.

Quadro 80 – Carregamento das telas no navegador a 10 Mbps, em milissegundos desde o início da navegação (n = 10 por linha)

Tela e cache	**FCP p75**	**LCP p75**	**LCP máximo**	**Última resposta da API p75**	**Máximo**
Lista de 50 Itens, sem cache	364	364	364	743	752
Lista de 50 Itens, com cache	188	188	192	301	321
Ficha própria com 15 Elementos, sem cache	300	476	1108	427	1070
Ficha própria com 15 Elementos, com cache	128	256	296	226	275

Fonte: Elaborada pelo autor

10.3.2 Requisitos Mensuráveis

RNF-02 (desempenho de carregamento). O requisito fixa três limites: três segundos para o carregamento de uma tela principal, dois segundos para as

respostas de leitura da API e três segundos para as operações de escrita. Para a tela, adota-se o maior elemento de conteúdo pintado (LCP), no percentil 75 dos carregamentos, convenção que acompanha a recomendação do web.dev, para o qual um LCP de até 2,5 segundos caracteriza um bom carregamento (WALTON; POLLARD, 2019). Para a API, adota-se o percentil 95, que afirma o limite para a quase totalidade das requisições, e não apenas para a típica. O maior percentil 95 entre as rotas de leitura foi de 160,2 milissegundos (lista de 50 Itens), e o maior entre as de escrita, de 371,9 milissegundos (cadastro de Item), valores cerca de doze e de oito vezes menores que os respectivos limites. No navegador, a 10 Mbps e sem cache, o caso mais desfavorável, o LCP no percentil 75 foi de 364 milissegundos na lista de Itens e de 476 milissegundos na ficha; com cache, caiu para 188 e 256 milissegundos. Na ficha, o maior elemento pintado é o texto de um Elemento, desenhado depois da chegada dos dados. Na lista, o LCP coincidiu com a primeira pintura em todos os carregamentos e ocorreu antes da chegada dos dados, porque o maior elemento pintado é a estrutura da tela, desenhada enquanto as respostas da API são aguardadas. Por isso, registra-se também o instante em que chegou a última resposta da API, a partir do qual os dados podem ser exibidos: na lista sem cache, 743 milissegundos no percentil 75, com máximo de 752. O maior valor observado em todas as medições foi o LCP de 1,1 segundo em um carregamento da ficha sem cache, quase três vezes abaixo do limite de três segundos. No navegador, foram medidas a lista de Itens e a ficha; as telas de Categorias e de Coleções, também citadas pelo requisito, foram medidas apenas pelas respostas da API, as mais rápidas do Quadro 79. Com essa ressalva de alcance, e nas condições do ambiente local descritas na Seção 8.7, a métrica é cumprida. Duas restrições do requisito, no entanto, não são atendidas: as imagens não são otimizadas pelo servidor, e as listagens de Categorias, Coleções e Itens não são paginadas, sendo devolvidas por inteiro em uma única resposta. O requisito é considerado **atendido com ressalva**.

RNF-06 (movimentação de Itens). O requisito exige que a mudança de Coleção se reflita na interface em até 500 milissegundos, que a interface a aplique de forma otimista, antes da confirmação do servidor, e que, em caso de falha, a reverta e informe o usuário. A interface aplica a movimentação de forma otimista:

ao confirmar, o Item sai da lista da Coleção de origem e a janela de escolha do destino se fecha antes da resposta da API; se o servidor recusar a movimentação, a lista volta ao estado anterior e a falha é informada na própria tela. Nas dez movimentações medidas a 10 Mbps, o quadro sem o Item foi pintado entre 41 e 57 milissegundos depois do clique, enquanto a resposta do servidor chegou entre 355 e 477 milissegundos; em todas, a tela mudou antes da resposta. O tempo percebido fica, assim, quase nove vezes abaixo do limite e deixa de depender da gravação, que é a mais lenta das rotas medidas. Essa lentidão se explica pelo número de operações executadas em sequência, cada uma atravessando a internet até o banco: a verificação da sessão, a confirmação de que o Item e a Coleção de destino pertencem ao usuário e a verificação de unicidade do nome, seguidas de uma transação que atualiza o Item, registra o evento e consulta quem acompanha a Coleção. Sem a atualização otimista, a interface teria de aguardar a gravação e reler a lista, e a soma das duas rotas, medida no mesmo dia, foi de 513,1 milissegundos na mediana e de 542,3 no percentil 95, acima do limite. A reversão em caso de falha não foi exercitada na medição, pois nenhuma movimentação falhou, e foi conferida apenas no código. A restrição de desfazer é atendida pelo botão que, por 30 segundos, devolve o Item à Coleção de origem. A medição foi feita sobre a versão da interface que introduziu a atualização otimista, incorporada ao repositório do *frontend* em 10 de outubro de 2026. O requisito é considerado **atendido**.

RNF-07 (desempenho do editor). O requisito limita a 200 milissegundos o tempo entre uma operação no editor e sua exibição. Esse valor coincide com o limite que o web.dev adota para uma boa responsividade na métrica INP, avaliada no percentil 75 das interações (WAGNER; POLLARD, 2022), que é, por isso, a medida adotada. Nas 30 interações registradas no editor, o tempo até a pintura do quadro seguinte foi de 32 milissegundos no percentil 75 e de 48 milissegundos no percentil 98, que, com essa quantidade de interações, coincide com o máximo; o gesto que não foi registrado ficou abaixo do piso de 16 milissegundos. O maior tempo medido ficou, portanto, cerca de quatro vezes abaixo do limite, com o editor exibindo entre 15 e 19 Elementos, o que atende também à restrição de suportar quinze Elementos sem degradação perceptível. A gravação no servidor não compõe esse tempo, pois ocorre de forma assíncrona, depois de uma pausa de 400 milissegundos,

sem bloquear a edição, o que atende à restrição de salvamento assíncrono; ainda assim, registra-se que ela levou 139,2 milissegundos no percentil 95. A restrição de fidelidade da prévia é sustentada pela inspeção que mostrou o mesmo componente desenhando a ficha no editor e no modo leitura (RF-14); a de renderização parcial não foi medida. A medição foi feita em um único computador de mesa, sem limitação do processador, e não representa aparelhos de menor desempenho. Como a métrica foi cumprida e nenhuma restrição foi comprovadamente descumprida, o requisito é considerado **atendido**, nos limites dessa máquina.

RNF-08 (disponibilidade). A disponibilidade de 99% só pode ser aferida por um serviço externo que consulte periodicamente a plataforma publicada, e esse acompanhamento não foi feito; o requisito permanece **não verificado**. A restrição de recuperação automática é atendida apenas em parte: a política de reinício do contêiner e os serviços habilitados na inicialização da máquina virtual, descritos no Capítulo 9, recuperam a API quando o processo ou a máquina param, mas não há verificação de saúde que detecte uma API em execução sem acesso ao banco, e um projeto do Supabase pausado por inatividade só volta por ação manual no painel. Três restrições não são atendidas independentemente da medição: as publicações da API interrompem o serviço por alguns instantes sem aviso prévio, ao contrário das manutenções comunicadas que a restrição exige; não há cópia local para uso sem conexão, e a interface não reenvia automaticamente operações feitas enquanto a conexão estava interrompida, com exceção das alterações do editor, cuja sincronização reenvia o que ficou pendente na tentativa seguinte.

10.3.3 Síntese da Validação

O Quadro 81 reúne a situação de todos os requisitos não funcionais, inclusive os que não dependem de medição de tempo.

Quadro 81 – Síntese da validação dos requisitos não funcionais

Requisito	**Instrumento**	**Resultado**	**Situação**
RNF-01 Usabilidade	Teste com usuários iniciantes (não realizado)	Nenhuma medição do tempo para criar uma Coleção e um Item	Não verificado
RNF-02 Carregamento	*Script* de latência; carregamento das telas no navegador a 10 Mbps	API: p95 máximo de 160,2 ms na leitura e de 371,9 ms na escrita. Telas sem cache: LCP p75 de até 476 ms e máximo de 1,1 s. Sem paginação e sem otimização de imagens	Atendido com ressalva
RNF-03 Segurança de senhas e dados	Inspeção do código; testes automatizados (Capítulo 8)	bcrypt, mensagem genérica no *login*, sessões revogáveis, HTTPS em produção e limite de cinco *logins* por minuto. Registro de erros inesperados com o objeto completo da exceção	Atendido com ressalva
RNF-04 Compatibilidade	Teste em navegadores e larguras de tela (não realizado)	Interface escrita para telas a partir de 320 px; matriz de navegadores não testada	Não verificado
RNF-05 Personalização	Inspeção do código e da interface	Variáveis CSS, alerta de contraste, aplicação imediata e persistência no perfil; métrica de qualquer combinação sem quebras não testada sistematicamente	Não verificado
RNF-06 Movimentação de Itens	Movimentação pela tela no navegador a 10 Mbps; *script* de latência	Atualização otimista: tela muda em até 57 ms, antes da resposta do servidor (até 477 ms); desfazer por 30 s via botão	Atendido
RNF-07 Editor	*Script* de latência; temporização de eventos no navegador	30 interações: p75 de 32 ms e máximo de 48 ms, com 15 a 19 Elementos; gravação assíncrona com p95 de 139,2 ms	Atendido
RNF-08 Disponibilidade	Monitoramento externo (não realizado)	Reinício automático do processo e da máquina; sem verificação de saúde; publicações sem aviso; sem cópia local para uso sem conexão	Não verificado
RNF-09 Acessibilidade	Inspeção do código e registros do projeto	Contraste das paletas padrão verificado e alerta de contraste; navegação do editor por teclado não implementada	Não atendido
RNF-10 Integridade	Inspeção do código; configuração do banco	Transações atômicas e confirmação antes de excluir; sem cópia de segurança automática no plano gratuito; sem registro de auditoria das exclusões	Não atendido

Fonte: Elaborada pelo autor

Quatro situações merecem comentário. O RNF-05 é considerado não verificado porque a sua métrica exige que qualquer combinação de tema e tipografia mantenha as telas sem quebras, e as combinações não foram testadas de forma sistemática; os mecanismos das restrições existem e foram inspecionados, mas não substituem a verificação da métrica. O RNF-03 é considerado atendido com ressalva porque os mecanismos que a métrica exige existem e estão verificados, mas a restrição de não expor dados sensíveis em registros do sistema não é cumprida por inteiro: os erros inesperados são registrados com o objeto completo da exceção, que pode conter valores de campos, e a falha no envio de um código de confirmação registra o e-mail do destinatário. O limite de tentativas de *login* por endereço, por sua vez, depende, atrás do Caddy, da configuração de confiança em um único intermediário descrita no Capítulo 9. O RNF-09 é considerado não atendido porque o requisito exige conformidade com o nível AA das WCAG e que todos os elementos interativos sejam acessíveis por teclado, enquanto a navegação do editor por teclado não foi implementada, de modo que posicionar um Elemento depende de um dispositivo apontador ou dos campos numéricos de posição; o critério de operação por teclado é de nível A, abaixo do AA exigido (CAMPBELL et al., 2024). As demais partes da interface não foram auditadas. Por fim, o RNF-10 é considerado não atendido porque a sua métrica principal, a de cópias de segurança diárias com retenção de 30 dias, não é cumprida no plano gratuito do Supabase, que não faz cópia automática e deixa a exportação dos dados a cargo do responsável pelo projeto (Supabase, c), e porque o registro em auditoria de toda exclusão também não é atendido: apenas as alterações de Itens em Coleções não privadas geram registro de evento.

O resultado de conjunto é coerente com a posição secundária desses requisitos no trabalho. Os tempos de resposta da API e os medidos no navegador, no carregamento das telas, na movimentação de Itens e nas interações do editor, ficaram com folga dentro dos limites no ambiente local em que foram medidos; as ressalvas vêm de restrições, como a paginação das listagens e a otimização das imagens, e não das métricas. Os requisitos que exigem avaliação com pessoas, como usabilidade e acessibilidade, testes sistemáticos de combinações, como a personalização, ou acompanhamento da plataforma publicada, como a disponibilidade, permanecem em aberto, e são essas as lacunas que a avaliação deixa.

10.4 Validação em Uso

Além da autora, duas pessoas que não participaram do desenvolvimento usaram a versão publicada da plataforma, e o uso foi acompanhado por observação.

Uma dessas pessoas pediu um guia de primeiros passos, porque esperava uma ferramenta de listas, parecida com uma planilha. O pedido mostrou que a relação entre o *layout*, a Coleção e as fichas dos Itens fica mais clara com uma apresentação inicial.

A plataforma oferece esse guia, especificado no RNF-01 e descrito na Seção 7.5. A página inicial pública explica o funcionamento com um diagrama que relaciona o *layout*, a Coleção e as fichas; no primeiro acesso, um tour apresenta a área autenticada, e uma lista de primeiros passos, com dicas em cada tela, acompanha o usuário. O efeito do guia sobre o uso não foi medido: a observação registrou apenas o pedido que o motivou.

10.5 Discussão e Implicações

O Capítulo 1 apresentou o problema que motivou este trabalho, que é a falta de uma plataforma que permita organizar coleções de qualquer tipo, definir a estrutura e a aparência de cada registro e compartilhar essas coleções com outras pessoas. A plataforma desenvolvida atende às três partes do problema.

- A organização de coleções de qualquer tipo é feita pela hierarquia de Categorias, Coleções e Itens, cujo tema é definido pelo usuário (RF-06 a RF-13).
- A estrutura e a aparência de cada registro são definidas no editor de *layout*, e a aparência da interface, pelas paletas e pela tipografia (RF-14 a RF-19).
- O compartilhamento é feito pela visita a perfis, pelo acompanhamento de Coleções, pelo *feed* e pelas notificações, sob a privacidade definida em cada Coleção (RF-08 e RF-20 a RF-24).

Esses resultados mostram que a proposta foi construída e está em operação, com os vinte e quatro requisitos funcionais atendidos tal como especificados, e com três comportamentos deixados para versões futuras (Subseção 5.2.4).

O uso relatado na Seção 10.4 mostra uma característica desse tipo de ferramenta. Como o próprio usuário define a estrutura de cada registro, ele precisa entender primeiro a relação entre o *layout* e as fichas, o que uma planilha, de estrutura fixa, dispensa. Por isso, o guia de primeiros passos faz parte do produto.

Duas decisões de projeto do editor trazem vantagens e custos. A primeira é a área de edição que desenha a ficha como ela aparecerá em modo leitura, descrita no Capítulo 6. A prévia é igual à ficha final, e para isso toda a ficha é medida em relação à largura da folha, por meio de consultas de contêiner, um recurso recente do CSS que depende do suporte do navegador. A segunda é a cópia própria do *layout* criada quando um Item é personalizado. Personalizar um Item nunca altera os demais, mas a cópia deixa de acompanhar as mudanças feitas depois no *layout* da Coleção. Para aproximar as duas, a plataforma oferece a mescla da ficha com o *layout* novo, feita apenas nos Itens que o usuário escolhe, que mantém na ficha os Elementos com valor que não encontram correspondência (RF-15).

A privacidade também tem um custo de manutenção. A regra de visibilidade é aplicada em cada rota que recebe o identificador de uma Coleção, de um Item ou de um perfil, e a resposta 404 esconde até a existência do recurso. Toda rota nova precisa aplicar a mesma regra, de modo que o esforço de manutenção cresce com o número de rotas.

A movimentação de Itens também combina uma vantagem e um custo. A interface retira o Item movido da Coleção de origem antes da confirmação do servidor, e por isso o tempo percebido deixa de somar as idas e voltas até o banco, que, como mostrou a Seção 10.3, ultrapassariam o limite do RNF-06. Em troca, a tela mostra por alguns instantes um estado que o banco ainda não confirmou, e precisa ser revertida quando a gravação falha.

11 CONCLUSÕES

Este trabalho propôs-se a desenvolver uma plataforma *web* para coleções digitais de qualquer natureza, organizadas em Categorias, Coleções e Itens, com a apresentação de cada Item definida pelo próprio usuário em um editor de *layouts*, a aparência da interface personalizável por paletas e tipografia e o compartilhamento das coleções com outras pessoas. A plataforma está em operação pública desde 3 de outubro de 2026. Este capítulo avalia o alcance desse propósito, registra as limitações do que foi afirmado, discute as ameaças à validade dos resultados e indica os desdobramentos possíveis.

11.1 Objetivos Alcançados

Os sete objetivos específicos enunciados na Seção 1.2.2 foram alcançados. Os requisitos funcionais e não funcionais, os casos de uso e o modelo de domínio foram especificados no Capítulo 5. O modelo de domínio, organizado em sete pacotes conceituais, deu origem ao projeto de dados do Capítulo 6, ao lado do projeto de interface, cujos esquemas foram elaborados na concepção, e a divisão em pacotes orientou os módulos da API, com os ajustes descritos na Seção 7.4. A organização do acervo em Categorias, Coleções e Itens, com a privacidade definida no nível da Coleção, foi implementada e atende aos requisitos do RF-06 ao RF-13. O editor de *layouts* oferece Elementos de seis tipos, associa um *layout* a Coleções e permite personalizar a ficha de um Item sem alterar os demais, atendendo aos requisitos do RF-14 ao RF-17, e respondeu às interações medidas em até 48 milissegundos, frente ao limite de 200 milissegundos do RNF-07. As paletas de cores e a tipografia são aplicadas a toda a interface e guardadas no perfil do usuário (RF-18 e RF-19). A amizade, o acompanhamento de Coleções, o *feed* de atualizações e as notificações operam sob a privacidade de cada Coleção (RF-20 a RF-24).

O sétimo objetivo, verificar o funcionamento do sistema, foi cumprido nos termos delimitados pelo Capítulo 8 e pela Seção 10.3. Os 520 testes de unidade e

os 26 testes de ponta a ponta da API, estes sobre um banco de dados real, foram aprovados, os vinte e quatro requisitos funcionais foram considerados atendidos tal como especificados, e as métricas de desempenho foram cumpridas com folga no ambiente local de medição: a 10 Mbps e sem cache, as telas medidas atingiram o maior elemento pintado em menos de meio segundo no percentil 75; a movimentação de um Item se refletiu na tela em menos de 60 milissegundos; e o editor respondeu às interações em até 48 milissegundos. A situação de cada requisito não funcional está reunida no Quadro 81. A ressalva é de alcance: a verificação cobre as regras e as telas exercitadas, e não toda combinação de uso possível.

Algumas decisões de projeto sustentam esse resultado. A área de edição desenha a ficha exatamente como ela aparecerá em modo leitura, de modo que o usuário vê o resultado final enquanto edita. A cópia própria do *layout*, criada quando a ficha de um Item é aberta pela primeira vez, isola a personalização de cada Item, e a mescla opcional com o *layout* novo da Coleção preserva os valores de quem personalizou. A regra de privacidade, aplicada em cada rota da API, responde a um recurso restrito como se ele não existisse. E a movimentação de Itens, aplicada de forma otimista, deixa de esperar a gravação no banco para atualizar a tela, decisão orientada pela própria medição, que mostrou que a espera somada à releitura da lista ultrapassaria o limite do RNF-06.

11.2 Limitações

As limitações a seguir incidem sobre o que o trabalho afirma.

O atendimento dos requisitos funcionais refere-se à especificação desta versão. Três comportamentos chegaram a ser especificados e foram retirados para os requisitos adiados (Subseção 5.2.4): a alteração do e-mail cadastrado, com verificação do novo endereço; a entrega imediata do *feed* e das notificações pelo servidor, hoje substituída por consulta periódica; e a ordenação dos Itens pela classificação, com filtros pelo conteúdo dos Elementos.

Os testes de ponta a ponta, que verificam a API junto com um banco de dados real, cobrem os fluxos de autenticação, de organização do acervo e de privacidade e interação social; as demais rotas, como as do editor de *layouts* e das

paletas, são verificadas pelos testes de unidade, com o banco de dados simulado. A interface, por sua vez, foi verificada manualmente e pelas medições no navegador, e não por testes automatizados.

As medições de desempenho foram feitas em um único computador de desenvolvimento, que executava a API e servia a interface, com a API conectada ao banco de dados do Supabase pela internet e sem limitação do processador. Elas caracterizam o código da API e da interface nesse computador, e não a máquina virtual de produção descrita no Capítulo 9, de capacidade muito menor, nem a rede de quem usa a plataforma publicada ou aparelhos de menor desempenho.

A reversão da movimentação de um Item, quando o servidor recusa a operação, foi implementada e conferida no código, mas não foi exercitada nas medições, pois nenhuma das movimentações medidas falhou.

11.3 Ameaças à Validade

Os resultados do Capítulo 10 apoiam-se em condições que convém explicitar. As medições de desempenho provêm de um único dia e de uma única máquina, com dez a cinquenta amostras por cenário, e não constituem série estatística. O uso da plataforma por pessoas que não participaram do desenvolvimento limitou-se a duas pessoas, acompanhadas por observação, o que não permite generalizar a percepção de uso para outros perfis de colecionador. Por fim, a verificação foi conduzida pela própria autora da implementação, sobre os aspectos que ela identificou como relevantes; uma verificação conduzida por terceiros poderia alcançar caminhos que este trabalho não considerou.

11.4 Trabalhos Futuros

Os desdobramentos identificados dividem-se em três grupos. O primeiro corresponde à evolução da plataforma para acervos maiores e para mais formas de interação: o carregamento progressivo das listagens de Categorias, Coleções e Itens, a otimização das imagens enviadas, a navegação do editor de *layouts* por teclado, um atalho de teclado para desfazer a movimentação de Itens e os requisitos

adiados da Subseção 5.2.4, a começar pela ordenação dos Itens pela classificação, pela alteração verificada do e-mail e pela entrega imediata de eventos ao navegador.

O segundo grupo amplia a verificação: a extensão dos testes de ponta a ponta às demais rotas da API e à interface, executada no navegador, uma avaliação de usabilidade com usuários iniciantes, testes em diferentes navegadores e larguras de tela e a repetição das medições de desempenho no ambiente de produção.

O terceiro diz respeito à operação da plataforma publicada: o acompanhamento contínuo de sua disponibilidade por um serviço de monitoramento externo e a adoção de cópias de segurança automáticas do banco de dados.

A verificação por terceiros, mencionada entre as ameaças à validade, é um desdobramento de natureza distinta dos anteriores, e seria o próximo passo natural para sustentar os resultados deste trabalho com independência maior do que a que ele próprio pôde oferecer.

REFERÊNCIAS BIBLIOGRÁFICAS

ABRAHÃO, S. et al. Model-based intelligent user interface adaptation: challenges and future directions. **Software and Systems Modeling**, v. 20, n. 5, p. 1335–1349, 2021. Citado 2 vezes nas páginas 27 e 28.

AICHNER, T. et al. Twenty-five years of social media: A review of social media applications and definitions from 1994 to 2019. **Cyberpsychology, Behavior, and Social Networking**, v. 24, n. 4, p. 215–222, 2021. Citado na página 32.

ALMEIDA, M. G. d.; CANEDO, E. D. Authentication and authorization in microservices architecture: A systematic literature review. **Applied Sciences**, v. 12, n. 6, p. 1–20, 2022. Artigo 3023. Citado na página 39.

ALMENARA, A. P. et al. Enhancing usability assessment with a novel heuristics-based approach validated in an actual business setting. **Journal on Interactive Systems**, v. 15, n. 1, p. 615–631, 2024. Citado na página 32.

ALVES, S. et al. Citizen-led personalization of user interfaces: Investigating how people customize interfaces for themselves and others. **Proceedings of the ACM on Human-Computer Interaction**, v. 8, n. CSCW2, p. 446:1–446:23, 2024. Citado 2 vezes nas páginas 27 e 43.

ALVES, S. et al. Exploring the role of interaction data to empower end-user decision-making in UI personalization. In: **Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems**. Barcelona: ACM, 2026. p. 1–21. Citado 2 vezes nas páginas 27 e 28.

ANTONELLI, L. et al. An approach to extract a conceptual model from natural language specifications. In: **Anais do Workshop em Engenharia de Requisitos (WER 2023)**. Porto Alegre: [s.n.], 2023. p. 163–169. Citado na página 37.

ARORA, C.; GRUNDY, J.; ABDELRAZEK, M. Advancing requirements engineering through generative AI: Assessing the role of LLMs. In: **Generative AI for Effective Software Development**. Cham: Springer, 2024. p. 129–148. Citado na página 35.

BANDY, J.; DIAKOPOULOS, N. More accounts, fewer links: How algorithmic curation impacts media exposure in Twitter timelines. **Proceedings of the**

ACM on Human-Computer Interaction, v. 5, n. CSCW1, p. 78:1–78:28, 2021. Citado na página 33.

BARBOSA, A.; MALCHER, P.; SANTOS, R. P. d. ReQSI-CI: Um catálogo de requisitos não-funcionais para sistemas de informação em cidades inteligentes sob a perspectiva de ecossistemas digitais. In: **Anais do Workshop em Engenharia de Requisitos (WER 2023)**. Porto Alegre: [s.n.], 2023. p. 122–135. Citado na página 36.

BEAUDOUIN-LAFON, M.; BØDKER, S.; MACKAY, W. E. Generative theories of interaction. **ACM Transactions on Computer-Human Interaction**, v. 28, n. 6, p. 45:1–45:54, 2021. Citado 2 vezes nas páginas 30 e 31.

BELK, R. W. et al. Collectors and collecting. In: HOUSTON, M. J. (Ed.). **Advances in Consumer Research**. Provo, UT: Association for Consumer Research, 1988. v. 15, p. 548–553. Citado na página 22.

BENNETT, D. et al. How does HCI understand human agency and autonomy? In: **Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems**. Hamburg: ACM, 2023. p. 1–18. Citado na página 28.

BOYD, d. m.; ELLISON, N. B. Social network sites: Definition, history, and scholarship. **Journal of Computer-Mediated Communication**, v. 13, n. 1, p. 210–230, 2007. Citado 2 vezes nas páginas 32 e 34.

Caddy Web Server. **Automatic HTTPS**. Documentação do Caddy. Disponível em: <https://caddyserver.com/docs/automatic-https>. Acesso em: 02 out. 2026. Citado 2 vezes nas páginas 247 e 251.

Caddy Web Server. **reverse_proxy (Caddyfile directive)**. Documentação do Caddy. Disponível em: <https://caddyserver.com/docs/caddyfile/directives/reverse_proxy>. Acesso em: 02 out. 2026. Citado na página 251.

CAMPBELL, A. et al. **Web Content Accessibility Guidelines (WCAG) 2.2**. 2024. W3C Recommendation. Disponível em: <https://www.w3.org/TR/2024/REC-WCAG22-20241212/>. Acesso em: 02 out. 2026. Citado 2 vezes nas páginas 29 e 270.

CAO, C. C.; BRUCKS, M.; REIMANN, M. Seeking structure in collections: Desire for control motivates engagement in collecting. **Journal of Consumer Research**, v. 52, n. 3, p. 480–501, 2025. Citado 2 vezes nas páginas 22 e 23.

CHAABEN, M. B. et al. On the utility of domain modeling assistance with large language models. **ACM Transactions on Software Engineering and Methodology**, v. 35, n. 4, p. 1–38, 2026. Citado na página 37.

CHARNSETHIKUL, P. et al. Navigating social media privacy: Awareness, preferences, and discoverability. **Proceedings on Privacy Enhancing Technologies**, v. 2025, n. 4, p. 620–638, 2025. Citado na página 34.

CHEN, A.-H.; MUHAMAD, N. Impact of color and polarity on visual resolution with varying contrast ratios and different text backgrounds. **Journal of Ophthalmic and Vision Research**, v. 17, n. 2, p. 217–224, 2022. Citado na página 29.

CHEN, S. et al. Accessible or not? an empirical investigation of Android app accessibility. **IEEE Transactions on Software Engineering**, v. 48, n. 10, p. 3954–3968, 2022. Citado na página 29.

CHOU, C. Y.; AYDEMIR, F. B.; DALPIAZ, F. A comparative study of large and small language models for domain model extraction. In: **Requirements Engineering: Foundation for Software Quality (REFSQ 2026)**. [S.l.]: Springer, 2026. (Lecture Notes in Computer Science), p. 336–351. Citado na página 37.

CODD, E. F. A relational model of data for large shared data banks. **Communications of the ACM**, v. 13, n. 6, p. 377–387, 1970. Citado na página 40.

CORREIA, G. F. A. et al. O despertar da "fortuna" colecionista nas memórias de colecionadores mineiros. **Revista Brasileira de Estudos Organizacionais**, v. 11, n. 2, p. 193–222, 2024. Citado na página 23.

CORREIA, G. F. A. et al. Identidades e identificações nas memórias de colecionadores brasileiros. **Cadernos EBAPE.BR**, v. 24, p. 1–16, 2026. E2025-0065. Citado 2 vezes nas páginas 22 e 23.

DAMASCENO, H. L. C.; NEVES, B. C. As pesquisas com a rede social de leitura Skoob: um mapeamento dos artigos científicos. **Verbo de Minas**, v. 23, n. 42, p. 73–100, 2022. Citado na página 34.

DENEGRI-KNOTT, J.; JENKINS, R.; LINDLEY, S. Valuing digital possessions: the role of affordances. **Journal of Computer-Mediated Communication**, v. 27, n. 6, p. 1–11, 2022. Zmac019. Citado na página 25.

DENEGRI-KNOTT, J. et al. Platformised possessions and relational labour. **Marketing Theory**, v. 24, n. 1, p. 23–44, 2024. Citado na página 25.

Docker. **Build secrets**. Documentação do Docker. Disponível em: <https://docs.docker.com/build/building/secrets/>. Acesso em: 05 out. 2026. Citado na página 249.

Docker. **Packet filtering and firewalls**. Documentação do Docker. Disponível em: <https://docs.docker.com/engine/network/packet-filtering-firewalls/>. Acesso em: 05 out. 2026. Citado na página 250.

DÖHMEN, T. et al. SchemaPile: A large collection of relational database schemas. **Proceedings of the ACM on Management of Data**, v. 2, n. 3, p. 172:1–172:25, 2024. Citado na página 41.

ELLISON, N. B.; BOYD, d. Sociality through social network sites. In: DUTTON, W. H. (Ed.). **The Oxford Handbook of Internet Studies**. Oxford: Oxford University Press, 2013. p. 151–172. Citado na página 33.

EPIFÂNIO, J. et al. Identifying knowledge gaps in requirements engineering: An empirical study with professionals in the brazilian software industry. In: **Anais do Workshop em Engenharia de Requisitos (WER 2023)**. Porto Alegre: [s.n.], 2023. p. 230–237. Citado na página 35.

EPSTEIN, D. A. et al. Mapping and taking stock of the personal informatics literature. **Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies**, v. 4, n. 4, p. 126:1–126:38, 2020. Citado na página 26.

FIELDING, R. T. **Architectural Styles and the Design of Network-based Software Architectures**. Tese (Doutorado) — University of California, Irvine, 2000. Disponível em: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Citado 3 vezes nas páginas 39, 45 e 53.

FIELDING, R. T.; NOTTINGHAM, M.; RESCHKE, J. **HTTP Semantics**. 2022. RFC 9110, STD 97. Internet Engineering Task Force (IETF). Disponível em: <https://www.rfc-editor.org/rfc/rfc9110.txt>. Acesso em: 02 out. 2026. Citado na página 39.

FINARDI, F. S.; SOUSA, R. P. L. d. Colecionismo moderno: conhecimento, memória e indústria cultural. **Tríades em Revista**, v. 9, n. 2, p. 84–99, 2020. Citado 3 vezes nas páginas 21, 22 e 23.

FONTELLES, M. J. et al. Metodologia da pesquisa científica: Diretrizes para a elaboração de um protocolo de pesquisa. **Revista Paraense de Medicina**, v. 23, n. 3, p. 1–8, 2009. Disponível em: <http://files.bvs.br/upload/S/0101-5907/2009/v23n3/a1967.pdf>. Citado na página 44.

FOWLER, M. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002. Citado 3 vezes nas páginas 45, 47 e 52.

FRANCH, X. et al. The state-of-practice in requirements specification: an extended interview study at 12 companies. **Requirements Engineering**, v. 28, n. 3, p. 377–409, 2023. Citado 3 vezes nas páginas 35, 36 e 37.

GAZIT, T. et al. The dark side of the interface: examining the influence of different background modes on cognitive performance. **Ergonomics**, v. 69, n. 5, p. 828–841, 2025. Publicado online em 2025; fascículo de 2026. Citado na página 28.

GERHARDT, T. E.; SILVEIRA, D. T. **Métodos de Pesquisa**. Porto Alegre: Editora da UFRGS, 2009. Citado na página 45.

GOLMOHAMMADI, A.; ZHANG, M.; ARCURI, A. Testing RESTful APIs: A survey. **ACM Transactions on Software Engineering and Methodology**, v. 33, n. 1, p. 1–41, 2023. Citado na página 39.

GRANOVETTER, M. S. The strength of weak ties. **American Journal of Sociology**, v. 78, n. 6, p. 1360–1380, 1973. Citado na página 33.

GRAY, J.; RUMPE, B. Modeling in the large: model libraries. **Software and Systems Modeling**, v. 20, n. 3, p. 591–593, 2021. Citado na página 38.

GUPTA, R. et al. LLM-assisted web application functional requirements generation: A case study of four popular LLMs over a mess management system. In: **Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE 2025)**. [S.l.]: ACM, 2025. p. 1076–1085. Citado na página 36.

HABIBULLAH, K. M.; GAY, G.; HORKOFF, J. Non-functional requirements for machine learning: understanding current use and challenges among practitioners. **Requirements Engineering**, v. 28, n. 2, p. 283–316, 2023. Citado na página 36.

HADDAD, S. et al. Good GUIs, bad GUIs: Affective evaluation of graphical user interfaces. In: **Proceedings of the 32nd ACM Conference on User Modeling, Adaptation and Personalization (UMAP '24)**. Cagliari: ACM, 2024. p. 232–243. Citado na página 29.

HIDER, P.; STEELE, G. LibraryThing and literary works revisited: Are social and library cataloging just as complementary as they were a decade ago? **Library Resources & Technical Services**, v. 65, n. 3, p. 113–125, 2021. Citado na página 34.

HOPPE, P. H. B.; SOUZA, V. E. S. Support for single page application frameworks on FrameWeb. In: **Proceedings of the 29th Brazilian Symposium on Multimedia and the Web (WebMedia '23)**. Ribeirão Preto: ACM, 2023. p. 260–268. Citado 3 vezes nas páginas 38, 40 e 42.

HOVEN, E. van den; ORTH, D.; ZIJLEMA, A. Possessions and memories. **Current Opinion in Psychology**, v. 39, p. 94–99, 2021. Citado na página 26.

HSU, H. et al. PosterLayout: A new benchmark and approach for content-aware visual-textual presentation layout. In: **2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)**. [S.l.]: IEEE, 2023. p. 6018–6026. Citado na página 32.

HUBER, S. P. et al. AiiDA 1.0, a scalable computational infrastructure for automated reproducible workflows and data provenance. **Scientific Data**, v. 7, n. 300, 2020. Citado na página 41.

HUSZÁR, F. et al. Algorithmic amplification of politics on Twitter. **Proceedings of the National Academy of Sciences**, v. 119, n. 1, p. e2025334119, 2022. Citado na página 33.

HUTCHINS, E. L.; HOLLAN, J. D.; NORMAN, D. A. Direct manipulation interfaces. **Human-Computer Interaction**, v. 1, n. 4, p. 311–338, 1985. Citado na página 31.

JIANG, L.; NAUMANN, F. Holistic primary key and foreign key detection. **Journal of Intelligent Information Systems**, v. 54, n. 3, p. 439–461, 2020. Citado na página 41.

JOLAK, R. et al. Software engineering whispers: The effect of textual vs. graphical software design descriptions on software design communication. **Empirical Software Engineering**, v. 25, n. 6, p. 4427–4471, 2020. Citado na página 37.

JONES, M. B.; BRADLEY, J.; SAKIMURA, N. **JSON Web Token (JWT)**. [S.l.], 2015. Disponível em: <https://www.rfc-editor.org/rfc/rfc7519>. Citado na página 39.

JORGENSEN, J. J.; SORENSEN, K.; SPILINEK, M. Motivations to collect: How consumers are socialized to build product collections. **Social Sciences**, v. 12, n. 12, p. 671, 2023. Citado na página 23.

JÚNIOR, E. W.; FARIAS, K.; SILVA, B. d. On the use of UML in the brazilian industry: A survey. **Journal of Software Engineering Research and Development**, v. 10, p. 10:1–10:16, 2022. Disponível em: <https://journals-sol.sbc.org.br/index.php/jserd/article/view/2554>. Citado na página 37.

KIM, B.; LEE, G.; SUK, H.-J. Color adjustment of brand logos for dark mode display. **PLOS ONE**, v. 21, n. 1, p. e0339392, 2026. Citado 2 vezes nas páginas 27 e 29.

KOMPERLA, V. et al. React: A detailed survey. **Indonesian Journal of Electrical Engineering and Computer Science**, v. 26, n. 3, p. 1710–1717, 2022. Citado na página 40.

KOTSTEIN, S.; BOGNER, J. Which RESTful API design rules are important and how do they improve software quality? a Delphi study with industry experts. In: BARZEN, J. (Ed.). **Service-Oriented Computing: SummerSOC 2021**. Cham: Springer, 2021. (Communications in Computer and Information Science, v. 1429), p. 154–173. Citado na página 39.

LARMAN, C. **Utilizando UML e Padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo**. 3. ed. Porto Alegre: Bookman, 2007. Citado 4 vezes nas páginas 45, 52, 62 e 184.

LAURINDO, A. d. O.; TERMOS, A. M.; CARDOSO, L. S. JSON web tokens: Um estudo sobre sua aplicação. In: **Anais do XXI Congresso Latino-Americano de Software Livre e Tecnologias Abertas (Latinoware 2024)**. [S.l.]: Sociedade Brasileira de Computação, 2024. p. 456–459. Citado na página 40.

LEE, C. et al. GUIComp: A GUI design assistant with real-time, multi-faceted feedback. In: **Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems**. [S.l.]: ACM, 2020. p. 1–13. Citado na página 32.

LEITE, H. G. d. A.; ALVES, V. H.; MANDAJI, C. F. d. S. Mujer en foco: interacciones con las películas de Greta Gerwig en Letterboxd. **Razón y Palabra**, v. 29, n. 122, p. 86–100, 2025. Citado na página 34.

LEMOS, A.; PASTOR, L. Experiência algorítmica: ação e prática de dado na plataforma Instagram. **Contracampo**, v. 39, n. 2, p. 132–146, 2020. Citado na página 33.

LI, I.; DEY, A.; FORLIZZI, J. A stage-based model of personal informatics systems. In: **Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '10)**. New York: ACM, 2010. p. 557–566. Citado na página 26.

LIMA, A. L. d. S. et al. A deep learning model for the assessment of the visual aesthetics of mobile user interfaces. **Journal of the Brazilian Computer Society**, v. 30, n. 1, p. 102–115, 2024. Citado na página 29.

LYALL, B. Narratives in numbers: Sociotechnical storytelling with self-tracking. **Memory, Mind & Media**, v. 3, p. 1–20, 2024. E1. Citado na página 26.

MARATHE, S.; SUNDAR, S. S. What drives customization? control or identity? In: **Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '11)**. Vancouver: ACM, 2011. p. 781–790. Citado na página 17.

MARDON, R.; DENEGRI-KNOTT, J.; MOLESWORTH, M. "kind of mine, kind of not": Digital possessions and affordance misalignment. **Journal of Consumer Research**, v. 50, n. 2, p. 255–281, 2023. Citado 3 vezes nas páginas 24, 25 e 43.

MARQUES, R. **Melhores práticas na elaboração de casos de teste**. 2018. Blog Cedro Technologies. Disponível em: <https://www.cedrotech.com/blog/melhores-praticas-na-elaboracao-de-casos-de-teste/>. Acesso em: 13 set. 2026. Citado na página 227.

MASSON, D. et al. DirectGPT: A direct manipulation interface to interact with large language models. In: **Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)**. [S.l.]: ACM, 2024. p. 1–16. Citado na página 31.

MDN Web Docs. **Mixed content**. Mozilla Developer Network. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/Security/Mixed_content>. Acesso em: 05 out. 2026. Citado na página 247.

MONTEIRO, J. C. d. S. A plataforma Skoob e os novos métodos de incentivo à leitura. **Asas da Palavra**, v. 17, n. 1, p. 61–69, 2020. Citado na página 34.

MOREWEDGE, C. K. et al. Evolution of consumption: A psychological ownership framework. **Journal of Marketing**, v. 85, n. 1, p. 196–218, 2021. Citado na página 25.

MORISSO, J. G. D.; FERREIRA, M. G. G. Relações entre experiência do usuário e design emocional na interação humano computador. **Arcos Design**, v. 14, n. 2, p. 46–61, 2021. Citado na página 29.

NEUMANN, A.; LARANJEIRO, N.; BERNARDINO, J. An analysis of public REST web service APIs. **IEEE Transactions on Services Computing**, v. 14, n. 4, p. 957–970, 2021. Citado na página 39.

NIELSEN, J. Enhancing the explanatory power of usability heuristics. In: **Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '94)**. [S.l.]: ACM, 1994. p. 152–158. Citado 2 vezes nas páginas 29 e 31.

NOURAEI, F. et al. Thinking outside the box: Non-designer perspectives and recommendations for template-based graphic design tools. In: **Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)**. Honolulu: ACM, 2024. p. 1–9. Citado na página 32.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (OMG UML), Version 2.5.1**. [S.l.], 2017. OMG Document Number formal/2017-12-05. Disponível em: <https://www.omg.org/spec/UML/2.5.1/>. Citado 2 vezes nas páginas 36 e 38.

ODOM, W. et al. Lost in translation: Understanding the possession of digital things in the cloud. In: **Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '12)**. New York: ACM, 2012. p. 781–790. Citado 3 vezes nas páginas 25, 26 e 34.

OLIVEIRA, C. Coleções e colecionadores: as práticas de colecionar, motivações e simbologias. **Museologia & Interdisciplinaridade**, v. 6, n. 12, p. 169–179, 2017. Citado 3 vezes nas páginas 21, 22 e 23.

OLLILA, R.; MÄKITALO, N.; MIKKONEN, T. Modern web frameworks: A comparison of rendering performance. **Journal of Web Engineering**, v. 21, n. 3, p. 789–813, 2022. Citado na página 40.

OLSSON, T.; SENTILLES, S.; PAPATHEOCHAROUS, E. A systematic literature review of empirical research on quality requirements. **Requirements Engineering**, v. 27, n. 2, p. 249–271, 2022. Citado na página 35.

Oracle. **Always Free Resources**. Documentação da Oracle Cloud Infrastructure. Disponível em: <https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm>. Acesso em: 05 out. 2026. Citado na página 245.

PALOMARES, I. et al. Reciprocal recommender systems: Analysis of state-of-art literature, challenges and opportunities towards social recommendation. **Information Fusion**, v. 69, p. 103–127, 2021. Citado na página 33.

PERRIG, S. A. C. et al. Smartphone app aesthetics influence users' experience and performance. **Frontiers in Psychology**, v. 14, p. 1113842, 2023. Citado na página 29.

POELL, T.; NIEBORG, D.; DIJCK, J. van. Plataformização. **Fronteiras: estudos midiáticos**, v. 22, n. 1, p. 2–10, 2020. Citado na página 33.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016. v. 8. ISBN 9788580555332. Citado 3 vezes nas páginas 45, 59 e 104.

RAVA, G. The 'dizziness' of accumulation: how digital collecting is undermining the very meaning of collection. **Punctum. International Journal of Semiotics**, v. 9, n. 1, p. 171–188, 2023. Citado 2 vezes nas páginas 22 e 23.

RAZALI, M. N.; ISHAK, W. H. W. A user-centered digital repository: Designing and evaluating HobbyHaven for collectors. **Journal of Digital System Development**, v. 3, n. 1, p. 68–76, 2025. Citado 2 vezes nas páginas 24 e 43.

RECUERO, R.; ZAGO, G. Em busca das "redes que importam": redes sociais e capital social no Twitter. **Líbero**, v. 12, n. 24, p. 81–94, 2009. Citado na página 33.

RHEU, M. et al. Jogging in your avatar's footsteps: The effects of avatar customization and control intuitiveness. **Frontiers in Virtual Reality**, v. 3, p. 873689, 2022. Citado na página 28.

RIBEIRO, C. M. F.; SALCEDO, D. A. Práticas de colecionismo filatélico brasileiro por meio das mídias digitais. **Informação em Pauta**, v. 7, p. 1–19, 2022. Citado na página 24.

RIGALT, D. C. El coleccionismo desde la prehistoria hasta el siglo XVI: entre la motivación religiosa, el deleite artístico, los códigos de exhibición y el negocio. **La Albolafia: Revista de Humanidades y Cultura**, n. 20, p. 13–35, 2020. Citado 2 vezes nas páginas 21 e 22.

SANTANDER, V. F. A. et al. Obtendo casos de uso a partir de modelos de processos de negócio: Uma revisão sistemática da literatura. In: **Anais do Workshop em Engenharia de Requisitos (WER 2022)**. Natal: [s.n.], 2022. Citado na página 36.

SARHAN, Q. I. et al. Software module clustering: An in-depth literature analysis. **IEEE Transactions on Software Engineering**, v. 48, n. 6, p. 1905–1928, 2022. Citado na página 38.

SATYANARAYAN, A. et al. Critical reflections on visualization authoring systems. **IEEE Transactions on Visualization and Computer Graphics**, v. 26, n. 1, p. 461–471, 2020. Citado na página 30.

SHAO, S. et al. Database-access performance antipatterns in database-backed web applications. In: **2020 IEEE International Conference on Software**

Maintenance and Evolution (ICSME). [S.l.]: IEEE, 2020. p. 58–69. Citado na página 42.

SHNEIDERMAN, B. Direct manipulation: A step beyond programming languages. **Computer**, v. 16, n. 8, p. 57–69, 1983. Citado 2 vezes nas páginas 30 e 31.

SILVA, H. H. C. d.; NUNES, M. R. F.; STREHLAU, V. I. A experiência estética no consumo de coleções: um estudo sobre colecionadores de estátuas e figuras de ação. **Signos do Consumo**, v. 12, n. 1, p. 94–111, 2020. Citado 2 vezes nas páginas 22 e 23.

SILVA, J. V. D. et al. Personalização de interfaces adaptativas baseada em análise de dados do usuário: uma revisão sistemática. In: **Anais do XXIV Simpósio Brasileiro sobre Fatores Humanos em Sistemas Computacionais (IHC 2025)**. [S.l.]: SBC, 2025. p. 1466–1489. Citado na página 28.

SILVA, M. O. et al. Brazilian reading preferences in Goodreads: Cross-state and cross-region analyses. **iSys: Revista Brasileira de Sistemas de Informação**, v. 15, n. 1, p. 25:1–25:20, 2022. Citado na página 24.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Tradução: Ivan Bosnic e Kalinka G. de O. Gonçalves. ISBN 9788579361081. Citado 6 vezes nas páginas 45, 51, 59, 64, 65 e 104.

SRINIVASAN, A. et al. InChorus: Designing consistent multimodal interactions for data visualization on tablet devices. In: **Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems**. [S.l.]: ACM, 2020. p. 1–13. Citado na página 31.

STERMAN, S.; NICHOLAS, M. J.; PAULOS, E. Towards creative version control. **Proceedings of the ACM on Human-Computer Interaction**, v. 6, n. CSCW2, p. 336:1–336:25, 2022. Citado na página 32.

STEWEN, L.; KLEPPMANN, M. Undo and redo support for replicated registers. In: **Proceedings of the 11th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC '24)**. [S.l.]: ACM, 2024. p. 1–7. Citado na página 32.

STOREY, V. C.; LUKYANENKO, R.; CASTELLANOS, A. Conceptual modeling: Topics, themes, and technology trends. **ACM Computing Surveys**, v. 55, n. 14s, p. 1–38, 2023. Citado 2 vezes nas páginas 37 e 38.

SUBRAMONYAM, H. et al. Bridging the gulf of envisioning: Cognitive challenges in prompt based interactions with LLMs. In: **Proceedings of the CHI**

Conference on Human Factors in Computing Systems (CHI '24). [S.l.]: ACM, 2024. p. 1–19. Citado na página 31.

SUNDAR, S. S. Rise of machine agency: A framework for studying the psychology of human-AI interaction (HAII). **Journal of Computer-Mediated Communication**, v. 25, n. 1, p. 74–88, 2020. Citado na página 27.

SUNDAR, S. S.; MARATHE, S. S. Personalization versus customization: The importance of agency, privacy, and power usage. **Human Communication Research**, v. 36, n. 3, p. 298–322, 2010. Citado na página 27.

SUNDSTRÖM, A. d. S. S.; ALBUQUERQUE, A. C. d. Colecionismo bibliográfico: contexto histórico, terminologia e perspectivas de estudo na ciência da informação. **Em Questão**, v. 26, n. 3, p. 250–275, 2020. Citado 2 vezes nas páginas 21 e 22.

Supabase. **About billing on Supabase**. Documentação do Supabase. Disponível em: <https://supabase.com/docs/guides/platform/billing-on-supabase>. Acesso em: 05 out. 2026. Citado na página 246.

Supabase. **Connect to your database**. Documentação do Supabase. Disponível em: <https://supabase.com/docs/guides/database/connecting-to-postgres>. Acesso em: 05 out. 2026. Citado na página 245.

Supabase. **Database Backups**. Documentação do Supabase. Disponível em: <https://supabase.com/docs/guides/platform/backups>. Acesso em: 05 out. 2026. Citado 2 vezes nas páginas 246 e 270.

Supabase. **Project Pausing**. Documentação do Supabase. Disponível em: <https://supabase.com/docs/guides/platform/free-project-pausing>. Acesso em: 05 out. 2026. Citado na página 246.

THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 18 Documentation: 5.5 Constraints**. [S.l.], 2025. Disponível em: <https://www.postgresql.org/docs/current/ddl-constraints.html>. Acesso em: 8 out. 2026. Citado na página 41.

THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 18 Documentation: 8.14 JSON Types**. [S.l.], 2025. Disponível em: <https://www.postgresql.org/docs/current/datatype-json.html>. Acesso em: 10 out. 2026. Citado na página 41.

THELWALL, M.; KOUSHA, K. Goodreads: A social network site for book readers. **Journal of the Association for Information Science and Technology**, v. 68, n. 4, p. 972–983, 2017. Citado na página 34.

TODI, K. et al. Adapting user interfaces with model-based reinforcement learning. In: **Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems**. Yokohama: ACM, 2021. p. 1–13. Citado na página 28.

TRACTINSKY, N.; KATZ, A. S.; IKAR, D. What is beautiful is usable. **Interacting with Computers**, v. 13, n. 2, p. 127–145, 2000. Citado na página 28.

VAITHILINGAM, P. et al. DynaVis: Dynamically synthesized UI widgets for visualization editing. In: **Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)**. [S.l.]: ACM, 2024. p. 1–17. Citado na página 31.

VALENTE, M. T. **Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade**. Belo Horizonte: Independente, 2020. ISBN 978-6500019504. Citado 6 vezes nas páginas 35, 36, 37, 38, 59 e 116.

VALIATI, V.; LUPINACCI, L.; SOARES, F. B. Wrap your head around it: algorithmic self-making and performances of taste on Spotify Wrapped. **Information, Communication & Society**, p. 1–22, 2026. Publicado online em 16 abr. 2026. Citado na página 26.

VEPSÄLÄINEN, J.; HEVERY, M.; VUORIMAA, P. Resumability: A new primitive for developing web applications. **IEEE Access**, v. 12, p. 9038–9046, 2024. Citado na página 40.

Vercel. **Deploying Git Repositories with Vercel**. Documentação da Vercel. Disponível em: <https://vercel.com/docs/git>. Acesso em: 05 out. 2026. Citado 2 vezes nas páginas 244 e 254.

Vercel. **Performing an Instant Rollback on a Deployment**. Documentação da Vercel. Disponível em: <https://vercel.com/docs/instant-rollback>. Acesso em: 05 out. 2026. Citado na página 255.

Vercel. **Vite on Vercel**. Documentação da Vercel. Disponível em: <https://vercel.com/docs/frameworks/frontend/vite>. Acesso em: 02 out. 2026. Citado na página 245.

WAGNER, J.; POLLARD, B. **Interaction to Next Paint (INP)**. 2022. Web.dev. Atualizado em 2 set. 2025. Disponível em: <https://web.dev/articles/inp>. Acesso em: 02 out. 2026. Citado na página 267.

WALSH, M.; ANTONIAK, M. The Goodreads “classics”: A computational study of readers, Amazon, and crowdsourced amateur criticism. **Journal of Cultural Analytics**, v. 6, n. 2, 2021. Citado na página 34.

WALTON, P.; POLLARD, B. **Largest Contentful Paint (LCP)**. 2019. Web.dev. Atualizado em 4 set. 2025. Disponível em: <https://web.dev/articles/lcp>. Acesso em: 02 out. 2026. Citado na página 266.

WANG, C. et al. Automatic generation of acceptance test cases from use case specifications: An NLP-based approach. **IEEE Transactions on Software Engineering**, v. 48, n. 2, p. 585–616, 2022. Citado 2 vezes nas páginas 36 e 37.

WATKINS, R. D.; SELLEN, A.; LINDLEY, S. E. Digital collections and digital collecting practices. In: **Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI ’15)**. Seoul: ACM, 2015. p. 3423–3432. Citado 4 vezes nas páginas 16, 23, 24 e 43.

WERNER, C. et al. Continuously managing NFRs: Opportunities and challenges in practice. **IEEE Transactions on Software Engineering**, v. 48, n. 7, p. 2629–2642, 2022. Citado na página 35.

World Wide Web Consortium. **Understanding Success Criterion 1.4.3: Contrast (Minimum)**. 2026. Understanding WCAG 2.2. Disponível em: <https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html>. Acesso em: 8 out. 2026. Citado na página 29.

YANG, J. et al. Managing data constraints in database-backed web applications. In: **Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE 2020)**. [S.l.]: ACM, 2020. p. 1098–1109. Citado 2 vezes nas páginas 38 e 42.

YANG, J. et al. Token time bomb: Evaluating JWT implementations for vulnerability discovery. In: **Proceedings of the Network and Distributed System Security Symposium (NDSS 2026)**. San Diego: Internet Society, 2026. Citado na página 39.

ZHADKO-BAZILEVYCH, S. Analysis of ORM framework approaches for Node.js. **Journal of Computer Sciences Institute**, v. 37, p. 426–430, 2025. Citado na página 42.

ZHANG, Y. et al. Automated and context-aware repair of color-related accessibility issues for Android apps. In: **Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations**

of Software Engineering (ESEC/FSE '23). San Francisco: ACM, 2023. p. 1255–1267. Citado na página 29.

ZHAO, L. et al. Natural language processing for requirements engineering: A systematic mapping study. **ACM Computing Surveys**, v. 54, n. 3, p. 1–41, 2021. Citado na página 35.